 © 978-88-08-19966-9

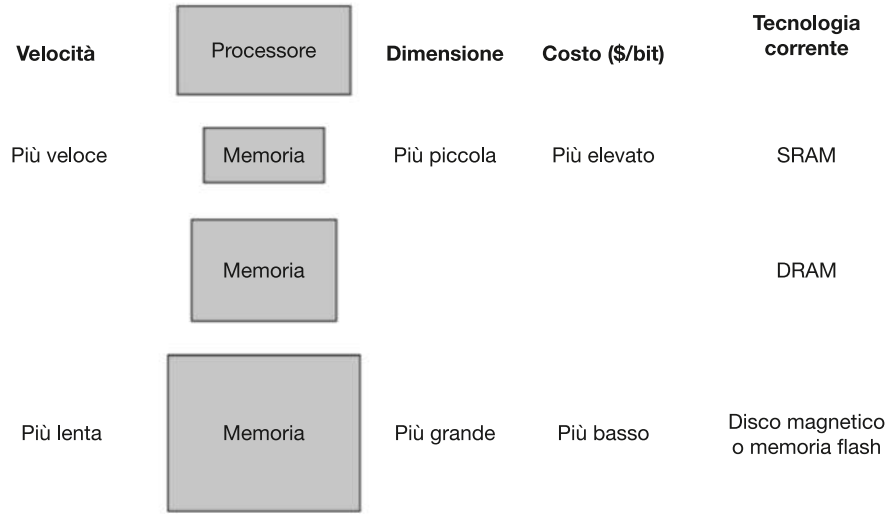


Figura 5.1 Struttura base di una gerarchia delle memorie. Realizzando il sistema di memoria in maniera gerarchica, l'utente ha l'impressione di utilizzare una memoria grande come quella del livello contenente la memoria di dimensioni maggiori, ma con una velocità di accesso pari a quella della memoria realizzata con la tecnologia più veloce. Le memorie flash hanno sostituito i dischi in molti dispositivi embedded e potrebbero portare in breve tempo all'introduzione di un nuovo livello di memoria nei calcolatori desktop e nei server (Paragrafo 5.2).

Anche i dati sono organizzati in modo gerarchico: un livello più vicino al processore contiene in generale un sottoinsieme dei dati memorizzati in ognuno dei livelli sottostanti, e tutti i dati si trovano memorizzati nel livello più basso della memoria. Si consideri ancora l'analogia con la biblioteca: i libri sulla scrivania rappresentano il sottoinsieme dei libri della biblioteca su cui lo studente sta lavorando; a sua volta la biblioteca è un sottoinsieme di tutte le biblioteche dell'università. Quanto più ci si allontana dal processore, tanto più aumenta il tempo necessario per accedere ai dati, proprio come accade per la consultazione dei libri in una gerarchia di biblioteche universitarie.

Blocco: detto anche linea, è l'unità minima di informazione che può essere presente o assente in una cache.

Una gerarchia delle memorie può essere composta da più livelli, ma i dati vengono di volta in volta trasferiti solo tra due livelli vicini; possiamo perciò concentrare la nostra attenzione su due soli livelli di memoria. Quello superiore, più vicino al processore, è più piccolo e veloce del livello inferiore, poiché utilizza una tecnologia più costosa. La più piccola quantità di informazione che può essere presente o assente in questa gerarchia su due livelli è denominata **blocco**, o **linea**, come illustrato nella Figura 5.2; nell'analogia con la biblioteca, il blocco di informazione corrisponde a un libro.

Se il dato richiesto dal processore è contenuto in uno dei blocchi presenti nel livello superiore, si dice che la richiesta ha avuto successo e si denota questo evento con il termine inglese *hit* (letteralmente, "colpito"); una hit corrisponde allo studente che trova le informazioni che cerca in uno dei libri che ha a disposizione sulla scrivania. Se il dato non viene trovato nel livello superiore della gerarchia, si dice che la richiesta fallisce e si indica questo evento con il termine *miss* (letteralmente, "mancato"). In questo secondo caso, per trovare il blocco che contiene il dato richiesto occorre accedere al livello inferiore della gerarchia. Proseguendo con l'analogia della biblioteca, lo studente si alza dalla scrivania e va a cercare il libro che gli serve sugli scaffali. La frequenza di hit, chiamata **hit rate** (letteralmente, "frequenza delle hit"), è la frazione degli accessi alla memoria nei quali il dato desiderato è stato trovato nel livello superiore; spesso l'hit rate viene utilizzato come indice delle prestazioni della gerarchia delle memorie. La frequenza delle miss, indicata con il termine

Hit rate: detto anche frequenza delle hit, è la frazione degli accessi alla memoria nei quali la parola cercata viene trovata in un certo livello della gerarchia.

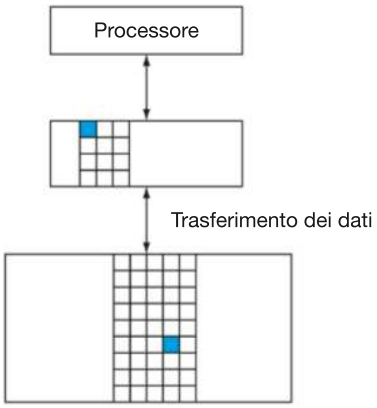


Figura 5.2 Ogni coppia di livelli in una gerarchia delle memorie può essere vista come formata da un livello superiore e da un livello inferiore. All'interno di ciascun livello, l'unità di informazione che è presente o assente è chiamata *blocco* o *linea*. Di solito trasferiamo un intero blocco quando copiamo qualcosa fra i due livelli.

miss rate, è pari a (1 – hit rate) ed è la frazione degli accessi alla memoria nei quali il dato desiderato non è stato trovato nel livello superiore.

Poiché il motivo principale che ha condotto all'organizzazione gerarchica delle memorie è l'aumento delle prestazioni, la velocità degli accessi è importante sia in caso di successo sia in caso di fallimento. Il **tempo di hit** è il tempo di accesso al livello superiore della gerarchia delle memorie e comprende anche il tempo necessario a stabilire se il tentativo di accesso si risolve in un successo o in un fallimento, cioè se produce una hit o una miss. Il tempo di hit è analogo al tempo necessario allo studente per passare in rassegna i libri che ha sulla scrivania. La **penalità di miss** è il tempo necessario a sostituire un blocco del livello superiore con un nuovo blocco, caricato dal livello inferiore della gerarchia, e a trasferire i dati contenuti in questo blocco al processore. La penalità di miss è il tempo che serve per prendere un nuovo libro dagli scaffali e metterlo sulla scrivania. Siccome il livello superiore è più piccolo ed è costruito utilizzando componenti più veloci, il tempo di hit sarà molto inferiore al tempo necessario ad accedere al secondo livello della gerarchia, che rappresenta la componente principale della penalità di miss. Proseguendo con l'analogia della biblioteca, il tempo richiesto per esaminare i libri sulla scrivania è molto inferiore a quello impiegato per alzarsi e andare a prelevare un altro libro dagli scaffali della biblioteca.

Come vedremo più avanti, i concetti utilizzati per realizzare i sistemi di memoria influenzano molti altri aspetti di un calcolatore, come le modalità di gestione della memoria e delle periferiche di I/O che possono essere adottate dal sistema operativo, la generazione del codice da parte dei compilatori e perfino la modalità di utilizzo del calcolatore da parte delle diverse applicazioni. Ovviamente, poiché tutti i programmi spendono molto del loro tempo di esecuzione in accessi alla memoria, il sistema di memoria è uno degli elementi cruciali che determinano le prestazioni del calcolatore nel suo complesso. La dipendenza delle prestazioni dall'utilizzo delle gerarchie delle memorie ha fatto sì che i programmatori – che hanno imparato a considerare la memoria come un contenitore dal quale prendere i dati con lo stesso tempo di accesso – debbano ora comprendere i meccanismi di funzionamento della memoria per ottenere prestazioni più elevate. Illustreremo più avanti quanto sia importante comprendere il funzionamento della gerarchia delle memorie con degli esempi: nella Figura 5.18 e nel Paragrafo 5.14 mostreremo come raddoppiare le prestazioni della moltiplicazione di matrici in doppia precisione.

Miss rate: detto anche frequenza delle miss, è la frazione degli accessi alla memoria nei quali la parola cercata non viene trovata in un certo livello della gerarchia.

Tempo di hit: il tempo richiesto per accedere a un livello della gerarchia delle memorie, compreso il tempo necessario a determinare se l'accesso ha avuto successo o meno, cioè se ha prodotto una hit o una miss.

Penalità di miss: il tempo richiesto per caricare un blocco dal livello inferiore della gerarchia delle memorie, comprendente il tempo per accedere al blocco, portarlo al livello superiore (che ha generato la miss) e scriverlo in quel livello di memoria.

 © 978-88-08-19966-9

Proprio perché i sistemi di memoria sono così critici per le prestazioni, sono stati oggetto di una grande attenzione da parte dei progettisti, i quali hanno sviluppato meccanismi molto sofisticati per migliorarli. In questo capitolo esamineremo le principali problematiche che stanno alla base dei sistemi di memoria; abbiamo scelto di adottare diverse semplificazioni e astrazioni per rendere i concetti il più possibile concisi e comprensibili.

QUADRO D'INSIEME

I programmi mostrano un comportamento locale sia dal punto di vista temporale, ossia hanno la tendenza a riutilizzare i dati che sono stati caricati di recente dalla memoria, sia da quello spaziale, cioè tendono a richiedere dati che si trovano in prossimità di quelli che sono stati appena letti o scritti. Le gerarchie delle memorie sfruttano la località temporale mantenendo i dati utilizzati più di recente nel livello più vicino al processore e sfruttano la località spaziale spostando verso i livelli superiori della gerarchia blocchi composti da più parole aventi indirizzi di memoria adiacenti. La Figura 5.3 mostra che una gerarchia delle memorie impiega la tecnologia più veloce con una capacità ridotta per realizzare i livelli più vicini al processore; in questo modo, gli accessi ai dati che si trovano nei livelli più alti, quando hanno successo, possono essere effettuati velocemente. Gli accessi che falliscono richiedono di caricare i dati dai livelli inferiori della gerarchia, che hanno dimensioni maggiori ma sono caratterizzati da un maggior tempo d'accesso. Se la frequenza di hit è sufficientemente alta, la gerarchia delle memorie ha un tempo medio di accesso vicino a quello del livello più alto (e più veloce) e una dimensione complessiva pari a quella del livello più basso (che ha una capacità maggiore). In molti sistemi, la memoria è una gerarchia vera e propria, nel senso che un dato non può essere presente nel livello i a meno che non si trovi anche nel livello $i + 1$. ■

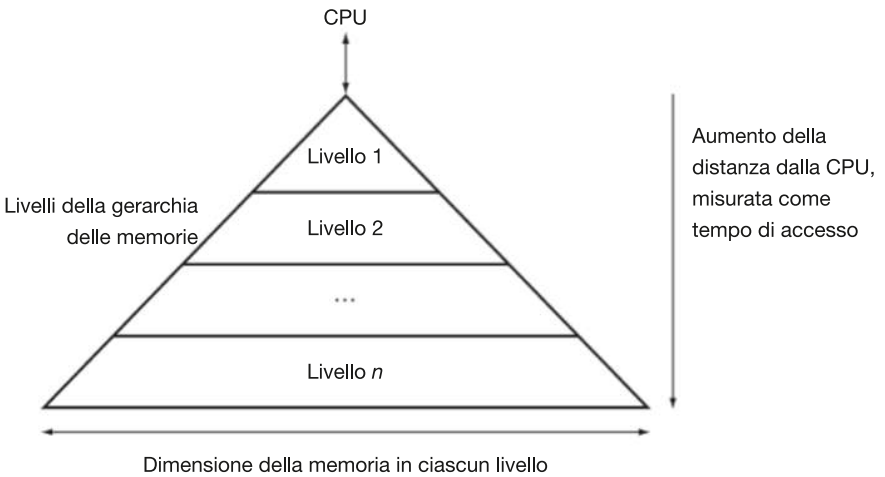


Figura 5.3 Schema della struttura di una gerarchia delle memorie: al crescere della distanza dal processore, cresce la dimensione della memoria. Questa struttura, con gli appropriati meccanismi di gestione, permette al processore di avere un tempo di accesso che è determinato principalmente dal livello 1 della gerarchia e allo stesso tempo di avere una memoria la cui dimensione è pari a quella del livello n. Il modo in cui si crea questa illusione è l'oggetto di questo capitolo. Sebbene la memoria di massa locale costituisca solitamente il livello inferiore della gerarchia, alcuni sistemi utilizzano nastri o file server su reti LAN (*Local Area Network*) come livelli addizionali della gerarchia.

Autovalutazione

Quali delle seguenti affermazioni sono generalmente vere?

1. Le gerarchie delle memorie sfruttano la località temporale.
2. Il valore restituito da una lettura dipende da quali blocchi si trovano nella cache.
3. La maggior parte del costo di una gerarchia delle memorie appartiene al livello più alto.
4. La maggior parte della capacità di una gerarchia delle memorie si trova nel livello più basso.

5.2 Tecnologie delle memorie

Al giorno d'oggi vengono utilizzate quattro principali tecnologie per costruire le gerarchie delle memorie. La memoria principale viene realizzata utilizzando DRAM (*Dynamic Random Access Memory*, memoria dinamica ad accesso casuale), mentre i livelli più vicini al processore (cache) sono composti da SRAM (*Static Random Access Memory*, memoria statica ad accesso casuale). La DRAM ha un costo per bit più basso della SRAM, ma il suo accesso è decisamente più lento. La differenza di costo dipende dal fatto che le memorie DRAM utilizzano meno transistor per ogni bit da memorizzare e riescono così a raggiungere una capacità superiore a parità di porzione di silicio utilizzata; la differenza di velocità è dovuta invece ad altri fattori, descritti nel Paragrafo A.9 dell'Appendice A. La terza tecnologia è quella delle memorie flash. Questa memoria non volatile viene utilizzata nei dispositivi mobili. La quarta tecnologia viene impiegata per implementare il livello di memoria più capiente e lento della gerarchia, ed è solitamente rappresentata dai dischi magnetici. Il tempo di accesso e il costo per bit variano ampiamente per ciascuna di queste tecnologie, come si vede dalla seguente tabella che riporta i costi e i tempi di accesso tipici, riferiti all'anno 2020:

Tecnologia di memoria	Tempo di accesso tipico (ns)	$ per GiB nel 2020
Memoria a semiconduttore SRAM	0,5–2,5 ns	$500–$1000
Memoria a semiconduttore DRAM	50–70 ns	$3–$6
Memoria flash a semiconduttore	5000–50 000 ns	$0,06–$0,12
Dischi magnetici	5 000 000–20 000 000 ns	$0,01–$0,02

Descriviamo ora le diverse tecnologie.

Tecnologia SRAM

Le SRAM sono semplicemente dei circuiti integrati organizzati come vettori di memoria che (di solito) hanno una sola porta di accesso che può fornire sia la lettura sia la scrittura. Le SRAM hanno uno stesso tempo di accesso per tutti i dati, anche se i tempi di accesso in lettura e scrittura possono essere diversi.

Le SRAM non hanno bisogno di essere rinfrescate (*refresh* è il termine inglese di uso comune), per cui il loro tempo di accesso è molto vicino alla durata del ciclo della memoria che è il tempo che intercorre tra due accessi alla memoria. Le SRAM utilizzano da sei a otto transistor per bit per evitare che l'informazione possa essere disturbata quando viene letta. Le SRAM hanno

bisogno di una potenza minima per mantenere la carica quando si trovano in modalità stand-by.

In passato, la maggior parte dei PC e dei server utilizzava chip di SRAM diversi per i diversi livelli di cache (primo, secondo e, in alcuni casi, terzo livello). Oggi, grazie alla legge di Moore, le cache di tutti i livelli sono integrate nel chip del processore, per cui il mercato delle SRAM singole è praticamente svanito.

Tecnologia DRAM

In una memoria SRAM il dato rimane memorizzato per tutto il tempo in cui l'alimentazione è attiva. In una memoria RAM *dinamica* (DRAM), invece, il dato viene memorizzato come carica in un condensatore e un solo transistor è sufficiente per leggere il dato o per sovrascriverlo. Dato che le DRAM utilizzano un solo transistor per bit memorizzato, sono molto più dense e il costo per bit è inferiore a quello delle SRAM. Tuttavia, dato che nelle DRAM l'informazione viene memorizzata in un condensatore, non rimane indefinitamente e occorre rinfrescarla periodicamente. La volatilità è il motivo per cui una memoria di questo tipo è chiamata dinamica, per distinguerla dalle memorie statiche delle celle SRAM.

Per rinfrescare una cella, occorre semplicemente leggerne il contenuto e riscriverlo. La carica può essere mantenuta per pochi millisecondi, per cui, se dovessimo leggere e riscrivere i diversi bit di una DRAM uno per uno, passeremmo tutto il tempo a fare il refresh della DRAM, senza avere il tempo per accedervi. Fortunatamente, le DRAM utilizzano un'architettura a due livelli che consente di rinfrescare un'intera *riga*, mediante un ciclo di lettura seguito immediatamente da un ciclo di riscrittura.

La **Figura 5.4** mostra l'organizzazione interna di una DRAM, mentre la **Figura 5.5** mostra come siano cambiati negli anni la densità, il costo e il tempo di accesso delle DRAM.

L'organizzazione per righe aiuta nel refresh della memoria e aiuta anche nelle prestazioni: le DRAM utilizzano un buffer per memorizzare una riga per consentire accessi ripetuti. Il buffer si comporta come una SRAM: cambiando opportunamente l'indirizzo si può accedere ai diversi bit del buffer fino a quando non si deve accedere ai dati di un'altra riga. Questa funzionalità migliora significativamente il tempo di accesso, dato che il tempo di accesso a bit diversi della stessa riga è molto più veloce. Anche costruire il chip più ampio migliora l'ampiezza di banda del chip. Quando la riga si trova nel buffer,

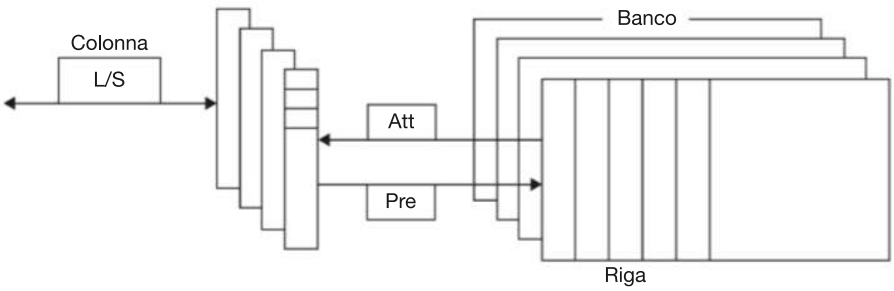


Figura 5.4 Organizzazione interna di una DRAM. Le DRAM moderne sono organizzate in banchi, tipicamente quattro per le DDR4. Ciascun banco consiste in una serie di righe. Inviando un comando di precaricamento (Pre) si apre o chiude un banco. L'indirizzo di una riga viene inviato con un comando di attivazione (Att), che attiva il trasferimento del contenuto della riga in un buffer. Quando la riga si trova nel buffer, il suo contenuto può essere trasferito specificando gli indirizzi successivi di gruppi di bit qualsiasi sia l'ampiezza della DRAM (tipicamente 4, 8 o 16 bit per le DDR4) o specificando l'indirizzo di partenza e che si intende trasferire l'intero blocco. Ciascun comando e il comando di trasferimento di un blocco sono sincronizzati con il clock.

Anno introduzione	Dimensione chip	$ per GiB	Tempo di accesso totale a nuova riga/colonna	Tempo medio di accesso alla riga esistente
1980	64 Kibibit	$6 480 000	250 ns	150 ns
1983	256 Kibibit	$1 980 000	185 ns	100 ns
1985	1 Mebibit	$720 000	135 ns	40 ns
1989	4 Mebibit	$128 000	110 ns	40 ns
1992	16 Mebibit	$30 000	90 ns	30 ns
1996	64 Mebibit	$9 000	60 ns	12 ns
1998	128 Mebibit	$900	60 ns	10 ns
2000	256 Mebibit	$840	55 ns	7 ns
2004	512 Mebibit	$150	50 ns	5 ns
2007	1 Gibibit	$40	45 ns	1,25 ns
2010	2 Gibibit	$13	40 ns	1 ns
2012	4 Gibibit	$5	35 ns	0,8 ns
2015	8 Gibibit	$7	30 ns	0,6 ns
2018	16 Gbibit	$6	25 ns	0,4 ns

Figura 5.5 La dimensione delle DRAM è quadruplicata approssimativamente ogni tre anni fino al 1996, mentre è cresciuta molto meno negli anni successivi. Il miglioramento del tempo di accesso è stato meno marcato ma continuo. L'andamento del costo segue da vicino l'incremento della densità, sebbene il costo sia spesso influenzato da altri elementi quali la disponibilità e la richiesta. Il costo per Gibibyte non tiene conto dell'inflazione. Prezzi ricavati da https:// jcmit.net/memoryprice.htm.

il suo contenuto può essere trasferito specificando gli indirizzi successivi di gruppi di bit qualsiasi sia l'ampiezza della DRAM (tipicamente 4, 8 o 16 bit) o specificando l'indirizzo di partenza e che si intende trasferire l'intero blocco.

Per migliorare ulteriormente l'interfaccia con i processori, alle DRAM è stato aggiunto il clock, per cui queste memorie sono chiamate più propriamente DRAM sincrone o SDRAM (*Synchronous* DRAM). Il vantaggio delle SDRAM sta nel fatto che il clock elimina il tempo necessario a sincronizzare la memoria con il processore. L'aumento di velocità delle DRAM sincrone proviene dalla possibilità di trasferire gruppi di bit adiacenti a raffica (*in burst*) senza dovere specificare altri indirizzi. Questa modalità di trasferimento è chiamata DDR (*Double Data Rate*, frequenza doppia dei dati) SDRAM. Questo termine indica che il trasferimento dei dati avviene sia sul fronte di salita sia di discesa del clock, ottenendo così il doppio della larghezza di banda rispetto alla banda calcolata a partire dall'ampiezza dei dati e dalla frequenza del clock. La versione più recente di questa tecnologia viene chiamata DDR4. Una DRAM DDR4-3200 può effettuare 3200 milioni di trasferimenti al secondo, che corrisponde a una frequenza di clock di 1600 MHz.

Sostenere una tale larghezza di banda richiede un'organizzazione particolarmente intelligente *all'interno* della DRAM. Invece di un unico buffer di riga più veloce, si possono organizzare internamente le DRAM per leggere o scrivere da diversi *banchi*, ciascuno con il suo buffer di riga. Inviare un indirizzo a più banchi consente di leggere o scrivere simultaneamente. Per esempio, con quattro banchi occorre un unico tempo di accesso alla memoria, dopodiché si possono indirizzare sequenzialmente a rotazione i quattro banchi, ottenendo così il quadruplo della larghezza di banda. Questo schema di accesso sequenziale a rotazione viene chiamato *indirizzamento con interleaving* (letteralmente, "intreccio").

Sebbene i dispositivi mobili come gli iPad (Capitolo 1) utilizzino DRAM singole, per i server vengono comunemente vendute delle schedine chiamate DIMM (*Dual Inline Memory Modules*, moduli di memoria in linea doppi, cioè

fronte e retro). Una DIMM tipicamente contiene 4–16 DRAM ed è generalmente organizzata in modo da offrire un'ampiezza di 8 byte per i server. Una DIMM che utilizza le SDRAM DDR4-3200 può arrivare a trasferire 8 × 3200 = 25 600 megabyte al secondo e prende il nome dalla sua ampiezza di banda: PC25600. Dato che una DIMM può avere così tanti chip che solo alcuni sono interessati da ogni trasferimento, occorre un termine per indicare il sottoinsieme di chip di una DRAM che condividono le linee di indirizzamento. Per evitare confusione con i termini che indicano la struttura interna, ovvero righe e banchi, utilizzeremo il termine *rango della memoria* (*memory rank*) per indicare questo sottoinsieme di chip di una DIMM.

Approfondimento. Un modo per misurare le prestazioni del sistema di memoria che sta dietro le cache è il *benchmark Stream* (cioè con flusso di dati continuo) [McCalpin, 1995], che misura le prestazioni di operazioni effettuate su vettori molto lunghi. Queste operazioni non presentano località temporale e accedono a vettori di dimensioni maggiori di quelle delle cache del calcolatore che viene esaminato.

Memorie flash

Le memorie flash sono un tipo di memoria a sola lettura, cancellabile elettricamente e programmabile (EEPROM, *Electrically Erasable Programmable Read-Only Memory*).

A differenza dei dischi e delle DRAM, ma come le altre tecnologie EEPROM, i bit delle memorie flash si deteriorano dopo un certo numero di scritture. Per far fronte a questa limitazione, la maggior parte dei dispositivi che utilizzano memorie flash contiene un controllore che distribuisce le scritture consentite rimappando i blocchi di memoria che sono stati scritti più spesso sui blocchi che sono stati scritti meno di frequente. Questa tecnica è chiamata *livellamento dell'usura* (*wear leveling*). Adottando questa tecnica è difficile che nei dispositivi mobili si possa superare il limite sul numero massimo di scritture per singola cella di memoria flash. I controllori che implementano il livellamento dell'usura fanno diminuire le prestazioni teoriche di una memoria flash, ma sono necessari, a meno che non sia il software di controllo al livello superiore a verificare l'usura dei blocchi di memoria. I controllori delle memorie flash possono anch'essi migliorare le prestazioni, identificando le celle di memoria costruite male e quindi mal funzionanti.

Memorie a disco

Come mostrato nella **Figura 5.6**, un disco magnetico è formato da un gruppo di dischi, detti *piatti*, che ruotano solidali a una velocità compresa tra i 5400 e i 15 000 giri al minuto (GPM). Ciascun piatto metallico è ricoperto su entrambe le facce da materiale magnetico registrabile simile a quello delle cassette audio o delle videocassette. Per leggere e scrivere dati su un disco rigido, poco al di sopra di ognuna delle due superfici di ogni disco è posizionato un *braccio mobile* contenente una piccola bobina elettromagnetica chiamata *testina di lettura/scrittura*. L'intera unità è sigillata per controllare i parametri fisici dell'ambiente interno; ciò consente anche di avvicinare molto di più le testine alla superficie del disco.

Traccia: uno delle migliaia di cerchi concentrici che compongono la superficie di un disco magnetico.

Settore: uno dei segmenti che costituiscono una traccia di un disco magnetico; un settore è la più piccola quantità di informazione letta o scritta sul disco.

La superficie di ogni disco è divisa in cerchi concentrici chiamati **tracce** (*tracks*): ci sono tipicamente decine di migliaia di tracce per ogni lato di un piatto[1]. Ogni traccia è a sua volta suddivisa in **settori** (*sectors*), che sono l'unità

[1] La struttura del singolo piatto e la testina di lettura assomigliano ai giradischi degli anni Settanta [*N.d.T.*].

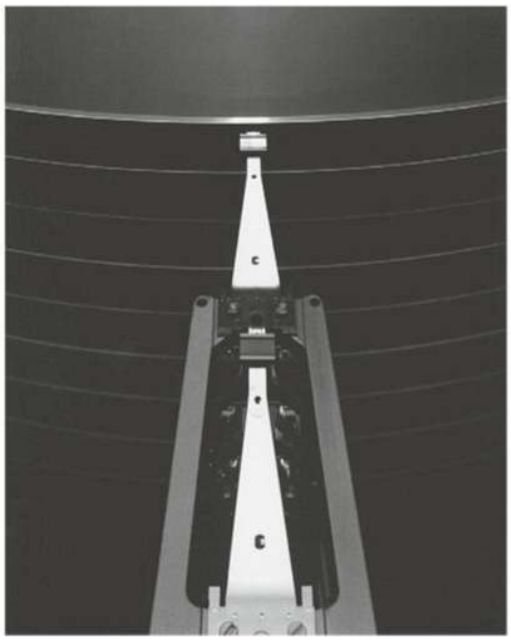


Figura 5.6 Disco rigido costituito da 10 piatti paralleli e dalle testine di lettura/scrittura. Il diametro dei dischi è oggi di 2,5 o 3,5 pollici e ci sono tipicamente uno o due piatti per ogni disco.

di memorizzazione delle informazioni: ciascuna traccia può contenere migliaia di settori. Ogni settore ha tipicamente una dimensione variabile da 512 a 4096 byte. La sequenza di informazioni registrata sul materiale magnetico è costituita dal numero del settore, seguito da uno spazio vuoto di separazione, dalle informazioni memorizzate nel settore, comprendenti il codice di correzione degli errori (Paragrafo 5.5), quindi da un altro spazio vuoto di separazione, dal numero del settore successivo ecc.

Le testine di lettura/scrittura associate a ognuna delle due superfici dei piatti sono collegate assieme e si muovono congiuntamente, così che ogni testina si trova in corrispondenza della stessa traccia su tutte le superfici. Il termine *cilindro* viene utilizzato per riferirsi a tutte le tracce dei diversi piatti che si trovano sotto le testine in un certo istante di tempo.

Per accedere ai dati, il sistema operativo deve guidare il disco attraverso un processo in tre passi. Il primo passo consiste nel posizionare la testina sulla traccia giusta. Questa operazione è chiamata **seek** (ricerca), e il tempo necessario per spostare la testina sopra la traccia desiderata viene chiamato *tempo di ricerca* (*seek time*).

Seek (ricerca): il processo di posizionamento della testina di lettura e scrittura su una data traccia del disco.

I costruttori di dischi riportano nei loro manuali il tempo minimo di ricerca, quello massimo e quello medio. I primi due sono facili da misurare, ma quello medio è aperto a diverse interpretazioni, perché dipende da quanto la testina si deve spostare. I costruttori hanno deciso di calcolare il tempo medio di ricerca come la somma dei tempi di ricerca di tutti i possibili settori diviso per il numero dei possibili settori. Il tempo medio di ricerca che viene di solito dichiarato dai costruttori è compreso tra i 3 e i 13 ms, ma, a seconda dell'applicazione e dell'ordine con cui arrivano le richieste di accesso a disco, il tempo medio effettivo di una ricerca si può ridurre fino a un valore compreso tra il 25 e il 33% del tempo di ricerca medio dichiarato, grazie alla località degli accessi al disco. Questa località può derivare sia dal fatto che gli accessi sono per parti adiacenti di uno stesso file, sia dal riordinamento degli accessi imposto dal sistema operativo.

Latenza di rotazione: detta anche ritardo di rotazione, è il tempo necessario perché, ruotando il disco, il settore desiderato passi sotto la testina di lettura/scrittura; in genere si suppone che sia pari a metà del tempo di rotazione.

Quando la testina di lettura/scrittura ha raggiunto la traccia corretta, deve aspettare che il settore desiderato passi sotto. Questo tempo di attesa viene chiamato **latenza di rotazione** o **ritardo di rotazione**. La latenza media corrisponde a mezzo giro del disco. I dischi ruotano a una velocità compresa tra i 5400 e i 15 000 GPM (giri/minuto). La latenza di rotazione media a 5400 GPM sarà:

$$\text{Latenza di rotazione media} = \frac{0{,}5 \text{ rotazione}}{5400 \text{ GPM}}$$

$$= \frac{0{,}5 \text{ rotazione}}{5400 \text{ GPM} \left(60 \frac{\text{secondi}}{\text{minuto}}\right)} = 0{,}0056 \text{ secondi} = 5{,}6 \text{ ms}$$

L'ultima componente del tempo di accesso al disco, il *tempo di trasferimento*, è il tempo impiegato per trasferire un blocco di bit; questo tempo è funzione della dimensione del settore, della velocità di rotazione e della densità di memorizzazione delle tracce. Le velocità di trasferimento dei dati nel 2020 erano tipicamente comprese tra i 150 e i 250 MB/s. Una delle complicazioni è che il controllore del disco contiene al suo interno una cache dove vengono memorizzati i dati letti dai diversi settori del disco e la velocità di trasmissione da questa cache è tipicamente più elevata e poteva arrivare a 1500 MB/s, ovvero 12 Gbit/s nel 2020. Il risultato è che la disposizione dei blocchi non è più intuitiva. L'ipotesi sottostante del modello basato su settori-tracce-cilindri descritto sopra è che blocchi contigui siano posizionati sulla stessa traccia, che i blocchi sullo stesso cilindro richiedano un tempo di accesso minore, dato che in questo caso il tempo di ricerca è nullo, e che alcune coppie di tracce siano più vicine tra loro di altre. Il motivo per cui questo modello sta perdendo consistenza è l'aumento delle prestazioni delle interfacce. Per incrementare la velocità degli accessi sequenziali, queste interfacce intelligenti organizzano i dischi in una forma più vicina a quella dei dispositivi a nastro che a quella delle memorie a semiconduttore. I blocchi logici sono disposti secondo una serpentina sulla superficie del disco, in modo da percorrere tutti i settori, i quali mantengono la stessa densità di bit. Di conseguenza, blocchi con indirizzi sequenziali possono trovarsi su tracce diverse.

Riassumendo, le due differenze principali tra i dischi magnetici e le memorie a semiconduttore sono che i dischi hanno un tempo di accesso superiore perché sono dispositivi meccanici (le memorie flash sono 1000 volte più veloci e le DRAM 100 000 volte più veloci dei dischi) ma hanno un costo per bit più basso perché offrono una grandissima capacità di memoria a un costo modesto (i dischi sono da 6 a 300 volte più economici). I dischi magnetici sono non volatili come le memorie flash, ma a differenza delle memorie flash non ci sono problemi di usura. Tuttavia, le memorie flash sono molto più robuste e quindi più adatte a sopportare gli urti inerenti all'utilizzo dei dispositivi mobili.

5.3 Principi base delle memorie cache

Cache: un posto sicuro dove nascondere o riporre le cose.
Webster's New World Dictionary of the American Language, Third College Edition, 1988

Nel nostro esempio della biblioteca la scrivania si comporta come una cache, ossia un posto sicuro dove riporre le cose (i libri) che dobbiamo esaminare. *Cache*[2] era il nome scelto per indicare il livello della memoria gerarchica che

[2] Il nome proviene dal termine francese *caché*, che significa "nascosto", e deriva dal fatto che la memoria cache e il suo utilizzo sono generalmente trasparenti agli occhi del programmatore e quindi nascosti [*N.d.T.*].

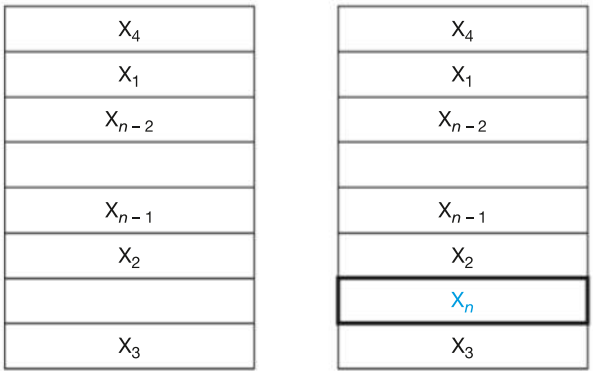
si trova tra il processore e la memoria principale nel primo calcolatore commerciale che utilizzava una gerarchia delle memorie. Le memorie contenute nell'unità di elaborazione descritta nel Capitolo 4 possono essere semplicemente sostituite da memorie cache. Oggi, anche se questo rimane l'utilizzo più comune, il termine cache viene usato anche per indicare i sistemi di memoria gestiti in modo tale da ottenere i massimi benefici dalla località degli accessi. La cache apparve per la prima volta in architetture sperimentali progettate nei primi anni '60 e nei calcolatori commerciali costruiti verso la fine di quel decennio; quasi tutti i calcolatori prodotti oggi, dai server ai processori embedded a basso consumo, contengono memorie cache.

In questo paragrafo esamineremo una cache molto semplice, dalla quale il processore carica una sola parola per volta e in cui i blocchi sono costituiti da una singola parola. I lettori che hanno già familiarità con i principi base delle cache possono passare direttamente alla lettura del Paragrafo 5.4. La **Figura 5.7** mostra questa semplice cache, prima e dopo la richiesta di un elemento che, all'inizio, non è ivi presente. Prima della richiesta, la cache contiene un insieme di elementi, quelli utilizzati più di recente: X_1, X_2, ..., X_{n-1}. Quando il processore richiede la parola X_n, che non è presente in cache, si produce una miss e la parola X_n viene prelevata dal livello inferiore della memoria e portata nella cache.

Osservando lo scenario rappresentato nella Figura 5.7, è naturale chiedersi: come si fa a sapere se un dato è presente nella cache? E se è contenuto nella cache, come facciamo a trovarlo? Le risposte a queste domande sono collegate. Se ogni parola può essere scritta in una sola posizione della cache, allora sappiamo dove trovarla, ammesso che la parola sia presente nella cache. La maniera più semplice per associare una sola locazione della cache a ogni parola della memoria consiste nel definire una corrispondenza tra l'*indirizzo in memoria* della parola e la locazione nella cache. Quest'organizzazione della cache è detta **a mappatura diretta** (*direct mapped cache*), dato che ogni locazione della memoria principale corrisponde, in modo univoco, a una locazione della cache. Per le cache a mappatura diretta, la corrispondenza tra indirizzi della memoria e locazioni della cache è di solito molto semplice. Per esempio, quasi tutte le cache a mappatura diretta utilizzano la seguente operazione per trovare il blocco che corrisponde a un dato indirizzo della memoria principale:

Cache a mappatura diretta: una cache in cui a ogni locazione della memoria principale corrisponde una (e una sola) locazione della cache.

(Indirizzo del blocco) modulo (numero di blocchi nella cache)



(a) Prima di richiedere il dato X_n **(b)** Dopo aver richiesto il dato X_n

Figura 5.7 Stato della cache immediatamente prima e immediatamente dopo la richiesta di una parola X_n che non è presente inizialmente nella cache. Questa richiesta provoca una miss che ha come conseguenza il caricamento della parola X_n dalla memoria principale e la sua scrittura nella cache.

 © 978-88-08-19966-9

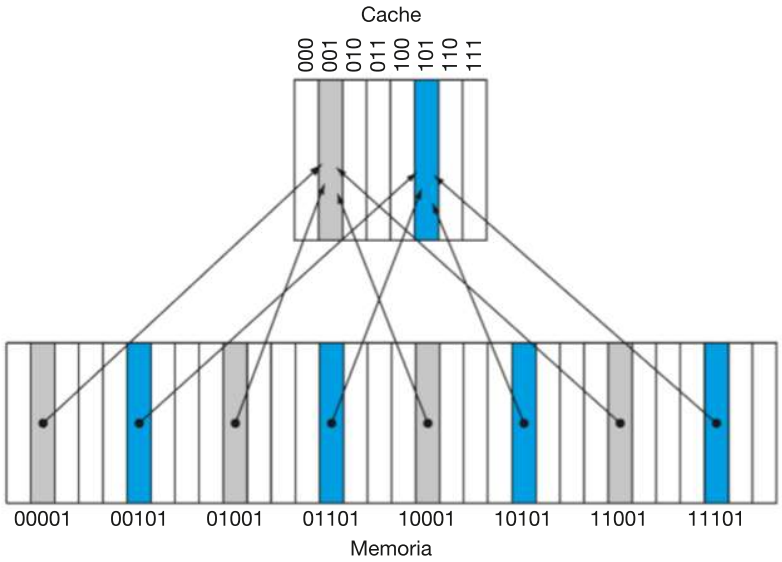


Figura 5.8 Esempio di cache a mappatura diretta contenente otto elementi e corrispondenza tra gli indirizzi della memoria principale, riferiti alla parola e compresi tra 0 e 31, e le locazioni della cache. Dato che la cache contiene 8 parole, un indirizzo X della memoria principale viene mappato sull'elemento X modulo 8 della cache a mappatura diretta. Cioè, i $\log_2(8) = 3$ bit meno significativi dell'indirizzo vengono utilizzati come indice della cache. Perciò gli indirizzi 00001_{due}, 01001_{due}, 10001_{due} e 11001_{due} vengono mappati tutti sull'elemento 001_{due} della cache, mentre gli indirizzi 00101_{due}, 01101_{due}, 10101_{due} e 11101_{due} vengono mappati tutti sull'elemento 101_{due}.

Se il numero di elementi della cache è una potenza di 2, l'operazione di modulo può essere eseguita semplicemente considerando il logaritmo in base due ($\log_2$) dei bit meno significativi dell'indirizzo, che corrispondono alla dimensione della cache in numero di blocchi. Quindi, una cache di 8 blocchi utilizzerà i 3 bit meno significativi per indirizzare il blocco, dato che $8 = 2^3$. Per esempio, la **Figura 5.8** mostra come gli indirizzi della memoria principale compresi tra 1_{dec} (00001_{due}) e 29_{dec} (11101_{due}) vengano mappati nelle locazioni comprese tra 1 (001_{due}) e 5 (101_{due}) di una cache a mappatura diretta contenente otto parole.

Tag (etichetta): un campo all'interno di una tabella utilizzata in una gerarchia delle memorie che contiene l'informazione necessaria per capire se il blocco di memoria associato al tag contenga o meno la parola richiesta.

Poiché ogni elemento della cache può contenere dati provenienti da diverse locazioni della memoria principale, come si riesce a capire se il dato presente nella cache corrisponde effettivamente alla parola desiderata? In altre parole, come facciamo a sapere se la parola richiesta si trova nella cache oppure no? È possibile risolvere questo problema aggiungendo alla cache un insieme di bit che costituiscono il campo **tag** (etichetta). I tag contengono le informazioni necessarie a verificare se una parola della cache corrisponda o meno alla parola cercata. Un tag contiene solamente la parte superiore dell'indirizzo della parola nella memoria principale, in particolare i bit dell'indirizzo che non vengono utilizzati come indice per individuare il blocco all'interno della cache. Per esempio, nella cache della Figura 5.8 occorre memorizzare nel campo tag solamente i 2 bit più significativi dei 5 bit totali dell'indirizzo, dato che i 3 bit inferiori servono per selezionare il blocco all'interno della cache. I bit di indice non vengono scritti nel campo tag perché sarebbero ridondanti, visto che, per definizione, l'indice associato a un qualsiasi blocco della cache coincide con il numero del blocco.

È necessario anche disporre di un metodo per sapere quando un blocco della cache non contiene informazioni valide. Per esempio, quando un processore viene avviato, la cache è vuota e i numeri contenuti nei campi tag non hanno alcun significato. Anche dopo aver eseguito molte istruzioni, alcune

delle locazioni della cache possono ancora essere vuote, come mostrato nella Figura 5.7. Occorre perciò sapere quando il campo tag associato a queste locazioni deve essere ignorato. La procedura più comune consiste nell'aggiungere un **bit di validità** (*valid bit*) per indicare se il corrispondente elemento della cache contenga dati validi. Se il bit non è impostato a 1, la richiesta di lettura dell'elemento associato non può avere successo, perché il contenuto del blocco della cache sarebbe privo di significato.

Bit di validità: un campo in una tabella di una gerarchia delle memorie che indica se il blocco di memoria associato contiene un dato valido.

Nel resto di questo paragrafo vedremo come viene gestita la lettura di una memoria cache. In generale, la lettura è un'operazione più semplice della scrittura, dato che essa non modifica i dati contenuti nella cache. Dopo aver esaminato i principi base del funzionamento della lettura e di come possano essere gestite le miss di una cache, analizzeremo le diverse organizzazioni di cache che si possono trovare nei calcolatori reali e illustreremo in dettaglio le modalità che queste adottano per gestire la scrittura.

QUADRO D'INSIEME

La memoria cache è forse l'esempio più importante della grande idea della **predizione**. Si basa sul principio di località, cerca di trovare i dati desiderati nei livelli più alti della gerarchia delle memorie e fornisce i meccanismi necessari per garantire che, quando una predizione si rivela errata, trovi e utilizzi i dati corretti presi dai livelli inferiori della gerarchia. La frequenza di hit della predizione nelle cache dei calcolatori moderni è spesso più alta del 95% (Figura 5.46). ■

Accesso alla cache

Nella tabella seguente viene riportata una sequenza di nove richieste di dati a una memoria cache di otto blocchi, inizialmente vuota, assieme all'azione associata a ogni richiesta. La **Figura 5.9** mostra come il contenuto della cache cambi dopo ogni miss. Dato che nella cache ci sono otto blocchi, i tre bit meno significativi dell'indirizzo forniscono il numero del blocco:

Indirizzo decimale del dato nella memoria principale	Indirizzo binario del dato nella memoria principale	Hit o miss nell'accesso alla cache	Blocco della cache corrispondente (dove trovare o scrivere il dato)
22	10110_{due}	Miss (5.9b)	$(10110_{due} \bmod 8) = 110_{due}$
26	11010_{due}	Miss (5.9c)	$(11010_{due} \bmod 8) = 010_{due}$
22	10110_{due}	Hit	$(10110_{due} \bmod 8) = 110_{due}$
26	11010_{due}	Hit	$(11010_{due} \bmod 8) = 010_{due}$
16	10000_{due}	Miss (5.9d)	$(10000_{due} \bmod 8) = 000_{due}$
3	00011_{due}	Miss (5.9e)	$(00011_{due} \bmod 8) = 011_{due}$
16	10000_{due}	Hit	$(10000_{due} \bmod 8) = 000_{due}$
18	10010_{due}	Miss (5.9f)	$(10010_{due} \bmod 8) = 010_{due}$
16	10000_{due}	Hit	$(10000_{due} \bmod 8) = 000_{due}$

Essendo la cache inizialmente vuota, alcuni dei primi accessi in lettura costituiscono delle miss; le azioni che vengono intraprese per ciascun accesso alla memoria sono descritte nella didascalia della Figura 5.9. All'ottavo accesso, si verifica un conflitto sul contenuto di un blocco: la parola contenuta all'indirizzo 18 (10010_{due}) della memoria principale dovrebbe essere scritta nel blocco 2

Indice	V	Tag	Dati
000	N		
001	N		
010	N		
011	N		
100	N		
101	N		
110	N		
111	N		

(a) Lo stato iniziale della cache dopo l'accensione del calcolatore

Indice	V	Tag	Dati
000	N		
001	N		
010	N		
011	N		
100	N		
101	N		
110	S	10_{due}	Memoria (10110_{due})
111	N		

(b) Dopo avere gestito una miss all'indirizzo (10110due)

Indice	V	Tag	Dati
000	N		
001	N		
010	S	11_{due}	Memoria (11010_{due})
011	N		
100	N		
101	N		
110	S	10_{due}	Memoria (10110_{due})
111	N		

(c) Dopo avere gestito una miss all'indirizzo 11010_{due}

Indice	V	Tag	Dati
000	S	10_{due}	Memoria (10000_{due})
001	N		
010	S	11_{due}	Memoria (11010_{due})
011	N		
100	N		
101	N		
110	S	10_{due}	Memoria (10110_{due})
111	N		

(d) Dopo avere gestito una miss all'indirizzo 10000_{due}

Indice	V	Tag	Dati
000	S	10_{due}	Memoria (10000_{due})
001	N		
010	S	11_{due}	Memoria (11010_{due})
011	S	00_{due}	Memoria (00011_{due})
100	N		
101	N		
110	S	10_{due}	Memoria (10110_{due})
111	N		

(e) Dopo avere gestito una miss all'indirizzo 00011_{due}

Indice	V	Tag	Dati
000	S	10_{due}	Memoria (10000_{due})
001	N		
010	S	10_{due}	Memoria (10010_{due})
011	S	00_{due}	Memoria (00011_{due})
100	N		
101	N		
110	S	10_{due}	Memoria (10110_{due})
111	N		

(f) Dopo avere gestito una miss all'indirizzo 10010_{due}

Figura 5.9 Contenuto della cache dopo ogni accesso che provoca una miss all'interno della sequenza riportata nella tabella all'inizio del paragrafo; i campi indice e tag sono in binario. All'inizio la cache è vuota e tutti i bit di validità (campo V) sono impostati a 0 (N). Il processore richiede i dati relativi ai seguenti indirizzi della memoria principale: 10110_{due} (miss), 11010_{due} (miss), 10110_{due} (hit), 11010_{due} (hit), 10000_{due} (miss), 00011_{due} (miss), 10000_{due} (hit) e 10010_{due} (miss). Le tabelle mostrano il contenuto della cache dopo la gestione di ognuna delle miss che si verificano nella sequenza di richieste. Quando il processore richiede il dato all'indirizzo 10010_{due} (18_{dec}) della memoria principale, il contenuto dell'elemento della cache corrispondente, che contiene in quel momento il dato con indirizzo 11010_{due} (26_{dec}), deve essere sostituito dal contenuto dell'indirizzo 10010_{due}; una successiva richiesta del dato contenuto all'indirizzo 11010_{due} provocherà una nuova miss. Si noti che il campo tag contiene soltanto la parte più significativa dell'indirizzo. L'indirizzo completo di una parola nel blocco *i*-esimo di questa cache a cui è associato un tag *j*, si ottiene come $8 \times j + i$, oppure, in maniera equivalente, lo si ricava dalla concatenazione del campo tag, *j*, e del campo indice, *i*. Per esempio, nella cache raffigurata nel pannello *f* al dato con indice 010_{due} è associata l'etichetta 10_{due}, per cui l'indirizzo del dato nella memoria principale sarà 10010_{due}.

della cache (010_{due}), sostituendo quindi la parola con indirizzo 26 (11010_{due}) scritta in precedenza. Questo comportamento consente alla cache di sfruttare la località temporale: le parole richieste più di recente sostituiscono quelle richieste in precedenza.

Questa situazione corrisponde esattamente a quella in cui lo studente ha bisogno di prendere un altro libro dagli scaffali della biblioteca, ma non ha più spazio sulla scrivania: alcuni dei testi sulla scrivania devono essere prima riportati sugli scaffali. In una cache a mappatura diretta c'è una sola locazione in cui poter scrivere il nuovo dato richiesto e quindi è uno solo il blocco della cache che può essere sostituito.

Sappiamo quale blocco della cache corrisponde a ogni indirizzo della memoria principale: i bit meno significativi dell'indirizzo vengono utilizzati per individuare l'unico elemento della cache a cui l'indirizzo può corrispondere. La **Figura 5.10** illustra come l'indirizzo della memoria principale venga suddiviso in:

- un *campo tag*, che viene confrontato con il contenuto del campo tag della cache;
- un *campo indice*, utilizzato per selezionare il blocco della cache.

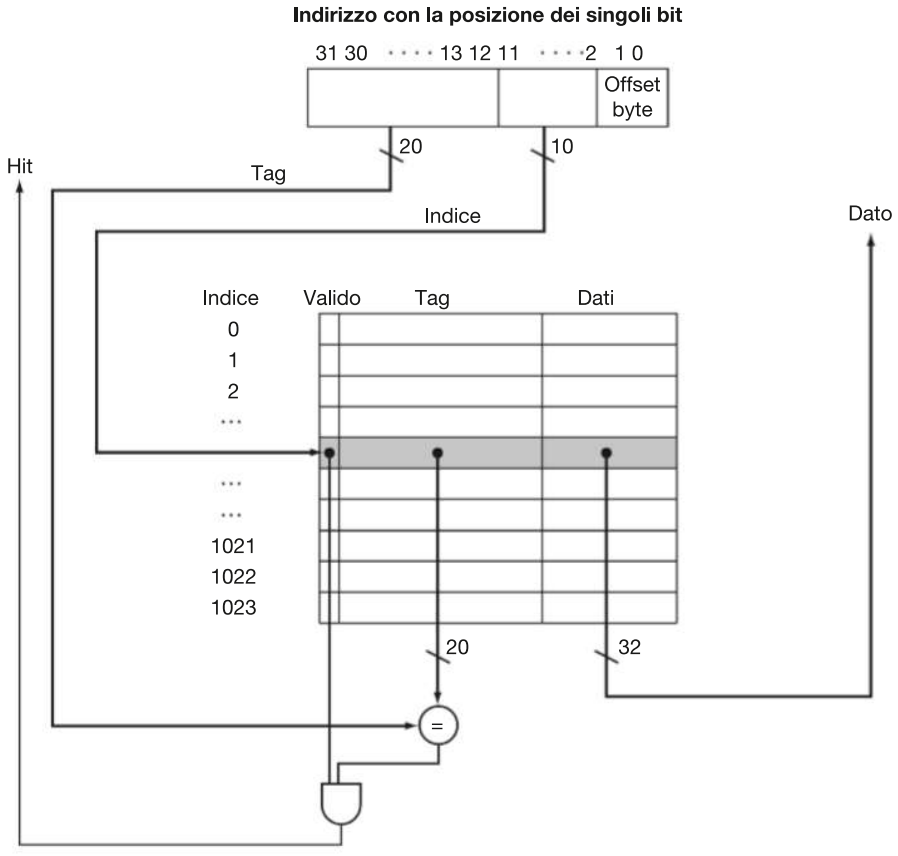


Figura 5.10 Per questa cache, la parte bassa dell'indirizzo viene utilizzata per selezionare un elemento della cache che consiste in un dato largo una parola e in un campo tag. Questa cache contiene 1024 parole o 4 KiB e in questo capitolo supponiamo indirizzi a 32 bit. Il tag della cache viene confrontato con la parte alta dell'indirizzo al fine di verificare se l'elemento della cache corrisponda all'elemento richiesto. Poiché la cache contiene 2^{10} (cioè 1024) parole e la dimensione di un blocco è di una parola, 10 bit dell'indirizzo della memoria principale vengono utilizzati per l'indirizzamento interno della cache, mentre $32 - 10 - 2 = 20$ bit vengono confrontati con il contenuto del campo tag. Se il campo tag è uguale ai 52 bit più significativi dell'indirizzo e se il bit di validità è impostato a 1, allora la ricerca del dato nella cache ha avuto successo, cioè si verifica una hit e il dato viene fornito al processore, altrimenti scatta una miss.

 © 978-88-08-19966-9

L'indice di un blocco della cache, insieme al contenuto del campo tag, specifica in maniera univoca l'indirizzo della memoria principale associato alla parola contenuta in quel blocco della cache. Dato che i bit del campo indice vengono utilizzati come indirizzo interno alla cache e che un campo di n bit contiene 2^n valori diversi, il numero totale degli elementi contenuti in una memoria cache a mappatura diretta deve essere una potenza di 2. Poiché le parole sono allineate a multipli di quattro byte e gli indirizzi sono riferiti ai byte, i due bit meno significativi dell'indirizzo specificano i byte all'interno di una parola. Quindi, se le parole sono allineate a multipli di 4 byte, si possono ignorare i 2 bit meno significativi quando si seleziona una parola all'interno di un blocco della cache. In questo capitolo supporremo che i dati siano allineati in memoria e discuteremo come gestire gli accessi a dati non allineati in una delle prossime sezioni di *Approfondimento*.

Il numero complessivo di bit necessari per realizzare una cache è funzione della dimensione della cache e della dimensione degli indirizzi, poiché la cache deve contenere spazio di memoria sia per i dati sia per i tag. La dimensione dei blocchi considerata finora era di una parola (4 byte), ma normalmente un blocco è costituito da più parole. Si consideri la seguente situazione:

- indirizzo su 32 bit;
- cache a mappatura diretta;
- dimensione della cache di 2^n blocchi, per cui n bit vengono utilizzati per l'indice;
- dimensione del blocco della cache di 2^m parole, ossia 2^{m+2} byte, per cui m bit vengono utilizzati per individuare una parola all'interno di un blocco, mentre due bit servono a individuare un byte all'interno di una parola.

In questo caso, la dimensione del campo tag è:

$$32 - (n + m + 2)$$

Il numero totale di bit contenuti in una cache a mappatura diretta è quindi:

$$2^n \times (\text{dimensione_blocco} + \text{dimensione_tag} + \text{bit_validità})$$

Tenendo presente che la dimensione del blocco è di 2^m parole, ossia 2^{m+2} byte che corrispondono a 2^{m+4} bit, e considerando il bit di validità, il numero di bit contenuti in questa cache è:

$$2^n \times [2^m \times 32 + (32 - n - m - 2) + 1] = 2^n \times (2^m \times 32 + 31 - n - m).$$

Sebbene questa sia la dimensione reale della cache, per convenzione si escludono dal calcolo i bit del campo tag e il bit di validità, per cui si considera solo la dimensione dei dati. Quella della Figura 5.10 è considerata quindi una cache da 4 KiB anche se contiene di fatto 1375 KiB di bit di validità e di tag oltre ai 4 KiB di dati.

Bit di una cache

ESEMPIO

Da quanti bit è costituita una cache a mappatura diretta contenente 16 KiB di dati e avente blocchi di 4 parole, ipotizzando un indirizzo su 64 bit?

(continua)

SOLUZIONE

Sappiamo che 16 KiB equivalgono a 4096 parole (cioè 2^{12} parole) e che la dimensione del blocco è di 4 parole (2^2); la cache conterrà quindi 1024 blocchi (2^{10} blocchi). Ogni blocco ha dimensione $4 \times 32 = 128$ bit di dati più la dimensione del campo tag, che è pari a $64 - 10 - 2 - 2 = 50$ bit, più il bit di validità. Quindi, la dimensione totale della cache è:

$$2^{10} \times [4 \times 32 + (64 - 10 - 2 - 2) + 1] = 2^{10} \times 179 = 179 \text{ Kibibit}$$

vale a dire 22,4 KiB per una cache che contiene 16 KiB di dati. In questo caso, il numero totale di bit della cache è circa 1,4 volte lo spazio richiesto per memorizzare i dati.

Mappatura di un indirizzo in una cache con blocchi composti da più parole

ESEMPIO

Si consideri una cache con 64 blocchi di 16 byte ciascuno. A quale numero di blocco corrisponde l'indirizzo 1200, espresso in byte, della memoria principale?

SOLUZIONE

In base alla relazione introdotta all'inizio del Paragrafo 5.3, il numero del blocco contenente il dato viene calcolato come:

(Indirizzo del blocco) modulo (numero dei blocchi nella cache)

dove l'indirizzo del blocco è:

$$\frac{\text{Indirizzo del dato in byte}}{\text{Byte del blocco}}$$

Si noti che le parole appartenenti al blocco individuato hanno indirizzo compreso tra:

$$\left[\frac{\text{Indirizzo del dato in byte}}{\text{Byte del blocco}}\right] \times \text{Byte per blocco}$$

e

$$\left[\frac{\text{Indirizzo del dato in byte}}{\text{Byte del blocco}}\right] \times \text{Byte per blocco} + (\text{Byte per blocco} - 1)$$

Perciò, dato che ci sono 16 byte in ogni blocco, l'indirizzo 1200 corrisponde all'indirizzo di blocco:

$$\left[\frac{1200}{16}\right] = 75$$

che corrisponde al blocco della cache avente indice (75 modulo 64) = 11. Infatti, sul blocco 11 vengono mappati tutti i dati contenuti negli indirizzi compresi tra 1200 e 1215 della memoria principale.

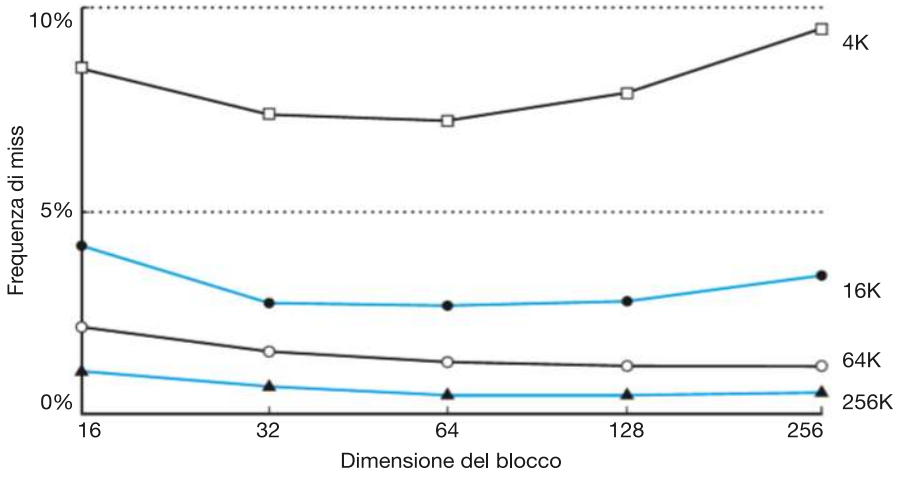


Figura 5.11 Frequenza delle miss in funzione della dimensione del blocco della cache. Si noti che la frequenza delle miss, in realtà, cresce se la dimensione del blocco è troppo grande rispetto alla dimensione della cache. Ognuna delle linee rappresenta una cache di dimensioni diverse (queste curve sono indipendenti dall'associatività, che analizzeremo a breve).

Blocchi di dimensioni maggiori sfruttano maggiormente la località spaziale e diminuiscono la frequenza delle miss. Come mostrato nella **Figura 5.11**, all'aumentare della dimensione dei blocchi di solito diminuisce la frequenza delle miss. Tuttavia, la frequenza delle miss può tornare a crescere se la dimensione dei blocchi diventa troppo grande rispetto alla dimensione della cache, perché in questo caso il numero dei blocchi che possono essere memorizzati nella cache diventa piccolo, mentre cresce la competizione per occuparli. Di conseguenza, i blocchi vengono scaricati dalla cache prima ancora che molti dei dati in essi contenuti siano stati utilizzati. In altre parole, la località spaziale tra le parole di un blocco diminuisce con blocchi di dimensioni troppo grandi e, di conseguenza, il miglioramento legato alla frequenza delle miss si riduce.

Un problema ancora più serio, associato all'aumento della dimensione dei blocchi, è la crescita del costo di una miss. La penalità di una miss, infatti, è determinata dal tempo necessario a prelevare un blocco dal livello inferiore della gerarchia e a scriverlo nella cache. Il tempo impiegato per prelevare il blocco è costituito da due parti: la latenza per ottenere la prima parola del blocco e il tempo di trasferimento del resto del blocco. Chiaramente, a meno che non si modifichi il sistema di memoria, il tempo di trasferimento, e quindi la penalità di miss, cresce con le dimensioni del blocco. Inoltre, il miglioramento nella frequenza delle miss comincia a diminuire man mano che aumenta la dimensione del blocco. Il risultato è che l'incremento della penalità di miss ha un impatto superiore alla diminuzione della frequenza delle miss e quindi le prestazioni della cache si abbassano. Ovviamente, se si progetta la memoria per trasferire in maniera più efficiente blocchi di dimensioni maggiori, è possibile aumentare la dimensione del blocco e ottenere un ulteriore incremento delle prestazioni della cache. Discuteremo questo aspetto nel prossimo paragrafo.

Approfondimento. Sebbene sia difficile fare qualcosa contro l'aumento della durata della componente preponderante della penalità di miss per blocchi di grandi dimensioni, è possibile nascondere una parte del tempo di trasferimento in modo da rendere effettivamente più breve la penalità di miss. Il metodo più semplice per ottenere tale risultato viene chiamato metodo della *ripartenza anticipata* (*early restart*), e consiste semplicemente nel riprendere l'esecuzione non appena è stata trasferita la parola richiesta, invece di aspettare che sia stato trasferito l'intero blocco. Molti

processori utilizzano questo meccanismo dove funziona meglio, ossia nell'accesso alla cache delle istruzioni. L'accesso alle istruzioni, infatti, è in gran parte sequenziale: ciò significa che se il sistema di memoria è in grado di trasferire una parola a ogni ciclo di clock, il processore può riprendere l'attività non appena la parola richiesta è stata trasferita, mentre il sistema di memoria continua a trasferire le altre istruzioni mettendole a disposizione in tempo. Questa tecnica, di solito, è meno efficace per le cache contenenti i dati, perché è facile che le parole di un blocco vengano richieste in maniera meno prevedibile e perché è alta la probabilità che il processore abbia bisogno di altre parole presenti in blocchi diversi della cache prima che il trasferimento di tutto il blocco sia stato completato. Se il processore non può accedere alla cache dei dati perché c'è un trasferimento in corso, allora deve generare uno stallo e fermare l'esecuzione.

Uno schema ancora più sofisticato consiste nell'organizzare la memoria in maniera tale che la parola richiesta venga trasferita per prima dalla memoria alla cache. Il resto del blocco viene poi trasferito a partire dall'indirizzo successivo a quello della parola richiesta, per poi tornare alla prima parola del blocco dopo avere trasferito l'ultima parola del blocco stesso. Questa tecnica, nota come tecnica della *prima parola richiesta* (*requested word first*), può risultare un po' più veloce della ripartenza anticipata, ma è limitata dagli stessi problemi che affliggono la ripartenza anticipata.

Gestione delle miss della cache

Prima di esaminare la struttura della cache di un sistema reale, vediamo in che modo l'unità di controllo gestisce le **miss della cache** (fallimenti della cache). Descriveremo in dettaglio il funzionamento del controllore della cache nel Paragrafo 5.9. L'unità di controllo deve riconoscere il fallimento del tentativo di accesso e provvedere a risolverlo andando a prelevare i dati dalla memoria principale, oppure, come vedremo più avanti, da una cache di livello inferiore. Se invece il tentativo di accesso va a buon fine, il calcolatore continua l'elaborazione utilizzando il dato come se nulla fosse accaduto.

Miss della cache: la richiesta di un dato alla cache che non può essere soddisfatta perché il dato non è presente nella cache.

Modificare l'unità di controllo del processore per gestire una hit è un'operazione banale. La gestione delle miss, invece, richiede del lavoro aggiuntivo e un'unità di controllo separata che collabora con il processore. Questa si occupa dell'accesso alla memoria principale e del "rifornimento" della cache. La risoluzione di una miss richiede lo stallo della pipeline (Capitolo 4) ed è differente da un interrupt o da un'eccezione, che implicherebbe il salvataggio dello stato di tutti i registri. A ogni miss della cache possiamo mettere in stallo l'intero processore, essenzialmente bloccando il contenuto dei registri temporanei e di quelli visibili al programmatore, per tutto il tempo necessario a caricare i dati dalla memoria principale. I processori più sofisticati, dotati di esecuzione fuori ordine, consentono l'esecuzione di altre istruzioni mentre attendono la risoluzione di una miss della cache, ma in questo paragrafo considereremo solamente i processori che eseguono il codice in sequenza; questi devono essere messi in stallo in seguito a una miss.

Vediamo più da vicino come vengono gestite le miss sulle istruzioni; gli stessi meccanismi si possono estendere alla gestione delle miss sui dati. Se l'accesso a un'istruzione si traduce in una miss, il contenuto del registro istruzioni non sarà più valido. Per caricare l'istruzione corretta nella cache dobbiamo poter dire al livello inferiore della gerarchia delle memorie di eseguire un'operazione di lettura. Dal momento che il program counter viene incrementato di 4 nel primo ciclo di clock di esecuzione, l'indirizzo dell'istruzione che ha generato la miss sarà uguale al contenuto del PC meno 4. Una volta costruito l'indirizzo corretto, si può chiedere alla memoria principale di effettuare la lettura dell'istruzione corrispondente; si deve poi attendere che la memoria abbia terminato la lettura, dato che l'accesso richiede più cicli di clock. Infine, occorre attendere che la parola contenente l'istruzione desiderata venga scritta nella cache.

Riassumiamo schematicamente i passi che devono essere eseguiti quando si verifica una miss nella cache istruzioni:

1. Inviare il valore originale del program counter alla memoria;
2. Comandare alla memoria principale di eseguire un'operazione di lettura e attendere che la memoria completi la lettura;
3. Scrivere la parola che proviene dalla memoria nella posizione opportuna del blocco della cache, aggiornare il campo tag corrispondente scrivendovi i bit più significativi dell'indirizzo presi direttamente dalla ALU e impostare a 1 il bit di validità, se non era stato già impostato a 1;
4. Far ripartire l'esecuzione dell'istruzione dall'inizio, ripetendo la fase di fetch, che questa volta troverà l'istruzione all'interno della cache.

Le operazioni di controllo svolte da una cache per l'accesso ai dati sono essenzialmente identiche: in caso di miss, mettiamo semplicemente in stallo il processore finché la memoria non risponde, restituendo il dato richiesto.

Gestione delle scritture

Write-through: uno schema secondo il quale un elemento viene scritto sia in cache sia nel livello inferiore della gerarchia delle memorie, assicurando così che i dati presenti nelle due memorie siano sempre coerenti tra di loro.

La scrittura funziona in modo un po' diverso. Supponiamo che un'operazione di store scriva il dato solamente nella cache dei dati, senza modificare la memoria principale; al termine della scrittura, la memoria principale avrebbe un contenuto diverso da quello della cache. In questo caso, si dice che la memoria e la cache sono *incoerenti*. La maniera più semplice per conservare la coerenza tra memoria e cache consiste nello scrivere sempre il dato in entrambe le memorie. Questo schema viene chiamato **write-through** ("scrivere attraverso").

L'altro elemento fondamentale della scrittura è rappresentato dalla gestione delle miss in scrittura. Dapprima occorre caricare dalla memoria principale le parole appartenenti al blocco interessato. Dopo avere caricato il blocco e averlo scritto in cache, possiamo sovrascrivere la parola del blocco che aveva causato la miss; questa parola viene scritta anche nella memoria principale utilizzando il suo indirizzo completo.

Sebbene il meccanismo sopra descritto consenta di gestire le scritture con molta semplicità, non offre buone prestazioni. L'utilizzo dello schema write-through comporta che a ogni scrittura la parola venga salvata anche nella memoria principale. Questa operazione richiede molto tempo, almeno 100 cicli di clock del processore, e può quindi rallentare il sistema in maniera considerevole. Per esempio, supponendo che il 10% delle istruzioni siano istruzioni di store, che il CPI senza miss sia pari a 1,0 e che ogni miss richieda 100 cicli di clock in più per ciascuna scrittura, il CPI del processore diventerebbe $1{,}0 + 100 \times 10\% = 11$, riducendo le prestazioni di più di un fattore 10!

Buffer di scrittura: una coda che contiene i dati che aspettano di essere scritti nella memoria principale.

Una possibile soluzione a questo problema consiste nell'utilizzare una memoria tampone, chiamata **buffer di scrittura**. Il buffer di scrittura memorizza i dati in attesa che essi vengano scritti in memoria: dopo aver salvato il dato nella cache e nel buffer di scrittura, il processore può proseguire l'esecuzione. Una volta completata la scrittura di un dato nella memoria principale, il corrispondente spazio nel buffer di scrittura viene liberato. Se il buffer di scrittura è pieno e il processore deve eseguire un'operazione di scrittura, il processore viene messo in stallo finché non si libera spazio nel buffer. Ovviamente, se la velocità con cui la memoria principale completa le scritture è inferiore a quella con cui il processore genera i dati da scrivere, per quanto grande possa essere il buffer di scrittura, prima o poi il processore dovrà comunque essere messo in stallo, poiché i dati da scrivere vengono generati più velocemente di quanto il sistema di memoria sia in grado di accettarli.

Anche se la frequenza con cui vengono effettuate le scritture fosse *inferiore* alla frequenza con cui la memoria può gestirle, si possono comunque

generare degli stalli. Ciò può succedere quando le scritture devono essere effettuate a raffica (*in burst*). Per ridurre il numero di stalli, i processori sono dotati solitamente di un buffer di scrittura in grado di contenere più di un singolo elemento.

Lo schema alternativo al write-through è chiamato **write-back**. In questo schema, quando si verifica una scrittura, il dato viene scritto solamente nel blocco corrispondente della cache e il blocco modificato viene salvato nel livello inferiore della gerarchia solo quando deve essere rimpiazzato. Lo schema write-back può produrre un miglioramento delle prestazioni, specialmente quando il processore può generare le scritture velocemente o più velocemente di quanto le scritture possano essere gestite dalla memoria principale; il write-back è comunque più complesso da implementare del write-through.

Nel resto di questo paragrafo descriveremo le cache di alcuni processori reali ed esamineremo come esse gestiscano sia le letture sia le scritture. Nel Paragrafo 5.8 esamineremo più dettagliatamente la gestione delle scritture.

Write-back: uno schema in cui un elemento viene scritto solo nel blocco di cache; il blocco corrispondente del livello inferiore della gerarchia viene scritto solamente quando il blocco della cache viene sostituito.

Approfondimento. La scrittura in cache introduce alcune problematiche che non sono presenti nella lettura. In questa sezione di approfondimento ne esamineremo due: la strategia adottata per le miss in scrittura e l'implementazione efficiente della scrittura in una cache con write-back.

Consideriamo una miss di una cache con write-through. La strategia adottata in molte cache di questo tipo, chiamata *write-allocate* (allocazione per la scrittura), è di allocare un blocco intero della cache. Il blocco intero viene prelevato dalla memoria e solamente la parte appropriata del blocco viene sovrascritta. Una strategia alternativa, chiamata *no-write-allocate*, è quella di aggiornare solo la parte appropriata del blocco direttamente nella memoria principale senza aggiornare la cache. Questi due metodi derivano dall'osservazione che a volte i programmi scrivono blocchi interi di dati, per esempio quando il sistema operativo inizializza a zero una pagina della memoria principale. In questi casi, il prelevamento del dato associato alla prima miss può risultare inutile. Alcuni calcolatori consentono una politica di write-allocate che può essere cambiata in base alla pagina di memoria considerata.

Implementare la scrittura in modo efficiente in una cache di tipo write-back è più complicato rispetto a una cache write-through. In una cache write-through possiamo scrivere il dato nella cache e leggere il campo tag; se questo non coincide con il corrispondente campo dell'indirizzo del dato, si verifica una miss. Dato che la cache è di tipo write-through, la sovrascrittura del blocco della cache non ha effetti catastrofici, perché la memoria principale contiene già il valore corretto. In una cache con write-back, quando un dato deve essere scritto nella cache e si ha una miss se il dato è stato modificato nella cache, occorre prima scrivere il blocco nella memoria principale. Se sovrascrivessimo semplicemente il blocco a seguito di un'istruzione di store prima ancora di sapere se l'accesso produca una hit (come si potrebbe fare nel caso di una cache di tipo write-through), distruggeremmo il contenuto del blocco, perché esso non è stato ancora copiato nel livello inferiore della gerarchia delle memorie.

In una cache di tipo write-back, poiché non si possono sovrascrivere i blocchi, ci sono due possibili soluzioni. Nella prima, le istruzioni di store richiedono due cicli di clock: un ciclo per verificare se si ottiene una hit, seguito da un secondo ciclo per eseguire effettivamente la scrittura. In alternativa occorre avere un buffer di scrittura per conservare temporaneamente il dato, consentendo così alla scrittura di impiegare un solo ciclo e inserendo, di fatto, l'operazione di scrittura in pipeline. Quando si utilizza un buffer di scrittura, il processore accede alla cache e scrive il dato nel buffer durante il normale ciclo di accesso alla cache. Nel caso in cui si verifichi una hit, il nuovo dato viene copiato dal buffer di scrittura in cache al primo ciclo di clock successivo in cui la cache non viene utilizzata.

In una cache di tipo write-through, invece, le scritture possono essere sempre eseguite in un solo ciclo di clock, nel quale viene letto il campo tag dell'indirizzo e viene salvato il dato nella porzione corrispondente del blocco della cache. Se il campo tag coincide con quello del blocco presente in cache, il processore può proseguire normalmente l'esecuzione, essendo stato aggiornato il blocco corretto; altrimenti, il processore genera una miss in scrittura per caricare le altre parole del blocco corrispondenti a quell'indirizzo.

Molte cache di tipo write-back contengono anche dei buffer di scrittura che vengono utilizzati per ridurre la penalità di miss dovuta alla sostituzione di un blocco che è stato modificato. In questi casi, il blocco modificato viene spostato in un buffer di write-back associato alla cache mentre il blocco richiesto viene letto dalla memoria. Il contenuto del buffer di write-back viene successivamente copiato nella memoria principale. Supponendo che una seconda miss non avvenga immediatamente, questa tecnica dimezza la penalità di miss associata alla sostituzione di un blocco modificato.

Un esempio di memoria cache: il processore FastMATH Intrinsity

Il processore FastMATH Intrinsity è un microprocessore embedded veloce che utilizza l'architettura MIPS e una semplice implementazione della cache. Iniziamo da questo esempio semplice (ma reale) per ragioni didattiche; più avanti esamineremo le strutture più complesse adottate per le cache dei microprocessori ARM e Intel. La Figura 5.12 mostra l'organizzazione della cache dati del processore FastMATH Intrinsity. Si noti che la dimensione dell'indirizzo in questo calcolatore è di soli 32 bit e non 64 bit come nel resto del libro.

Questo processore ha una pipeline a 12 stadi. Quando opera a pieno regime, il processore può richiedere sia una parola di dati sia una parola di istruzioni a ogni ciclo di clock. Per soddisfare queste richieste senza generare stalli nella pipeline, vengono utilizzate una cache dati e una cache istruzioni separate. Ciascuna cache è di 16 KiB, ossia di 4096 parole, con 16 parole per blocco.

Le richieste di lettura dalla cache non destano problemi. Poiché la cache dati e la cache istruzioni sono separate, occorrono segnali diversi per leggere e scrivere in ognuna delle due memorie; si ricordi, inoltre, che occorre aggiornare la cache istruzioni ogni volta che si verifica una miss. Quindi, i passi

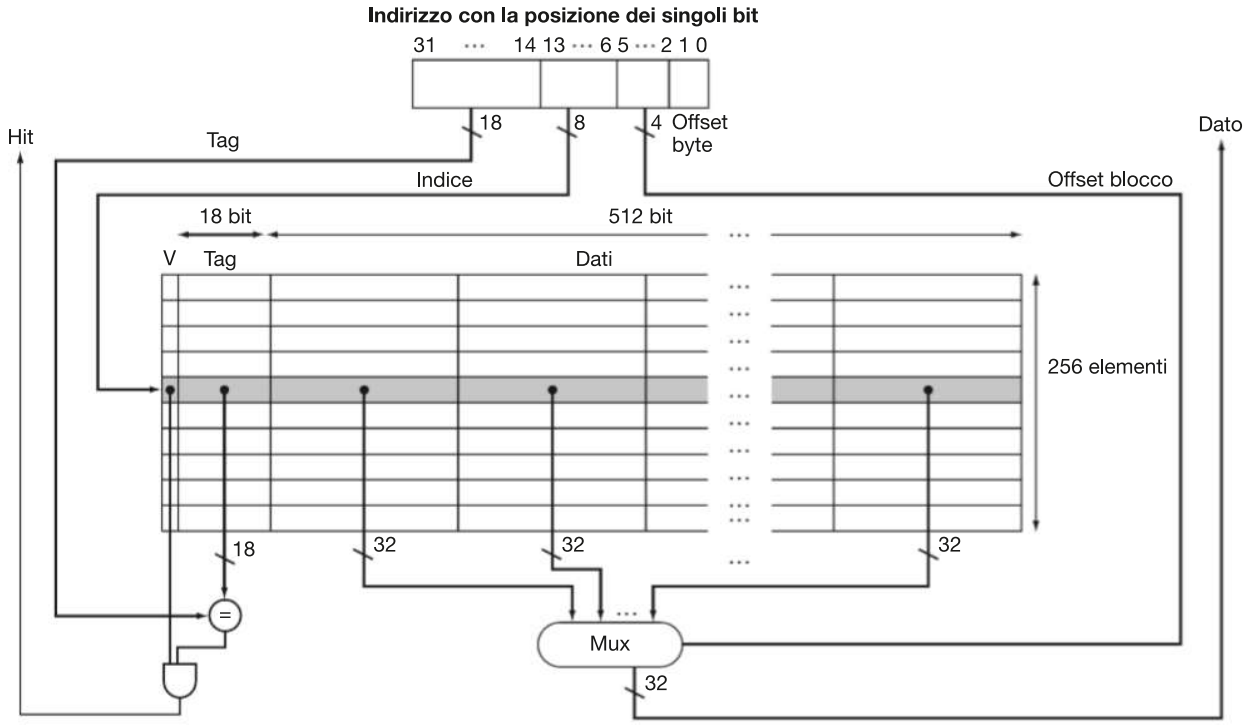


Figura 5.12 Le due cache del processore FastMATH Intrinsity hanno una dimensione di 16 KiB: ognuna contiene 256 blocchi con 16 parole ciascuno. Si noti che l'ampiezza dell'indirizzo è di soli 32 bit in questo computer. Il campo tag è di 18 bit, il campo indice è di 8 bit, mentre un campo di 4 bit (i bit dal 5 al 2) viene utilizzato per indicizzare il blocco e selezionare la parola all'interno del blocco mediante un multiplexer 16:1. In pratica, per eliminare il multiplexer, le cache utilizzano una RAM separata di grandi dimensioni per i dati e una RAM più piccola per i tag e l'offset del blocco fornisce i bit aggiuntivi per indirizzare ila parola all'interno della RAM più grande. In questo caso, la RAM dati ha parole di 32 bit ciascuna e deve contenere 16 parole per ogni blocco.

Frequenza di miss sulle istruzioni	Frequenza di miss sui dati	Frequenza di miss totale
0,4%	11,4%	3,2%

Figura 5.13 Frequenza approssimativa delle miss sulle istruzioni e sui dati in un processore FastMATH Intrinsity quando vengono eseguiti i benchmark SPEC CPU2000. La frequenza di miss totale è la frequenza di miss effettiva rilevata per una configurazione che prevede una cache dati da 16 KiB e una cache istruzioni da 16 KiB. Questa viene ottenuta pesando la frequenza di miss delle istruzioni e dei dati con la frequenza di accesso alla cache dati e alla cache istruzioni.

associati a una richiesta di lettura, validi per entrambe le memorie, sono i seguenti.

1. Inviare l'indirizzo alla cache appropriata. L'indirizzo si ricava dal PC (per le istruzioni) oppure dalla ALU (per i dati).
2. Se la cache segnala una hit, la parola richiesta è disponibile sulle linee dati. Poiché ci sono 16 parole all'interno del blocco selezionato, occorre scegliere la parola giusta. Il campo offset del blocco viene utilizzato per controllare il multiplexer mostrato in basso nella figura, il quale seleziona la parola richiesta tra le 16 parole del blocco selezionato.
3. Se la cache genera una miss, l'indirizzo della parola viene inviato alla memoria principale. Quando la memoria restituisce il dato richiesto, esso viene scritto nella cache e quindi trasferito dalla cache al processore per soddisfare la richiesta.

Per le operazioni di scrittura, il FastMATH Intrinsity offre sia la modalità write-through sia la modalità write-back, lasciando al sistema operativo il compito di decidere quale strategia utilizzare per l'applicazione corrente. La dimensione del buffer di scrittura è di un singolo elemento.

Quale percentuale di miss ci possiamo aspettare da una cache con una struttura come quella utilizzata dal FastMATH Intrinsity? La **Figura 5.13** mostra la frequenza di miss per le istruzioni e per i dati. La frequenza di miss totale è la frequenza di miss effettiva degli accessi a cache calcolata per ciascun programma tenendo conto delle differenti frequenze di accesso ai dati e alle istruzioni.

Sebbene la frequenza delle miss sia una caratteristica importante da considerare durante la progettazione di una cache, l'indicatore fondamentale è l'impatto del sistema di memoria sul tempo di esecuzione dei programmi; vedremo tra poco come frequenza delle miss e tempo di esecuzione siano collegati tra loro.

Approfondimento. Una cache unica di dimensioni pari alla somma delle due **split cache** (cache separate) di solito ha una frequenza di hit superiore. Questo accade perché, utilizzando una memoria unica, i blocchi non sono separati rigidamente tra istruzioni e dati. Nonostante ciò, oggi quasi tutte le architetture adottano cache separate per dati e istruzioni, al fine di aumentare la *larghezza di banda* della cache, per soddisfare le esigenze delle moderne pipeline. Inoltre, questa struttura permette di ottenere un numero minore di miss per conflitto (Paragrafo 5.8).

Split cache: uno schema in cui un livello della gerarchia delle memorie è composto da due cache indipendenti che operano in parallelo: una cache gestisce le istruzioni e l'altra i dati.

Di seguito riportiamo alcune misure ottenute su cache tipiche del processore FastMATH Intrinsity e su cache unificate che abbiano dimensioni pari alla somma delle due cache:

- dimensione complessiva della cache: 32 KiB;
- frequenza effettiva delle miss per le cache separate: 3,24%;
- frequenza delle miss per la cache unica: 3,18%.

La frequenza delle miss è di poco inferiore nel caso di cache separate.

Il vantaggio ottenuto raddoppiando la larghezza di banda della cache, conseguenza del supporto all'accesso simultaneo ai dati e alle istruzioni, compensa abbondantemente lo svantaggio di avere una frequenza di miss leggermente più elevata. Questa è un'ulteriore conferma del fatto che non è possibile utilizzare solo la frequenza delle miss come indice delle prestazioni di una cache, come verrà mostrato nel Paragrafo 5.4.

Riepilogo

Abbiamo iniziato il paragrafo esaminando la più semplice delle cache: una cache a mappatura diretta con blocchi di ampiezza pari a quella di una parola. In una cache di questo tipo sia le hit sia le miss sono semplici da gestire, visto che ogni parola può andare esattamente in una sola locazione della cache e per ciascuna di esse è previsto un tag separato. Per mantenere la coerenza tra memoria principale e cache, si può utilizzare uno schema write-through nel quale ogni scrittura in cache provoca l'aggiornamento anche della memoria principale. L'alternativa allo schema write-through è lo schema write-back, nel quale un blocco della cache viene ricopiato nella memoria principale solamente quando deve essere sostituito; quest'ultimo schema verrà discusso in modo più approfondito nei prossimi paragrafi.

Per sfruttare la località spaziale, una cache deve avere blocchi di dimensioni superiori a una parola. L'utilizzo di blocchi di dimensioni maggiori consente di diminuire la frequenza delle miss e di migliorare l'efficienza della cache, riducendo la quantità di memoria dedicata ai campi tag a favore dei dati. Anche se l'utilizzo di blocchi di dimensioni maggiori riduce la frequenza delle miss, questa strategia può causare un aumento della penalità di miss. Se la penalità di miss crescesse linearmente con la dimensione dei blocchi, l'utilizzo di blocchi di dimensioni maggiori porterebbe rapidamente a un peggioramento delle prestazioni.

Per evitare una caduta delle prestazioni, la banda della memoria principale viene aumentata in modo che il trasferimento dei blocchi dalla cache avvenga nel modo più efficiente possibile.

I due metodi più utilizzati per aumentare la larghezza di banda sono: rendere la memoria più larga oppure organizzarla in blocchi interallacciati. Nel tempo i progettisti delle DRAM hanno continuamente migliorato l'interfaccia tra processore e memoria per aumentare la larghezza di banda dei trasferimenti a raffica (in modalità *burst*), in modo da diminuire la latenza del trasferimento di blocchi di cache più ampi.

Autovalutazione

La velocità del sistema di memoria condiziona il progettista nel decidere quale dimensione adottare per il blocco della cache. Quali delle seguenti linee guida possono essere considerate generalmente valide nella progettazione di una cache?

1. Più piccola è la latenza della memoria, più piccolo sarà il blocco della cache.
2. Più piccola è la latenza della memoria principale, più grande sarà il blocco della cache.
3. Più larga è la banda della memoria, più piccolo sarà il blocco della cache.
4. Più larga è la banda della memoria, più grande sarà il blocco della cache.

5.4 Come misurare e migliorare le prestazioni di una cache

Iniziamo questo paragrafo esaminando diversi modi per misurare e analizzare le prestazioni di una cache; esploreremo quindi due diverse tecniche per incrementare le prestazioni. La prima è basata sulla riduzione della frequenza delle

miss, ottenuta riducendo la probabilità che due differenti blocchi di memoria principale entrino in conflitto per la stessa locazione della cache.

La seconda tecnica riduce la penalità delle miss introducendo nella gerarchia un livello aggiuntivo. Questa secondo metodo, chiamato *cache multilivello* (*multilevel caching*), fu introdotto per la prima volta nel 1990 in calcolatori di fascia alta, venduti a una cifra superiore ai 100 000 dollari; da allora è divenuto comune anche nei dispositivi mobili, venduti per alcune centinaia di dollari!

Il tempo di CPU può essere suddiviso nei cicli di clock che la CPU spende per eseguire un programma e in quelli che la CPU trascorre in attesa di una risposta dal sistema di memoria. Di solito si suppone che il costo degli accessi alla cache che producono una hit rientri nei cicli di clock della normale esecuzione della CPU. Si ottiene quindi:

$$\begin{aligned}\text{Tempo di CPU} = {} & (\text{Cicli di esecuzione CPU} + \text{Cicli di stallo della memoria}) \\ & \times \text{Durata del ciclo di clock}\end{aligned}$$

I cicli di stallo della memoria sono dovuti principalmente alle miss della cache. In questo capitolo considereremo valida questa ipotesi e restringeremo la discussione a un modello semplificato del sistema di memoria. Nei processori reali, gli stalli generati da operazioni di lettura e scrittura possono risultare anche molto complessi e una previsione accurata delle prestazioni richiede di solito che siano effettuate simulazioni molto dettagliate del processore e del sistema di memoria.

I cicli di stallo dovuti alla memoria possono essere definiti come la somma degli stalli provocati dalle letture e di quelli provocati dalle scritture:

$$\begin{aligned}\text{Cicli di stallo della memoria} = {} & (\text{Cicli di stallo in lettura} \\ & + \text{Cicli di stallo in scrittura})\end{aligned}$$

I cicli di stallo dovuti alle letture possono, a loro volta, essere definiti in termini di numero di accessi in lettura fatti dal programma, di penalità di miss per ogni operazione di lettura (in cicli di clock) e di frequenza di miss per le letture:

$$\begin{aligned}\text{Cicli di stallo in lettura} = {} & \frac{\text{Letture}}{\text{Programma}} \times \text{Frequenza di miss in lettura} \times \\ & \times \text{Penalità di miss in lettura}\end{aligned}$$

Il discorso diventa più complesso per la scrittura. In uno schema write-through, sono due le cause che generano uno stallo del processore: le miss in scrittura, che di solito richiedono di prelevare dalla memoria principale l'intero blocco prima di continuare la scrittura (per maggiori dettagli sulla gestione delle scritture *vedi Approfondimento* nel sottoparagrafo *Gestione della scrittura* del Paragrafo 5.3), e la saturazione del buffer di scrittura, che si verifica quando viene richiesta una scrittura ma il buffer di scrittura risulta pieno. Quindi, il numero di cicli di stallo provocati dalle scritture si ottiene come somma di queste due componenti:

$$\begin{aligned}\text{Cicli di stallo in scrittura} = {} & \left[\frac{\text{Scritture}}{\text{Programma}} \times \text{Frequenza di miss in scrittura}\right. \\ & \left.\times \text{Penalità di miss in scrittura}\right] \\ & + \text{Numero di stalli del buffer di scrittura}\end{aligned}$$

Poiché gli stalli del buffer di scrittura dipendono da quanto ravvicinate nel tempo sono le scritture, e non solo dalla loro frequenza, non è possibile fornire una semplice equazione per calcolare il numero di stalli. Fortunatamente, nei sistemi provvisti di buffer di scrittura sufficientemente profondi (per es. quattro o più parole) e di una memoria capace di scrivere dati a una frequenza molto superiore (per es. di un fattore 2) alla frequenza media delle richieste di scrittura dei programmi, gli stalli dovuti al buffer saranno molto pochi e si possono quindi trascurare. Se un sistema non dovesse soddisfare questi criteri, sarebbe un sistema progettato male: il progettista avrebbe dovuto utilizzare un buffer di scrittura più profondo oppure adottare una politica di scrittura di tipo write-back.

Gli schemi di scrittura di tipo write-back hanno un'ulteriore possibile causa di stallo, che nasce dalla necessità di scrivere un blocco della cache nella memoria principale quando il blocco della cache viene rimpiazzato. Discuteremo più in dettaglio questo problema nel Paragrafo 5.8.

Nell'organizzazione di molte cache di tipo write-through, la penalità delle miss in lettura e in scrittura è uguale e corrisponde al tempo richiesto per prelevare un blocco dalla memoria principale. Se supponiamo che gli stalli provocati dal buffer di scrittura siano trascurabili, possiamo considerare una sola frequenza e una sola penalità di miss per le letture e le scritture:

$$\text{Cicli di stallo della memoria} = \frac{\text{Accessi alla memoria}}{\text{Programma}} \times \text{Frequenza di miss} \times \text{Penalità di miss}$$

che si può scrivere anche come:

$$\text{Cicli di stallo della memoria} = \frac{\text{Istruzioni}}{\text{Programma}} \times \frac{\text{Miss}}{\text{Istruzioni}} \times \text{Penalità di miss}$$

Consideriamo ora un semplice esempio per comprendere l'impatto delle prestazioni delle cache sulle prestazioni complessive di un processore.

Determinazione delle prestazioni di una cache

ESEMPIO

Si supponga una frequenza di miss del 2% per una cache istruzioni e del 4% per una cache dati. Si supponga anche che il processore abbia un CPI di 2 quando non si verificano stalli dovuti alla memoria e che la penalità di miss sia pari a 100 cicli di clock per tutte le miss. Determinare di quanto sarebbe più veloce il processore se fosse dotato di una cache ideale che non provochi mai miss. Si ipotizzi anche che la percentuale delle istruzioni di load e di store sul totale delle istruzioni sia del 36%.

SOLUZIONE

Il numero di cicli di clock spesi per gestire le miss per le istruzioni, espresso in numero di istruzioni, I, è:

$$\text{Cicli di miss per le istruzioni} = I \times 2\% \times 100 = 2{,}00 \times I$$

Dato che la percentuale delle istruzioni di load e di store è del 36%, possiamo calcolare il numero di cicli di clock spesi per le miss per l'accesso ai dati:

$$\text{Cicli di miss per i dati} = I \times 36\% \times 4\% \times 100 = 1{,}44 \times I$$

(continua)

Il numero totale di cicli di clock di stallo dovuti alla memoria è quindi pari a 2,0 × I + 1,44 × I = 3,44 × I, che è più di 3 cicli di clock per istruzione. Di conseguenza, il CPI totale, che tiene conto anche degli stalli dovuti alla memoria, è pari a 2 + 3,44 = 5,44. Dato che non muta né il numero di istruzioni né la frequenza di clock, il rapporto tra i tempi di esecuzione della CPU è:

$$\frac{\text{Tempo di CPU con stalli}}{\text{Tempo di CPU con cache ideale}} = \frac{\text{I} \times \text{CPI}_{\text{stallo}} \times \text{Periodo di clock}}{\text{I} \times \text{CPI}_{\text{ideale}} \times \text{Periodo di clock}} = \frac{\text{CPI}_{\text{stallo}}}{\text{CPI}_{\text{ideale}}} = \frac{5{,}44}{2}$$

Perciò, le prestazioni del processore dotato di una cache ideale saranno migliori di un fattore pari a:

$$\frac{5{,}44}{2} = 2{,}72$$

(continua)

Che cosa succede se il processore è reso più veloce, ma il sistema di memoria rimane invariato? In questo caso, la quantità di tempo spesa negli stalli dovuti alla memoria aumenterà e diventerà la parte preponderante del tempo totale di esecuzione. La legge di Amdahl, che abbiamo esaminato nel Capitolo 1, spiega questo meccanismo. Pochi semplici esempi consentono di mostrare quanto questo problema sia importante. Immaginiamo di accelerare il calcolatore considerato nell'esempio precedente riducendo il suo CPI da 2 a 1 senza modificare la frequenza di clock (cosa che può essere ottenuta migliorando la pipeline). Il sistema, considerando le miss della cache, avrà ora un CPI pari a 1 + 3,44 = 4,44, mentre il sistema dotato di una cache ideale sarà:

$$\frac{4{,}44}{1} = 4{,}44 \text{ volte più veloce}$$

La quantità di tempo di esecuzione spesa in stalli dovuti alla memoria salirebbe quindi da:

$$\frac{3{,}44}{5{,}44} = 63\%$$

a:

$$\frac{3{,}44}{4{,}44} = 77\%$$

Analogamente, aumentare la frequenza di clock senza modificare il sistema di memoria aumenterebbe la perdita di prestazioni dovuta alle miss della cache.

Gli esempi e le equazioni viste in precedenza sono basati sull'ipotesi che il tempo di hit non sia un fattore significativo nel calcolo delle prestazioni di una cache. Chiaramente, se il tempo di hit aumentasse, il tempo totale richiesto per accedere a una parola del sistema di memoria aumenterebbe, facendo quindi crescere verosimilmente la durata del periodo di clock del processore. Anche se analizzeremo più avanti altre situazioni in cui aumenta il tempo di hit, introduciamo ora uno degli elementi cruciali: la dimensione della cache. Una cache più grande, infatti, avrà bisogno di un tempo di accesso maggiore. Tornando all'analogia con la biblioteca, è come se la scrivania a cui siede lo studente fosse molto grande: occorrerebbe più tempo per trovare un libro rispetto a una scrivania più piccola, perché ci sarebbero più libri da consultare.

Un aumento del tempo di hit è facile che comporti la necessità di aggiungere un altro stadio alla pipeline, poiché occorrerebbero più cicli di clock per accedere e trasferire i dati da una cache a seguito di una hit. Sebbene sia più difficile calcolare l'impatto sulle prestazioni di una pipeline più profonda, ci sarà comunque una dimensione della cache oltre la quale l'aumento del tempo di hit supera il miglioramento ottenuto nella frequenza di hit, portando a un peggioramento delle prestazioni globali del processore.

Per tenere conto del fatto che il tempo di accesso ai dati influenza le prestazioni sia nel caso delle hit sia delle miss, i progettisti utilizzano a volte il *tempo medio di accesso alla memoria* (AMAT, *Average Memory Access Time*) per esaminare e confrontare tra loro cache diverse. Questo tempo è dato dalla seguente relazione:

$$\text{AMAT} = \text{Durata di una hit} + \text{Frequenza di miss} \times \text{Penalità di miss}$$

Calcolo del tempo medio di accesso alla memoria

ESEMPIO

Determinare l'AMAT di un processore avente un periodo di clock di 1 ns, una penalità di miss di 20 cicli di clock, una frequenza di miss di 0,05 miss per istruzione e un tempo di accesso alla cache, comprendente la generazione del segnale di hit, di 1 ciclo di clock. Si supponga che la penalità di miss sia la stessa per le letture e le scritture e si ignorino gli altri stalli in scrittura.

SOLUZIONE

Il tempo medio di accesso alla memoria per singola istruzione è

$$\begin{aligned}\text{AMAT} &= \text{Durata di una hit} + \text{Frequenza di miss} \times \text{Penalità di miss}\\ &= 1 + 0{,}05 \times 20\\ &= 2 \text{ cicli di clock}\end{aligned}$$

ossia 2 ns.

Nel prossimo paragrafo analizzeremo le possibili diverse organizzazioni di una cache che consentono di diminuire la frequenza di miss; il prezzo da pagare, a volte, è l'aumento del tempo di hit. Discuteremo altri esempi nel P aragrafo 5.16.

Riduzione delle miss di una cache utilizzando un posizionamento più flessibile dei blocchi

Finora abbiamo utilizzato uno schema semplice per determinare la posizione di un blocco che deve essere caricato in cache in cui il blocco può trovarsi in una sola posizione. Come già detto in precedenza, questo schema viene chiamato schema a *mappatura diretta*, poiché esiste una mappatura diretta degli indirizzi della memoria principale sulle singole locazioni del livello superiore della gerarchia. In realtà, esiste un ventaglio di schemi per determinare la posizione di un blocco. A un'estremità sta la mappatura diretta, all'estremità opposta si trova uno schema nel quale un blocco della memoria principale può essere scritto in una *qualsiasi* locazione della cache. Una cache che implementi tale schema è chiamata **cache completamente associativa**, perché un blocco della memoria può essere associato a un qualsiasi blocco della cache. Per trovare un blocco in una cache di questo tipo, la ricerca deve essere effettuata su

Cache completamente associativa: una cache in cui un blocco della memoria principale può essere scritto o letto da un qualsiasi blocco della cache.

tutti gli elementi della cache, dato che il blocco può trovarsi in una qualsiasi posizione. Per renderla efficace, la ricerca viene svolta in parallelo utilizzando un comparatore per ogni blocco della cache. Tali comparatori incrementano significativamente il costo dell'hardware, rendendo realizzabile lo schema totalmente associativo, di fatto, solo per cache con un numero ridotto di blocchi.

Le cache che adottano schemi intermedi tra la mappatura diretta e quella completamente associativa vengono chiamate **cache set-associative**. In una cache set-associativa ogni blocco della memoria principale può essere caricato in un numero prefissato di posizioni alternative (almeno due), e una cache con n possibili scelte è chiamata set-associativa a n vie. Una cache set-associativa a n vie è costituita da un certo numero di insiemi ciascuno costituito da n blocchi o linee. Ciascun blocco della memoria principale viene mappato su un'unica linea della cache, individuata dal campo indice, e il blocco di dati può essere scritto in *uno qualsiasi* degli insiemi costituenti la linea. Quindi una cache set-associativa combina la mappatura diretta con il posizionamento completamente associativo di un blocco: un blocco di memoria principale viene associato a una linea della cache in maniera diretta, e i blocchi di tutti gli insiemi vengono esaminati per verificare se contengono l'elemento cercato. La **Figura 5.14** mostra dove può essere scritto il blocco 12 della memoria principale in una cache costituita da otto blocchi in totale secondo i tre diversi schemi di posizionamento.

Cache set-associativa: una cache che possiede un numero prestabilito di locazioni alternative (almeno due) nelle quali può essere scritto o letto ogni blocco della memoria principale.

Come abbiamo già visto, in una cache a mappatura diretta la posizione di un blocco della memoria principale è data da:

(Numero del blocco) modulo (Numero dei *blocchi* nella cache)

Nella cache set-associativa, la linea che contiene il blocco viene individuata da:

(Numero del blocco) modulo (Numero delle *linee* della cache)

Dato che il blocco di dati può essere scritto in uno qualsiasi degli insiemi, la ricerca deve essere effettuata *analizzando il campo tag dei blocchi di tutti gli insiemi della linea*. In una cache completamente associativa, il blocco può essere messo in un qualsiasi insieme della cache, pertanto deve essere analizzato il campo tag di tutti i blocchi della cache durante la ricerca.

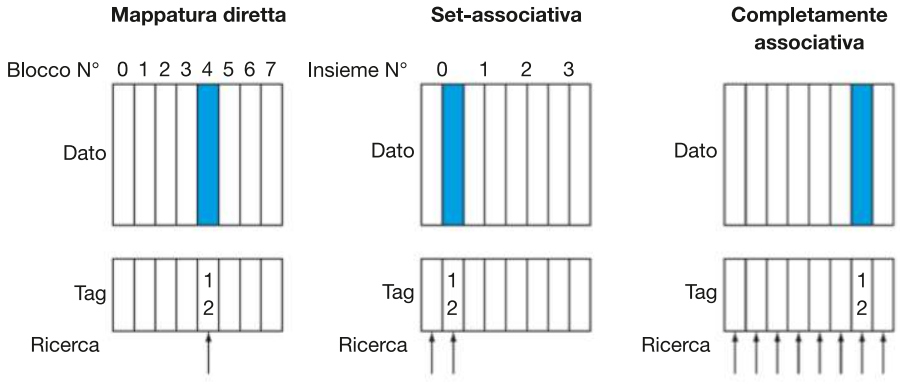


Figura 5.14 La posizione del blocco della memoria principale avente indirizzo 12 in una cache a otto blocchi varia a seconda che la cache sia a mappatura diretta, set-associativa o completamente associativa. Nella cache a mappatura diretta, il blocco 12 può essere scritto in un solo blocco della cache e il numero di questo blocco si ottiene dall'operazione (12 modulo 8) = 4. La cache set-associativa a due vie è costituita da quattro linee e il blocco 12 della memoria dovrà trovarsi nella linea (12 modulo 4) = 0, cioè in uno dei due blocchi che costituiscono questa linea. Nella cache completamente associativa, il blocco di indirizzo 12 può trovarsi in uno qualsiasi degli otto blocchi.

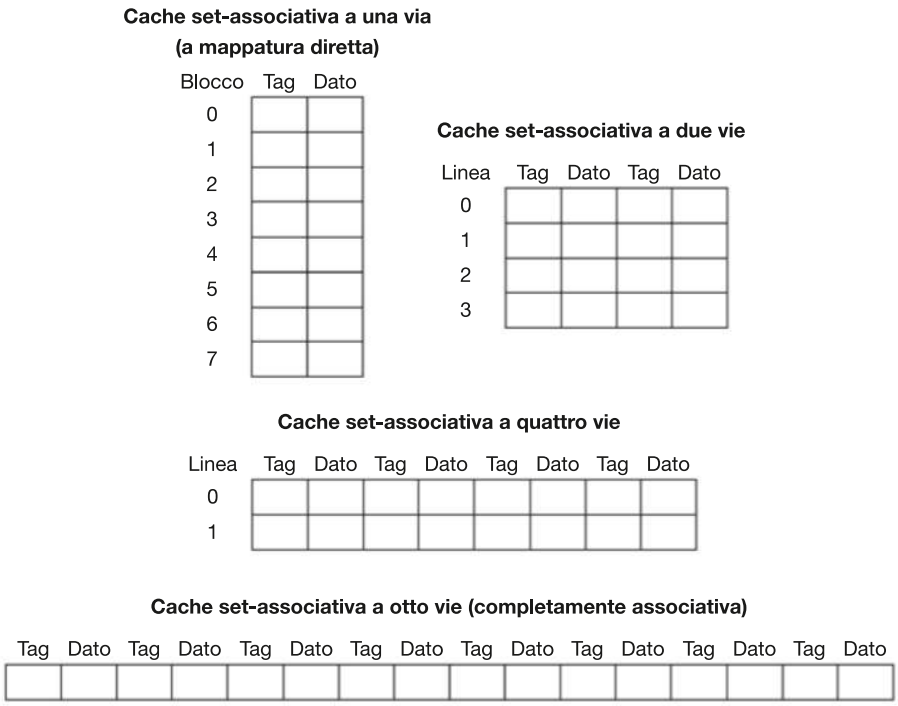


Figura 5.15 Cache di otto blocchi configurata come cache a mappatura diretta, set-associativa a due vie, set-associativa a quattro vie e completamente associativa. La dimensione complessiva della cache in termini di blocchi è uguale al numero di linee moltiplicato per il grado di associatività. Perciò, definita la dimensione della cache, al crescere dell'associatività diminuisce il numero delle linee, mentre cresce il numero dei blocchi sulla stessa linea. Con otto blocchi, una cache set-associativa a otto vie è equivalente a una cache completamente associativa.

I diversi schemi di corrispondenza si possono anche vedere come varianti della politica set-associativa.

La Figura 5.15 mostra le possibili strutture, con diverso grado di associatività, che possono essere adottate per una cache costituita da otto blocchi. Una cache a mappatura diretta è semplicemente una cache set-associativa a una sola via: ogni elemento della cache contiene un unico blocco, così come ogni linea contiene un solo blocco. Una cache di m elementi, completamente associativa, è semplicemente una cache set-associativa a m vie: ha un'unica linea contenente m blocchi e un elemento può trovarsi in uno qualsiasi dei blocchi.

Il vantaggio dell'aumento del grado di associatività è che, in genere, diminuisce la frequenza delle miss, come mostra l'esempio seguente. Lo svantaggio principale, del quale discuteremo più avanti, è il potenziale aumento del tempo di hit.

Miss e associatività nelle cache

ESEMPIO

Consideriamo tre cache di piccole dimensioni, ciascuna costituita da quattro blocchi di una parola: la prima cache è completamente associativa, la seconda è set-associativa a due vie e la terza è a mappatura diretta. Determinare il numero di miss nei tre casi quando vengono richiesti i blocchi con la seguente sequenza di indirizzi: 0, 8, 0, 6, 8.

(continua)

(continua)

SOLUZIONE

Il caso della mappatura diretta è il più semplice. Innanzitutto occorre determinare a quale blocco della cache corrispondano i diversi indirizzi:

Indirizzo del blocco	Blocco della cache
0	(0 modulo 4) = 0
6	(6 modulo 4) = 2
8	(8 modulo 4) = 0

Ora è possibile compilare il contenuto della cache dopo l'indirizzamento di ciascun blocco, come è mostrato nella seguente tabella. I blocchi non validi non contengono nulla; il testo in blu mostra il blocco appena inserito nella cache, mentre il testo in nero mostra gli elementi precedentemente inseriti nella cache:

Indirizzo del blocco richiesto della memoria principale	Hit o miss	Contenuto dei blocchi della cache dopo l'indirizzamento			
		0	1	2	3
0	Miss	Memoria[0]			
8	Miss	Memoria[8]			
0	Miss	Memoria[0]			
6	Miss	Memoria[0]		Memoria[6]	
8	Miss	Memoria[8]		Memoria[6]	

La cache a mappatura diretta genera cinque miss per i cinque accessi.

La cache set-associativa è costituita da due insiemi, aventi indice 0 e 1, ciascuno contenente due blocchi. Come prima cosa occorre individuare a quale linea corrisponda ciascun indirizzo di blocco:

Indirizzo del blocco	Linea della cache
0	(0 modulo 2) = 0
6	(6 modulo 2) = 0
8	(8 modulo 2) = 0

Dato che, in caso di miss, si può scegliere il blocco di quale insieme sostituire, è necessario stabilire una modalità di sostituzione. Di solito le cache set-associative sostituiscono il blocco di quale insieme utilizzato meno di recente; discuteremo in dettaglio altri metodi di sostituzione più avanti. Utilizzando questa regola di sostituzione, il contenuto della cache set-associativa dopo ogni accesso risulterà essere:

Indirizzo del blocco richiesto della memoria principale	Hit o miss	Contenuto dei blocchi della cache dopo l'indirizzamento			
		Linea 0	Linea 0	Linea 1	Linea 1
0	Miss	Memoria[0]			
8	Miss	Memoria[0]	Memoria[8]		
0	Hit	Memoria[0]	Memoria[8]		
6	Miss	Memoria[0]	Memoria[6]		
8	Miss	Memoria[8]	Memoria[6]		

(continua)

(continua)

Si noti che quando viene indirizzato il blocco 6, questo va a sostituire il blocco 8, poiché il blocco 8 è stato indirizzato meno di recente rispetto al blocco 0. La cache set-associativa a due vie genera in totale 4 miss, una in meno della cache a mappatura diretta.

La cache completamente associativa ha quattro blocchi, distribuiti su una singola linea: ogni blocco della memoria principale può quindi essere memorizzato in un qualsiasi blocco della cache. Le prestazioni di questo schema sono le migliori, dato che si verificano solamente 3 miss, come si può vedere dalla tabella seguente:

Indirizzo del blocco richiesto della memoria principale	Hit o miss	Contenuto dei blocchi della cache dopo l'accesso			
		Blocco 0	Blocco 1	Blocco 2	Blocco 3
0	Miss	Memoria[0]			
8	Miss	Memoria[0]	Memoria[8]		
0	Hit	Memoria[0]	Memoria[8]		
6	Miss	Memoria[0]	Memoria[8]	Memoria[6]	
8	Hit	Memoria[0]	Memoria[8]	Memoria[6]	

Per questa sequenza di accessi, 3 miss è il risultato migliore che si può ottenere, poiché vengono richiesti dati aventi 3 indirizzi di blocco distinti. Si noti che se la cache avesse avuto otto blocchi, nella struttura set-associativa a due vie non sarebbero state necessarie sostituzioni (come potete verificare voi stessi) e si sarebbe ottenuto un numero di miss pari a quello della cache completamente associativa.

Analogamente, se i blocchi fossero stati sedici, tutte e tre le cache avrebbero avuto lo stesso numero di miss. Questo esempio, sebbene molto semplice, dimostra che la dimensione della cache e il suo grado di associatività non sono parametri indipendenti nel determinarne le prestazioni.

Di quanto si può ridurre la frequenza di miss sfruttando l'associatività? La **Figura 5.16** mostra il miglioramento che si ottiene in un'architettura con una cache dati da 64 KiB e blocchi di 16 parole per diversi gradi di associatività, passando dalla mappatura diretta all'organizzazione set-associativa a otto vie. Il passaggio del grado di associatività da una a due vie migliora già di circa il 15% la frequenza di miss, la quale migliora leggermente aumentando ancora il grado di associatività.

Associatività	Frequenza di miss
1	10,3%
2	8,6%
4	8,3%
8	8,1%

Figura 5.16 Frequenza di miss in una cache dati con associatività variabile da una via a otto vie, avente un'organizzazione simile a quella del processore FastMATH Intrinsity, valutata utilizzando come benchmark gli SPEC CPU2000. Questi dati, relativi a 10 programmi del benchmark SPEC CPU2000, provengono da Hennessy e Patterson [2003].

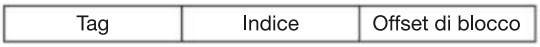


Figura 5.17 Le tre parti di un indirizzo in una cache set-associativa o a mappatura diretta. L'indice viene utilizzato per selezionare la linea, mentre per selezionare il blocco all'interno della linea viene comparato il tag dell'indirizzo con i tag dei blocchi. L'offset di blocco rappresenta la posizione del dato desiderato all'interno del blocco.

Come trovare un blocco nella cache

Consideriamo ora il problema di come si possa trovare un blocco all'interno di una cache set-associativa. Proprio come in una cache a mappatura diretta, ogni blocco di una cache set-associativa contiene un campo tag che permette di individuarne l'indirizzo riferito alla memoria principale. Il campo tag di ciascuno dei blocchi associati ai diversi insiemi, contenuti sulla stessa linea della cache viene controllato per verificare se corrisponda a quello dell'indirizzo del dato o dell'istruzione richiesta dal processore. La **Figura 5.17** mostra come si scompone l'indirizzo. Il contenuto del campo indice serve a selezionare la linea che può contenere l'elemento cercato: viene esaminato il campo tag di tutti gli insiemi appartenenti alla linea selezionata. Dato che la velocità di esecuzione è fondamentale, il contenuto del campo tag viene esaminato in parallelo per tutti i blocchi della linea selezionata associati ai diversi insiemi: una ricerca sequenziale renderebbe eccessivo il tempo di hit di una cache set-associativa.

Se si mantiene costante la dimensione totale della cache, al crescere del grado di associatività aumenta il numero dei blocchi contenuti in una singola linea, e quindi anche il numero di confronti che devono essere condotti simultaneamente per effettuare la ricerca in parallelo: ogni volta che raddoppia il grado di associatività, raddoppia anche il numero di blocchi contenuti in una linea e si dimezza il numero di linee. Analogamente, ogni incremento dell'associatività di un fattore 2 fa diminuire di 1 bit la dimensione del campo indice e fa aumentare di 1 bit la dimensione del campo tag. Perciò, in una cache completamente associativa c'è una sola linea e tutti i blocchi devono essere esaminati in parallelo: il campo indice non esiste e tutto l'indirizzo, a parte l'offset del blocco, deve essere confrontato con il tag di ogni blocco. In altre parole, la ricerca viene effettuata su tutta la cache, senza indicizzazione.

Una cache a mappatura diretta richiede un unico comparatore, dato che l'elemento cercato può trovarsi in una sola posizione; in questo caso, si accede alla cache utilizzando solamente il campo indice. In una cache set-associativa a quattro vie, come quella mostrata nella **Figura 5.18**, sono necessari quattro comparatori, oltre a un multiplexer con quattro ingressi e un'uscita, richiesto per scegliere uno dei quattro possibili blocchi della linea selezionata. L'accesso alla cache, quindi, si effettua utilizzando l'indice per individuare la linea appropriata e poi si esaminano i diversi tag della linea. Il costo di una cache set-associativa consiste nei comparatori aggiuntivi e nel ritardo imposto dalla necessità di confrontare i campi tag con l'indirizzo e di selezionare il blocco desiderato tra i blocchi della linea associati ai diversi insiemi.

In ogni gerarchia delle memorie, la scelta tra lo schema a mappatura diretta, lo schema set-associativo e quello completamente associativo dipende dal confronto tra il costo di una miss e il costo necessario a realizzare l'associatività, sia dal punto di vista del tempo di elaborazione sia in relazione ai componenti hardware aggiuntivi.

Approfondimento. Una *memoria indirizzabile per contenuto* (CAM, *Content Addressable Memory*) è un circuito che combina la comparazione e la memorizzazione in un unico dispositivo. Invece di ricevere un indirizzo e leggere una parola, come nelle RAM, una CAM riceve un dato, cerca al suo interno una copia del dato in ingresso

 © 978-88-08-19966-9

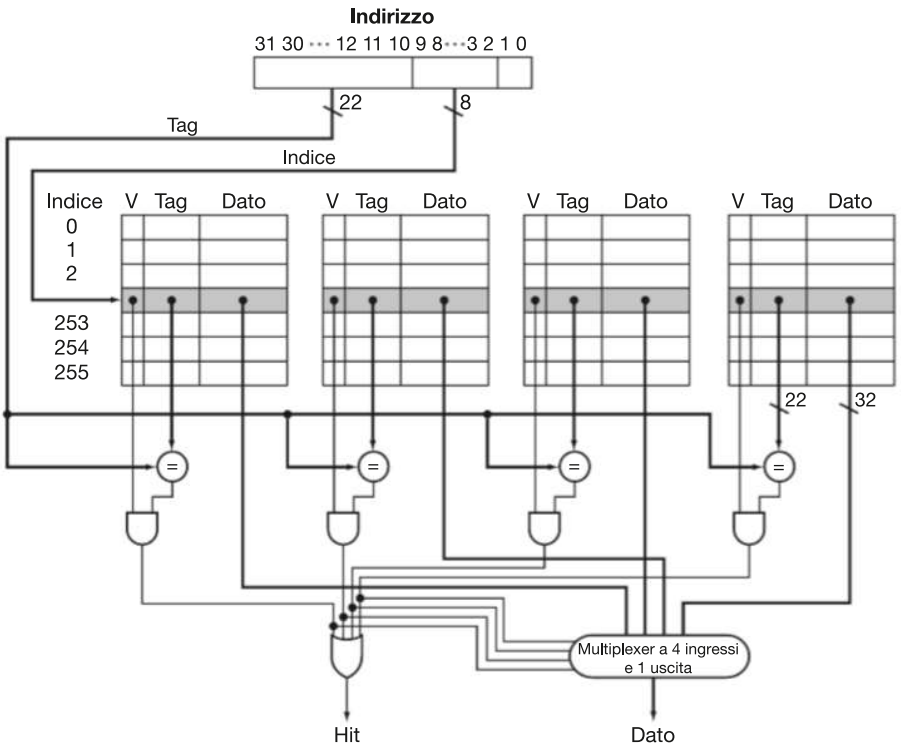


Figura 5.18 La realizzazione di una cache set-associativa a quattro vie richiede quattro comparatori e un multiplexer con 4 ingressi e 1 uscita. I comparatori individuano il blocco di quale insieme, se esiste, corrisponde al tag dell'indirizzo. L'uscita dei comparatori viene utilizzata per selezionare il dato da uno dei quattro blocchi della linea selezionata, attraverso un multiplexer pilotato dai segnali di hit dei diversi blocchi. In alcune realizzazioni della cache, il segnale di abilitazione dell'uscita, che si trova nella parte del chip dedicata ai dati, può essere utilizzato per selezionare il blocco della linea che deve essere portato in uscita. Il segnale di abilitazione dell'uscita proviene direttamente dai comparatori e consente di portare in uscita il blocco il cui tag corrisponde a quello dell'indirizzo del dato richiesto. Questa struttura elimina la necessità del multiplexer.

e restituisce l'indice della riga nella quale ha trovato il dato (se lo trova). Le memorie CAM consentono ai progettisti di realizzare un grado di associatività ancora maggiore di quello che si riesce a ottenere utilizzando SRAM e comparatori. Nel 2020, l'esistenza di CAM più grandi e potenti ha fatto sì che le memorie associative a due e quattro vie fossero costruite ancora con comparatori e blocchi di SRAM standard, mentre le cache a otto vie o con un grado di associatività ancora superiore vengono costruite con memorie CAM.

Come scegliere il blocco da sostituire

Quando si verifica una miss in una cache a mappatura diretta, il blocco richiesto può essere scritto in un'unica posizione e quello che occupava precedentemente questa posizione deve essere sostituito. In una cache associativa, possiamo scegliere dove scrivere il blocco richiesto e quindi dobbiamo decidere quale sia il blocco da sostituire. In una cache completamente associativa, tutti i blocchi costituiscono un potenziale candidato per la sostituzione. Se la cache è set-associativa, possiamo scegliere tra i blocchi contenuti nella stessa linea della cache.

Lo schema di sostituzione più comunemente utilizzato è detto **utilizzo meno recente** (**LRU**, *Least Recently Used*) ed è quello che abbiamo adottato nel precedente esempio. Nello schema LRU, il blocco sostituito è quello che è rimasto inutilizzato più a lungo; nel nostro esempio abbiamo quindi sostituito il blocco Memoria(0) invece del blocco Memoria(6).

Utilizzo meno recente (LRU): uno schema di sostituzione nel quale il blocco sostituito è quello che è rimasto inutilizzato per più tempo.

La sostituzione tramite schema LRU viene implementata tenendo traccia della sequenza di impiego dei vari blocchi della linea. Per una cache set-associativa a due vie, è sufficiente un unico bit per ogni linea per memorizzare quale dei due blocchi è stato utilizzato più di recente; questo bit viene impostato in maniera opportuna ogni volta che si utilizza uno dei due blocchi della linea. All'aumentare del grado di associatività, l'implementazione dello schema LRU diventa sempre più difficile; nel Paragrafo 5.8 analizzeremo uno schema di sostituzione alternativo.

Dimensione del campo tag e grado di associatività

ESEMPIO

Incrementare l'associatività di una cache comporta più comparatori e più bit per il campo tag di ogni blocco della cache. Considerando una cache costituita da 4096 blocchi, con blocchi da quattro parole e 32 bit di indirizzo, determinare il numero totale di linee e il numero di bit del campo tag per una cache a mappatura diretta, una cache set-associativa a due vie e a quattro vie e per una cache completamente associativa.

SOLUZIONE

Dato che ci sono 16 (= 2^4) byte per blocco e l'indirizzo è su 32 bit, 32 – 4 = 28 bit devono essere utilizzati per l'indice e il tag. La cache a mappatura diretta ha un numero di linee pari al numero di blocchi; occorrono quindi 12 bit per l'indice, essendo $\log_2(4096) = 12$. Il numero totale di bit per il campo tag è perciò di $(28 - 12) \times 4096 = 16 \times 4096 = 66$ K.

Ogni volta che raddoppia il grado di associatività, diminuisce il numero di linee di un fattore 2; decresce quindi di 1 il numero di bit utilizzati per l'indice e aumenta di 1 il numero di bit del campo tag. Quindi, per una cache set-associativa a due vie, ci sono 2048 linee e il numero totale di bit del campo tag è pari a $(28 - 11) \times 2 \times 2048 = 34 \times 2048 = 70$ Kbit. Per una cache set-associativa a quattro vie, il numero totale di linee è pari a 1024 e il numero totale di bit del campo tag è $(28 - 10) \times 4 \times 1024 = 72 \times 1024 = 74$ K.

In una cache completamente associativa esiste un'unica linea costituita da 4096 blocchi e il campo tag è costituito da 60 bit; pertanto il numero totale di bit utilizzati per il campo tag è $28 \times 4096 \times 1 = 115$ K.

Ridurre la penalità di miss utilizzando una cache multilivello

Tutti i moderni calcolatori fanno uso delle cache. Per ridurre ulteriormente la differenza tra l'elevata frequenza di clock dei processori moderni e il tempo di accesso relativamente lungo delle DRAM, molti microprocessori supportano un livello addizionale di cache. Questo secondo livello si trova di solito sullo stesso chip del processore e viene utilizzato quando si verifica una miss all'interno della cache primaria. Se il secondo livello di cache contiene il dato desiderato, la penalità di miss per il primo livello di cache sarà essenzialmente il tempo di accesso al secondo livello di cache, molto inferiore al tempo di accesso alla memoria principale. Se né la cache primaria né quella secondaria contengono il dato, viene richiesto un accesso alla memoria principale e, inevitabilmente, si incorre in una penalità di miss elevata.

Quanto può essere significativo l'incremento di prestazioni dovuto all'utilizzo di una cache secondaria? Rispondiamo a questa domanda attraverso il prossimo esempio.

 © 978-88-08-19966-9

Prestazioni di una cache multilivello

ESEMPIO

Supponiamo di disporre di un processore con frequenza di clock pari a 4 GHz e con un CPI di base pari a 1,0, misurato quando tutti gli accessi alla cache primaria producono una hit. La memoria principale ha un tempo di accesso di 100 ns che comprende anche tutto il tempo richiesto per gestire una miss. Supponiamo anche che la frequenza di miss della cache primaria sia pari al 2% delle istruzioni. Di quanto sarà più veloce il processore se introduciamo una cache secondaria con tempo di accesso di 5 ns, sia per le hit sia per le miss, grande a sufficienza da ridurre la frequenza di miss della memoria principale allo 0,5%?

SOLUZIONE

La penalità di miss della memoria principale è pari a:

$$\frac{100 \text{ ns}}{0{,}25 \dfrac{\text{ns}}{\text{ciclo di clock}}} = 400 \text{ cicli di clock}$$

Il CPI effettivo, considerando un livello di cache, è dato da:

$$\text{CPI totale} = \text{CPI base} + \text{Numero di cicli di stallo per istruzione}$$

Per il processore in questione, con un livello di cache si ha:

$$\begin{aligned}\text{CPI totale} &= 1{,}0 + \text{Numero di cicli di stallo per istruzione} \\ &= 1{,}0 + 2\% \times 400 = 9\end{aligned}$$

Con due livelli di cache, una miss della cache primaria (o di primo livello) può essere soddisfatta dalla cache secondaria o dalla memoria principale. La penalità di miss per un accesso al secondo livello di cache è pari a:

$$\frac{5 \text{ ns}}{0{,}25 \dfrac{\text{ns}}{\text{ciclo di clock}}} = 20 \text{ cicli di clock}$$

Se la miss viene risolta dalla cache di secondo livello, questo tempo costituisce la penalità di miss totale; se la miss richiede invece di accedere alla memoria principale, il tempo totale di miss sarà dato dalla somma del tempo di accesso alla cache secondaria e di quello alla memoria principale. Quindi, per una cache a due livelli, il CPI totale è la somma del CPI di base e dei cicli di stallo introdotti da entrambi i livelli di cache:

$$\begin{aligned}\text{CPI totale} &= 1 + \text{stalli primari per istruzione} + \text{stalli secondari per istruzione} \\ &= 1 + 2\% \times 20 + 0{,}5\% \times 400 = 1 + 0{,}4 + 2 = 3{,}4\end{aligned}$$

Perciò, il processore con il secondo livello di cache è più veloce di un fattore:

$$\frac{9{,}0}{3{,}4} = 2{,}6$$

In alternativa, avremmo potuto calcolare il numero di cicli di stallo totali sommando i cicli di stallo dovuti agli accessi alla cache secondaria che hanno prodotto una hit [(2% – 0,5%) × 20 = 0,3] e gli accessi alla memoria principale ai quali bisogna aggiungere anche il costo degli accessi alla cache secondaria [0,5% × (20 + 400) = 2,1]. La somma 1,0 + 0,3 + 2,1 dà ancora 3,4.

Le considerazioni da fare in fase di progettazione sono molto diverse per la cache primaria e per la cache secondaria, perché la presenza di una seconda cache modifica le specifiche. In particolare, una cache a due livelli permette di progettare la cache primaria focalizzandosi sulla minimizzazione del tempo di hit, in modo da ottenere un periodo di clock più breve oppure un numero minore di stadi di pipeline e permette di progettare la cache secondaria focalizzandosi sulla frequenza delle miss per ridurre la penalità dovuta ai lunghi tempi di accesso alla memoria principale.

L'effetto di queste modifiche sulle cache a due livelli si può osservare confrontando ciascuna cache con la cache ottimale a singolo livello. Rispetto a questa, la cache primaria di una **cache multilivello** è spesso di dimensioni ridotte e utilizza di solito blocchi di grandezza inferiore, che consentono una dimensione complessiva inferiore della cache e una penalità di miss ridotta. La cache secondaria, invece, è molto più grande della cache a un solo livello, dato che il tempo di accesso di una cache secondaria è meno critico; avendo una dimensione maggiore, potrà utilizzare, in generale, blocchi più ampi di quelli adatti a una cache a singolo livello. Inoltre, può utilizzare un grado più elevato di associatività della cache primaria, poiché l'obiettivo della cache secondaria è la riduzione della frequenza di miss.

Cache multilivello: una gerarchia delle memorie composta da più livelli di cache, invece che da una sola cache e dalla memoria principale.

Capire le prestazioni dei programmi

Il problema dell'ordinamento (*sorting*) di un insieme di elementi è stato ampiamente studiato e ha portato all'ideazione di algoritmi ad ampia diffusione, tra cui: Bubble Sort, Quicksort, Radix Sort e così via. Il numero di istruzioni eseguite per ogni elemento cercato è mostrato nella **Figura 5.19a** per gli algoritmi Quicksort e Radix Sort. Come si poteva immaginare, per vettori di grandi dimensioni Radix Sort mostra un vantaggio nei confronti di Quicksort in termini di numero di operazioni. La **Figura 5.19b** mostra invece il tempo impiegato per ogni ordinamento invece che il numero di istruzioni eseguite. Vediamo che le curve hanno un andamento simile a quello della Figura 5.19a quando gli elementi da ordinare sono pochi, ma la curva relativa a Radix Sort, dopo una discesa iniziale, tende a risalire man mano che aumenta il numero degli elementi da ordinare. Come si spiega questo comportamento? La **Figura 5.19c** fornisce la risposta, mostrando il numero di miss della cache per ogni elemento che viene ordinato: Quick Sort genera in maniera consistente meno miss per elemento ordinato.

Purtroppo l'analisi standard degli algoritmi ignora spesso l'impatto della gerarchia delle memorie. Come l'incremento della frequenza di clock e la legge di Moore consentono ai progettisti dei calcolatori di ottenere le migliori prestazioni dall'esecuzione di una sequenza di istruzioni, così l'uso efficiente della gerarchia delle memorie è un elemento cruciale per il miglioramento delle prestazioni del sistema. Come abbiamo visto nell'introduzione, comprendere il comportamento della gerarchia delle memorie è quindi fondamentale per valutare le prestazioni dei programmi in un calcolatore moderno.

Ottimizzazione software mediante elaborazione a blocchi

Dato l'impatto delle gerarchie delle memorie sulle prestazioni dei programmi, molte ottimizzazioni software sono state inventate per migliorare notevolmente le prestazioni riutilizzando i dati già contenuti in cache, riducendo la frequenza di miss attraverso l'aumento della località temporale.

Quando si devono utilizzare delle matrici, si possono ottenere buone prestazioni dal sistema di memoria se si memorizzano le matrici in modo tale che l'accesso in memoria ai loro elementi sia sequenziale. Supponiamo di dovere gestire più matrici e che ad alcune di esse si acceda per righe e ad altre per colonne. La memorizzazione delle matrici per riga (detto *ordinamento per riga*)

 © 978-88-08-19966-9

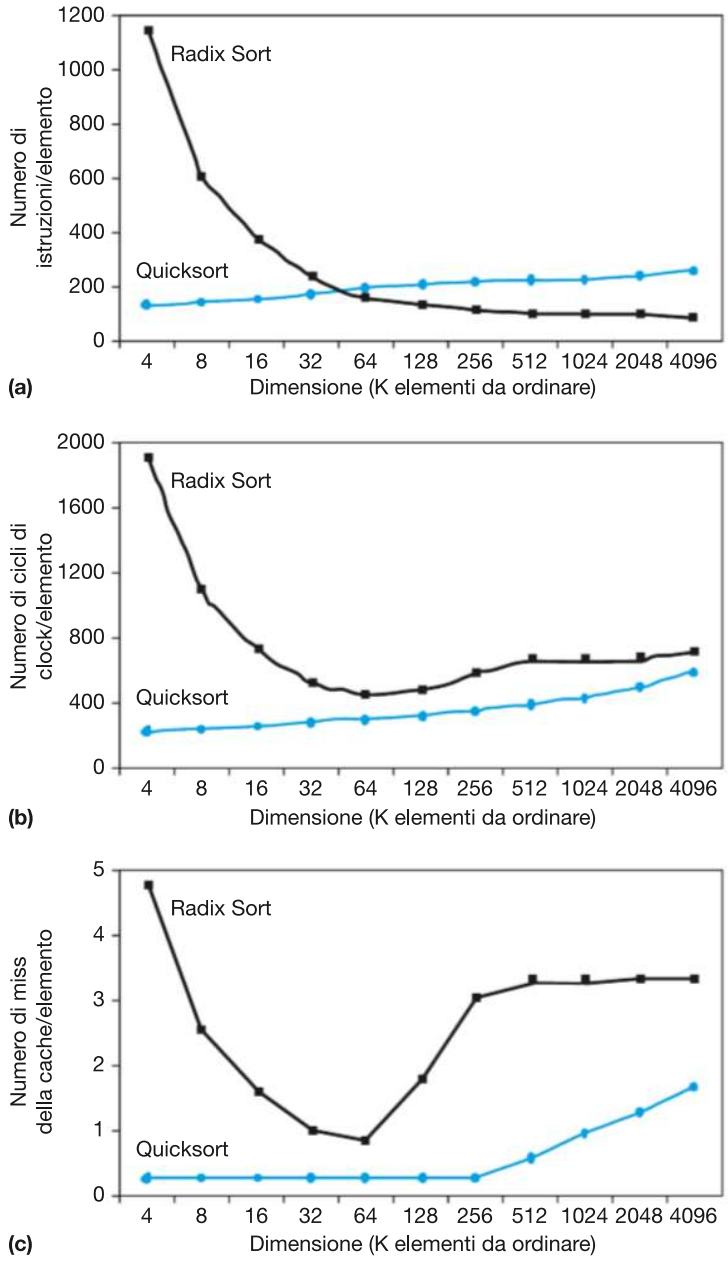


Figura 5.19 Confronto tra Quicksort e Radix Sort in termini di numero di istruzioni eseguite per elemento ordinato (a), tempo per elemento ordinato (b) e miss della cache per elemento ordinato (c). Questi dati provengono da un articolo di LaMarca e Ladner [1996]. Anche se i numeri potranno essere diversi per i nuovi calcolatori, il concetto non cambia. In seguito a questi risultati sono state realizzate nuove versioni dell'algoritmo Radix Sort, che tengono in considerazione anche la gerarchia delle memorie al fine di riguadagnare il suo vantaggio algoritmico (Paragrafo 5.15). L'idea base delle ottimizzazioni sull'uso della cache è quella di utilizzare tutti i dati all'interno del blocco ripetitivamente prima della sua sostituzione in seguito a una miss.

o per colonna (detto *ordinamento per colonna*) non risolve il problema, perché sia le righe sia le colonne vengono utilizzate in ogni iterazione del ciclo.

Invece di operare su intere righe o intere colonne della matrice, gli algoritmi *a blocchi* operano su sottomatrici o *blocchi*. L'obiettivo è massimizzare l'accesso ai dati caricati nella cache prima che debbano essere sostituiti, cioè migliorare la località temporale per ridurre le miss della cache.

Per esempio, i cicli interni della funzione DGEMM (dalla linea 4 alla linea 9 della Figura 3.22) sono:

```
for (int j = 0;  j < n; j++)
    {
     double cij = C[i+j*n]; /* cij = C[i][j] */
     for (int k = 0; k < n; k++)
      cij += A[i+k*n]*B[k+j*n]; /* cij += A[i][k]*B[k][j] */
     C[i+j*n] = cij; /* C[i][j] = cij */
    }
}
```

Questa parte di codice legge tutti gli `N` per `N` elementi di `B`, legge tutti gli elementi della *i*-esima riga di `A` e scrive il risultato nella *i*-esima riga di `N` elementi di `C` (i commenti rendono facile capire a quali righe e colonne ci stiamo riferendo). La **Figura 5.20** fotografa l'accesso alle tre matrici: le celle grigio scuro indicano un accesso recente, le celle grigio chiaro un accesso precedente e le celle bianche indicano che non ci sono stati ancora accessi.

Il numero delle miss per capacità dipende chiaramente da `N` e dalla dimensione della cache. Se la cache potesse contenere tutte e tre le matrici di dimensioni `N` per `N`: `A`, `B` e `C`, allora non ci sarebbero conflitti sulla cache. Abbiamo opportunamente scelto la dimensione delle matrici utilizzate in DGEMM negli esempi dei Capitoli 3 e 4 per rientrare in questa situazione.

Se la cache potesse contenere una matrice `N` per `N` e una riga di `N` elementi, allora almeno l'*i*-esima riga di `A` e la matrice `B` potrebbero essere contenute in cache. Se la cache non può contenere neppure queste, si possono verificare miss anche per le matrici `B` e `C`. Nel caso peggiore, si accede a $2\,N^3 + N^2$ parole di memoria per N^3 operazioni.

Per essere sicuri che gli elementi a cui si accede siano contenuti nella cache, modifichiamo il codice originale per effettuare i calcoli su sottomatrici. Chiameremo quindi ripetutamente la versione della funzione DGEMM riportata nella Figura 4.78 su matrici di dimensioni `DIM_BLOCCO` per `DIM_BLOCCO`, dove `DIM_BLOCCO` è chiamato *fattore di blocco* (*blocking factor*).

La **Figura 5.21** mostra la versione a blocchi della funzione DGEMM. La funzione `esegui_blocco` è uguale alla funzione della Figura 3.22 con tre parametri aggiuntivi: `si`, `sj` e `sk`, che specificano la posizione iniziale delle sottomatrici in `A`, `B` e `C`. I due cicli interni di `esegui_blocco` eseguono ora i calcoli su `DIM_BLOCCO` elementi invece che su tutta la lunghezza delle righe e colonne di `B` e `C`. L'ottimizzatore del compilatore gcc rimuove ogni aggiunta richiesta dalle

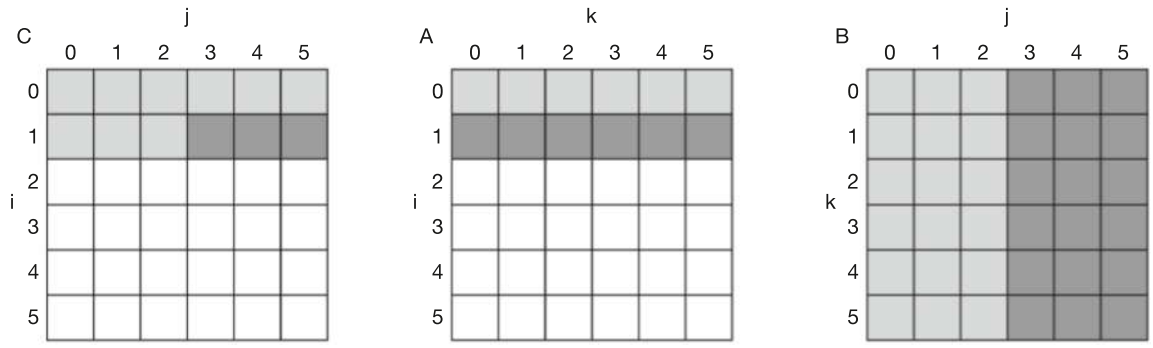


Figura 5.20 Fotografia delle tre matrici `C`, `A` e `B` quando `N` = 6 e `i` = 1. Il livello di grigio degli elementi delle matrici rappresenta quanto di recente si è verificato l'ultimo accesso a quell'elemento: grigio chiaro indica accessi più remoti, grigio scuro indica accessi più recenti. Gli elementi bianchi non hanno ancora ricevuto accessi. Rispetto alla Figura 5.22, gli elementi di `A` e `B` vengono letti ripetutamente per calcolare il nuovo elemento di `C`. Le variabili `i`, `j` e `k` vengono mostrate sulle righe e colonne a seconda di come vengono utilizzate per accedere agli elementi delle matrici.

```
1  #define DIM_BLOCCO 32
2  void esegui_blocco (int n, int si, int sj, int sk, double *A, double
3  *B, double *C)
4  {
5     for (int i = si; i < si+DIM_BLOCCO; ++i)
6        for (int j = sj; j < sj+DIM_BLOCCO; ++j)
7           {
8              double cij = C[i+j*n];/* cij = C[i][j] */
9              for( int k = sk; k < sk+DIM_BLOCCO; k++ )
10                cij += A[i+k*n] * B[k+j*n];/* cij+=A[i][k]*B[k][j] */
11             C[i+j*n] = cij;/* C[i][j] = cij */
12          }
13 }
14 void dgemm (int n, double* A, double* B, double* C)
15 {
16    for ( int sj = 0; sj < n; sj += DIM_BLOCCO )
17       for ( int si = 0; si < n; si += DIM_BLOCCO )
18          for ( int sk = 0; sk < n; sk += DIM_BLOCCO )
19             esegui_blocco(n, si, sj, sk, A, B, C);
20 }
```

Figura 5.21 Versione a blocchi della funzione DGEMM della Figura 3.21. Si supponga che la matrice `C` sia stata inizializzata a zero. La funzione `esegui_blocco` è praticamente uguale alla funzione del Capitolo 3, con tre parametri aggiuntivi che specificano la posizione iniziale delle sottomatrici di dimensione `DIM_BLOCCO`. L'ottimizzatore del compilatore gcc rimuove ogni aggiunta richiesta dalle chiamate a funzione, inserendo la funzione direttamente nel codice.

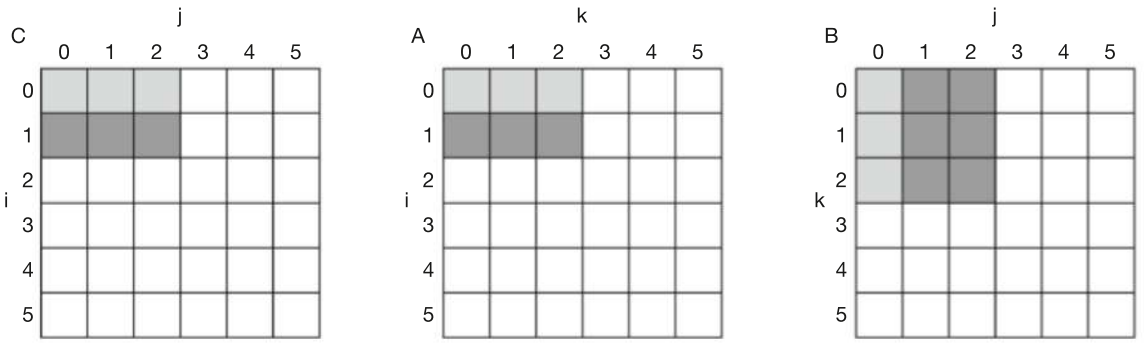


Figura 5.22 Tempo dall'ultimo accesso per le matrici `C`, `A` e `B` quando `DIM_BLOCCO` = 3. Si noti che, a differenza della Figura 5.20, pochi elementi hanno ricevuto accessi.

chiamate a funzione inserendo la funzione direttamente nel codice ("inlining" della funzione) per evitare le istruzioni che gestiscono il passaggio dei parametri e il ritorno alla procedura chiamante.

La Figura 5.22 illustra gli accessi alle tre matrici utilizzando l'elaborazione a blocchi. Analizzando solamente le miss causate dalla dimensione, il numero totale di parole a cui si accede in memoria è sceso a $2\ N^3/DIM_BLOCCO + N^2$, che costituisce un miglioramento di un fattore `DIM_BLOCCO` approssimativamente. Quindi, l'elaborazione a blocchi sfrutta una combinazione di località spaziale e temporale, dato che `A` beneficia della località spaziale e `B` di quella temporale. A seconda del calcolatore e delle dimensioni delle matrici, l'elaborazione a blocchi può migliorare le prestazioni da un fattore di circa 2 a più di un fattore 10.

Sebbene il nostro obiettivo fosse ridurre le miss della cache, l'elaborazione a blocchi aiuta anche nell'allocazione dei registri. Scegliendo una dimensione

del blocco sufficientemente piccola per cui i blocchi possono essere contenuti nei registri, possiamo minimizzare il numero di operazioni del programma di trasferimento da e verso la memoria, cosa che migliora ulteriormente le prestazioni.

Approfondimento. Le cache multilivello creano diverse complicazioni. Anzitutto vengono introdotti altri tipi di miss, che hanno una loro frequenza di miss. Nell'esempio a pagina 374, abbiamo introdotto la frequenza di miss della cache primaria e la **frequenza di miss globale**, cioè la percentuale di accessi che falliscono in tutti i livelli di cache. C'è anche una frequenza di miss della cache secondaria, che è il rapporto tra tutti i fallimenti nella cache secondaria e il numero totale di accessi a essa. Questa frequenza di miss è chiamata **frequenza di miss locale** della cache secondaria. Poiché la cache primaria filtra gli accessi, specialmente quelli con una buona località spaziale e temporale, la frequenza di miss locale della cache secondaria è molto più alta della frequenza di miss globale. Per l'esempio a pagina 374, la frequenza di miss locale della cache secondaria risulta pari a 0,5%/2% = 25%! Fortunatamente è la frequenza di miss globale a indicare quanto spesso dobbiamo accedere alla memoria principale.

Frequenza di miss globale: la percentuale degli accessi alla memoria che falliscono in tutti i livelli di una cache multilivello.

Frequenza di miss locale: la percentuale degli accessi a un livello di cache che falliscono; viene utilizzata in una gerarchia multilivello.

Approfondimento. Nei processori dotati di esecuzione fuori ordine (Capitolo 4) il calcolo delle prestazioni diviene più complesso, dal momento che essi possono eseguire delle istruzioni anche durante la penalità di miss. Invece della frequenza di miss per istruzioni e dati, si utilizza il numero di miss per istruzione calcolato attraverso la seguente formula:

$$\frac{\text{Cicli di stallo dovuti alla memoria}}{\text{Istruzione}} =$$

$$= \frac{\text{Miss}}{\text{Istruzione}} \times (\text{Latenza totale delle miss} - \text{Latenza delle miss sovrapposte})$$

Non esiste un modo generale per calcolare la latenza delle miss sovrapposte, quindi la valutazione della gerarchia delle memorie per i processori fuori ordine richiede inevitabilmente la simulazione del processore e della gerarchia delle memorie. Solamente osservando l'esecuzione di un processore durante ogni miss possiamo capire se il processore rimane in stallo, in attesa del dato, oppure semplicemente trova altro lavoro da compiere. In generale, la regola è che un processore riesce spesso a nascondere la penalità di miss della cache L1 se il dato viene trovato nella cache L2, ma raramente nasconde una miss della cache L2.

Approfondimento. La sfida, per quel che concerne le prestazioni degli algoritmi, è rappresentata dal fatto che le gerarchie delle memorie variano nelle differenti implementazioni hardware della stessa architettura per dimensioni delle cache, grado di associatività, dimensione del blocco e numero di livelli di cache. Per gestire questa variabilità, alcune recenti librerie numeriche rendono parametrici i loro algoritmi ed esplorano lo spazio dei parametri durante l'esecuzione per trovare la miglior combinazione adatta al particolare tipo di calcolatore utilizzato. Questo approccio viene chiamato *autocalibrazione* (*autotuning*).

Autovalutazione

Quale delle seguenti affermazioni è generalmente vera per quel che riguarda la progettazione di cache multilivello?

1. Le cache di primo livello sono focalizzate principalmente sul tempo di accesso, mentre le cache di secondo livello sono focalizzate sulla frequenza di miss.
2. Le cache di primo livello sono focalizzate principalmente sulla frequenza di miss, mentre le cache di secondo livello sono focalizzate sul tempo di accesso.

Riepilogo

In questa prima parte del capitolo abbiamo discusso tre argomenti principali: le prestazioni delle cache, utilizzando il grado di associatività per ridurre la frequenza delle miss, l'utilizzo di gerarchie delle memorie costituite da cache multilivello per ridurre la penalità di miss e le ottimizzazioni software per migliorare l'efficacia delle cache. Il sistema di memoria ha un impatto significativo sul tempo di esecuzione dei programmi. Il numero totale di cicli di stallo dovuti alla memoria dipende sia dalla frequenza sia dalla penalità di miss. La sfida, come vedremo nel Paragrafo 5.8, è ridurre uno di questi due fattori critici della gerarchia delle memorie senza alterare significativamente l'altro.

Per ridurre la frequenza di miss abbiamo esaminato gli schemi associativi di corrispondenza tra memoria principale e cache. Tali schemi possono ridurre la frequenza delle miss di una cache permettendo una più flessibile collocazione dei blocchi all'interno della cache. Gli schemi completamente associativi permettono di scrivere i blocchi della memoria principale in un qualsiasi blocco della cache, ma richiedono anche di controllare tutti i blocchi della cache a ogni accesso; il costo elevato che ne risulta rende praticamente irrealizzabili cache completamente associative di grandi dimensioni. Le cache set-associative sono un'alternativa praticabile, dal momento che qui la ricerca deve essere effettuata solamente tra i blocchi di una stessa linea, selezionata attraverso il campo indice. Le cache set-associative hanno una frequenza di miss più elevata, ma con tempi di accesso inferiori. Il grado di associatività che produce le migliori prestazioni dipende sia dalla tecnologia sia dal tipo di implementazione.

Infine, abbiamo visto come le cache multilivello costituiscano una tecnica che può essere utilizzata per ridurre la penalità di miss, permettendo a una cache secondaria di dimensioni maggiori di gestire le miss della cache primaria. Le cache di secondo livello sono divenute comuni da quando i progettisti si sono resi conto che il limitato spazio disponibile sul chip e la necessità di mantenere elevata la frequenza del clock impediscono alle cache primarie di essere troppo grandi. La cache secondaria, che ha dimensioni 10 o più volte superiori di una cache primaria, gestisce molti degli accessi falliti alla cache primaria. In tal caso, la penalità di miss è pari al tempo di accesso alla cache secondaria (tipicamente minore di 10 cicli di clock del processore), che è molto inferiore al tempo di accesso alla memoria principale (tipicamente maggiore di 100 cicli di clock del processore). Come per il grado di associatività, il compromesso tra dimensione e tempo di accesso nella progettazione di una cache secondaria dipende da un gran numero di fattori legati all'implementazione della gerarchia delle memorie.

Infine, data l'importanza delle gerarchie delle memorie per le prestazioni, abbiamo esaminato come modificare un algoritmo per migliorare il comportamento delle cache: l'elaborazione a blocchi è una tecnica molto importante quando si devono elaborare matrici di grandi dimensioni.

5.5 Affidabilità delle gerarchie delle memorie

L'ipotesi alla base delle discussioni dei paragrafi precedenti è che le memorie non dimentichino. Avere una memoria veloce ma non affidabile non è per nulla attraente. Come abbiamo visto nel Capitolo 1, una grande idea per ottenere sistemi **affidabili** è la ridondanza. In questo paragrafo esamineremo i termini associati all'affidabilità per definire delle misure associate ai malfunzionamenti; vedremo poi come la ridondanza possa rendere praticamente infinita la durata delle memorie.

Definizione di malfunzionamento

Partiamo dall'ipotesi che sappiate quali siano le specifiche di un servizio adeguato. Gli utenti potranno valutare il sistema in base a due possibili condizioni di erogazione del servizio rispetto alle specifiche del servizio stesso:

1. *Erogazione corretta del servizio*, quando il servizio viene erogato secondo le specifiche;
2. *Interruzione del servizio*, quando il servizio fornito differisce dalle specifiche definite per quel servizio.

Le transizioni dalla condizione 1 alla condizione 2 sono causate da *malfunzionamenti*, mentre le transizioni dalla condizione 2 alla condizione 1 sono chiamate *ripristino*. I malfunzionamenti possono essere permanenti o intermittenti e i secondi costituiscono il caso più arduo: è più difficile diagnosticare un problema quando un sistema oscilla tra i due stati. I malfunzionamenti permanenti sono invece più facili da diagnosticare. Queste definizioni conducono alla definizione di due termini collegati: affidabilità e disponibilità.

L'*affidabilità* è una misura della continuità con cui viene fornito il servizio a partire da un certo istante; ossia, in maniera equivalente, è il tempo che intercorre prima che si verifichi il primo malfunzionamento. Quindi, il *tempo medio di funzionamento prima di un malfunzionamento* (MTTF, *Mean Time To Failure*) è una misura di affidabilità. Una misura collegata è la *frequenza annua media dei malfunzionamenti* (AFR, *Annual Failure Rate*) che rappresenta la percentuale di dispositivi che, in un anno, presentano un malfunzionamento, dato un certo MTTF. Quando l'MTTF diventa grande può diventare fuorviante e perciò l'AFR rappresenta una misura migliore.

MTTF e AFR dei dischi

ESEMPIO

Alcuni dischi recenti dichiarano un MTTF di 1 000 000 di ore. Dato che 1 000 000 di ore corrisponde a 1 000 000/(365 × 24) = 114 anni, sembrerebbe che questi dischi non si guastino mai. I grandi centri di calcolo (Paragrafo 6.7) che forniscono i servizi Internet come la ricerca su web possono essere dotati anche di 50 000 server. Si supponga che ciascun server abbia 2 dischi. Utilizzare l'AFR per calcolare quanti dischi ci si può aspettare che presentino malfunzionamenti in un anno.

SOLUZIONE

Un anno è costituito da 365 × 24 = 8760 ore. Un MTTF di 1 000 000 di ore corrisponde a un AFR di 8760/1 000 000 = 0,876%. Avendo il centro di calcolo 100 000 dischi, possiamo attenderci che in un anno presentino malfunzionamenti 876 dischi, cioè in media più di due dischi al giorno!

L'interruzione di un servizio viene misurata attraverso il *tempo medio di riparazione* (MTTR, *Mean Time To Repair*), mentre il *tempo medio tra due guasti* (MTBF, *Mean Time Between Failures*) è semplicemente la somma di MTTF e MTTR. Sebbene l'MTBF sia la misura più diffusa, l'MTTF spesso è il termine più adeguato. La *disponibilità* è quindi una misura relativa della corretta erogazione di un servizio ed è data rispetto all'alternanza delle condizioni di erogazione corretta e interruzione del servizio. La disponibilità viene misurata, in modo statistico, come:

$$\text{Disponibilità} = \frac{\text{MTTF}}{(\text{MTTF} + \text{MTTR})}$$

Si noti che affidabilità e disponibilità sono quantità misurabili nella pratica e non solo sinonimi di sicurezza. Diminuire l'MTTR può fare aumentare l'affidabilità, così come aumentare l'MTTF. Per esempio, strumenti per l'identificazione dei guasti, per la diagnosi e la riparazione possono aiutare a ridurre i tempi di riparazione e quindi migliorare la disponibilità.

Solitamente si vuole che l'affidabilità sia molto alta. Un modo per valutarla è attraverso il numero di "nove" nella percentuale di affidabilità per anno. Per esempio, un servizio Internet molto buono oggi offre da 4 a 5 "nove di affidabilità". Vediamo come questa indicazione si traduce in numeri. Considerando che ci sono 365 giorni in un anno e quindi 365 × 24 × 60 = 526 000 minuti, otteniamo:

Un nove:	90%	→ 36,5 giorni di riparazione/anno
Due nove:	99%	→ 3,65 giorni di riparazione/anno
Tre nove:	99,9%	→ 526 minuti di riparazione/anno
Quattro nove:	99,99%	→ 52,6 minuti di riparazione/anno
Cinque nove:	99,999%	→ 5,26 minuti di riparazione/anno

e così via (cinque nove significa 5 minuti di riparazioni all'anno, che serve da promemoria).

Per incrementare l'MTTF si può migliorare la qualità dei componenti o del progetto del sistema, in modo da poter continuare a svolgere le operazioni anche quando un componente presenta dei malfunzionamenti; i malfunzionamenti devono perciò essere definiti rispetto al loro contesto: il guasto di un componente potrebbe non implicare il malfunzionamento dell'intero sistema. Perché sia chiara questa distinzione, useremo in seguito il termine *guasto* (*fault*) per indicare un malfunzionamento permanente o transitorio di un componente. Ci sono tre modi per migliorare l'MTTF.

1. *Evitare il guasto*: costruire il dispositivo in modo da evitare il verificarsi del guasto stesso.
2. *Tolleranza ai guasti*: utilizzare la ridondanza per fare sì che il servizio soddisfi le specifiche anche quando si verifica un guasto.
3. *Previsione dei guasti*: **prevedere** la presenza di guasti o la possibilità che questi si generino, *prima* che si verifichino, in modo da sostituire il componente *prima* che si guasti.

Codice di Hamming per la correzione di errori singoli e identificazione di errori doppi (SEC/DED)

Richard Hamming ha inventato uno schema molto utilizzato per ridondare la memoria, per il quale ha ricevuto il Premio Turing nel 1968. Per capire i codici ridondanti, occorre definire una misura di "vicinanza" tra due insiemi di bit: definiamo *distanza di Hamming* il minimo numero di bit che devono essere commutati perché i due insiemi risultino uguali. Per esempio, la distanza tra 011011 e 001111 è due.

Che cosa succede se la distanza tra due insiemi di bit è due e si ha un errore su un bit? In questo caso l'insieme sul quale si è verificato l'errore non sarà più un numero valido. Perciò, se possiamo riconoscere se un numero è valido o no, possiamo rilevare gli errori su un singolo bit e possiamo dire di avere un **codice di rilevamento degli errori** su un singolo bit.

Codice di rilevamento degli errori: un codice che consente la rilevazione di un errore nei dati. Non consente però di identificare la posizione dell'errore all'interno di una parola e quindi di correggerlo.

Hamming ha utilizzato il *codice di parità* per la rilevazione degli errori. In un codice di parità, viene contato il numero di 1 in una parola; si dice che la parola ha parità dispari se il numero di 1 è dispari e che ha parità pari se il numero di 1 è pari. Quando una parola viene scritta in memoria, viene aggiunto il bit di parità (1 per un numero dispari di 1, 0 per un numero pari di 1), di modo che la parità della parola di N + 1 bit sarà sempre pari. Quando la parola viene letta, viene letto e confrontato il bit di parità con il numero di 1 presente nei primi N bit. Se il numero di 1 è pari e il bit di parità indica un numero dispari di 1, o viceversa, significa che si è verificato un errore.

ESEMPIO

Calcolare la parità associata a un byte che contiene il numero 31_{dec} e mostrare la stringa di bit che viene memorizzata. Si supponga che il bit di parità sia sulla destra. Si supponga che il bit più significativo commuti mentre il byte risiede in memoria e che leggiate successivamente il byte. Siete in grado di rilevare l'errore? Che cosa succede se i primi due bit più significativi commutano?

SOLUZIONE

31_{dec} corrisponde a 00011111_{due}, che contiene cinque 1. Per scrivere un numero pari di 1, dobbiamo scrivere un 1 nel bit di parità e quindi: 000111111_{due}. Se il bit più significativo cambia segno, quando leggiamo il byte otterremo: 100111111_{due}, che contiene sette 1. Dato che ci aspettavamo una parità pari mentre il numero ha una parità dispari, siamo in grado di rilevare l'errore. Se invece commutassero i primi *due* bit più significativi, leggeremmo 110111111_{due}, che contiene otto 1, cioè un numero pari di 1, e *non* saremmo in grado di rilevare l'errore.

Se si verifica un errore su 2 bit, quindi, uno schema di parità a 1 bit non sarebbe in grado di rilevare nessun errore, dato che il bit di parità codifica la stessa parità quanto il numero contenente due errori. (In realtà il codice di parità a 1 bit consente di rilevare un numero dispari di errori, ma la probabilità di avere tre errori è molto inferiore della probabilità di averne due, per cui, in pratica, il codice di parità su 1 bit è limitato al riconoscimento di errori sul singolo bit.)

Naturalmente il codice di parità non può correggere gli errori, mentre Hamming voleva arrivare a questo. Se utilizzassimo un codice di errore che abbia una distanza 3 tra due numeri validi, allora ogni errore sul singolo bit produrrebbe un numero più vicino al numero corretto che a ogni altro numero valido. Sulla base di queste osservazioni, Hamming propose una trasformazione dei dati in un codice a distanza 3, chiamato *codice di correzione degli errori* (ECC, *Error Correction Code*) *di Hamming* in suo onore. Utilizzeremo i bit di parità aggiunti per consentire di identificare la posizione in cui si è verificato l'errore singolo. Questi sono i passi per calcolare la codifica ECC di Hamming.

1. Iniziare a contare i bit a partire da sinistra, invece di contarli da destra come si fa solitamente.
2. Riservare tutte le posizioni che sono potenze di 2 ai bit di parità (posizioni 1, 2, 4, 8, 16, …).
3. Tutte le altre posizioni possono essere utilizzate per memorizzare i bit del dato (posizioni 3, 5, 6, 7, 9, 10, 11, 12, 13, 14, 15, 17, …).
4. I bit inseriti nelle posizioni dei bit di parità determinano la sequenza dei bit dei dati da controllare (la **Figura 5.23** mostra questo meccanismo graficamente per un numero su otto bit):

Posizione bit		1	2	3	4	5	6	7	8	9	10	11	12
Bit codificati		p_1	p_2	d_1	p_4	d_2	d_3	d_4	p_8	d_5	d_6	d_7	d_8
Copertura del bit di parità	p_1	X		X		X		X		X		X	
	p_2		X	X			X	X			X	X	
	p_4				X	X	X	X					X
	p_8								X	X	X	X	X

Figura 5.23 **I bit di dati (d_i) e i bit di parità (p_i) con la loro copertura per il codice ECC di Hamming su dati di otto bit.**

- bit 1 (0001_{due}) controlla i bit (1, 3, 5, 7, 9, 11, …), che sono i bit il cui indirizzo contiene un 1 nel bit più a destra (0001_{due}, 0011_{due}, 0101_{due}, 0111_{due}, 1001_{due}, 1011_{due}, …);
- bit 2 (0010_{due}) controlla la parità dei bit (2, 3, 6, 7, 10, 11, 14, 15, …), che sono i bit per i quali il secondo bit a destra dell'indirizzo è un 1;
- bit 4 (0100_{due}) controlla la parità dei bit (4–7, 12–15, 20–23, …), che sono i bit per i quali il terzo bit a destra dell'indirizzo è un 1;
- bit 8 (1000_{due}) controlla la parità dei bit (8–15, 24–31, 40–47, …), che sono i bit per i quali il quarto bit a destra dell'indirizzo è un 1.

Si noti che ciascun bit del dato è coperto da uno o più bit di parità.

5. Impostare il bit di parità per ottenere una parità pari in ciascun gruppo.

Sembra una magia, ma ciò vi consente di determinare se i bit sono commutati controllando i bit di parità. Utilizzando il codice a 12 bit della Figura 5.23, se il valore dei quattro bit di parità (p_8, p_4, p_2, p_1) è 0000, vuol dire che non si sono verificati errori. Se, invece, i quattro bit assumono, per esempio, il valore 1010, cioè 10_{dec}, allora il codice ECC di Hamming ci dice che l'errore è sul bit 10 (d_6). Possiamo correggere l'errore semplicemente commutando il valore del bit 10.

ESEMPIO

Si supponga che un dato di ampiezza un byte sia 10011010_{due}. Mostrare il codice ECC di Hamming per il dato. Invertire poi il bit 10_{dec} e mostrare che il codice ECC è in grado di correggere l'errore.

SOLUZIONE

Lasciando lo spazio per i bit di parità, la stringa su 12 bit è:

_ _ 1 _ 0 0 1 _ 1 0 1 0

La posizione 1 verifica la parità dei bit 1, 3, 5, 7, 9 e 11, che evidenziamo: **__1_001_1010**. Per rendere pari il numero di 1 in questo gruppo, occorre che il bit di parità assuma valore 0.

La posizione 2 verifica la parità dei bit 2, 3, 6, 7, 10 e 11, che evidenziamo: 0**_1_001_1010**. Per rendere pari il numero di 1 in questo gruppo, occorre che il bit di parità assuma valore 1.

La posizione 4 verifica la parità dei bit 4, 5, 6, 7 e 12: 011**_001_**101. Per rendere pari il numero di 1 in questo gruppo, occorre che il bit di parità assuma valore 1.

(continua)

La posizione 8 verifica la parità dei bit 8, 9, 10, 11 e 12: 0111001_**1010**. Per rendere pari il numero di 1 in questo gruppo, occorre che il bit di parità assuma valore 0.

La stringa di bit finale è 011100101010. Invertiamo ora il bit 10; otteniamo: 011100101**1**10.

Il bit di parità 1 è 0 (**0111001011**10 contiene quattro 1 nel gruppo associato, parità pari, e quindi i bit di questo gruppo non contengono errori).

Il bit di parità 2 è 1 (0**111**0**01**011**1**0 contiene cinque 1 nel gruppo associato, parità dispari, e quindi si è verificato un errore in uno di questi bit).

Il bit di parità 4 è 1 (011**1001**0111**0** contiene due 1 nel gruppo associato, parità pari, e quindi i bit di questo gruppo non contengono errori).

Il bit di parità 8 è 1 (0111001**01110** contiene tre 1 nel gruppo associato, parità dispari, e quindi si è verificato un errore in uno di questi bit).

I bit di parità 2 e 8 segnalano un errore, che si trova in posizione 10, cioè 8 + 2. Possiamo quindi invertire il contenuto del bit 10 e correggere l'errore: 011100101**0**10.

(continua)

Hamming non si fermò al codice di errore per la correzione dell'errore su un bit. Con l'aggiunta di un altro bit, possiamo aumentare a un minimo di quattro la distanza tra due numeri validi e possiamo quindi *rilevare l'errore su due bit* e correggere l'errore su un bit. L'idea è quella di aggiungere un bit di parità che viene calcolato sull'intera parola. Utilizziamo un dato su quattro bit come esempio. Il codice richiede in questo caso 7 bit per il rilevamento di un errore singolo. I bit di parità del codice di Hamming, H (p_1, p_2 e p_3), vengono calcolati (supponendo una parità pari) e viene anche calcolato il bit di parità globale, p_4:

1	2	3	4	5	6	7	8
p_1	p_2	d_1	p_3	d_2	d_3	d_4	$\mathbf{p_4}$

L'algoritmo per correggere un errore e per identificarne due richiede solamente di calcolare la parità sui gruppi ECC (H) come visto in precedenza e sull'intero gruppo p4. Si distinguono quattro casi:

1. H è pari e p_4 è pari: non si è verificato nessun errore;
2. H è dispari e p_4 è dispari: si è verificato un errore singolo che può essere corretto (p_4 calcola la parità dispari quando si è verificato 1 errore);
3. H è pari e p_4 è dispari: si è verificato un errore in p_4, ma non negli altri bit, quindi occorre commutare p_4;
4. H è dispari e p_4 è pari: si sono verificati due errori (p_4 calcola la parità pari quando si sono verificati 2 errori).

Oggi, la correzione di errori singoli e il rilevamento di errori doppi (SEC/DED, *Single Error Correction/Double Error Detection*) è comune nelle memorie dei server: il SEC/DED per blocchi di dati ampi otto byte richiede un solo byte in più e questo è il motivo per cui molte DIMM hanno un'ampiezza di 72 bit.

Approfondimento. Per calcolare quanti bit occorrono per il SEC, supponiamo che p sia il numero totale dei bit di parità, d il numero di bit dei dati e $p + d$ l'ampiezza delle parole di memoria. Se p bit servono per identificare e correggere gli errori su

 © 978-88-08-19966-9

un bit ($p + d$ combinazioni) e un bit serve per indicare che non si sono verificati errori, occorrono:

$$2^p \geq p + d + 1 \text{ bit e quindi } p \geq \log(p + d + 1)$$

Per esempio, per dati su 8 bit, $d = 8$ e $2^p \geq p + 8 + 1$ e quindi $p = 4$. Analogamente, $p = 5$ per dati su 16 bit, 6 per dati su 32 bit, 7 per dati su 64 bit ecc.

Approfondimento. Nei sistemi di grandissime dimensioni, la probabilità di errori multipli e del malfunzionamento di tutto un chip di memoria diventa significativa. L'IBM ha introdotto *chipkill* per ovviare a questo problema, e molti altri sistemi di grosse dimensioni hanno adottato questo sistema (Intel ha chiamato la sua versione SDDC). È un approccio simile ai dischi RAID esaminati nel Paragrafo 5.11 . Chipkill distribuisce i dati e le informazioni sull'ECC, in modo tale che il malfunzionamento di un intero chip di memoria possa essere gestito ricostruendo i dati mancanti a partire dai dati contenuti negli altri chip di memoria. Supponendo di avere un cluster di 10 000 processori, con 4 GiB di memoria per processore, IBM ha calcolato la seguente frequenza di errori di memoria *irrecuperabili* nell'arco di tre anni:

- solo parità: circa 90 000, ovvero un errore irrecuperabile (o non rilevato) ogni 17 minuti;
- SEC/DED solamente: circa 3500, ovvero circa un errore irrecuperabile (o non rilevato) ogni 7,5 ore;
- chipkill: 6, ovvero un errore irrecuperabile (o non rilevato) ogni 2 mesi.

Quindi la funzionalità chipkill è diventata un requisito fondamentale per i calcolatori per i grandi centri di calcolo (*vedi* Paragrafo 6.7).

Approfondimento. Se gli errori su singolo bit o su due bit sono tipici per i sistemi di memoria, le reti possono avere errori a raffica. Una soluzione è il controllo periodico: *Cyclic Redundancy Check* (controllo periodico della ridondanza). Per un blocco di k bit, un trasmettitore genera una sequenza di n-k bit di controllo: trasmette un numero su n bit che risulta esattamente divisibile per un certo numero. Il ricevitore divide il numero ricevuto per questo numero e controlla il resto: se è uguale a zero non si sono verificati errori. Se il resto è diverso da zero, il ricevitore rigetta il messaggio e chiede la ritrasmissione. Come abbiamo visto nel Capitolo 3, è facile calcolare la divisione per una potenza di due mediante un registro a scorrimento, cosa che ha permesso la diffusione del codice CRC anche quando l'hardware era più prezioso. Facendo un ulteriore passo avanti, i codici di Reed-Solomon utilizzano i campi di Galois per correggere errori di trasmissione su più bit; in questo caso i dati sono considerati i coefficienti di un polinomio e lo spazio dei codici diventa lo spazio dei valori del polinomio. Il calcolo del codice di Reed-Solomon è notevolmente più complicato della divisione binaria!

5.6 Macchine virtuali

Le *macchine virtuali* (VM, *Virtual Machines*) sono state sviluppate a partire dagli anni '60 e negli anni successivi hanno sempre costituito una parte importante dei processi di calcolo nei calcolatori mainframe. Sebbene siano state ignorate negli anni '80 e '90 nei PC a uso singolo, recentemente stanno guadagnando popolarità per i seguenti motivi:

- l'aumento dell'importanza di isolamento e sicurezza nei sistemi moderni;
- i problemi di sicurezza e affidabilità dei sistemi operativi standard;
- la condivisione di uno stesso calcolatore tra diversi utenti non collegati tra loro, in particolare per il cloud computing;
- l'aumento esponenziale della velocità dei processori, che ha reso il sovraccarico di lavoro delle VM più accettabile.

Nella definizione più ampia di VM sono compresi praticamente tutti i metodi di emulazione che forniscono un'interfaccia software standard, come la VM

di Java. In questo paragrafo esamineremo le VM che forniscono un ambiente di sistema completo, in grado di gestire, a livello binario, una certa implementazione architetturale di un insieme di istruzioni (ISA). Sebbene alcune VM possano emulare diverse ISA sul loro hardware nativo, supporremo che una VM corrisponda sempre a un particolare processore. Queste VM sono chiamate *macchine virtuali di sistema* (operativo); alcuni esempi sono il VM/370 di IBM, VMware, il Server ESX e Xen.

Le macchine virtuali di sistema forniscono agli utenti l'illusione di avere a loro disposizione l'intero calcolatore, compresa una copia del sistema operativo. Un singolo calcolatore può eseguire più VM e può supportare diversi *sistemi operativi* (SO). Su una piattaforma convenzionale, un singolo SO "possiede" tutte le risorse hardware, ma con una VM più SO possono condividere le stesse risorse.

Il software che supporta una VM è chiamato *monitor della macchina virtuale* (VMM, *Virtual Machine Monitor*), o *hypervisor*; il VMM è il cuore della tecnologia delle macchine virtuali. La piattaforma su cui gira un VMM è chiamata *host* (letteralmente, "ospitante"), e le sue risorse vengono condivise dalle diverse VM *guest* (gli "ospiti"). Il VMM determina come mappare le risorse virtuali nelle risorse fisiche: una risorsa fisica può essere condivisa in istanti diversi di tempo (in *time sharing*), partizionata o addirittura emulata dal software. Il VMM è molto più piccolo di un sistema operativo tradizionale: il software di un VMM può contenere circa 10 000 linee di codice. Sebbene il nostro interesse in questo capitolo sia focalizzato sull'utilizzo delle VM per aumentare la protezione del calcolatore, una VM fornisce altri due vantaggi significativi dal punto di vista commerciale.

1. *Gestione del software*. Le VM forniscono un'astrazione che può eseguire programmi appartenenti a tutti gli strati del software e può eseguire perfino vecchi sistemi operativi, come il DOS. In una installazione tipica, alcune VM possono emulare una vecchia versione di alcuni SO, altre VM possono emulare la versione stabile attuale del SO e alcune VM possono persino emulare versioni beta del SO.
2. *Gestione dell'hardware*. Uno dei motivi per cui conviene utilizzare server multipli è la possibilità di eseguire ciascuna applicazione con la versione appropriata del sistema operativo su calcolatori diversi, dato che la separazione aumenta l'affidabilità. Le VM consentono di eseguire indipendentemente le diverse applicazioni con la loro versione del SO utilizzando lo stesso hardware, riducendo così il numero di server richiesti. Un'altra applicazione di un VMM è il supporto alla migrazione di una VM in esecuzione su un altro calcolatore, per bilanciare il carico di lavoro o per abbandonare un server che si sta per guastare.

Interfaccia hardware/software

Amazon fornisce i suoi servizi Web EC2 (AWS, *Amazon Web Services*) utilizzando le macchine virtuali della sua "nuvola" per cinque motivi.

1. Consente al sistema AWS di proteggere gli utenti l'uno dall'altro mentre condividono lo stesso server.
2. Semplifica la distribuzione del software all'interno dei sistemi della dimensione di un grosso centro di calcolo. Il cliente installa l'immagine di una macchina virtuale configurata mediante un software adeguato e AWS lo distribuisce a tutti i calcolatori che un cliente vuole utilizzare.
3. I clienti (e AWS) possono affidabilmente terminare una VM per controllare l'utilizzo delle risorse quando i clienti hanno completato il loro lavoro.
4. Le macchine virtuali nascondono l'identità dell'hardware sul quale il cliente sta eseguendo i propri programmi; questo vuol dire che AWS può continuare a utilizzare i vecchi server *e anche* introdurre nuovi server più performanti. Il cliente,

per esempio, si può aspettare che le prestazioni siano equivalenti alle specifiche riportate per le Unità di Calcolo EC2, che, secondo AWS, "forniscono prestazioni equivalenti a una CPU AMD Opteron del 2007 da 1,0–1,2 GHz o di uno Xeon Intel del 2007". I server più recenti solitamente offrono un numero maggiore di Unità di Calcolo EC2 dei server più vecchi, ma AWS può continuare a noleggiare i server vecchi fino a quando rimangono economici.

5. I monitor delle macchine virtuali possono controllare la frequenza con cui una VM utilizza un processore, la rete e lo spazio su disco, cosa che consente all'AWS di offrire prezzi diversi per esecuzioni di tipo diverso che pure vengono eseguite dallo stesso server fisico. Per esempio, nel 2020, AWS offriva più di 200 tipi di esecuzione: da meno di mezzo centesimo di dollaro americano all'ora (t3a nano a 0,0047 US$) a più di 25 (ottimizzazione della memoria, x1 e 32xlarge, a 26,69 US$): un rapporto tra i prezzi di più di 5000:1.

In generale, il costo della virtualizzazione di un processore dipende dal carico di lavoro. I programmi utente che utilizzano principalmente il processore non hanno costi aggiuntivi di virtualizzazione; dato che il SO viene raramente richiamato, tutto continua a essere eseguito alla velocità originale. Invece, i programmi che prevedono un flusso di I/O rilevante, in genere, usano in maniera intensiva anche il sistema operativo, dato che devono eseguire molte chiamate di sistema e istruzioni privilegiate; queste possono produrre un elevato costo aggiuntivo di virtualizzazione. D'altra parte, se il carico di lavoro dell'I/O contiene principalmente *operazioni di I/O*, il costo della virtualizzazione del processore può essere completamente nascosto, poiché il processore si trova spesso nella condizione di attesa dei segnali di I/O. Il costo aggiuntivo dipende sia dal numero di istruzioni che devono essere emulate dal VMM sia dal tempo richiesto per emularle. Perciò, quando la VM ospite esegue la stessa ISA dell'host, come supponiamo qui, l'obiettivo dell'architettura e del VMM è di eseguire praticamente tutte le istruzioni direttamente sull'hardware nativo.

Requisiti del monitor di una macchina virtuale

Che cosa deve fare il monitor di una VM? Deve fornire un'interfaccia software al software ospitato, deve isolare uno dall'altro lo stato dei diversi software ospitati e deve proteggere se stesso dal software ospitato (compresi i SO ospitati). I requisiti qualitativi sono:

- il software ospitato dovrebbe comportarsi sulla VM esattamente come se venisse eseguito sull'hardware nativo, fatta eccezione per le caratteristiche legate alle prestazioni o alle limitazioni delle risorse prefissate che sono condivise tra le diverse VM;
- il software ospitato non dovrebbe essere in grado di modificare direttamente l'allocazione delle risorse reali del sistema.

Per "virtualizzare" un processore, il VMM deve poter controllare praticamente tutto: accessi allo stato privilegiato, traduzione degli indirizzi, I/O, eccezioni e interrupt, anche quando la VM ospitata e il SO in esecuzione li stanno utilizzando temporaneamente. Per esempio, se si verifica un interrupt del timer, il VMM sospenderà la VM ospitata correntemente in esecuzione, salverà lo stato di questa VM, gestirà l'interrupt, determinerà quale VM tra quelle ospitate deve riprendere l'esecuzione e infine caricherà lo stato della nuova VM. Le VM ospitate, il cui funzionamento è legato agli interrupt del timer, saranno dotate di un timer virtuale e riceveranno dal VMM un interrupt che emula l'interrupt del timer.

Per svolgere questa attività, il VMM deve avere un livello di privilegio superiore a quello della VM ospitata, che generalmente viene eseguita in modalità utente; ciò assicura anche che l'esecuzione di tutte le istruzioni privilegiate sia gestita dal VMM. I requisiti di base delle macchine virtuali di sistema sono:

- almeno due modalità di esecuzione del processore: sistema e utente;
- un insieme privilegiato di istruzioni, disponibili solamente nella modalità di sistema, che provocano un'eccezione di trap se vengono eseguite in modalità utente; tutte le risorse di sistema devono essere controllabili solamente attraverso queste istruzioni.

Mancanza di supporto alle macchine virtuali da parte dell'architettura dell'insieme di istruzioni

Se le VM fossero già previste durante la progettazione dell'ISA, sarebbe relativamente semplice sia ridurre il numero di istruzioni che devono essere eseguite da un VMM, sia aumentare la velocità di emulazione. Un'architettura che consente di eseguire una VM direttamente in hardware viene detta *virtualizzabile*; per esempio, le architetture IBM 370 e RISC-V sono di questo tipo.

Tuttavia, dato che le VM sono state introdotte solamente di recente nelle applicazioni per i PC e per i server, la maggior parte degli insiemi di istruzioni, compreso l'x86 e molte architetture RISC (tra le quali gli ARMv7 e i MIPS), è stata creata senza pensare alla virtualizzazione.

Dato che il VMM deve assicurare che il sistema ospitato interagisca solamente con le risorse virtuali, un tipico SO ospitato deve essere eseguito come un programma in modalità utente sopra il VMM. Quindi, se un sistema operativo ospitato cerca di accedere o di modificare informazioni legate alle risorse hardware utilizzando le istruzioni privilegiate, per esempio per leggere o scrivere il bit che abilita gli interrupt, verrà generata un'eccezione di trap che verrà gestita dal VMM. Sarà quindi il VMM a eseguire le modifiche richieste alle risorse fisiche interessate.

Se tutte le istruzioni che cercano di leggere o scrivere queste informazioni delicate provocano un'eccezione di trap quando vengono eseguite in modalità utente, il VMM può intercettare e può supportare una versione virtuale di queste informazioni delicate, gestendole come le gestirebbe il SO ospitato.

Senza questo supporto, occorrerebbe prendere altri provvedimenti: il VMM dovrebbe adottare precauzioni particolari per individuare tutte le istruzioni problematiche e assicurarsi che si comportino correttamente quando vengono eseguite da un SO ospitato, aumentando quindi la complessità del VMM e riducendo le prestazioni della VM in esecuzione.

Protezione e architettura dell'insieme delle istruzioni

La protezione deriva da uno sforzo congiunto dell'architettura e del sistema operativo, ma i progettisti, quando si sono diffuse le memorie virtuali, hanno dovuto modificare alcuni dettagli problematici delle architetture degli insiemi di istruzioni esistenti.

Per esempio, l'istruzione POPF dell'x86 carica i registri di flag dalla cima dello stack in memoria. Uno dei flag è il flag di abilitazione degli interrupt (IE, *Interrupt Enable*). Quando l'istruzione di POPF viene eseguita in modalità utente, invece di generare un'eccezione di trap modifica semplicemente il valore di tutti i flag a eccezione di IE, mentre se viene eseguita in modalità di sistema modifica anche IE. I sistemi operativi ospitati in una VM vengono eseguiti in modalità utente e questo rappresenta un problema, poiché ci aspetteremmo di vedere modificato anche il flag IE.

Storicamente, si possono identificare tre tappe nello sviluppo dell'hardware dei mainframe IBM e dei VMM per migliorare le prestazioni delle macchine virtuali:

1. Riduzione del costo della virtualizzazione del processore;
2. Riduzione dei costi addizionali degli interrupt dovuti alla virtualizzazione;
3. Riduzione dei costi degli interrupt pilotando gli interrupt verso la VM appropriata, invece che richiedere l'intervento del VMM.

Nel 2006 nuove proposte di AMD e Intel hanno cercato di affrontare il primo punto, riducendo il costo della virtualizzazione del processore. Sarà interessante vedere quante generazioni di architetture e quante modifiche al VMM vedranno la luce prima che questi tre aspetti siano affrontati insieme, e quanto tempo dovrà passare prima che le macchine virtuali del XXI secolo diventino efficienti come i mainframe IBM e i VMM degli anni '70.

Approfondimento. I RISC-V intercettano tutte le istruzioni privilegiate quando funzionano in modalità utente, per cui supportano la *virtualizzazione classica*, nella quale il SO ospitato funziona in modalità utente mentre il VMM funziona in modalità supervisore.

Approfondimento. L'ultima parte delle architetture da virtualizzare è il sotto-sistema di I/O. Questo è di gran lunga la parte di un sistema più difficile da virtualizzare per il continuo aumento del numero di dispositivi di I/O che vengono collegati a un calcolatore e per la loro diversità. Un'altra difficoltà è rappresentata dalla condivisione di un dispositivo reale tra più VM, e un'altra ancora deriva dalla necessità di supportare la miriade di driver dei dispositivi presenti. Questo viene soprattutto richiesto quando SO ospitati differenti devono essere supportati dalla stessa VM. L'illusione della VM può essere mantenuta fornendo a ciascuna VM una versione generica del driver di ogni tipo di dispositivo di I/O per poi lasciare al VMM il compito di gestire le reali transazioni di I/O.

5.7 Memoria virtuale

È stato progettato un sistema per fare in modo che la memoria centrale appaia al programmatore come una memoria a un solo livello, dato che la gestione dei trasferimenti richiesti viene effettuata automaticamente.

Kilburn *et al., One-level storage system (Sistema di memoria a un livello),* 1962

Nei paragrafi precedenti abbiamo visto come le cache forniscano un accesso veloce alle porzioni dei dati e del codice di un programma utilizzate più di recente. Allo stesso modo, la memoria principale può agire come una cache della memoria di massa, che di solito è costituita da dischi magnetici; questa tecnica è chiamata **memoria virtuale**. Storicamente esistono due motivazioni principali che hanno portato alla realizzazione della memoria virtuale: consentire una condivisione efficiente e sicura della memoria da parte di più programmi e liberare chi scrive i programmi dall'onere di dover gestire una memoria principale piccola e limitata. Quattro decenni dopo la sua invenzione, la prima motivazione è oggi predominante.

Naturalmente, affinché più macchine virtuali possano condividere la stessa memoria, dobbiamo essere in grado di proteggere le macchine virtuali una dall'altra, assicurando che un programma possa leggere e scrivere solamente le porzioni di memoria principale che gli sono state assegnate. La memoria principale conterrà solamente le parti attive delle diverse macchine virtuali, proprio come la cache contiene soltanto le parti attive di un singolo programma. Quindi, il principio di località giustifica la presenza sia della memoria virtuale sia della cache, e la memoria virtuale ci permette una condivisione efficiente del processore e della memoria principale.

Memoria virtuale: una tecnica che utilizza la memoria principale come se fosse una "cache" per la memoria di massa.

Non possiamo sapere quali macchine virtuali condivideranno la memoria con altre macchine virtuali, quando le compiliamo. Infatti, le macchine virtuali che condividono la memoria principale cambiano dinamicamente durante l'esecuzione. A causa di questa interazione dinamica, sarebbe opportuno

compilare ciascun programma nel suo *spazio di indirizzamento* privato: un insieme di locazioni di memoria riservate, accessibili soltanto a quel programma. La memoria virtuale implementa la traduzione degli indirizzi appartenenti allo spazio di indirizzamento privato di un programma in **indirizzi fisici**. Questo processo di traduzione garantisce la **protezione** dello spazio di indirizzamento di un programma dalle altre macchine virtuali.

Il secondo motivo che giustifica l'uso della memoria virtuale è consentire ai programmi utente di superare la dimensione fisica della memoria principale. Precedentemente, se un programma diventava troppo grande da poter essere contenuto nella memoria fisica, era compito del programmatore modificarlo in modo che ci potesse stare. I programmatori suddividevano il programma in porzioni e poi identificavano le parti tra loro indipendenti. Questi moduli, detti *overlay*, ossia "sovrapposti" in memoria, venivano caricati o scaricati dalla memoria principale sotto il controllo del programma durante la sua esecuzione; era compito del programmatore garantire che il programma non cercasse mai di accedere a un modulo che non fosse stato già caricato in memoria e che i moduli caricati non superassero mai la dimensione complessiva della memoria. I moduli di overlay erano tradizionalmente organizzati in modo da contenere sia il codice sia i dati e la chiamata di una procedura appartenente a un modulo diverso da quelli presenti in memoria richiedeva di caricare quel modulo in memoria a spese di uno dei moduli già presenti.

Come si può ben immaginare, questa operazione comportava un ingente carico di lavoro per il programmatore; la memoria virtuale fu inventata per alleggerire i programmatori da questo carico, gestendo automaticamente i due livelli della gerarchia costituiti dalla memoria principale, chiamata anche *memoria fisica* (per distinguerla dalla memoria virtuale), e dalla memoria di massa.

Sebbene i concetti che stanno alla base della memoria virtuale e delle cache siano gli stessi, le diverse origini storiche hanno portato all'utilizzo di una differente terminologia. Un blocco della memoria virtuale è chiamato *pagina* e le miss di una memoria virtuale vengono chiamate **page fault** (letteralmente, "mancanza di pagina"). Quando si utilizza la memoria virtuale, il processore genera un **indirizzo virtuale**; questo viene tradotto da una combinazione di componenti hardware e software in un *indirizzo fisico*, che può essere utilizzato per accedere alla memoria principale. La Figura 5.24 mostra

Indirizzo fisico: l'indirizzo di una cella della memoria principale.

Protezione: un insieme di meccanismi che garantiscono che più processi che condividono lo stesso processore, la stessa memoria o gli stessi dispositivi di I/O non interferiscano tra loro, intenzionalmente o involontariamente, evitando così che un programma possa leggere o scrivere i dati di un altro programma. Questi meccanismi consentono anche di isolare il sistema operativo dai processi utente.

Page fault: un evento che si verifica quando la pagina a cui si accede non è presente nella memoria principale.

Indirizzo virtuale: un indirizzo che corrisponde a una locazione nello spazio virtuale convertito in un indirizzo fisico mediante il processo di mappatura su un indirizzo fisico quando si accede alla memoria.

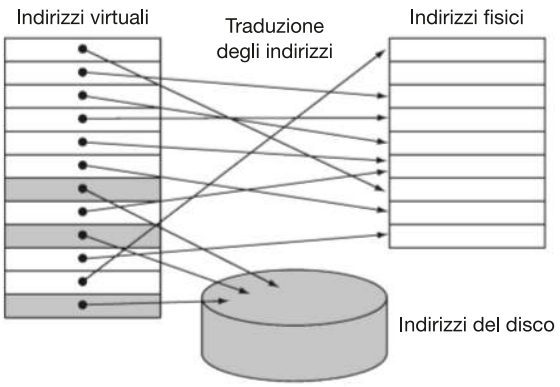


Figura 5.24 Nella memoria virtuale i blocchi della memoria (chiamati *pagine*) vengono mappati da un insieme di indirizzi (detti *virtuali*) a un altro insieme di indirizzi (detti *fisici*). Il processore produce indirizzi virtuali mentre l'accesso alla memoria avviene utilizzando gli indirizzi fisici. Sia la memoria virtuale sia quella fisica sono suddivise in pagine, in modo tale che una pagina virtuale viene mappata su una pagina fisica. Naturalmente è possibile che una pagina virtuale non sia presente nella memoria principale e che quindi non possa essere mappata su una pagina fisica; in tal caso si troverà sul disco. Le pagine fisiche possono essere condivise facendo in modo che due indirizzi virtuali puntino allo stesso indirizzo fisico. Questa possibilità permette a due diversi programmi di condividere dati e parti di codice.

Traduzione dell'indirizzo: detta anche mappatura dell'indirizzo, è il processo tramite il quale un indirizzo virtuale viene mappato sull'indirizzo utilizzato per accedere alla memoria.

la memoria indirizzata in modo virtuale e la mappatura delle pagine virtuali sulle pagine della memoria principale. Questo processo prende il nome di **traduzione dell'indirizzo** o *mappatura dell'indirizzo*. Attualmente, i due livelli della gerarchia delle memorie che vengono controllati utilizzando la memoria virtuale sono di solito le DRAM e le memorie a stato solido per i dispositivi mobili e le DRAM e i dischi magnetici nei server (Paragrafo 5.2). Considerando di nuovo l'analogia con la biblioteca, possiamo pensare che l'indirizzo virtuale sia il titolo di un libro e che l'indirizzo fisico sia la posizione di quel libro sullo scaffale.

La memoria virtuale semplifica anche la fase di caricamento di un programma prima della sua esecuzione, fornendo un meccanismo di *rilocazione*. La rilocazione mappa gli indirizzi virtuali utilizzati da un programma nei corrispondenti indirizzi fisici prima ancora che gli indirizzi vengano effettivamente utilizzati per accedere alla memoria. Questa mappatura consente il caricamento dei programmi in una qualsiasi locazione della memoria principale. Inoltre, tutti i meccanismi di gestione della memoria virtuale attualmente in uso rilocano il programma su un insieme di blocchi di dimensione prefissata, detti pagine, eliminando così la necessità di individuare un unico grande blocco di memoria contigua sufficientemente ampio da potervi caricare il programma; il sistema operativo deve solamente trovare un numero sufficiente di pagine della memoria fisica disponibili.

In una memoria virtuale, l'indirizzo viene suddiviso in due parti: *numero della pagina virtuale* e *offset nella pagina*. La **Figura 5.25** mostra la traduzione del numero della pagina virtuale nel corrispondente *numero della pagina fisica*. La versione del RISC-V descritta in questo libro fa uso di indirizzi a 32 bit e nella figura si suppone una memoria fisica di 1 GB che richiede 30 bit di indirizzamento (esiste anche una versione del RISC-V con indirizzi su 64 bit che offre una memoria fisica e una memoria virtuale di dimensioni molto maggiori).

Il numero della pagina fisica è definito dai bit più significativi dell'indirizzo fisico, mentre l'offset della pagina, che non viene modificato, definisce i bit meno significativi dell'indirizzo fisico. Il numero di bit riservati al campo

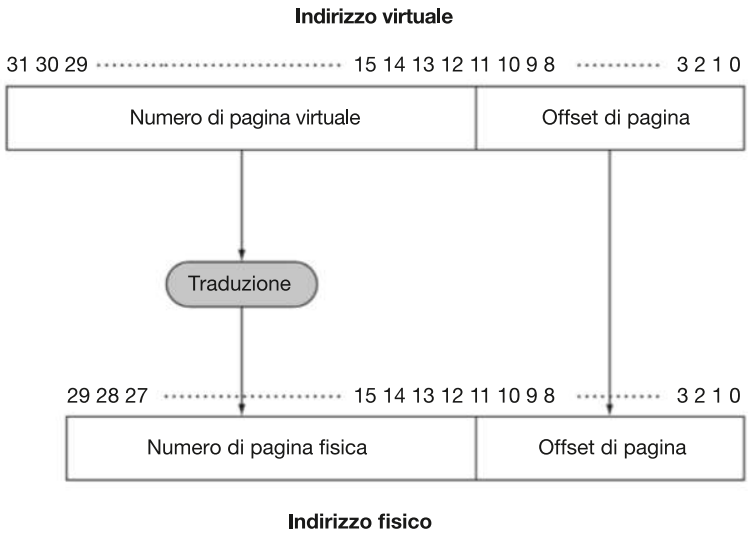


Figura 5.25 Mappatura di un indirizzo virtuale in un indirizzo fisico. La dimensione della pagina è di 2^{12} = 4 KiB. Il numero di pagine fisiche che possono risiedere in memoria è 2^{18}, dato che il numero di pagina fisica è codificato su 18 bit. Questo significa che la memoria principale può contenere al massimo 1 GiB, mentre lo spazio di indirizzamento virtuale è di 4 GiB.

offset determina la dimensione della pagina. Non è necessario che il numero di pagine indirizzabili attraverso l'indirizzo virtuale corrisponda al numero di pagine indirizzabili attraverso l'indirizzo fisico: avere un numero di pagine virtuali maggiore di quello delle pagine fisiche sta alla base dell'illusione di possedere una memoria virtuale di capacità maggiore.

Nei sistemi di memoria virtuale, molte scelte di progetto sono motivate dall'elevato costo di una miss, che nella memoria virtuale viene tradizionalmente chiamata *page fault*. Un page fault richiede milioni di cicli di clock per essere risolto: la tabella riportata all'inizio del Paragrafo 5.2 mostra che la memoria principale è circa 100 000 volte più veloce di un disco rigido. Questa enorme penalità di miss, che per dimensioni normali della pagina è costituita principalmente dal tempo necessario a caricare la prima parola del blocco, condiziona diverse scelte chiave nella progettazione dei sistemi di memoria virtuale.

- Le pagine devono essere sufficientemente grandi da ammortizzare il lungo tempo di accesso. Attualmente le dimensioni tipiche variano tra 4 KiB e 64 KiB. I nuovi sistemi desktop e server vengono progettati per supportare pagine da 32 KiB a 64 KiB, ma i nuovi sistemi embedded stanno andando nella direzione opposta, cioè verso pagine da 1 KiB.
- Le strutture che riducono la frequenza dei page fault sono molto attraenti. La tecnica principale che viene utilizzata consiste nel permettere un posizionamento flessibile delle pagine nella memoria principale mediante tecniche completamente associative.
- I page fault possono essere gestiti via software, perché l'incremento del tempo di elaborazione è piccolo se confrontato con il tempo di accesso al disco. Inoltre, il software può consentire l'utilizzo di algoritmi più intelligenti nella scelta del posizionamento delle pagine, poiché anche una piccola diminuzione della frequenza delle miss ripaga il tempo speso per eseguire questi algoritmi.
- La tecnica di scrittura di tipo write-through non funziona per la memoria virtuale, perché richiederebbe troppo tempo; i sistemi di memoria virtuale utilizzano quindi tecniche di scrittura in write-back.

Esamineremo questi elementi di progetto di una memoria virtuale nei prossimi paragrafi.

Approfondimento. Abbiamo introdotto la necessità di avere una memoria virtuale attraverso diverse macchine virtuali che devono condividere la stessa memoria, ma la memoria virtuale è stata inventata originariamente affinché programmi diversi potessero condividere un calcolatore, utilizzandolo per porzioni di tempo, in time sharing. Dato che molti lettori oggi non hanno esperienza dei sistemi in time sharing, abbiamo utilizzato le macchine virtuali per introdurre questo paragrafo.

Approfondimento. I processori con 32 bit di indirizzamento sono problematici per i server, per i PC e persino per i telefoni cellulari. Sebbene siamo inclini a pensare che lo spazio degli indirizzi virtuali sia molto più grande dello spazio degli indirizzi fisici, si può verificare la situazione opposta quando l'ampiezza dell'indirizzo, imposta dal processore, è piccola rispetto alla tecnologia corrente della memoria. In questo caso, il singolo programma non può trarre beneficio da una maggiore memoria fisica, ma un insieme di programmi, eseguiti contemporaneamente in parallelo, può beneficiare del fatto che i programmi non debbano essere caricati e scaricati dalla memoria, o che possano essere eseguiti su processori paralleli.

Approfondimento. In questo libro la discussione relativa alla memoria virtuale è focalizzata sulla tecnica della paginazione, nella quale vengono utilizzati blocchi di dimensione prefissata. Esiste anche uno schema con blocchi di dimensione variabile, detto **segmentazione**. Nella segmentazione un indirizzo è formato da due parti: un numero di segmento e un offset interno al segmento. Per determinare l'indirizzo fisico

Segmentazione: uno schema di mappatura a dimensione variabile in cui un indirizzo viene suddiviso in due parti: un numero di segmento, che viene mappato in un indirizzo fisico, e un offset interno al segmento.

effettivo, il contenuto del registro di segmento viene mappato in un indirizzo fisico a cui viene poi *sommato* l'offset. Dato che i segmenti possono avere dimensioni diverse, è necessario verificare che l'offset non superi i limiti del segmento. L'utilizzo principale della segmentazione è il supporto a tecniche più potenti per la protezione e la condivisione di uno spazio di indirizzamento comune. Molti libri sui sistemi operativi contengono una descrizione estesa della segmentazione (confrontata con la paginazione) e del suo utilizzo per condividere lo spazio di indirizzamento a livello logico. Lo svantaggio principale della segmentazione consiste nel dover suddividere lo spazio di indirizzamento in due parti logicamente separate, che vengono rappresentate e gestite utilizzando due campi diversi dell'indirizzo: il numero di segmento e l'offset. La paginazione invece rende invisibili al programmatore e al compilatore i confini tra il numero di pagina e l'offset della pagina.

I segmenti sono stati utilizzati anche come metodo per estendere lo spazio di indirizzamento, senza dover modificare la dimensione della parola del calcolatore. Questi tentativi non hanno avuto successo a causa della difficoltà di implementazione e della diminuzione delle prestazioni, dovute alla gestione di un indirizzo formato da due parti, di cui i programmatori e i compilatori devono necessariamente tenere conto.

Molte architetture dividono lo spazio di indirizzamento in grossi blocchi di dimensioni prefissate, in modo da rendere più semplice la protezione tra sistema operativo e programmi utente e aumentare l'efficienza nell'implementazione della paginazione. Sebbene queste partizioni vengano spesso chiamate «segmenti», questo meccanismo è molto più semplice della segmentazione in blocchi di dimensioni variabili e non è visibile ai programmi utente; esamineremo in dettaglio questa soluzione più avanti.

Come individuare la posizione di una pagina e come ritrovarla

A causa dell'altissima penalità dei page fault, i progettisti cercano di ridurne la frequenza ottimizzando la posizione delle pagine. Se si consente di mappare una pagina virtuale su una qualsiasi pagina fisica, il sistema operativo potrà scegliere di sostituire una qualsiasi pagina fisica quando si verifica un errore di page fault. Per esempio, il sistema operativo può utilizzare degli algoritmi sofisticati associati a strutture dati complesse per seguire l'utilizzo delle diverse pagine fisiche e cercare di scegliere di sostituire quella pagina fisica che si presume non verrà richiesta per molto tempo. L'utilizzo di uno schema di sostituzione intelligente e flessibile riduce la frequenza dei page fault e semplifica l'utilizzo di uno schema di posizionamento delle pagine completamente associativo.

Come menzionato nel Paragrafo 5.4, la difficoltà che si incontra nell'utilizzare uno schema completamente associativo è nell'individuazione dell'elemento, dato che esso può trovarsi ovunque all'interno del livello superiore della gerarchia e una ricerca esaustiva è impraticabile. In un sistema di memoria virtuale la pagina fisica corrispondente a una pagina virtuale viene individuata attraverso una tabella che indicizza la memoria; questa tabella viene chiamata **tabella delle pagine** (*page table*) e risiede nella memoria principale.

La tabella delle pagine ha come indice il numero di pagina contenuto nell'indirizzo virtuale e specifica il numero della corrispondente pagina fisica. Ogni programma possiede la propria tabella delle pagine, che mappa lo spazio di indirizzamento virtuale del programma nella memoria fisica.

Tabella delle pagine: la tabella che contiene la traduzione degli indirizzi virtuali in indirizzi fisici in un sistema di memoria virtuale. Essa è memorizzata nella memoria principale e viene tipicamente indicizzata tramite il numero della pagina virtuale: ogni elemento della tabella contiene il numero della pagina fisica corrispondente alla pagina virtuale, sempre che la pagina sia presente nella memoria principale.

Tornando all'analogia con la biblioteca, la tabella delle pagine corrisponde al catalogo, che contiene la corrispondenza tra i titoli dei libri e la loro posizione sugli scaffali. Come il catalogo della biblioteca può contenere informazioni relative anche ai libri che stanno in altre biblioteche, così la tabella delle pagine può contenere informazioni relative a pagine che non sono presenti nella memoria principale. Per individuare la posizione della tabella delle pagine nella memoria, l'architettura prevede un registro hardware che indica l'indirizzo iniziale della tabella; questo registro è chiamato *registro della tabella delle pagine*. Ipotizzeremo per il momento che la tabella delle pagine si trovi in un'area di memoria prefissata e contigua.

Interfaccia hardware/software

La tabella delle pagine, insieme al program counter e ai registri, specifica lo *stato* di una macchina virtuale; se si vuole consentire a un'altra macchina virtuale di utilizzare il processore, è necessario salvare lo stato, cioè il contenuto di questi componenti. In seguito, dopo avere ripristinato lo stato, il programma potrà continuare l'esecuzione dal punto in cui era stato interrotto. Spesso si chiama *processo* questo particolare stato; il processo viene considerato *attivo* quando è padrone del processore, altrimenti si dice che è *inattivo*. Il sistema operativo può rendere attivo un processo caricandone lo stato, incluso il program counter, in modo da far riprendere l'esecuzione del programma a partire dall'indirizzo contenuto nel program counter, precedentemente salvato.

Lo spazio di indirizzamento del processo è quindi l'insieme di tutti i dati della memoria a cui il processo può accedere ed è definito dalla sua tabella delle pagine, che risiede anch'essa in memoria. Invece di salvare l'intera tabella delle pagine, il sistema operativo carica semplicemente solo il registro della tabella delle pagine, in modo tale da potere indirizzare la tabella appartenente al processo che vuole rendere attivo. Ogni processo ha la propria tabella delle pagine, dato che processi differenti possono utilizzare gli stessi indirizzi virtuali. Il sistema operativo è responsabile dell'allocazione della memoria fisica e dell'aggiornamento della tabella delle pagine, che deve essere fatta in modo tale che gli spazi virtuali di due processi differenti non entrino in conflitto. Come vedremo tra breve, l'utilizzo di tabelle separate per processi differenti permette di proteggere un processo dagli altri.

Nella **Figura 5.26** è mostrato il modo in cui l'hardware forma l'indirizzo fisico a partire dal registro della tabella delle pagine, la relativa tabella delle pagine e l'indirizzo virtuale. In ogni elemento della tabella è presente anche un bit di validità, proprio come nelle cache. Se il bit è a 0, la pagina non è presente nella memoria fisica e si verifica un errore di page fault; se invece il bit assume il valore 1, la pagina è presente e il corrispondente elemento della tabella delle pagine contiene il numero della pagina fisica.

Poiché la tabella delle pagine contiene la traduzione di ogni possibile pagina virtuale, non c'è bisogno di aggiungere un campo tag. Utilizzando la terminologia delle cache, l'indice utilizzato per accedere alla tabella delle pagine è l'indirizzo completo di un blocco e corrisponde al numero della pagina virtuale.

Page fault

Se il bit di validità associato a una pagina virtuale è impostato a 0, si verifica un errore di page fault e il controllo deve essere trasferito al sistema operativo. Il trasferimento viene effettuato attraverso il sollevamento di un'eccezione, come verrà descritto più avanti in questo paragrafo (*vedi* anche Capitolo 4). Quando il sistema operativo prende il controllo, deve scoprire dove si trova la pagina richiesta nel livello inferiore della gerarchia, solitamente costituita da una memoria flash o da un disco magnetico, e decidere dove copiare la pagina nella memoria principale. L'indirizzo virtuale da solo non consente di individuare immediatamente la posizione della pagina nella memoria secondaria. Tornando all'analogia con la biblioteca, non è possibile individuare la posizione di un libro sugli scaffali conoscendone soltanto il titolo, ma è necessario consultare il catalogo per ricavare la sua posizione. Analogamente, in un sistema di memoria virtuale occorre tenere traccia della posizione nella memoria secondaria di ogni pagina definita nello spazio di indirizzamento virtuale.

Dato che non possiamo sapere a priori quando una pagina della memoria principale verrà selezionata per essere sostituita, di solito il sistema operativo riserva sulla memoria flash o sul disco uno spazio sufficiente per tutte le

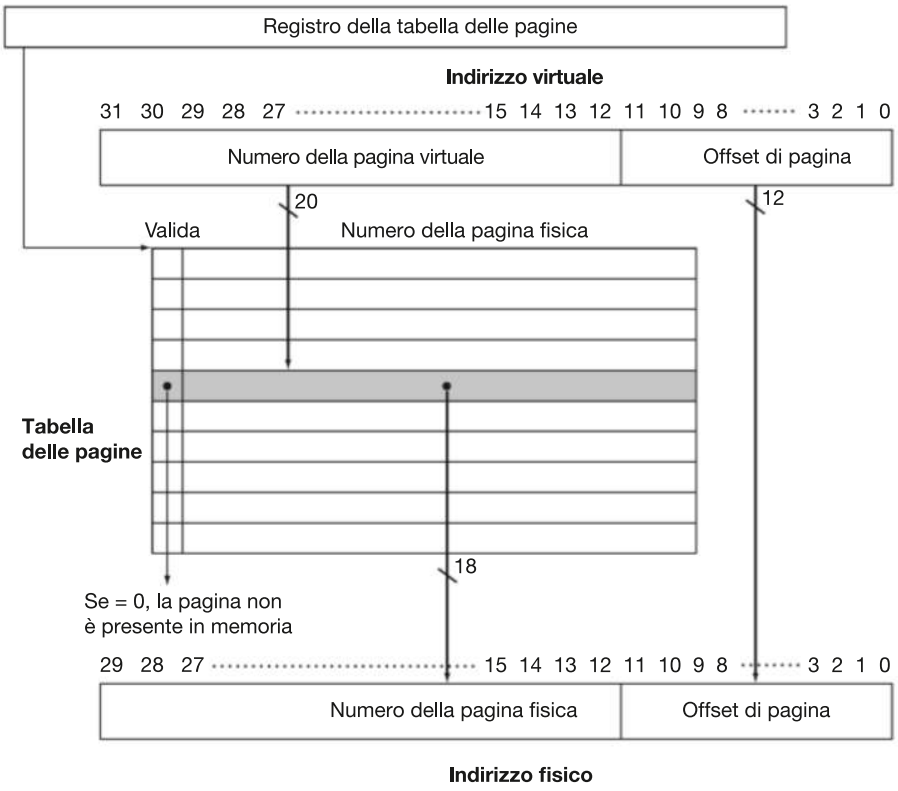


Figura 5.26 La tabella delle pagine ha come indice il numero della pagina virtuale e contiene la parte corrispondente (più significativa) dell'indirizzo fisico. Si suppone che gli indirizzi siano su 32 bit. L'indirizzo di partenza della tabella delle pagine è contenuto nel registro della tabella delle pagine. Nella figura, la dimensione della pagina è di 2^{12} byte o 4 KiB. Lo spazio di indirizzamento virtuale è di 2^{32} byte, cioè 4 GiB, mentre lo spazio di indirizzamento fisico è di 2^{30} byte, che consente alla memoria fisica di arrivare fino a 1 GiB. Il numero di elementi della tabella è pari a 2^{20}, cioè circa 1 milione. Il bit di validità associato a ogni elemento indica se l'indirizzo associato è valido; se il bit di validità è impostato a 0, la pagina richiesta non è presente nella memoria principale. Sebbene ogni elemento della tabella delle pagine qui mostrata richieda 19 bit, l'ampiezza degli elementi viene tipicamente arrotondata a 32 bit per ottenere un indirizzamento più facile. I bit aggiuntivi possono essere utilizzati per memorizzare informazioni supplementari relative a ogni pagina, come la protezione.

Spazio di swap: lo spazio che viene riservato su disco all'intero spazio di memoria virtuale di un processo.

pagine di un processo nel momento in cui crea il processo stesso; questo spazio viene chiamato **spazio di swap** (spazio di scambio). Contemporaneamente, il SO crea anche una struttura dati in cui registra la posizione su disco di ciascuna pagina virtuale. Questa struttura dati può far parte della tabella delle pagine, oppure può essere una struttura dati a se stante, indicizzata come la tabella delle pagine. La Figura 5.27 mostra l'organizzazione della memoria virtuale nel caso in cui si utilizzi una sola tabella che contiene il numero della pagina fisica o l'indirizzo della memoria secondaria, corrispondente a ogni pagina virtuale.

Il sistema operativo crea inoltre una struttura dati che tiene traccia dei processi e degli indirizzi virtuali associati a ciascuna pagina fisica. Quando si verifica un errore di page fault, se tutte le pagine della memoria fisica sono in uso il sistema operativo deve scegliere una pagina da sostituire. Poiché si vogliono minimizzare i page fault, molti sistemi operativi cercano di scegliere una pagina che suppongono di non dover utilizzare poco dopo. Analizzando il passato per predire il futuro, i sistemi operativi seguono uno schema di sostituzione di tipo LRU (*Least Recently Used*, già introdotto nel Paragrafo 5.4). Il sistema operativo cerca la pagina utilizzata meno di recente, ipotizzando che una pagina inutilizzata da molto tempo abbia una minore probabilità di

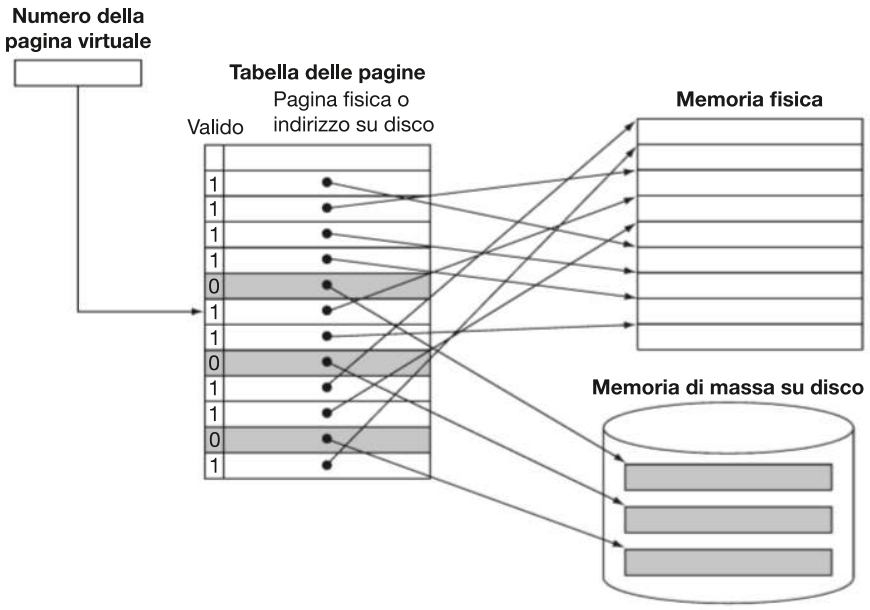


Figura 5.27 La tabella delle pagine mappa ciascuna pagina della memoria virtuale in una pagina della memoria principale oppure in una pagina memorizzata su disco, che è il livello successivo della gerarchia. Il numero della pagina virtuale viene utilizzato come indice della tabella delle pagine. Se il bit di validità è impostato a 1, la tabella delle pagine fornisce il numero della pagina fisica corrispondente alla pagina virtuale, ossia l'indirizzo della memoria fisica a partire dal quale è memorizzata la pagina virtuale. Se invece il bit di validità è impostato a 0, la pagina in quel momento si trova solamente sul disco, all'indirizzo specificato. In molti sistemi, le tabelle degli indirizzi delle pagine fisiche e delle pagine su disco, sebbene costituiscano una sola tabella dal punto di vista logico, vengono memorizzate in due strutture dati separate. L'utilizzo di tabelle separate è in parte giustificato dal fatto che è necessario conservare gli indirizzi su disco di tutte le pagine, anche di quelle che sono correntemente presenti nella memoria fisica. Ricordiamo che le pagine della memoria principale e le pagine su disco hanno dimensioni identiche.

essere utilizzata rispetto alle altre. Le pagine sostituite vengono salvate nello spazio di swap del disco. Forse vi state domandando come venga trattato il sistema operativo; esso è analogo a ogni altro processo e le tabelle utilizzate dal sistema operativo per gestire la memoria virtuale sono residenti nella memoria principale; una spiegazione dettagliata di questa apparente contraddizione verrà data a breve.

Interfaccia hardware/software

Implementare uno schema di tipo LRU completamente corretto è troppo dispendioso, poiché richiederebbe l'aggiornamento della struttura dati a *ogni* accesso alla memoria. Per questo motivo molti sistemi operativi adottano una strategia LRU approssimata, tenendo traccia delle pagine che sono state utilizzate da poco. Per aiutare il sistema operativo a individuare la pagina richiesta meno di recente, i computer che adottano RISC-V forniscono un **bit di indirizzamento** (*reference bit*), detto anche **bit di utilizzo** (*use bit*) o **bit di accesso** (*access bit*), che viene impostato a 1 ogni volta che si accede alla pagina corrispondente. Il sistema operativo azzera periodicamente tutti i bit di utilizzo, per poi impostare a 1 il bit di utilizzo associato a ogni pagina indirizzata da quel momento in poi. Analizzando questo bit, il sistema operativo può determinare quali pagine siano state toccate durante un certo lasso di tempo e può quindi utilizzare questa informazione per selezionare una pagina tra quelle richieste meno di recente, cioè tra quelle il cui bit di utilizzo è impostato a 0. Se questo bit non viene fornito dall'hardware, il sistema operativo deve trovare un altro metodo per individuare quali siano le pagine richieste di recente.

Bit di indirizzamento (detto anche bit di utilizzo o bit di accesso): è un campo che viene impostato a 1 ogni volta che si accede alla pagina associata e che viene utilizzato per implementare la politica LRU o altri schemi di sostituzione.

Memoria virtuale per un insieme ampio di indirizzi virtuali

Con un indirizzo virtuale su 32 bit, ed elementi della tabella delle pagine di 4 byte, la dimensione dalla tabella delle pagine sarebbe di 4 MiB. Questo implica che dovremmo utilizzare 4 MiB della memoria principale per ciascun programma in esecuzione in ogni istante. La quantità di memoria non è così tanta per un singolo processo. Ma cosa succederebbe se ci fossero centinaia di processi in esecuzione, ciascuno con la sua tabella delle pagine? E come potremmo gestire indirizzi su 64 bit, che richiederebbero Terabyte per la tabella delle pagine di ciascun programma?

Esistono molte tecniche per ridurre la quantità di memoria richiesta da una tabella delle pagine. I seguenti cinque metodi hanno lo scopo di ridurre la quantità totale di memoria richiesta e di minimizzare la quantità di memoria principale dedicata alla tabella delle pagine.

1. La tecnica più semplice consiste nell'utilizzare un registro limite che riduca la dimensione della tabella delle pagine di un certo processo. Se il numero di pagine virtuali cresce oltre il valore contenuto nel registro limite, occorre aggiungere nuovi elementi alla tabella delle pagine. Questa tecnica permette di aumentare la dimensione della tabella delle pagine mano a mano che un processo richiede più spazio in memoria. In questo modo, la tabella diventa grande solo se il processo utilizza molte pagine dello spazio di indirizzamento virtuale. Questa tecnica richiede che lo spazio di indirizzamento si espanda in una sola direzione (crescente).
2. Consentire la crescita della memoria in una sola direzione non è sufficiente, poiché la maggior parte dei linguaggi di programmazione richiede almeno due aree di memoria che possano crescere secondo necessità: un'area che contiene lo stack e l'altra che contiene i dati dinamici ed è chiamata *heap*. Per questo motivo, è conveniente suddividere la tabella delle pagine in due parti e lasciare crescere verso il basso la parte superiore e verso l'alto la parte inferiore. Questo significa che ci saranno due tabelle delle pagine separate e due limiti distinti. L'utilizzo di due tabelle delle pagine divide lo spazio di indirizzamento in due segmenti: il bit più significativo di un indirizzo virtuale determina generalmente il segmento e quindi quale delle due tabelle delle pagine debba essere utilizzata per tradurre quell'indirizzo. Dato che il segmento viene specificato dal bit più significativo dell'indirizzo, ognuno dei due segmenti può occupare al massimo metà dello spazio di indirizzamento. Un registro limite associato a ogni segmento contiene la dimensione corrente del segmento, che cresce di una unità per ogni nuova pagina. A differenza della segmentazione precedentemente descritta, questo tipo di segmentazione è invisibile ai programmi applicativi ma non al sistema operativo. Il principale svantaggio di questo schema è che non funziona bene quando lo spazio degli indirizzi viene utilizzato in modo "sparso" e non come un insieme contiguo di indirizzi virtuali.
3. Un altro approccio per ridurre la dimensione della tabella delle pagine consiste nell'applicare all'indirizzo virtuale una funzione di corrispondenza (*hashing function*), in modo tale che la tabella delle pagine abbia dimensione pari soltanto al numero delle pagine fisiche della memoria principale. Una struttura di questo tipo viene chiamata *tabella inversa delle pagine*. Ovviamente il processo di ricerca in una tabella inversa delle pagine è leggermente più complesso, perché non è più sufficiente utilizzare il numero della pagina virtuale come indice della tabella.
4. Per ridurre la quantità di memoria principale consumata dalle tabelle delle pagine, la maggior parte dei sistemi moderni prevede che la tabella delle pagine sia, a sua volta, suddivisa in pagine. Sebbene questo sembri un trucco, funziona utilizzando gli stessi concetti che sono alla base della memoria

virtuale, a patto di consentire alla tabella delle pagine di risiedere nello spazio di indirizzamento virtuale. Inoltre, occorre evitare alcuni problemi piccoli ma critici, come la generazione di una catena infinita di errori di page fault. Il modo in cui questi problemi vengono superati è molto particolare ed è di solito strettamente legato alla macchina. In breve, questi problemi si evitano posizionando tutte le tabelle delle pagine all'interno dello spazio di indirizzamento del sistema operativo e inserendo almeno qualcuna delle tabelle delle pagine utilizzate dal sistema operativo in una porzione della memoria principale che viene indirizzata fisicamente, le cui pagine sono sempre presenti in memoria e quindi non si troveranno mai nella memoria secondaria.

5. Per ridurre la quantità totale di memoria richiesta per memorizzare una tabella delle pagine, si possono utilizzare anche tabelle delle pagine su più livelli, e questa è la soluzione adottata dal RISC-V per ridurre l'occupazione di memoria dovuta alla traduzione degli indirizzi. La **Figura 5.28** mostra i due livelli di traduzione richiesti per passare da un indirizzo virtuale su 32 bit a un indirizzo fisico su 32 bit per pagine di 4 KiB. La traduzione di un indirizzo inizia guardando nella tabella di livello 0, utilizzando i bit più significativi dell'indirizzo. Se questo indirizzo si trova nella tabella, viene utilizzato il gruppo successivo di bit più significativi per indicizzare la tabella delle pagine indicata dall'elemento della tabella indicante il segmento, e così via. In questo modo, la tabella di livello 0 mappa l'indirizzo virtuale in regioni di 4 MiB (2^{22} byte). La tabella di livello 1 a sua volta mappa l'indirizzo virtuale sulla pagina di memoria di 4 KiB (2^{12} byte). Questo schema consente di utilizzare lo spazio di indirizzamento in maniera sparsa: segmenti multipli non contigui possono essere particolarmente utili per spazi di indirizzamento molto ampi, in particolare per la versione del RISC-V a 64 bit, e nei sistemi software che richiedono non contigua delle pagine. Il principale svantaggio di questa tecnica a più livelli è la maggiore complessità del processo di traduzione degli indirizzi virtuali.

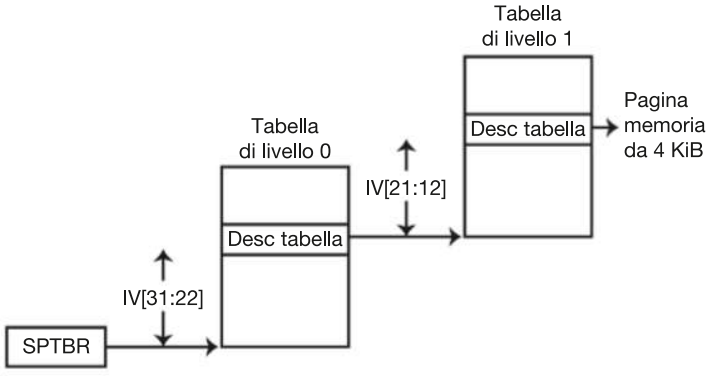


Figura 5.28 Il RISC-V utilizza due livelli di tabelle per tradurre un indirizzo virtuale su 32 bit in un indirizzo fisico su 32 bit. Invece di aver bisogno di 1 milione di elementi in una singola tabella delle pagine come nella Figura 5.26, l'approccio gerarchico ne richiede solamente una piccola frazione. Ciascun passo della traduzione utilizza 10 bit dell'indirizzo virtuale per identificare la tabella del livello successivo, fino quando i bit più significativi dell'indirizzo virtuale non sono stati mappati sull'indirizzo fisico della pagina desiderata di 4 KiB. Ciascun elemento di una tabella della pagine RISC-V è di 4 byte, per cui i 1024 elementi di una tabella riempiono una singola pagina di 4 KiB. Il Registro Base della Tabella delle Pagine del Supervisore (SPTBR, *Supervisor Page Table Base Register*) fornisce l'indirizzo di partenza della prima tabella delle pagine. Anche la versione a 64 bit del RISC-V utilizza pagine di 4 KiB e tre o quattro livelli di tabelle delle pagine radix-512 (dato che gli elementi della tabella delle pagine sono di 8 Byte, poiché gli indirizzi virtuali che possono arrivare a 64 bit), con indirizzi virtuali su 39 o 48 bit. Uno spazio di indirizzamento fisico di 1 Terabyte (2^{40}) indirizzato attraverso un indirizzo virtuale su 48 bit era sufficiente per i sistemi dell'anno 2020 (i 16 bit più significativi dei 64 bit dell'indirizzo virtuale venivano ignorati).

Che cosa succede in scrittura?

La differenza tra il tempo di accesso a una cache e il tempo di accesso alla memoria principale è dell'ordine delle decine o centinaia di cicli di clock; risulta quindi possibile utilizzare uno schema di scrittura di tipo write-through per le cache, anche se occorre introdurre un buffer di scrittura per nascondere al processore la latenza della scrittura. In un sistema di memoria virtuale, la scrittura al livello inferiore della gerarchia, il disco, richiede milioni di cicli di clock; perciò, utilizzare un buffer di scrittura per consentire al processore di scrivere direttamente sul disco sarebbe del tutto inutile. I sistemi di memoria virtuale devono invece utilizzare lo schema write-back, in cui il singolo dato viene scritto nella pagina della memoria principale; l'intera pagina viene poi copiata nella memoria secondaria quando questa deve essere sostituita in memoria principale.

Interfaccia hardware/software

Lo schema write-back ha un altro grande vantaggio per i sistemi di memoria virtuale. Dato che il tempo di trasferimento su disco è piccolo rispetto al tempo di accesso al disco, copiare un'intera pagina su disco è molto più efficiente che non scrivere le singole parole. Tuttavia, un'operazione di scrittura su disco è comunque costosa anche utilizzando lo schema di scrittura write-back, che è più efficiente del trasferimento di parole singole. Perciò è opportuno sapere se sia effettivamente *necessario* copiare una pagina sul disco quando si sceglie di sostituirla. Per sapere se il sistema abbia scritto su una certa pagina a partire dall'istante in cui la pagina è stata caricata in memoria, si aggiunge un bit, detto *dirty bit* (bit sporco), alla tabella delle pagine. Il dirty bit viene impostato a 1 nel momento in cui una qualsiasi parola della pagina viene scritta. Quando il sistema operativo sceglie di sostituire la pagina, il dirty bit indica se è necessario scrivere la pagina sul disco prima di rilasciare lo spazio che occupava in memoria per destinarlo a una nuova pagina. Una pagina il cui contenuto è stato modificato viene chiamata *dirty page* (pagina sporca).

Come rendere più veloce la traduzione degli indirizzi: il TLB

Dato che la tabella delle pagine deve essere salvata nella memoria principale, ogni accesso alla memoria da parte di un programma richiede almeno il doppio del tempo, dovendo compiere un accesso alla memoria per ottenere l'indirizzo fisico e un secondo accesso per avere il dato richiesto. La chiave per migliorare le prestazioni in termini di tempo di accesso è lo sfruttamento della località degli accessi alla tabella delle pagine. Quando si utilizza la traduzione del numero di una certa pagina virtuale, è molto probabile che la stessa traduzione sia richiesta di nuovo poco dopo, dato che le parole contenute in una pagina sono caratterizzate da località sia spaziale sia temporale.

Translation Lookaside Buffer (TLB): una cache che tiene traccia degli indirizzi virtuali tradotti più di recente per evitare l'accesso alla tabella delle pagine.

Basandosi su queste considerazioni, molti processori moderni contengono una cache speciale che tiene traccia delle traduzioni utilizzate più di recente. Questa cache speciale, che specifica la traduzione degli indirizzi virtuali utilizzati più di recente, è chiamata **Translation Lookaside Buffer** (**TLB**), letteralmente "buffer di traduzione consultato a parte", sebbene sia più corretto chiamarla *cache di traduzione*. Tornando all'analogia con la biblioteca, il TLB corrisponde a quel pezzettino di carta che utilizziamo per scrivere la posizione dei libri che abbiamo cercato nel catalogo: invece di consultare ogni volta l'intero schedario, annotiamo la posizione dei diversi libri e utilizziamo i nostri appunti come se fossero una cache del catalogo della biblioteca.

La **Figura 5.29** mostra che il campo tag del TLB contiene una parte del numero della pagina virtuale, mentre il campo dati contiene il numero di una

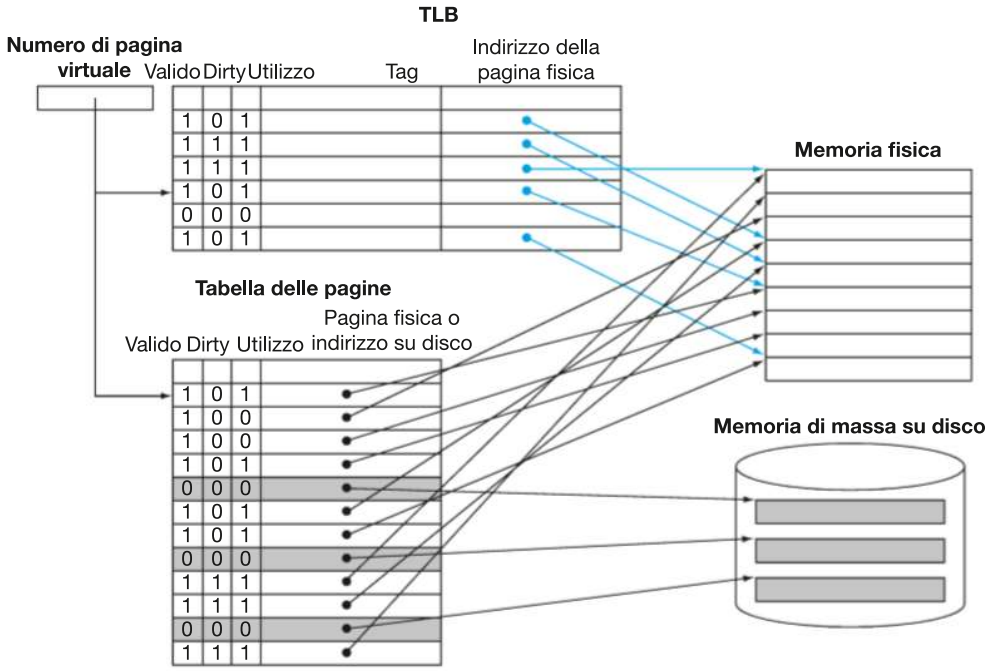


Figura 5.29 Il TLB si comporta come una cache della tabella delle pagine solo per le pagine mappate su una pagina fisica. Il TLB contiene un sottoinsieme delle corrispondenze tra gli indirizzi virtuali e gli indirizzi fisici che sono memorizzati nella tabella delle pagine. Le corrispondenze definite nel TLB sono disegnate in blu. Dato che il TLB è una cache, deve contenere il campo tag. Se nessun elemento del TLB corrisponde alla pagina richiesta, si rende necessario esaminare la tabella delle pagine; questa può fornire il numero della corrispondente pagina fisica (che può essere inserito assieme alla pagina virtuale corrispondente nel TLB), oppure indica che la pagina si trova sul disco (nel qual caso si genera un page fault). Poiché la tabella delle pagine contiene un elemento per ogni pagina virtuale, essa non richiede alcun campo etichetta; in altre parole, a differenza del TLB, la tabella delle pagine *non* è una memoria cache.

pagina fisica. Poiché non c'è più bisogno di accedere alla tabella delle pagine a ogni richiesta di accesso alla memoria (dato che stiamo utilizzando il TLB), nelle linee del TLB devono essere presenti altri bit di stato: il bit di utilizzo e il dirty bit. Anche se la Figura 5.29 mostra una tabella delle pagine singola, i TLB funzionano bene anche con le tabelle delle pagine multilivello. Il TLB semplicemente carica l'indirizzo fisico e i tag di protezione associati alla tabella delle pagine di primo livello.

Quando viene richiesta una pagina, per prima cosa si cerca il numero della pagina virtuale nel TLB. Se la ricerca ha avuto successo, il numero di pagina fisica corrispondente viene utilizzato per costruire l'indirizzo e il bit di utilizzo viene impostato a 1; se il processore sta effettuando una scrittura, anche il dirty bit viene impostato a 1. Se l'accesso al TLB genera una miss, occorre verificare se si tratti di un vero page fault, oppure soltanto di una miss del TLB. Se la pagina è presente nella memoria principale, allora la miss del TLB indica solamente che manca la traduzione. In tal caso, il processore può gestire la miss del TLB caricando dalla tabella delle pagine (dell'ultimo livello) nel TLB la traduzione dell'indirizzo virtuale, per poi ripetere l'accesso al TLB. Se invece la pagina non si trova nella memoria principale, allora la miss del TLB indica che si è verificato un vero e proprio page fault. In questo caso, il processore richiede l'intervento del sistema operativo, sollevando un'eccezione. Dato che il TLB contiene molti meno elementi rispetto al numero di pagine della memoria principale, le miss del TLB saranno molto più frequenti dei page fault effettivi.

Le miss del TLB possono essere gestite via hardware oppure via software. La differenza di prestazioni tra le due soluzioni è minima, poiché le operazioni fondamentali che devono essere eseguite sono le stesse in entrambi i casi.

Quando si verifica una miss del TLB e la traduzione mancante è stata recuperata dalla tabella delle pagine, occorre individuare quale elemento del TLB debba essere sostituito. Dato che il bit di utilizzo e il dirty bit fanno parte di ogni elemento del TLB, quando un elemento del TLB viene sostituito occorre copiare questi bit nel blocco corrispondente della tabella delle pagine. Questi bit rappresentano l'unica parte di un elemento del TLB che può essere modificata. Utilizzare una strategia di write-back, ossia copiare questi bit quando si verifica una miss invece di farlo ogni volta che vengono modificati, è molto efficiente, poiché ci aspettiamo che la frequenza di miss del TLB sia bassa. Alcuni sistemi utilizzano altre tecniche per impostare in modo approssimato il bit di utilizzo e il dirty bit, eliminando la necessità di scrivere nel TLB se non quando viene caricato un nuovo elemento a seguito di una miss.

Alcuni tipici valori di un TLB possono essere:

- dimensione del TLB: 16–512 elementi;
- dimensione del blocco: 1–2 elementi della tabella delle pagine (tipicamente 4–8 byte ciascuno);
- tempo di hit: 0,5–1 ciclo di clock;
- penalità di miss: 10–100 cicli di clock;
- frequenza di miss: 0,01–1%.

I progettisti hanno utilizzato una vasta gamma di gradi di associatività nei TLB. Molti sistemi utilizzano piccoli TLB completamente associativi, perché una corrispondenza di questo tipo consente una frequenza di miss più bassa; inoltre, viste le ridotte dimensioni del TLB, il costo della mappatura completamente associativa non è troppo elevato. Altri sistemi utilizzano TLB di grandi dimensioni, spesso con basso grado di associatività. Con una mappatura completamente associativa, scegliere l'elemento da sostituire diventa complicato, dal momento che l'implementazione hardware di uno schema LRU diventa troppo costoso. Inoltre, essendo le miss del TLB molto più frequenti dei page fault e dovendo quindi essere gestite in maniera più efficiente, non si può accettare l'utilizzo di un algoritmo software pesante da un punto di vista computazionale, come invece è possibile fare per la sostituzione delle pagine a seguito di un page fault. Di conseguenza, molti sistemi prevedono di sostituire un elemento del TLB scelto a caso. Esamineremo gli schemi di sostituzione con maggior dettaglio nel Paragrafo 5.8.

TLB del processore FastMATH Intrinsity

Per vedere come vengono applicati questi concetti a un processore reale, esaminiamo più da vicino il TLB del processore FastMATH Intrinsity. Il sistema di memoria utilizza pagine di 4 KiB e uno spazio di indirizzamento a soli 32 bit; quindi, il numero della pagina virtuale viene codificato su 20 bit. L'indirizzo fisico ha la stessa dimensione dell'indirizzo virtuale. Il TLB contiene 16 elementi, è completamente associativo e contiene sia dati che istruzioni; ogni elemento del TLB è di 64 bit e comprende un campo tag di 20 bit (contenente il numero della pagina virtuale associata all'elemento del TLB), il numero della pagina fisica associata (anch'esso su 20 bit), un bit di validità, un dirty bit e altri bit richiesti per la gestione della pagina virtuale. Come la maggior parte delle architetture MIPS, le miss del TLB vengono gestite via software.

La **Figura 5.30** mostra il TLB e una delle cache, mentre la **Figura 5.31** illustra i passi necessari per soddisfare una richiesta di lettura o scrittura. Quando avviene una miss del TLB, l'hardware provvede a salvare il numero di pagina associata all'indirizzo richiesto in un registro speciale e solleva un'eccezione; l'eccezione attiva il sistema operativo, il quale gestisce la miss via software. Per trovare l'indirizzo fisico, la procedura software di risposta all'eccezione di

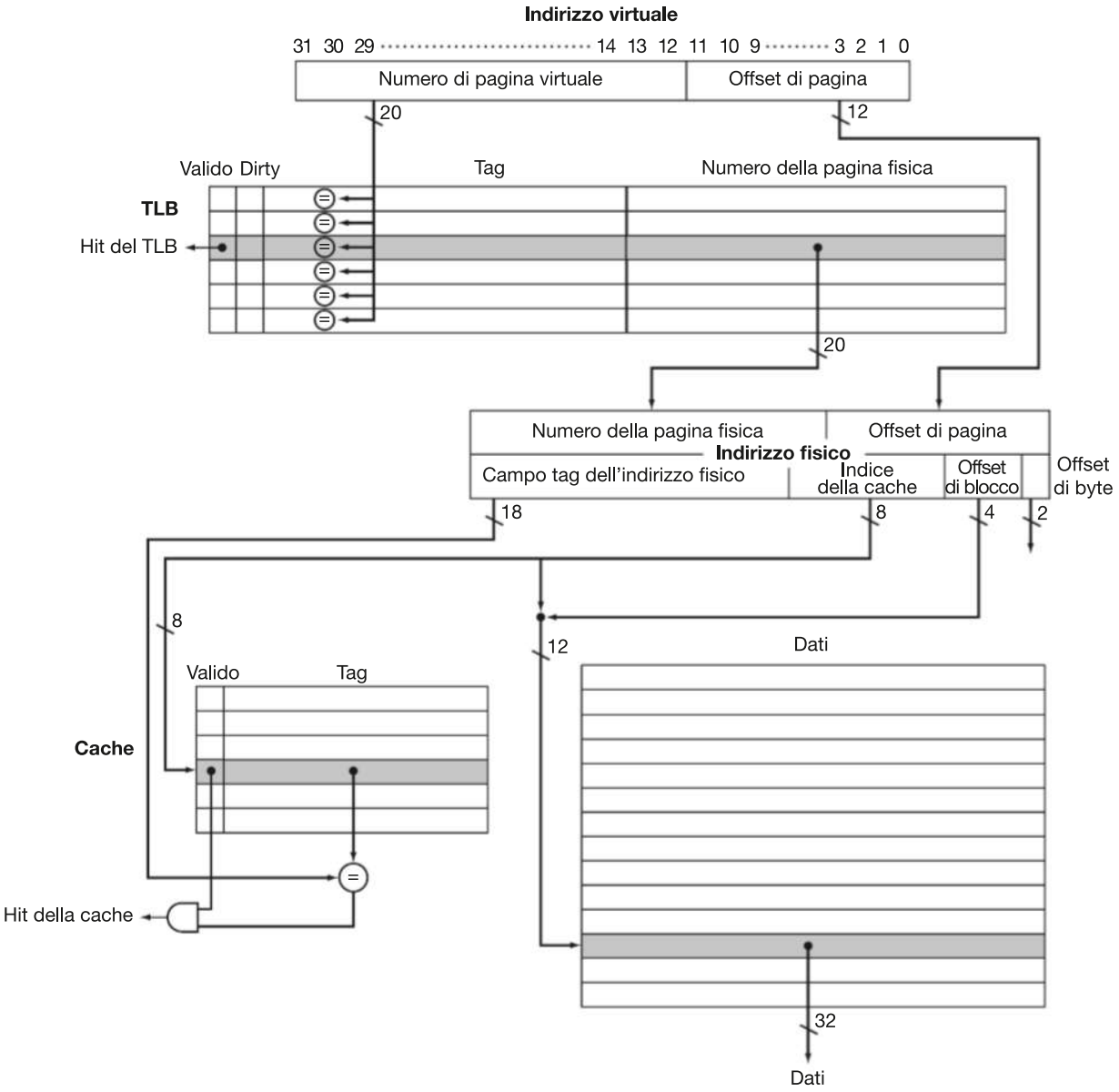


Figura 5.30 Il TLB e la cache implementano, nel processore FastMATH Intrinsity, il meccanismo che consente di trasformare un indirizzo virtuale nel dato corrispondente. Questa figura mostra l'organizzazione del TLB e della cache dati supponendo una dimensione della pagina di 4 KiB. Si noti che gli indirizzi per questo computer sono ampi solo 32 bit. Lo schema si focalizza sulla lettura; la Figura 5.31 descriverà come vengono gestite le scritture. Si noti che, diversamente dalla Figura 5.12, la memoria che contiene i campi tag e quella che contiene i dati sono separate. Indirizzando la memoria dati (RAM), lunga e stretta, concatenando i campi dell'indice e dell'offset di blocco, viene selezionata la parola del blocco desiderata senza aver bisogno di un multiplexer 16:1. Mentre la cache è a mappatura diretta, il TLB è completamente associativo. Implementare un TLB completamente associativo richiede che ogni tag del TLB venga confrontato con il numero della pagina virtuale, dato che l'elemento di interesse può essere ovunque nel TLB (*vedi* la sezione *Approfondimento* di pagina 371). Se il bit di validità dell'elemento identificato è impostato a 1, l'accesso si traduce in una hit del TLB e i bit del numero della pagina fisica insieme ai bit provenienti dall'offset della pagina concorrono a formare l'indice che viene utilizzato per accedere alla cache.

miss indicizza la tabella delle pagine utilizzando il numero di pagina dell'indirizzo virtuale e il registro della tabella delle pagine che specifica l'indirizzo di partenza della tabella delle pagine associata al processo attivo. Attraverso uno speciale insieme di istruzioni di sistema che aggiornano il TLB, il sistema operativo scrive nel TLB l'indirizzo fisico preso dalla tabella delle pagine. Una miss del TLB può essere gestita in circa 13 cicli di clock, supponendo che il codice di risposta all'eccezione e l'elemento della tabella delle pagine siano

 © 978-88-08-19966-9

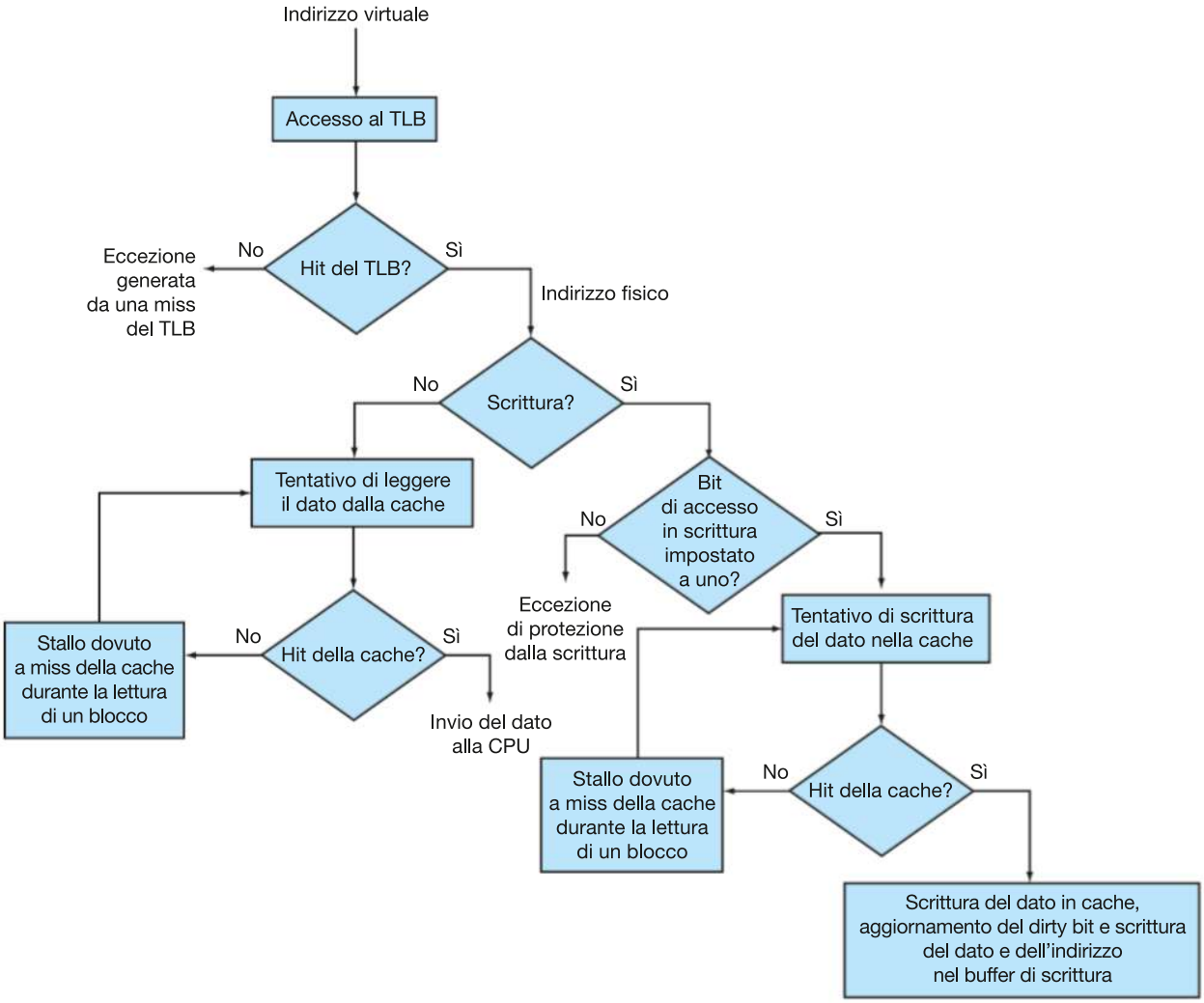


Figura 5.31 Gestione di una lettura o di una scrittura di tipo write-through nel TLB e nella cache nel processore FastMATH Intrinsity. Se l'accesso al TLB produce una hit, l'accesso alla cache avviene con l'indirizzo fisico ottenuto. In lettura, la cache può generare una hit o una miss e quindi fornire il dato richiesto o causare uno stallo che dura il tempo necessario perché il dato sia prelevato dalla memoria principale. In scrittura, se l'accesso genera una hit, una parte dell'elemento della cache selezionato viene sovrascritto e il dato viene inviato al buffer di scrittura, poiché utilizziamo un approccio di tipo write-through. Una miss in scrittura è simile a una miss in lettura, a eccezione del fatto che il blocco viene modificato dopo che è stato letto dalla memoria principale. La politica write-back richiederebbe di impostare a 1 (a ogni scrittura) il dirty bit associato al blocco della cache e di caricare nel buffer di scrittura l'intero blocco solo quando si verifica una miss in lettura o scrittura e il blocco che deve essere sostituito è stato modificato. Si noti inoltre che una hit del TLB e una hit della cache sono eventi indipendenti, ma una hit nella cache può verificarsi solo se è avvenuta una hit del TLB, il che significa che il dato deve essere presente nella memoria principale. La relazione tra miss del TLB e miss della cache verrà esaminata nel prossimo esempio e negli esercizi alla fine del capitolo. Si noti che gli indirizzi per questo computer sono ampi solo 32 bit.

contenuti rispettivamente nella cache istruzioni e nella cache dati. Un vero e proprio page fault si verifica se il blocco individuato nella tabella delle pagine non contiene un indirizzo fisico valido. L'hardware mantiene un indice che segnala quale elemento del TLB sia opportuno sostituire; in questo caso l'elemento da sostituire è scelto a caso.

Esiste una complicazione aggiuntiva per le richieste di scrittura, dovuta al fatto che occorre controllare il bit di accesso in scrittura del TLB. Questo bit impedisce a un programma di scrivere nelle pagine alle quali può accedere solamente in lettura. Se il programma cerca di scrivere ma il bit di accesso in scrittura è impostato a 0, viene generata un'eccezione. Il bit di accesso in scrittura fa parte del meccanismo di protezione che discuteremo tra breve.

Integrazione della memoria virtuale, dei TLB e delle cache

La memoria virtuale e le cache lavorano insieme, in un'unica gerarchia, cosicché un dato non può trovarsi all'interno della cache a meno che non sia presente anche nella memoria principale.

Il sistema operativo gioca un ruolo importante nel sostenere questa gerarchia, scaricando il contenuto delle pagine presenti in cache quando decide di spostare queste pagine sul disco. Allo stesso tempo, il sistema operativo modifica la tabella delle pagine e il TLB, cosicché un tentativo di accesso a un qualsiasi dato all'interno di una pagina spostata su disco genera un errore di page fault.

Nella migliore delle ipotesi, un indirizzo virtuale viene tradotto dal TLB e inviato alla cache, dove il dato cercato viene individuato, letto e inviato al processore. Nel caso peggiore l'accesso si traduce in una miss di tutti e tre i componenti della gerarchia delle memorie: il TLB, la tabella delle pagine e la cache. Il seguente esempio illustra più in dettaglio l'interazione fra i tre componenti.

Complesso delle operazioni di una gerarchia delle memorie

ESEMPIO

In una gerarchia delle memorie come quella della Figura 5.30, che comprende un TLB e una cache, un accesso alla memoria può generare tre differenti tipi di miss: una miss del TLB, un page fault e una miss della cache.

Considerare tutte le possibili combinazioni di questi tre tipi di miss, tenendo presente che una miss può essere generata da uno o più componenti, per un totale di sette combinazioni diverse. Per ogni combinazione, si dica se l'evento può realmente accadere e in quali situazioni.

SOLUZIONE

La Figura 5.32 mostra tutte le combinazioni possibili e se queste possano presentarsi o meno nella realtà.

TLB	Tabella delle pagine	Cache	Possibile? Se sì, in quale situazione?
Hit	Hit	Miss	Possibile, sebbene la tabella delle pagine non venga mai realmente controllata se si verifica una hit del TLB
Miss	Hit	Hit	Miss del TLB, ma l'elemento si trova nella tabella delle pagine; al secondo tentativo, il dato viene trovato nella cache
Miss	Hit	Miss	Miss del TLB, ma l'elemento si trova nella tabella delle pagine; al secondo tentativo, si verifica però una miss della cache
Miss	Miss	Miss	Miss del TLB, seguita da un page fault; al secondo tentativo, l'accesso al dato deve provocare una miss della cache
Hit	Miss	Miss	Impossibile: il TLB non può fornire la traduzione di una pagina che non è presente in memoria
Hit	Miss	Hit	Impossibile: il TLB non può fornire la traduzione di una pagina che non è presente in memoria
Miss	Miss	Hit	Impossibile: i dati non possono trovarsi nella cache se la pagina non è presente in memoria

Figura 5.32 Possibili combinazioni di eventi nel TLB, nel sistema di memoria virtuale e nella cache. Tre di queste combinazioni sono impossibili; una combinazione è teoricamente possibile, ma in pratica non si verifica mai: hit del TLB, hit della memoria virtuale e miss della cache.

Approfondimento. Nella Figura 5.32 si presuppone che tutti gli indirizzi di memoria siano tradotti in indirizzi fisici prima che avvenga l'accesso alla cache. In questa organizzazione, la cache è *indicizzata fisicamente* e contiene *tag fisici* (il campo tag e il campo indice sono riferiti all'indirizzo fisico e non a quello virtuale). In tale schema, la quantità di tempo richiesta per accedere alla memoria, in caso di hit della cache, deve essere sufficiente per accedere sia al TLB sia alla cache; ovviamente questi due accessi possono essere eseguiti in **pipeline**.

Cache indirizzata virtualmente: una cache a cui si accede tramite l'indirizzo virtuale invece che tramite l'indirizzo fisico.

In alternativa, il processore può indirizzare la cache utilizzando un indirizzo completamente o parzialmente virtuale. In questo caso si parla di **cache indirizzata virtualmente** e contiene campi tag associati a indirizzi virtuali; quindi, una cache di questo tipo viene *indicizzata virtualmente* e contiene *tag virtuali*. In questo schema, la traduzione hardware dell'indirizzo virtuale in indirizzo fisico non viene utilizzata nei normali accessi alla cache, dato che gli accessi avvengono attraverso l'indirizzo virtuale che non è ancora stato tradotto in indirizzo fisico. Questo porta il TLB al di fuori del cammino critico, riducendo così la latenza della cache. Tuttavia, quando si verifica una miss della cache, il processore ha comunque bisogno di tradurre l'indirizzo virtuale in indirizzo fisico, in modo tale da poter prelevare dalla memoria principale il blocco da scrivere nella cache.

Aliasing: una situazione in cui si accede allo stesso oggetto attraverso due indirizzi; può verificarsi nella memoria virtuale quando esistono due indirizzi virtuali per la stessa pagina fisica.

Quando si accede alla cache con un indirizzo virtuale e alcune pagine fisiche sono condivise tra diversi programmi (i quali possono accedere a queste pagine utilizzando indirizzi virtuali diversi), esiste la possibilità che si verifichi **aliasing** (sovrapposizione). L'aliasing si verifica quando lo stesso oggetto ha due nomi diversi; in questo caso due indirizzi virtuali diversi per la stessa pagina. Questa ambiguità diventa critica, perché una parola appartenente a una pagina di questo tipo può essere caricata in due diverse locazioni della cache, corrispondenti a due indirizzi virtuali diversi, consentendo quindi a un programma di scrivere nella pagina senza che l'altro programma possa accorgersi che i dati vengono modificati. Le cache a indirizzamento virtuale richiedono che siano imposti dei vincoli di progetto sulla cache e sul TLB per ridurre l'aliasing, oppure richiedono che il sistema operativo, ed eventualmente l'utente, facciano in modo che l'aliasing non si possa mai verificare.

Cache indirizzata fisicamente: una cache che viene indirizzata utilizzando l'indirizzo fisico.

Un compromesso spesso utilizzato, che si trova a cavallo fra le due scelte di progetto estreme, è quello di utilizzare cache che contengono nei campi tag indirizzi fisici ma che vengono indicizzate virtualmente, utilizzando a volte solo l'offset interno alla pagina dell'indirizzo virtuale (che è anche l'offset dell'indirizzo fisico, poiché non viene tradotto). Queste cache, che sono quindi *indicizzate virtualmente ma contengono nei campi tag indirizzi fisici*, hanno l'obiettivo di raggiungere le prestazioni delle cache indicizzate virtualmente sfruttando la semplicità architetturale delle **cache indirizzate fisicamente**.

In questo tipo di cache, per esempio, il problema dell'aliasing non esiste. Nella Figura 5.30 si presuppone una dimensione della pagina di 4 KiB, ma in realtà la dimensione è di 16 KiB, per cui il FastMATH Intrinsity può implementare questo stratagemma; tuttavia, perché funzioni, ci deve essere un attento coordinamento tra la dimensione minima della pagina, la dimensione della cache e il grado di associatività. Il RISC-V richiede che la cache si comporti come se avesse indici e tag fisici, ma non richiede che sia implementata in questo modo. Per esempio, cache con indici virtuali e tag fisici potrebbero utilizzare della logica aggiuntiva per garantire che il software non si accorga della differenza.

Meccanismi di protezione basati sulla memoria virtuale

Oggigiorno, la funzione più importante della memoria virtuale è forse quella di consentire a più processi di condividere un'unica memoria principale, fornendo allo stesso tempo un meccanismo di protezione della memoria ai diversi processi e al sistema operativo. Il meccanismo di protezione deve garantire che, anche se più processi condividono la stessa memoria principale, non sia possibile per un processo scrivere nello spazio d'indirizzamento di un altro processo utente o nello spazio riservato al sistema operativo, di fatto non rispettando le regole in modo intenzionale o accidentale. Il bit di accesso in scrittura del TLB permette di proteggere la pagina associata, evitando che questa possa essere scritta. Senza questo livello di protezione, i virus dei calcolatori sarebbero ancora più diffusi.

Interfaccia hardware/software

Per permettere al sistema operativo di implementare i meccanismi di protezione nel sistema di memoria virtuale, l'hardware deve fornire almeno le tre seguenti funzionalità di base. Si noti che le prime due sono le stesse richieste per le macchine virtuali (Paragrafo 5.6).

1. Supportare almeno due modalità di funzionamento: la prima associata all'esecuzione di processi utente e la seconda all'esecuzione di processi del sistema operativo; questi ultimi vengono chiamati anche **processi kernel**, **processi supervisore** o *processi executive*.
2. Mettere a disposizione dei processi utente una parte dello stato del processore che i processi potranno leggere ma non scrivere. Questa parte dello stato comprende il bit che determina la modalità di funzionamento, utente o kernel, il puntatore alla tabella delle pagine e il TLB. Per scrivere in questi componenti il processore utilizza delle istruzioni speciali disponibili solo in modalità kernel.
3. Fornire dei meccanismi che permettano al processore di passare dalla modalità utente alla modalità kernel e viceversa. Il primo passaggio viene solitamente realizzato mediante un'eccezione di **chiamata di sistema** (*system call*). Questa eccezione viene sollevata da un'istruzione speciale, che nell'insieme di istruzioni RISC-V è l'istruzione `ecall`; essa trasferisce il controllo a un indirizzo predeterminato nello spazio di indirizzamento della modalità supervisore. Come per tutte le altre eccezioni, il contenuto del program counter associato all'istruzione `ecall` viene salvato nel program counter delle eccezioni del supervisore, SEPC (*Supervisor Exception Program Counter*), e il processore inizia a lavorare in modalità supervisore. Per ritornare alla modalità utente, al termine della risposta all'eccezione, viene utilizzata l'istruzione di *ritorno da eccezione del supervisore* (`sret`, *Supervisor exception RETurn*), che ripristina la modalità utente e continua l'esecuzione a partire dall'indirizzo memorizzato nel SEPC.

Mediante l'utilizzo di questi meccanismi e memorizzando la tabella delle pagine nello spazio di indirizzamento del sistema operativo, il sistema operativo può modificare le tabelle delle pagine e, allo stesso tempo, vietare ai processi utente di modificarle, garantendo quindi che un processo utente possa solo accedere allo spazio di memoria a esso assegnato dal sistema operativo.

Modalità kernel: detta anche **modalità supervisore**, è una modalità di funzionamento nella quale il processo in esecuzione è un processo del sistema operativo.

Chiamata di sistema: è un'istruzione speciale che trasferisce il controllo dalla modalità utente a un indirizzo predeterminato nello spazio dei processi di supervisione, sollevando un'eccezione.

Occorre anche evitare che un processo legga i dati di un altro processo. Per esempio, non vogliamo che il programma di uno studente legga il testo di un esame prima del giorno dell'esame quando questo si trova nella memoria del processore. Se si permette di condividere la memoria principale, occorre fornire a ogni processo la possibilità di proteggere i propri dati dalla lettura e scrittura da parte di altri processi; in caso contrario, la condivisione della memoria si potrebbe rivelare una maledizione!

Ricordiamo che ciascun processo ha il proprio spazio di indirizzamento virtuale; perciò se il sistema operativo organizza le tabelle delle pagine in modo che pagine virtuali indipendenti corrispondano a pagine fisiche differenti, un processo non sarà in grado di accedere ai dati di un altro processo. Ovviamente, questo richiede anche che un processo utente non possa alterare la traduzione delle pagine virtuali contenuta nella tabella delle pagine. Il sistema operativo può garantire la protezione se può impedire ai processi utente di modificare la propria tabella delle pagine; tuttavia, il sistema operativo deve poter modificare le tabelle delle pagine. Inserendo le tabelle delle pagine nello spazio di indirizzamento protetto del sistema operativo, si possono soddisfare entrambi i requisiti. Quando i processi vogliono condividere delle informazioni, devono essere assistiti dal sistema operativo, dato che accedere alle informazioni di un altro processo implica cambiare la tabella delle pagine per il processo che effettua l'accesso. Si può utilizzare il bit d'accesso in scrittura per limitare la condivisione alla sola lettura; tale bit, come il resto della tabella delle pagine, può essere modificato solo dal sistema operativo. Per consentire

a un processo, per esempio P1, di leggere una pagina del processo P2, P2 dovrebbe chiedere al sistema operativo di creare un nuovo elemento della tabella delle pagine contenente una pagina virtuale nello spazio di indirizzamento di P1; questo nuovo elemento punterà alla pagina fisica che P2 vuole condividere. Il sistema operativo può usare il bit di protezione in scrittura per impedire a P1 di scrivere dei dati in questa pagina fisica, se questo è il desiderio di P2. Tutti i bit che determinano i diritti di accesso a una pagina devono essere contenuti sia nella tabella delle pagine sia nel TLB, dato che la tabella delle pagine viene consultata solo in corrispondenza delle miss del TLB.

Cambio di contesto: la modifica dello stato interno del processore che consente a un processo di subentrare a un altro processo nell'utilizzo del processore; comprende il salvataggio dello stato, necessario a ritornare in seguito al processo che era precedentemente in esecuzione.

Approfondimento. L'azione che compie il sistema operativo quando decide di cambiare il processo in esecuzione, da P1 a P2, è chiamata **cambio di contesto** (*context switch*) o anche *cambio di processo* (*process switch*). Il cambio di contesto deve garantire che P2 non possa accedere alla tabella delle pagine di P1, perché ciò potrebbe compromettere la protezione dei dati di P1. Se non c'è un TLB, è sufficiente modificare il registro della tabella delle pagine in modo che punti alla tabella delle pagine di P2, anziché a quella di P1. Se invece è presente un TLB, occorre invalidare gli elementi del TLB, i quali corrispondono a P1, sia per proteggere i dati di P1 sia per forzare il TLB a caricare gli elementi di P2. Se la frequenza con cui avvengono i cambi di contesto fosse elevata, questo meccanismo potrebbe rivelarsi facilmente inefficiente. Per esempio, P2 potrebbe fare in tempo a caricare solo pochi elementi del TLB prima che il sistema operativo restituisca di nuovo il controllo a P1; P1 a quel punto si troverebbe nella situazione in cui tutti gli elementi del TLB sono stati cancellati e dovrebbe ricaricarli di nuovo attraverso una sequenza di miss del TLB. Questo problema sorge perché gli indirizzi virtuali utilizzati da P1 e da P2 possono essere gli stessi e occorre, quindi, svuotare il TLB per evitare confusione tra essi.

Un'alternativa comunemente utilizzata è quella di estendere lo spazio di indirizzamento virtuale aggiungendo un *identificatore di processo*, o *identificatore di task*. Per esempio, il FastMATH Intrinsity prevede a questo scopo un campo per l'identificazione dello spazio di indirizzamento su 8 bit, chiamato ASID (*Address Space IDentifier*). Questo piccolo campo identifica il processo correntemente in esecuzione ed è contenuto in un registro apposito, che viene aggiornato dal sistema operativo ogni volta che cambia il processo in esecuzione. Il RISC-V offre anche un ASID per ridurre i flush del TLB quando si verifica una commutazione di contesto. L'identificatore di processo viene concatenato con il campo tag del TLB, in modo tale che una hit del TLB può verificarsi solo nel caso in cui sia il numero di pagina sia l'identificatore del processo corrispondono a quelli della pagina cercata. Questa soluzione permette di eliminare la necessità di cancellare gli elementi del TLB, che saranno cancellati solamente in rare occasioni, per esempio quando si ricicla un ASID.

Possono nascere dei problemi di natura simile anche per la cache, poiché quando cambia il processo in esecuzione la cache contiene i dati del processo precedentemente in esecuzione. I problemi per le cache indirizzate fisicamente sono diversi da quelli delle cache indirizzate virtualmente e diverse soluzioni, come quella degli identificatori di processo, possono essere utilizzate per garantire che ciascun processo possa accedere ai propri dati.

Gestione delle miss del TLB e dei page fault

Mentre la traduzione degli indirizzi virtuali in indirizzi fisici attraverso il TLB è semplice da capire quando l'accesso al TLB produce una hit, la gestione delle miss del TLB e dei page fault è più complessa. Una miss del TLB si verifica quando nessuno degli elementi del TLB corrisponde all'indirizzo virtuale e può essere dovuta a una delle due seguenti cause:

1. La pagina è presente in memoria e bisogna soltanto inserire nel TLB l'elemento corrispondente;
2. La pagina non è presente in memoria ed è quindi necessario trasferire il controllo al sistema operativo per gestire il page fault che viene generato.

Per gestire una miss del TLB o un page fault si utilizza il meccanismo della risposta alle eccezioni: viene interrotto il processo attivo e si trasferisce il

controllo al sistema operativo, per poi riprendere l'esecuzione del processo interrotto. Il page fault viene identificato all'interno del ciclo di clock in cui si accede alla memoria. Per riprendere l'esecuzione dell'istruzione dopo avere gestito l'errore di page fault, è necessario salvare il contenuto del program counter dell'istruzione che ha provocato il page fault. Il registro *SEPC*, *program counter delle eccezioni del supervisore*, viene utilizzato per mantenere questo valore.

Inoltre una miss del TLB, o un'eccezione di page fault, deve essere sollevata prima della fine del ciclo di clock in cui si verifica l'accesso alla memoria, in modo tale che il programma di risposta all'eccezione possa iniziare già nel ciclo di clock immediatamente successivo, invece di continuare la normale esecuzione delle istruzioni. Se il page fault fosse riconosciuto più tardi, un'eventuale istruzione di load potrebbe modificare il contenuto di un registro con conseguenze disastrose nel momento in cui il processo viene fatto ripartire dall'istruzione interrotta. Per esempio, si consideri l'istruzione `lb x10,0(x10)`: il calcolatore deve evitare la scrittura nella pipeline del dato letto dalla memoria, altrimenti non sarebbe più possibile far ripartire correttamente il programma, poiché il contenuto originale del registro `x10` verrebbe distrutto. Un problema simile si verifica con le istruzioni di store: occorre evitare che la scrittura in memoria venga effettivamente completata quando si verifica un page fault; di solito ciò viene ottenuto impostando a 0 il segnale di controllo della scrittura della memoria.

Interfaccia hardware/software

Nel lasso di tempo che intercorre tra l'istante in cui il sistema operativo inizia l'esecuzione del programma di risposta alle eccezioni e quello in cui termina di salvare tutto lo stato del processo, il sistema operativo è particolarmente vulnerabile. Per esempio, se si dovesse verificare una seconda eccezione mentre il sistema operativo sta ancora gestendo la prima, l'unità di controllo sovrascriverebbe il program counter delle eccezioni rendendo impossibile il ritorno all'istruzione che ha causato l'errore di page fault. È possibile evitare questa situazione disastrosa fornendo un meccanismo di **disabilitazione** e **abilitazione delle eccezioni**. Quando si verifica la prima eccezione, il processore imposta a 1 un bit che disabilita tutte le altre eccezioni; questo può avvenire nello stesso istante in cui il processore imposta a 1 il bit relativo alla modalità supervisore. Il sistema operativo salverà quindi quella parte dello stato sufficiente a far ripartire l'esecuzione anche se dovesse arrivare un'altra eccezione, vale a dire il program counter delle eccezioni del supervisore (SEPC) e il *registro causa del supervisore* (SCAUSA), che, come abbiamo visto nel Capitolo 4, memorizza la causa dell'eccezione. Nel RISC-V, SEPC e SCAUSA sono due degli speciali registri di controllo che vengono utilizzati nella gestione delle eccezioni, delle miss del TLB e dei page fault. Al termine del salvataggio dello stato, il sistema operativo può riabilitare le eccezioni. Questo meccanismo fa in modo che le eccezioni non causino la perdita dello stato di nessun processo, cosa che renderebbe impossibile il riavvio dell'esecuzione a partire dall'istruzione interrotta.

Abilitazione delle eccezioni: o abilitazione degli interrupt, è un segnale o un'azione che controlla se il processo debba o meno rispondere a un'eccezione; è necessario per evitare che si verifichino eccezioni prima che il processo abbia salvato in modo sicuro lo stato richiesto per riprendere l'esecuzione.

Dopo che il sistema operativo ha individuato l'indirizzo virtuale che ha provocato il page fault, deve eseguire i seguenti tre passi:

1. Esaminare l'elemento della tabella delle pagine individuato tramite l'indirizzo virtuale e identificare la posizione nella memoria secondaria della pagina desiderata;
2. Scegliere la pagina fisica da sostituire; se la pagina scelta è stata modificata, cioè il dirty bit è impostato a 1, la pagina deve essere salvata nella memoria secondaria prima di copiare una nuova pagina virtuale sulla pagina fisica;
3. Avviare la lettura dalla memoria secondaria e trasferire la pagina richiesta alla memoria principale nella pagina fisica individuata.

Ovviamente quest'ultimo passo richiede milioni di cicli di clock del processore, come pure il secondo nel caso in cui la pagina da sostituire sia stata modificata; di conseguenza il sistema operativo, di solito, sceglie un altro processo da fare eseguire al processore finché l'accesso al disco non è stato completato. Dato che il sistema operativo ha salvato l'intero stato del processo corrente, può tranquillamente assegnare il controllo del processore a un altro processo.

Quando la lettura della pagina dalla memoria secondaria è stata completata, il sistema operativo può ripristinare lo stato del processo che aveva generato il page fault ed eseguire l'istruzione di ritorno dall'eccezione. Questa istruzione fa tornare il processore dalla modalità di funzionamento kernel alla modalità utente e ripristina il contenuto del program counter. A questo punto, il processo utente può rieseguire l'istruzione che aveva provocato il page fault, riuscendo ad accedere alla pagina desiderata, e può quindi continuare la sua esecuzione.

La risposta alle eccezioni di page fault causate dagli accessi ai dati è difficile da implementare correttamente in un processore, per la combinazione delle seguenti tre caratteristiche:

1. Queste eccezioni sono generate durante l'esecuzione di un'istruzione, a differenza dei page fault sulle istruzioni;
2. L'istruzione non può essere completata prima di aver gestito l'eccezione che è stata generata;
3. Dopo avere gestito l'eccezione, l'istruzione deve riprendere come se nulla fosse accaduto.

Istruzione riavviabile: un'istruzione che può riprendere l'esecuzione dopo che l'eccezione è stata risolta, senza che ciò influenzi il risultato dell'istruzione.

Fare in modo che un'**istruzione** sia **riavviabile** (*restartable instruction*) vuol dire poter continuare l'esecuzione dopo avere gestito l'eccezione; ciò è relativamente semplice in un'architettura come il RISC-V. Dato che ogni istruzione scrive solo un dato e che la scrittura del dato avviene alla fine del ciclo di esecuzione dell'istruzione, è sufficiente evitare che l'istruzione venga completata, impedendo la scrittura, per poi riprendere l'esecuzione dell'istruzione dall'inizio.

Approfondimento. Per i calcolatori dotati di istruzioni più complesse, che possono richiedere l'accesso a molte celle di memoria e scrivere molti dati, è molto più difficile fare in modo che le istruzioni siano riavviabili.

Una singola istruzione può generare diversi page fault nel corso della sua esecuzione. Per esempio, i calcolatori x86 sono dotati di istruzioni di spostamento di blocchi di memoria che spostano dati costituiti da migliaia di parole. In questi processori spesso non è possibile far ripartire un'istruzione dall'inizio, come si fa per le istruzioni RISC-V; l'istruzione deve essere interrotta e poi fatta ripartire dallo stadio di esecuzione nel quale era stata fermata. Riprendere l'esecuzione di un'istruzione interrotta a metà di solito richiede che venga salvato uno stato speciale, che venga risolta l'eccezione e che venga poi ripristinato questo stato speciale. Per fare in modo che tutto questo funzioni correttamente è necessario garantire un preciso coordinamento tra l'hardware e quella parte di codice del sistema operativo che si occupa della gestione delle eccezioni.

Approfondimento. Piuttosto che pagare il costo di un passo in più di indirizzamento a ogni accesso a memoria, il Monitor della Memoria Virtuale (Paragrafo 5.6) mantiene una *tabella delle pagine ombra* (*shadow page table*) che mappa direttamente gli indirizzi dallo spazio virtuale dei sistemi ospitati negli indirizzi fisici dell'hardware.

Intercettando tutte le modifiche alla tabella delle pagine dei sistemi ospitati, il VMM è in grado di assicurare che il contenuto della tabella delle pagine ombra utilizzato dall'hardware per la mappatura corrisponda al contenuto delle pagine del SO ospitato, con la differenza che il numero della pagina reale è sostituito dal numero di pagina fisica nelle tabelle delle pagine dei SO ospitati. Quindi, il VMM deve sollevare

un'eccezione di trap a ogni tentativo di un SO ospitato di modificare la sua tabella delle pagine o di accedere al puntatore alla tabella delle pagine. Questo viene di solito ottenuto proteggendo dalla scrittura le tabelle delle pagine dei sistemi ospitati e impedendo ai SO ospitati l'accesso al puntatore alla tabella delle pagine. Come notato in precedenza, la seconda condizione è ottenuta naturalmente se l'accesso al puntatore della tabella delle pagine è un'operazione privilegiata.

Approfondimento. Oltre alla virtualizzazione dell'insieme di istruzioni, un'altra sfida per una macchina virtuale è rappresentata dalla virtualizzazione della memoria virtuale, dato che ciascun SO ospitato in ogni VM gestisce le proprie tabelle delle pagine. Per fare funzionare il tutto, il VMM distingue tra *memoria reale* e *memoria fisica*, anche se queste sono spesso trattate come se fossero la stessa cosa, e rende la memoria reale un livello intermedio separato tra la memoria virtuale e la memoria fisica; alcuni chiamano questi tre livelli: *memoria virtuale*, *memoria fisica* e *memoria macchina*.

Il SO ospitato mappa la memoria virtuale sulla memoria reale attraverso le proprie tabelle delle pagine e il VMM mappa la memoria reale del sistema ospitato sulla memoria fisica attraverso la tabella delle pagine del VMM. L'architettura della memoria virtuale può essere definita attraverso la tabella delle pagine, come nell'IBM VM/370, nell'x86 e nel RISC-V.

Riepilogo

"Memoria virtuale" è il nome che viene dato al livello della gerarchia delle memorie che si occupa del trasferimento dei dati tra la memoria principale e il disco. La memoria virtuale permette ai singoli programmi di espandere il proprio spazio di indirizzamento oltre i limiti della memoria principale. Caratteristica ancora più importante è il fatto che la memoria virtuale supporta la condivisione della memoria principale tra più processi attivi simultaneamente, in modo protetto.

La gestione della coppia di memorie costituita da memoria principale e disco, all'interno della gerarchia delle memorie è molto complessa per l'alto costo dei page fault. Per ridurre la frequenza delle miss sono state introdotte alcune tecniche:

1. Vengono definite pagine di grandi dimensioni per trarre vantaggio dalla località spaziale e per ridurre così la frequenza delle miss;
2. La mappatura tra indirizzi virtuali e indirizzi fisici, contenuta nella tabella delle pagine, viene resa completamente associativa, in modo che una pagina virtuale possa essere scritta in una pagina qualsiasi della memoria principale;
3. Il sistema operativo utilizza tecniche come LRU e il bit di utilizzo per scegliere quale pagina sostituire.

Le scritture su disco sono costose; per questo motivo la memoria virtuale utilizza uno schema write-back e tiene anche traccia delle pagine che non sono state modificate, attraverso il dirty bit, per evitare di salvarle su disco.

Il meccanismo della memoria virtuale fornisce la traduzione degli indirizzi dallo spazio di indirizzamento virtuale utilizzato dal programma allo spazio degli indirizzi fisici utilizzati per accedere alla memoria principale. Tale traduzione permette la condivisione protetta della memoria principale e offre altri vantaggi, tra cui la semplificazione dell'allocazione della memoria. Per garantire che i processi siano reciprocamente protetti, è necessario che solo il sistema operativo possa modificare la traduzione degli indirizzi. Ciò viene ottenuto impedendo ai programmi utente di modificare le tabelle delle pagine.

Si può realizzare una condivisione controllata delle pagine tra diversi processi con l'aiuto del sistema operativo e dei bit di accesso contenuti nella tabella

delle pagine, i quali indicano se il programma utente abbia diritto di lettura o scrittura su una pagina.
Se il processore dovesse accedere a una tabella delle pagine residente in memoria per tradurre ogni accesso, il carico di lavoro introdotto dalla memoria virtuale sarebbe troppo elevato e l'utilizzo delle cache risulterebbe inutile. Per questo motivo si utilizza un TLB, che agisce come una cache e memorizza alcune delle mappature contenute nella tabella delle pagine. Gli indirizzi vengono quindi tradotti da virtuali a fisici dal TLB.

Le cache, la memoria virtuale e il TLB sono caratterizzati dallo stesso insieme di princìpi e modalità di gestione; nel prossimo paragrafo descriveremo tale visione unificata.

Capire le prestazioni dei programmi

Sebbene la memoria virtuale sia stata inventata per consentire a una memoria di piccole dimensioni di comportarsi come una memoria più grande, la differenza di prestazioni tra disco e memoria implica che, se un programma accede spesso alla memoria virtuale invece che alla memoria fisica, verrà eseguito molto lentamente. Tale programma scambierà continuamente le pagine tra memoria e disco, generando un fenomeno che prende il nome di *thrashing*.

Il thrashing comporta prestazioni disastrose, ma per fortuna si verifica molto raramente. Se un programma incorre nel thrashing, la soluzione più semplice è eseguirlo su un calcolatore dotato di molta memoria oppure acquistare della memoria aggiuntiva. Una soluzione più complessa consiste nel riesaminare l'algoritmo e le strutture dati del programma per vedere se è possibile modificare la definizione dei dati e delle funzioni con lo scopo di aumentare la località e ridurre quindi il numero di pagine che vengono utilizzate simultaneamente dal programma. Questo insieme di pagine viene informalmente chiamato *insieme di lavoro* (*working set*).

Un problema più comune che affligge le prestazioni è quello delle miss del TLB. Dato che il TLB può gestire solo 32–64 pagine alla volta, un programma può facilmente avere una frequenza di miss del TLB elevata, dato che il processore può accedere direttamente al massimo a un quarto di mebibyte (64 × 4 KiB = 0,25 MiB). Per esempio, le miss nel TLB costituiscono un grosso problema per l'algoritmo Radix Sort. Per cercare di risolverlo, molte architetture di calcolatori supportano una dimensione della pagina variabile. Per esempio, in aggiunta alla dimensione minima di pagina di 4 KiB, l'hardware RISC-V supporta pagine di dimensioni di 2 MiB e 1 GiB. Quindi, se un programma utilizza una dimensione di pagina più grande, può accedere direttamente a una porzione di memoria maggiore senza incorrere in una miss del TLB.

La sfida, dal punto di vista pratico, è fare in modo che il sistema operativo riesca a far selezionare ai programmi pagine di dimensioni variabili. I progettisti dei sistemi operativi sono preoccupati della frammentazione della memoria principale con pagine più grandi quando un server deve eseguire centinaia di processi. Ancora una volta, la soluzione più complessa per ridurre le miss del TLB consiste nel riesaminare l'algoritmo e le strutture dati del programma, con lo scopo di ridurre il numero di pagine che costituiscono l'insieme di lavoro. Data l'importanza delle prestazioni degli accessi a memoria e della frequenza delle miss del TLB, alcuni programmi caratterizzati da insiemi di lavoro di grandi dimensioni sono stati ristrutturati per raggiungere questo obiettivo.

Approfondimento. Il RISC-V supporta pagine di dimensioni più ampie attraverso la tabella delle pagine multilivello della Figura 5.28. Inoltre, per puntare alla tabella delle pagine successiva nei livelli 1 e 2, utilizza una *traduzione "superpage"* per mappare l'indirizzo virtuale su un indirizzo fisico di 1 GiB (se la traduzione del blocco è al livello 1) o su un indirizzo fisico di 2 MiB (se la traduzione è al livello 2). Linus Torvolds canta le virtù delle pagine da 4 KiB a questo indirizzo: https://yarchive.net/comp/linux/page_sizes.html.

5.8 Schema comune per le gerarchie delle memorie

Finora abbiamo visto come i diversi tipi di gerarchie delle memorie abbiano molte caratteristiche in comune. Sebbene molti aspetti differiscano dal punto di vista quantitativo, molte delle caratteristiche che definiscono il funzionamento della gerarchia sono simili da un punto di vista qualitativo. La Figura 5.33 mostra gli intervalli di variazione di alcuni parametri caratteristici delle gerarchie. In questo paragrafo esamineremo le soluzioni operative più comuni adottate per le gerarchie delle memorie e come queste ne determinino il comportamento. Esamineremo queste strategie attraverso quattro domande che possono essere formulate per ogni coppia di livelli di una gerarchia delle memorie, anche se, per semplicità, adotteremo principalmente la terminologia utilizzata per le cache.

Domanda 1: dove può essere posizionato un blocco?

Abbiamo visto che si può utilizzare una vasta gamma di schemi per identificare un blocco nel livello superiore della gerarchia: dalla mappatura diretta a quella set-associativa, a quella completamente associativa. Come si è detto in precedenza, i diversi schemi possono essere visti come una variazione dello schema set-associativo, nel quale il numero di linee e il numero di blocchi contenuti in una linea variano:

Nome dello schema	Numero di linee	Blocchi per linea
Mappatura diretta	Numero di blocchi della cache	1
Set-associativo	$\frac{\text{Numero di blocchi della cache}}{\text{Grado di associatività}}$	Grado di associatività (tipicamente 2–16)
Completamente associativo	1	Numero di blocchi della cache

Il vantaggio che deriva dall'aumento del grado di associatività è che generalmente diminuisce la frequenza delle miss, perché si riduce il numero di miss dovute agli accessi che competono per la stessa locazione. Esamineremo questo aspetto in maggior dettaglio tra breve. Ci focalizziamo ora sul guadagno dovuto all'aumento di associatività: la Figura 5.34 mostra la frequenza di miss al variare del grado di associatività, da quello di una cache a mappatura diretta a quello di una cache set-associativa a otto vie. Il guadagno maggiore si

Caratteristica	Valori tipici per una cache L1	Valori tipici per una cache L2	Valori tipici per una memoria a pagine	Valori tipici per un TLB
Dimensione totale in blocchi	250-2000	2500-25 000	16 000–250 000	40–1024
Dimensione totale in kilobyte	16-64	125-2000	1 000 000–1 000 000 000	0,25–16
Dimensione del blocco in byte	16-64	64-128	4000–64 000	4–32
Penalità di miss in cicli di clock	10-25	100-1000	10 000 000–100 000 000	10–1000
Frequenza di miss (globale per L2)	2-5%	0,1-2%	0,00001–0,0001%	0,01–2%

Figura 5.33 Intervallo dei parametri caratterizzanti i principali livelli di memoria di una gerarchia delle memorie di un calcolatore. Questi sono i tipici valori dei parametri riferiti all'anno 2020. Si noti come il campo di variazione dei parametri sia ampio; ciò è dovuto in parte al fatto che molti dei parametri, che sono cambiati nel tempo, sono tra loro correlati; per esempio, al crescere delle dimensioni delle cache, per evitare penalità di miss più alte, anche la dimensione dei blocchi deve aumentare. Anche se non mostrato in tabella, i microprocessori utilizzati nei server hanno anche una cache L3, di dimensioni variabili tra 4 e 50 MiB, che può contenere molti più blocchi della cache L1 o L2. Le cache L3 abbassano la penalità di miss a 30–40 cicli di clock.

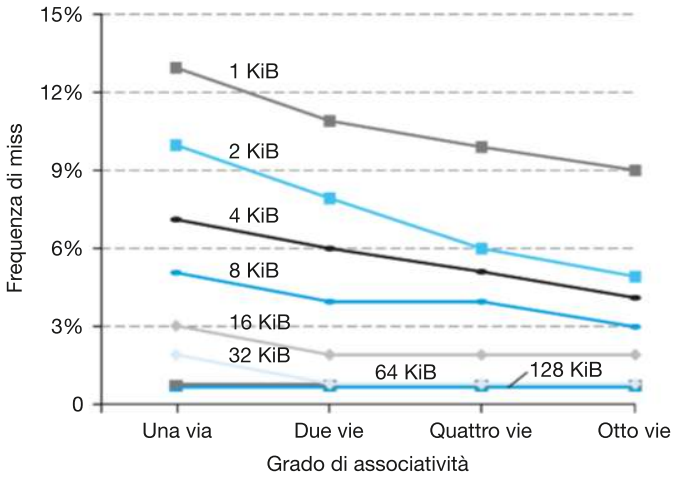


Figura 5.34 La frequenza delle miss sui dati di una cache per ognuna delle otto diverse dimensioni considerate diminuisce al crescere del grado di associatività. Mentre il beneficio ottenuto passando da una cache set-associativa a una via (a mappatura diretta) a una cache a due vie è significativo, il miglioramento ottenuto con un incremento ulteriore del grado di associatività è ridotto. Per esempio, il miglioramento nel passaggio da due a quattro vie è compreso tra l'1 e il 10%, contro il 20–30% di miglioramento nel passaggio da una a due vie. Il miglioramento è ancora inferiore passando da quattro a otto vie, dove ci si avvicina molto alla frequenza di miss di una cache completamente associativa. Cache di dimensioni ridotte ottengono un miglioramento della frequenza di miss assoluta maggiore all'aumentare del grado di associatività, perché la frequenza di base delle miss di una cache piccola è più elevata. La didascalia della Figura 5.16 spiega in che modo sono stati raccolti questi dati.

ottiene passando da una cache a mappatura diretta a una cache set-associativa a due vie: questo passaggio produce una riduzione della frequenza di miss compresa tra il 20 e il 30%. Al crescere della dimensione della cache, il miglioramento relativo aumenta di poco con l'aumento del grado di associatività della cache: dato che è minore la frequenza globale delle miss in una cache di dimensioni maggiori, l'opportunità di migliorare ulteriormente la frequenza di miss diminuisce e il miglioramento assoluto della frequenza di miss dovuto al grado di associatività si riduce significativamente. I potenziali svantaggi dell'aumento di associatività, come già menzionato, sono l'incremento del costo e dei tempi di accesso.

Domanda 2: come si individua un blocco?

La scelta di come individuare un blocco dipende dallo schema utilizzato per la disposizione dei blocchi, poiché esso definisce il numero di possibili locazioni. Possiamo riassumere gli schemi come segue:

Associatività	Metodo di ricerca	Numero di confronti richiesti
Mappatura diretta	Indice	1
Set-associativa	Indice per la linea, ricerca tra gli elementi della linea	Grado di associatività
Completamente associativa	Ricerca in tutti gli elementi all'interno della cache	Dimensione della cache
	Tramite una tabella di lookup separata	0

In ogni gerarchia delle memorie la scelta tra mappatura diretta, set-associativa o completamente associativa dipende dal confronto tra il costo delle miss e

il costo dell'implementazione dell'associatività, sia in termini di tempo sia in termini di hardware aggiuntivo. Includere la cache L2 all'interno del chip del processore permette di utilizzare un grado di associatività molto maggiore, poiché il tempo di hit non è più così critico e il progettista non deve più utilizzare i chip standard di SRAM come blocchi di base. Le cache completamente associative sono proibitive eccetto che per piccole dimensioni: in questo caso, il costo dei comparatori non è così determinante e il miglioramento della frequenza assoluta di miss è massimo.

Nei sistemi di memoria virtuale si utilizza una tabella separata, detta tabella delle pagine, per indicizzare la memoria. Oltre allo spazio richiesto per memorizzarla, la tabella delle pagine richiede un doppio accesso alla memoria. La scelta di associatività totale per la tabella delle pagine e l'utilizzo di una tabella aggiuntiva sono motivati da questi tre aspetti:

1. L'associatività totale è un beneficio, essendo le miss molto dispendiose;
2. L'associatività totale permette al software di utilizzare schemi di sostituzione molto raffinati, progettati al fine di ridurre la frequenza di miss;
3. L'intera tabella può essere facilmente indicizzata senza hardware aggiuntivo e non serve implementare alcuna ricerca al suo interno.

Per questi motivi, i sistemi di memoria virtuale utilizzano di solito una disposizione delle pagine completamente associativa.

La disposizione set-associativa viene spesso utilizzata per le cache e per i TLB, nei quali l'accesso richiede l'indicizzazione unita alla ricerca all'interno di un piccolo insieme. Alcuni sistemi impiegano cache a mappatura diretta per i vantaggi che ne derivano in termini di tempo d'accesso e semplicità. Il vantaggio relativo al tempo d'accesso è dovuto al fatto che per trovare il blocco richiesto non c'è bisogno di alcun confronto. Tali scelte di progetto dipendono da molti dettagli dell'implementazione hardware, per esempio dal fatto che la cache sia contenuta nel chip del processore o meno, dalla tecnologia utilizzata per implementare la cache e dal ruolo critico che ha il tempo d'accesso alla cache nel determinare il periodo di clock del processore.

Domanda 3: quale blocco deve essere sostituito in caso di miss della cache?

Quando si verifica una miss in una cache associativa, bisogna decidere quale blocco sostituire. In una cache completamente associativa ogni blocco è un potenziale candidato alla sostituzione, mentre se la cache è set-associativa bisogna scegliere tra i blocchi contenuti sulla stessa linea. Ovviamente la scelta diventa facile in una cache a mappatura diretta, perché c'è solo un candidato.

Sono due le principali strategie utilizzate per la scelta del blocco da sostituire nelle cache set-associative o completamente associative:

- *casuale:* il blocco è scelto a caso, eventualmente utilizzando il supporto di alcuni componenti hardware.
- *blocco utilizzato meno di recente* (LRU): il blocco prescelto è quello che è rimasto inutilizzato da più tempo.

Nella pratica la politica LRU è troppo costosa da implementare per le memorie che hanno un grado di associatività non basso (tipicamente da due a quattro), poiché è costoso tenere traccia dell'utilizzo dei diversi blocchi. Anche negli schemi set-associativi a quattro vie la strategia LRU viene spesso approssimata, per esempio suddividendo le linee in due coppie di blocchi, tenendo traccia della coppia che è stata utilizzata meno di recente (serve un solo bit) e memorizzando poi il blocco nella coppia che è stato a sua volta utilizzato meno di recente (anche questa operazione richiede un solo bit per ogni coppia).

Quando il grado di associatività cresce, la strategia LRU viene sempre approssimata oppure si preferisce una scelta casuale del blocco. Nelle cache l'algoritmo di sostituzione è implementato in hardware, il che significa che l'algoritmo deve poter essere realizzato in maniera semplice. La sostituzione casuale è facile da realizzare in hardware, e in una cache set-associativa a due vie la sostituzione casuale produce una frequenza di miss pari a circa 1,1 volte quella ottenuta con la strategia LRU. Al crescere delle dimensioni della cache, la frequenza delle miss diminuisce con entrambi gli approcci e la differenza in termini assoluti diventa minima; la sostituzione casuale può certe volte dare risultati migliori delle semplici approssimazioni della politica LRU che possono essere facilmente implementate in hardware.

Per la memoria virtuale si adotta sempre una politica di tipo LRU approssimata, dato che anche una minima riduzione della frequenza di miss può essere importante (essendo così alto il costo di una miss). Il bit di utilizzo o modalità equivalenti di verifica dell'uso di una pagina vengono spesso impiegati per aiutare il sistema operativo a tenere traccia delle pagine richieste meno di recente. Poiché le miss sono così costose e relativamente infrequenti, approssimare queste informazioni principalmente via software è considerato accettabile.

Domanda 4: come vengono gestite le scritture?

Una caratteristica fondamentale di tutte le gerarchie delle memorie è la gestione della scrittura. Abbiamo già visto le due opzioni di base:

- *write-through*: l'informazione viene scritta sia nel blocco della cache sia nel blocco del livello inferiore della gerarchia delle memorie, che per la cache è la memoria principale (le cache presentate nel Paragrafo 5.3 adottavano questo schema);
- *write-back*: l'informazione viene scritta solo nel blocco della cache; i blocchi che sono stati modificati vengono poi scritti nel livello inferiore della gerarchia solo quando devono essere sostituiti. I sistemi di memoria virtuale utilizzano sempre lo schema write-back per le ragioni illustrate nel Paragrafo 5.7.

Entrambi gli schemi hanno vantaggi e svantaggi. I vantaggi principali della modalità write-back sono i seguenti:

- le singole parole possono essere scritte dal processore alla frequenza con cui la cache, e non la memoria principale, è in grado di scriverle;
- scritture multiple all'interno dello stesso blocco richiedono una sola scrittura nel livello inferiore della gerarchia;
- quando i blocchi vengono scritti nel livello inferiore della gerarchia, il sistema può sfruttare appieno la banda molto larga del trasferimento, dato che viene trasferito un intero blocco.

I vantaggi principali della modalità write-through sono i seguenti:

- le miss sono più semplici da gestire e meno costose, perché non richiedono mai la scrittura di un blocco intero nel livello inferiore;
- lo schema write-through è più facile da implementare dello schema di write-back anche se, per essere efficace, una cache con write-through deve essere dotata anche di un buffer di scrittura.

Nei sistemi di memoria virtuale l'unica modalità praticabile è quella di write-back a causa della lunga latenza richiesta dall'operazione di scrittura nel livello inferiore (disco). La frequenza con cui vengono generate le scritture da parte del processore, in genere, supera la frequenza con cui il sistema di memoria può gestirle, anche quando la memoria è più larga sia nella dimensione

fisica che in quella logica, e viene utilizzata la modalità di trasferimento a raffica. Di conseguenza, attualmente le cache del livello inferiore della gerarchia utilizzano solitamente la strategia write-back.

QUADRO D'INSIEME

Anche se cache, TLB e memoria virtuale possono inizialmente apparire fra loro estremamente diversi, si basano sui medesimi due principi di località. Il loro funzionamento si può comprendere rispondendo alle seguenti quattro domande:

Domanda 1: Dove può essere posizionato un blocco?
Risposta: In una sola posizione (mappatura diretta), in poche posizioni (set-associatività) o in una qualsiasi posizione (associatività completa).

Domanda 2: Come si individua un blocco?
Risposta: Sono disponibili quattro metodi: con indice (come in una cache a mappatura diretta), con una ricerca limitata (come in una cache set-associativa), con una ricerca completa (come in una cache completamente associativa) o tramite una tabella di lookup separata (come nella tabella delle pagine).

Domanda 3: Quale blocco deve essere sostituito in caso di miss?
Risposta: Di solito il blocco utilizzato meno di recente oppure un blocco scelto a caso.

Domanda 4: Come vengono gestite le scritture?
Risposta: Ogni livello della gerarchia può utilizzare lo schema write-through oppure lo schema write-back. ■

Le tre C: un modello intuitivo per comprendere il comportamento delle gerarchie delle memorie

Modello delle tre C: un modello di cache in cui tutte le miss vengono classificate in base alle seguenti tre categorie: miss obbligate, miss di capacità e miss di conflitto.

In questo paragrafo esamineremo un modello che aiuta a comprendere la causa delle miss in una gerarchia delle memorie e le conseguenze sulle miss di eventuali modifiche alla gerarchia. Spiegheremo i diversi concetti facendo riferimento alle cache, ma gli stessi concetti si possono estendere a tutti gli altri livelli della gerarchia. In questo modello tutte le miss vengono classificate in base alle tre seguenti categorie (**modello delle tre C**):

- **miss obbligate** (*compulsory misses*): queste miss sono causate dal primo accesso a un blocco della cache che non era mai stato caricato in precedenza; sono dette anche **miss da partenza a freddo** (*cold start misses*);
- **miss di capacità** (*capacity misses*): queste miss sono generate quando la cache non è in grado di contenere tutti i blocchi di memoria necessari durante l'esecuzione di un programma; si verificano quando un blocco viene sostituito per essere ricaricato più tardi;
- **miss di conflitto** (*conflict misses*): queste miss sono generate nelle cache a mappatura diretta o set-associative quando più blocchi competono per la stessa linea; possono essere eliminate utilizzando una cache completamente associativa della stessa dimensione. Queste miss sono anche dette **miss di collisione** (*collision misses*).

Miss obbligate: dette anche **miss da partenza a freddo**, sono le miss della cache causate dal primo accesso a un blocco che non era mai stato caricato in cache.

Miss di capacità: sono le miss che si verificano perché la cache, anche se fosse completamente associativa, non potrebbe contenere tutti i blocchi necessari a soddisfare le richieste.

Miss di conflitto o miss di collisione: indicano le miss che avvengono in una cache set-associativa o a mappatura diretta quando più blocchi competono per la stessa linea. Le miss di conflitto possono essere eliminate utilizzando una cache completamente associativa della stessa dimensione.

La **Figura 5.35** mostra che le miss possono essere suddivise in base a tre cause principali; l'incidenza di ciascuna causa può essere ridotta modificando alcuni

 © 978-88-08-19966-9

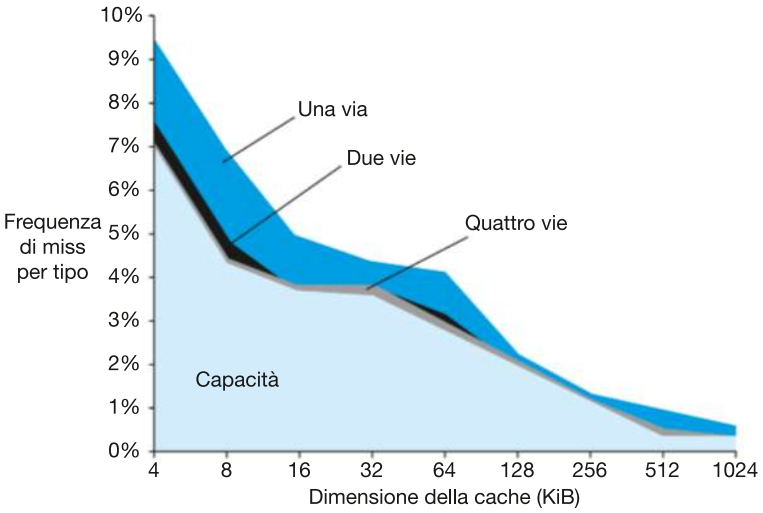


Figura 5.35 La frequenza delle miss può essere scomposta in tre diverse componenti. Questo grafico mostra la frequenza complessiva delle miss e le sue componenti in funzione della dimensione della cache. Questi dati sono relativi ai benchmark SPEC CPU2000 per numeri interi e in virgola mobile, e provengono dalla stessa fonte dei dati della Figura 5.34. Le miss obbligate sono pari allo 0,006% e non sono visibili in questo grafico. La seconda componente è rappresentata dalle miss di capacità, le quali dipendono dalla dimensione della cache. La frequenza delle miss di conflitto, che dipende sia dal grado di associatività sia dalle dimensioni della cache, viene mostrata per un grado di associatività che varia da una a otto vie: la parte del grafico etichettata corrisponde all'incremento della frequenza di miss quando il grado di associatività passa dal livello superiore al livello indicato dalla scritta. Per esempio, la parte del grafico più scura (*due vie*) indica la frequenza di miss in più che si ottiene quando la cache passa da un grado di associatività pari a quattro a un grado di associatività pari a due. Quindi, la differenza della frequenza di miss di una cache a mappatura diretta rispetto a quella di una cache completamente associativa della stessa dimensione è data dalla somma delle aree indicate come *quattro vie*, *due vie* e *una via*. La differenza tra otto vie e quattro vie è così piccola che è difficilmente visibile nel grafico.

aspetti del progetto della cache. Siccome le miss di conflitto nascono direttamente dalla competizione per lo stesso blocco della cache, incrementare il grado di associatività ne riduce l'incidenza. Tuttavia, l'aumento dell'associatività può causare un aumento del tempo di accesso, provocando una diminuzione delle prestazioni complessive.

Le miss di capacità si possono facilmente ridurre aumentando la dimensione della cache; infatti, la capacità delle cache di secondo livello negli ultimi anni è cresciuta con un tasso costante. Ovviamente, quando si progettano cache più grandi, occorre anche prestare attenzione all'aumento del tempo d'accesso, che può provocare una diminuzione delle prestazioni globali; per questo motivo la dimensione delle cache di primo livello è cresciuta con un tasso molto inferiore, o addirittura è rimasta invariata.

Dato che le miss obbligate sono provocate dal primo accesso a un blocco di memoria, il sistema migliore che si può adottare per ridurre questo tipo di miss consiste nell'aumentare la dimensione del blocco. Questo accorgimento permette di ridurre il numero di accessi ai blocchi di memoria che contengono un programma, visto che il programma sarà formato da un minor numero di blocchi. Come spiegato in precedenza, però, aumentare troppo la dimensione dei blocchi può avere un effetto negativo sulle prestazioni, perché determina anche un aumento della penalità di miss.

La suddivisione delle miss secondo il modello delle tre C è utile dal punto di vista qualitativo. Nella progettazione delle cache reali, molte delle scelte progettuali interagiscono tra loro e la modifica di una caratteristica della cache influenza spesso le altre caratteristiche, che a loro volta hanno conseguenze sulla frequenza delle miss. Nonostante questi limiti, il modello è utile per prevedere le prestazioni che potrà avere una cache.

Cambiamento nel progetto	Effetto sulla frequenza di miss	Eventuale effetto negativo sulle prestazioni
Aumento della dimensione della cache	Diminuiscono le miss di capacità	Può aumentare il tempo di accesso
Aumento del grado di associatività	Diminuisce la frequenza delle miss causate da conflitti	Può aumentare il tempo di accesso
Aumento della dimensione del blocco	Diminuisce la frequenza delle miss per un'ampia gamma di dimensioni dei blocchi a causa della località spaziale	Aumenta la penalità di miss. Blocchi molto grandi potrebbero aumentare la frequenza di miss

Figura 5.36 Problemi da affrontare nella progettazione delle memorie.

QUADRO D'INSIEME

La sfida nella progettazione di una gerarchia delle memorie consiste nel fatto che ogni modifica che può migliorare la frequenza di miss può anche influenzare in maniera negativa le prestazioni globali, come è qui riassunto nella Figura 5.36. Questa combinazione di effetti positivi e negativi rende interessante la progettazione di una gerarchia delle memorie. ■

Autovalutazione

Quali delle seguenti affermazioni (se ce ne sono) sono generalmente vere?

1. Non c'è modo di ridurre le miss obbligate.
2. Le cache completamente associative non hanno miss di conflitto.
3. Il grado di associatività è più importante della capacità se si vuole ridurre il numero di miss.

5.9 Come utilizzare una macchina a stati finiti per controllare una cache semplificata

Possiamo implementare ora l'unità di controllo di una cache, esattamente come abbiamo fatto nel Capitolo 4 per l'unità di controllo dell'unità di elaborazione a singolo ciclo di quella con pipeline. Questo paragrafo inizia con la definizione di una cache semplificata, per poi descrivere le macchine a stati finiti (FSM, *Finite-State Machines*) e terminare con la sintesi della FSM che controlla questa cache. Questo argomento viene approfondito nel Paragrafo 5.12 , in cui viene mostrata la cache e la sua unità di controllo in un nuovo linguaggio di descrizione dell'hardware.

Una cache semplificata

Ci accingiamo a progettare l'unità di controllo di una cache semplificata che ha le seguenti caratteristiche:

- cache a mappatura diretta;
- write-back con modalità write-allocate;
- dimensione del blocco di 4 parole (16 byte, ossia 128 bit);
- dimensione della cache di 16 KiB; la cache contiene, quindi, 1024 blocchi;
- indirizzi su 32 bit;
- presenza del bit di validità e del dirty bit per ogni blocco.

Da quanto visto nel Paragrafo 5.3, possiamo ora calcolare la dimensione dei campi che compongono l'indirizzo della cache:

- l'indice della cache è di 10 bit;
- l'offset di blocco è di 4 bit;
- la dimensione del tag è quindi di 32 – (10 + 4) = 18 bit.

I segnali tra il processore e la cache sono:

- segnale di lettura e scrittura su 1 bit ciascuno;
- segnale di validità su 1 bit: indica se si sta eseguendo un'operazione sulla cache o meno;
- indirizzi su 32 bit;
- dati trasferiti fra il processore e la cache su 32 bit;
- dati trasferiti fra la cache e il processore su 32 bit;
- segnale di cache ready (cache pronta) su 1 bit: segnala quando la cache ha terminato l'attività.

L'interfaccia tra memoria principale e cache ha gli stessi campi dell'interfaccia esistente tra cache e processore, fatta eccezione per il campo dati che qui è largo 128 bit. Questa maggiore larghezza della memoria si trova facilmente negli attuali microprocessori, che utilizzano parole di 32 o 64 bit nel processore, mentre i controllori delle DRAM gestiscono spesso dati su 128 bit. Rendere il blocco della cache largo quanto la larghezza della DRAM semplifica il progetto. Questi sono i segnali utilizzati tra cache e memoria principale:

- segnale di lettura e scrittura su 1 bit ciascuno;
- segnale di validità su 1 bit: indica se si sta eseguendo un'operazione sulla memoria o meno;
- indirizzi su 32 bit;
- dati trasferiti da cache a memoria su 128 bit;
- dati trasferiti da memoria a cache su 128 bit;
- segnale di cache ready (cache pronta) su 1 bit: indica se la memoria ha terminato il suo lavoro.

Si noti che l'interfaccia con la memoria non richiede un numero fisso di cicli di clock, ma si suppone che il controllore della memoria notifichi alla cache tramite il segnale di ready quando è terminata la lettura o la scrittura della memoria.

Prima di descrivere il controllore della cache, dobbiamo rivedere i concetti su cui si basano le macchine a stati finiti, che permettono di controllare operazioni che possono durare più cicli di clock.

Macchine a stati finiti

Per costruire l'unità di controllo dell'unità di elaborazione a singolo ciclo, abbiamo utilizzato una serie di tabelle della verità che specificano il valore che dovevano assumere i segnali di controllo in funzione del tipo di istruzione. Nel caso di una cache l'unità di controllo è più complessa, perché le operazioni di lettura e scrittura sono costituite da una successione di passi elementari. L'unità di controllo della cache deve sia impostare i segnali di controllo che vengono utilizzati a ogni passo sia determinare quale sarà il passo successivo.

Il metodo di controllo più diffuso per operazioni che si sviluppano su più passi è basato sulle **macchine a stati finiti**. Queste vengono di solito rappresentate mediante grafi: per definire una macchina a stati finiti occorre specificare un insieme di stati e per ogni stato bisogna dire quale sarà lo stato successivo. La scelta dello stato successivo viene effettuata da una particolare funzione,

Macchina a stati finiti: una macchina astratta che implementa una logica sequenziale. Essa è definita da un insieme di ingressi, di uscite e di stati, da una funzione stato prossimo che mappa gli ingressi e lo stato corrente in un nuovo stato e da una funzione di uscita che mappa lo stato corrente (ed eventualmente gli ingressi) in un insieme di segnali in uscita.

detta **funzione stato prossimo**, che mappa lo stato corrente e gli ingressi in quello successivo. Quando si utilizza una macchina a stati finiti per realizzare l'unità di controllo, occorre specificare anche l'insieme delle uscite che devono essere asserite in corrispondenza di ciascuno stato. L'implementazione circuitale di una macchina a stati finiti viene realizzata supponendo che quando un'uscita non viene esplicitamente asserita deve essere deasserita. In modo analogo, il funzionamento corretto dell'unità di elaborazione implica che un segnale non esplicitamente asserito deve essere deasserito.

Funzione stato prossimo: una funzione di logica combinatoria che determina lo stato successivo della macchina a partire dagli ingressi e dallo stato corrente.

Il segnale di controllo di un multiplexer funziona in modo leggermente diverso, poiché esso seleziona uno degli ingressi a seconda che assuma il valore 0 o 1. Perciò, in una macchina a stati finiti dobbiamo sempre impostare il segnale di controllo dei multiplexer che ci servono. Quando realizziamo una macchina a stati finiti con la logica digitale, impostare un segnale di controllo a 0 può essere la scelta di default e può quindi non richiedere porte logiche. Un semplice esempio di macchina a stati finiti è riportato nell'Appendice A; se non avete familiarità con i concetti su cui si basano le macchine a stati finiti, potete leggere l'Appendice A prima di proseguire con la lettura di questo paragrafo.

Una macchina a stati finiti può essere implementata con un registro temporaneo che mantiene lo stato corrente e un blocco di logica combinatoria che determina sia i segnali dell'unità di elaborazione che devono essere asseriti sia lo stato successivo. La Figura 5.37 mostra lo schema di una macchina a stati finiti e l'Appendice B descrive in dettaglio come si possa realizzare una macchina a stati finiti siffatta. Nel Paragrafo A.3 la logica di controllo combinatoria di una macchina a stati finiti viene implementata utilizzando una ROM (*Read-Only Memory*, memoria a sola lettura) o una PLA (*Programmable Logic Array*, matrice logica programmabile). *Vedi* l'Appendice A anche per una descrizione di questi componenti logici.

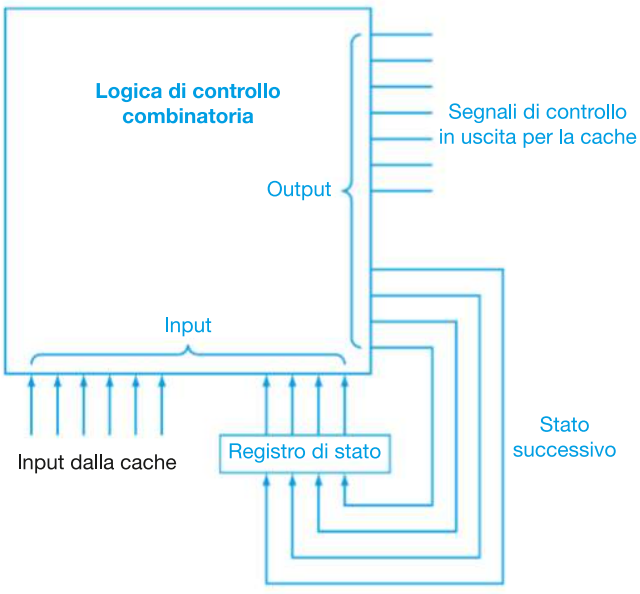


Figura 5.37 I controllori basati su macchine a stati finiti vengono tipicamente implementati utilizzando un blocco di logica combinatoria e un registro per salvare lo stato corrente (modello di Huffman). Le uscite della logica combinatoria sono lo stato successivo e i segnali di controllo che devono essere asseriti per ogni stato. Gli ingressi alla logica combinatoria sono lo stato corrente e tutti gli ingressi necessari per determinare lo stato successivo. Si noti che nella macchina a stati finiti utilizzata in questo capitolo le uscite dipendono solamente dallo stato corrente, non dagli ingressi. La seconda delle sezioni *Approfondimento* di pagina 422 spiega la macchina a stati finiti più in dettaglio.

 © 978-88-08-19966-9

Approfondimento. Si noti che la cache associata alla Figura 5.39 è una *cache bloccante*, cioè il processore deve attendere che la cache termini la sua richiesta. Il Paragrafo 5.12 descrive l'alternativa, rappresentata dalle *cache non bloccanti*.

Approfondimento. Il tipo di macchina a stati finiti utilizzato in questo libro è chiamato macchina di Moore, dal nome del suo inventore, Edward Moore. La sua caratteristica distintiva è che le uscite dipendono solamente dallo stato corrente. In una macchina di Moore il riquadro etichettato "Logica di controllo combinatoria" può essere suddiviso in due parti: una parte riceve solamente lo stato presente in ingresso e fornisce i segnali di controllo in uscita, mentre l'altra riceve in ingresso lo stato presente e gli ingressi e fornisce in uscita solamente lo stato successivo.

Un tipo alternativo di macchina a stati finiti è la macchina di Mealy, dal nome del suo inventore, George Mealy. La macchina di Mealy consente di utilizzare sia lo stato corrente sia gli ingressi per determinare le uscite. Le macchine di Moore hanno un vantaggio teorico in termini di velocità e dimensioni dell'unità di controllo: la maggiore velocità è dovuta al fatto che i segnali di controllo forniti in uscita, che devono essere utilizzati dall'unità di elaborazione all'inizio del ciclo di clock, non dipendono dagli ingressi ma solamente dallo stato corrente. Nell'Appendice A, dove lo schema della macchina a stati finiti viene tradotto in un circuito di porte logiche, il vantaggio in termini di dimensioni diventa chiaramente visibile. Lo svantaggio teorico di una macchina di Moore è dovuto al fatto che può richiedere un numero maggiore di stati. Per esempio, nel caso in cui due sequenze di stati differiscano per un solo stato, la macchina di Mealy può unificare gli stati rendendo le uscite dipendenti dagli ingressi.

FSM per il controllore semplificato della cache

La Figura 5.38 mostra i quattro stati del controllore della nostra cache semplificata.

- *Inattivo* (*idle*): questo è lo stato in cui si trova il controllore quando è in attesa di una richiesta valida di lettura o scrittura dal processore. Quando questa richiesta arriva, lo stato passa a *confronto dei tag*.

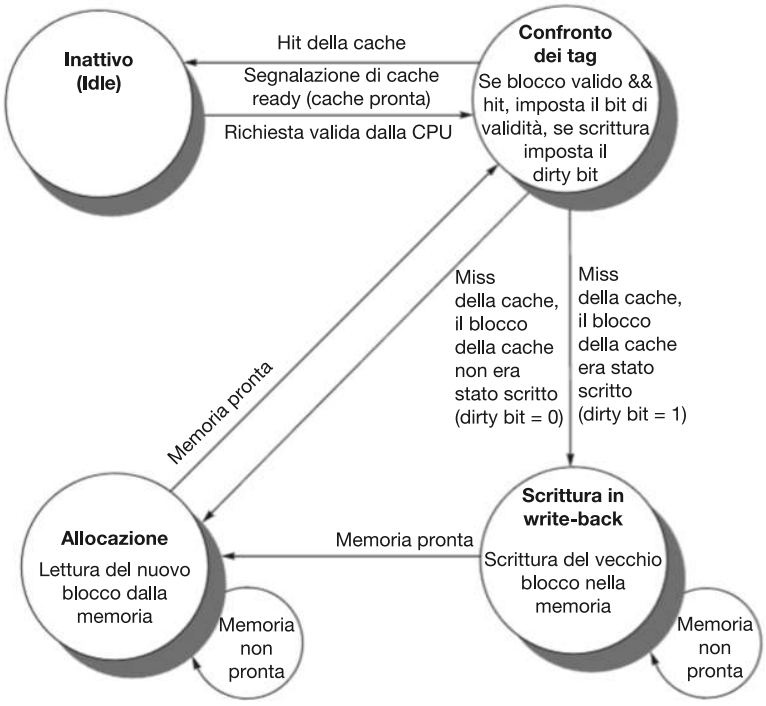


Figura 5.38 I quattro stati del controllore semplificato della cache.

- *Confronto dei tag*: come suggerito dal nome, in questo stato il controllore verifica se la richiesta di lettura o scrittura si risolve in una hit o in una miss: utilizza la parte dell'indirizzo riservata all'indice per selezionare la linea della cache che occorre analizzare. Se il bit di validità della linea è impostato a 1 e il campo tag della linea coincide con quello dell'indirizzo, si verifica una hit. In questo caso la parola contenente il dato viene letta se si tratta di una load o scritta se si tratta di una store e il bit *cache ready* viene impostato a 1. Se si tratta di una scrittura, occorre impostare a 1 anche il dirty bit. Si noti che in una hit in scrittura occorre anche impostare a 1 il bit di validità e scrivere il contenuto del campo tag; anche se a prima vista ciò sembrerebbe non necessario, occorre farlo perché i bit accessori vengono gestiti come un'unica struttura, per cui se si modifica il contenuto del dirty bit verrà modificato anche il contenuto del campo tag e del bit di validità. Se si verifica una hit e il blocco è valido, la FSM ritorna allo stato *inattivo*. Se invece si verifica una miss, per prima cosa si aggiorna il campo tag e quindi si passa o allo stato *scrittura in write-back* (se il blocco indirizzato ha il dirty bit impostato a 1), o allo stato *allocazione* (se il blocco indirizzato ha il dirty bit uguale a 0).
- *Scrittura in write-back*: in questo stato viene trasferito alla memoria il blocco di 128 bit della cache selezionato utilizzando l'indirizzo composto dal campo tag e dall'indice. Si rimane in questo stato in attesa del segnale di ready dalla memoria. Quando la memoria ha terminato la scrittura, la FSM passa allo stato *allocazione*.
- *Allocazione*: il nuovo blocco viene prelevato dalla memoria e si rimane in questo stato in attesa del segnale ready dalla memoria. Quando la memoria ha terminato la lettura, la FSM passa allo stato *confronto dei tag*. Per completare l'operazione saremmo potuti andare in uno stato diverso da *confronto dei tag*; tuttavia le operazioni da eseguire sono in gran parte le stesse, compreso l'aggiornamento della parola del blocco corretta, quando l'accesso viene effettuato in scrittura.

Questo semplice modello può essere facilmente esteso introducendo altri stati per cercare di migliorare le prestazioni. Per esempio, nello stato *confronto dei tag* il controllore confronta i campi tag e legge o scrive il dato in cache, tutto in un singolo ciclo di clock. Tuttavia, spesso, confronto e accesso ai dati della cache vengono eseguiti in stati separati per poter ridurre la durata del periodo del clock. Un'altra ottimizzazione consiste nell'aggiunta di un buffer di scrittura nel quale salvare il blocco se era stato modificato, prima di leggere il nuovo blocco; in questo modo, il processore non deve aspettare due accessi alla memoria quando si verifica una miss su un blocco modificato, ma la cache scriverà il dato nel buffer di scrittura mentre il processore continuerà con l'elaborazione dei dati che aveva richiesto.

Il Paragrafo 5.12 contiene ulteriori dettagli sulle FSM, mostrando il controllore completo definito in un linguaggio di descrizione dell'hardware e uno schema a blocchi di questa cache semplificata.

5.10 Parallelismo e gerarchie delle memorie: coerenza delle cache

Dato che i multiprocessori multicore contengono più processori sullo stesso chip, è verosimile che questi processori condividano lo stesso spazio di indirizzamento fisico. La scrittura in cache di dati condivisi introduce un nuovo problema: poiché ciascun processore vede la memoria attraverso la propria cache, senza opportune precauzioni due processori finirebbero per "vedere" allo stesso indirizzo due dati diversi. La **Figura 5.39** illustra questo problema e mostra

Passo	Evento	Contenuto della cache della CPU A	Contenuto della cache della CPU B	Contenuto della memoria nella locazione X
0				0
1	CPU A legge X	0		0
2	CPU B legge X	0	0	0
3	CPU A scrive 1 in X	1	0	1

Figura 5.39 Il problema della coerenza delle cache, illustrato con una locazione di memoria (X), che viene letta e scritta da due processori (A e B). Supponiamo inizialmente che nessuna delle due cache contenga dei dati e che la locazione X della memoria principale contenga il valore 0. Supponiamo anche che la cache sia di tipo write-through; una cache di tipo write-back aggiunge alcuni problemi, ma della stessa natura. Dopo che la CPU A ha scritto il valore 1 nella locazione X della memoria, la memoria e la cache A contengono entrambe il valore aggiornato, mentre la cache B contiene ancora 0: se B richiedesse il contenuto della locazione X, leggerebbe quindi il valore 0!

come due processori diversi possano avere dati diversi nella stessa posizione di memoria. Questo problema è chiamato *problema della coerenza della cache*.

In modo informale si può affermare che un sistema di memoria è coerente se la lettura di un dato restituisce il valore che è stato scritto più di recente in quella cella di memoria. Questa definizione, sebbene possa apparire intuitivamente chiara, è vaga e semplicistica; la realtà è molto più complessa. Nella definizione di coerenza sono contenuti due diversi aspetti del comportamento di un sistema di memoria, che sono entrambi critici per la scrittura in una memoria condivisa. Il primo aspetto, chiamato *coerenza*, definisce *quale valore* debba essere restituito da una lettura. Il secondo aspetto, chiamato *consistenza*, determina *quando* un valore che viene scritto in memoria possa essere restituito da una lettura.

Esaminiamo anzitutto la coerenza. Un sistema di memoria risulta coerente quando:

1. Un processore P legge una locazione X della memoria dopo che P stesso ha scritto nella locazione X, senza che altri processori abbiano scritto in X tra la scrittura e la lettura di P e il valore ottenuto è quello che è stato scritto da P; perciò nella Figura 5.39 la CPU A, dopo il passo 3, deve leggere da X il valore 1;
2. Un processore legge una locazione X della memoria dopo che un altro processore ha scritto in quella locazione X e il valore letto corrisponde al valore scritto dall'altro processore, se la lettura e la scrittura sono sufficientemente distanti nel tempo e non intervengono altre scritture nella locazione X tra l'accesso in scrittura e quello in lettura; perciò nella Figura 5.39 avremmo bisogno di un meccanismo che scriva 1 nella cache della CPU B al posto del valore 0 quando la CPU A scrive il valore 1 all'indirizzo X della memoria al passo 3;
3. Le scritture nella stessa locazione di memoria vengono *serializzate*, ossia due scritture nella stessa locazione fatte da due processori diversi devono essere viste nella stessa sequenza anche dagli altri processori; per esempio, se la CPU B scrivesse 2 nella locazione X della memoria dopo il passo 3, i processori non dovrebbero mai leggere dalla locazione X prima il valore 2 e poi il valore 1.

La prima proprietà semplicemente preserva l'ordine delle istruzioni e ci aspettiamo che sia soddisfatta nei processori a singolo core. La seconda proprietà definisce che cosa significhi avere una visione coerente della memoria: se un processore continuasse a leggere il vecchio contenuto della memoria, potremmo affermare con sicurezza che la memoria è incoerente.

La necessità di *serializzare le scritture* è più sottile, ma ugualmente importante. Supponiamo di non avere serializzato le scritture e che un processore P1 scriva nella locazione X e subito dopo un altro processore P2 scriva anch'esso nella locazione X. La serializzazione delle scritture garantisce che, prima o poi, ciascun processore possa vedere il dato scritto in memoria da P2. Se non serializzassimo le scritture, si potrebbe verificare la situazione in cui uno dei processori vede il dato scritto da P2 prima di vedere quello scritto da P1; in tal caso manterrebbe indefinitamente nella sua cache il dato scritto da P1. Il modo più semplice per evitare questo problema è assicurare che tutte le scritture nella stessa locazione di memoria siano viste nello stesso ordine; questa proprietà viene chiamata *serializzazione delle scritture*.

Schemi di base per garantire la coerenza

In un multiprocessore in cui viene garantita la coerenza, le cache devono consentire la *migrazione* e la *replicazione* dei dati condivisi.

- *Migrazione*: un dato può essere trasferito in una cache locale e utilizzato in quella cache in modo trasparente. La migrazione riduce sia la latenza di accesso ai dati condivisi allocati nella memoria remota sia la larghezza di banda richiesta dalla memoria condivisa.
- *Replicazione*: quando un dato condiviso viene letto simultaneamente da più processori, le cache fanno una copia del dato nella cache locale. La replicazione riduce sia la latenza di accesso sia la competizione per la lettura dei dati condivisi.

Il supporto alla migrazione e alla replicazione è critico per ottenere buone prestazioni nell'accesso ai dati condivisi, per cui molti multiprocessori introducono un protocollo hardware per mantenere la coerenza delle cache; questo protocollo viene chiamato *protocollo di coerenza delle cache*. L'elemento chiave per l'implementazione di questi protocolli è il tracciamento dello stato della condivisione di tutti i blocchi.

Il protocollo più diffuso che viene utilizzato per mantenere la coerenza della cache è chiamato *protocollo di snooping* (letteralmente, "spiare"). Ogni cache che contiene una copia dei dati contenuti in un blocco della memoria fisica ha anche una copia dello stato della condivisione di quel blocco; non viene mantenuta, invece, una copia centralizzata dello stato di condivisione del blocco. Le cache sono tutte accessibili attraverso un dispositivo di collegamento, un bus o una rete, e i controllori di tutte le cache controllano o "spiano" il traffico sul dispositivo di collegamento per determinare se uno dei propri blocchi sia richiesto da un altro dispositivo collegato al bus o alla rete.

Nel prossimo paragrafo spiegheremo come la coerenza delle cache venga implementata su un bus condiviso utilizzando lo snooping, ma qualsiasi altro dispositivo di comunicazione che trasmetta le miss di una cache a tutti i processori può essere utilizzato per implementare uno schema di coerenza basato su snooping. La trasmissione a tutte le cache rende semplici da implementare i protocolli di snooping, ma limita anche la loro scalabilità.

Protocolli di snooping

Un metodo per garantire la coerenza consiste nell'assicurare che il processore abbia accesso esclusivo a un dato prima di scriverlo. Questo tipo di protocollo viene chiamato *write invalidate protocol* (protocollo di invalidazione in scrittura), perché la copia di un dato contenuto nelle altre cache viene invalidata quando si verifica una scrittura di quel dato. L'accesso esclusivo assicura che non ci siano copie del dato disponibili né in lettura né in scrittura nel

Attività del processore	Attività del bus	Contenuto della cache della CPU A	Contenuto della cache della CPU B	Contenuto della locazione X della memoria
				0
CPU A legge X	Miss cache per X	0		0
CPU B legge X	Miss cache per X	0	0	0
CPU A scrive 1 in X	Invalidazione di X	1		0
CPU B legge X	Miss cache per X	1	1	1

Figura 5.40 Esempio di protocollo di invalidazione su un bus con snooping per l'accesso a un singolo blocco della cache, corrispondente alla locazione X della memoria, con una cache di tipo write-back. Si suppone che all'inizio nessuna delle due cache contenga il dato della locazione X della memoria principale e che X contenga inizialmente il valore 0. Il contenuto della CPU e della memoria viene mostrato dopo che il processore e il bus hanno entrambi terminato le loro attività. Lo spazio vuoto indica che non si è verificata alcuna attività o che non è stata scritta in cache alcuna copia del dato. Quando si verifica la seconda miss, relativa alla cache B, la CPU A risponde alla richiesta fornendo il dato e cancellando la richiesta della memoria. Inoltre, il contenuto sia della cache B sia della locazione X della memoria viene aggiornato. Questa modalità di aggiornamento della memoria, che si verifica quando un blocco viene condiviso, semplifica il protocollo; è comunque possibile tenere traccia del processore che aggiorna i dati e forzare la scrittura in memoria solamente quando il blocco deve essere sostituito. Ciò richiede l'aggiunta di un altro stato, chiamato *owner* (proprietario), che indica che un blocco può essere condiviso, ma che il processore proprietario è responsabile dell'aggiornamento della cache degli altri processori e della memoria quando modifica il contenuto del blocco o lo deve sostituire.

momento in cui il dato viene scritto; tutte le altre copie presenti nelle altre cache vengono invalidate.

La Figura 5.40 mostra un esempio di protocollo di invalidazione che funziona su un bus con snooping, con una cache di tipo write-back. Per vedere come questo protocollo assicuri la coerenza, si consideri la scrittura di un dato seguita dalla sua lettura da parte di un altro processore: poiché la scrittura richiede l'accesso al dato in modo esclusivo, deve essere invalidata la copia del dato contenuto nella cache dell'altro processore che deve effettuare la lettura del dato, da cui il nome del protocollo. Perciò, quando l'altro processore cerca di leggere il dato, viene generata una miss e la cache sarà costretta a prelevare la nuova copia dalla memoria principale. In caso di scrittura occorre che il processore abbia l'accesso esclusivo al dato e bisogna evitare che gli altri processori cerchino di scrivere lo stesso dato simultaneamente; solo uno dei processori può vincere l'eventuale competizione per la scrittura del dato (*data race*) e invaliderà la copia del dato contenuta nella cache degli altri processori. Perché possano completare la loro scrittura, gli altri processori devono ottenere prima una nuova copia del dato, che conterrà il valore aggiornato. Questo protocollo forza perciò anche la sequenzialità delle scritture.

Interfaccia hardware/software

Falsa condivisione: quando due variabili condivise, non in relazione tra loro, sono situate nello stesso blocco e l'intero blocco viene scambiato tra due processori, anche se questi, in realtà, accedono a variabili diverse.

Si può intuire che la dimensione dei blocchi della cache gioca un ruolo fondamentale nel meccanismo di coerenza. Per esempio, consideriamo il caso di snooping su una cache con blocchi della dimensione di otto parole e di due processori che alternativamente leggono e scrivono la stessa parola. Quasi tutti i protocolli scambiano interi blocchi tra processori e richiedono quindi una notevole larghezza di banda per mantenere la coerenza.

I blocchi di grandi dimensioni possono anche causare quella che viene definita **falsa condivisione**: quando due variabili condivise non in relazione tra loro sono situate nello stesso blocco della cache, l'intero blocco viene scambiato tra i processori, anche se questi, in realtà, stanno lavorando su variabili diverse. I programmatori e i compilatori devono stare molto attenti a come dispongono i dati per evitare false condivisioni.

Approfondimento. Sebbene le tre proprietà descritte all'inizio del Paragrafo 5.10 siano sufficienti ad assicurare la coerenza, sapere quando il valore di un dato scritto in cache verrà visto dagli altri processori è un aspetto altrettanto importante. Per capirne il motivo, notiamo che non si può pensare che il dato scritto in una locazione X da un processore sia immediatamente disponibile per la lettura (Figura 5.39). Per esempio, se un processore scrive nella locazione X poco prima che un altro processore legga in quella stessa locazione, può diventare impossibile assicurare che la lettura restituisca il valore corretto, perché al momento della lettura il dato può non aver lasciato ancora il processore. I *modelli di consistenza della memoria* si occupano di definire esattamente *quando* un valore scritto può essere visto e letto correttamente.

Formuliamo le due ipotesi seguenti. Innanzitutto, una scrittura non può essere considerata completata (consentendo quindi la scrittura successiva) finché tutti i processori non vedono il suo effetto. In secondo luogo, un processore non deve cambiare l'ordine delle scritture rispetto alla sequenza degli accessi alla memoria. Queste due condizioni implicano che se un processore scrive prima nella locazione X e poi nella locazione Y, i processori che vedono il nuovo valore di Y devono vedere anche il nuovo valore di X. Questi vincoli consentono ai processori di riordinare le letture, ma forzano i processori a completare le scritture nel giusto ordine.

Approfondimento. Dato che l'input può modificare la memoria che sta sotto le cache e che l'output può avere bisogno del valore più recente contenuto in una cache di tipo write-back, ci imbattiamo in un problema di coerenza della memoria anche tra l'I/O e la cache, anche nel caso di processore singolo, esattamente come tra le cache di un multiprocessore. Il problema della coerenza delle cache per i multiprocessori e per l'I/O (Capitolo 6), sebbene abbia la stessa origine, ha caratteristiche differenti, che fanno sì che la soluzione più appropriata per risolverlo possa essere diversa. A differenza dell'I/O, in cui la replicazione di uno stesso dato è un evento raro (da evitare il più possibile), un programma che viene eseguito su un sistema multiprocessore dispone normalmente di una copia degli stessi dati nelle diverse cache.

Approfondimento. Oltre al protocollo di coerenza delle cache basato su snooping, in cui lo stato dei blocchi condivisi è distribuito, esiste un protocollo di coerenza delle cache *basato su directory*, il quale mantiene lo stato della condivisione di ogni blocco della memoria fisica in un unico posto, chiamato *directory*. L'implementazione della coerenza basata su directory ha un costo leggermente superiore allo snooping, ma può ridurre il traffico tra le cache e può quindi essere utilizzata anche su sistemi costituiti da molti processori.

5.11 Parallelismo e gerarchie delle memorie: i dischi RAID

Questo paragrafo, disponibile online, descrive come utilizzando molti dischi assieme si possa ottenere un throughput maggiore, che è stata la motivazione che ha portato all'invenzione dei RAID (*Redundant Arrays of Inexpensive Disks*, insieme ridondante di dischi economici). La popolarità dei RAID, tuttavia, è dovuta all'affidabilità molto maggiore ottenuta introducendo un piccolo numero di dischi aggiuntivi. Questo paragrafo mostra le differenze in prestazioni, costi e **affidabilità** tra i livelli RAID.

5.12 Argomenti avanzati: come implementare i controllori delle cache

Questo paragrafo, disponibile online, mostra come implementare il controllore di una cache in modo del tutto simile a quanto abbiamo fatto nel Capitolo 4 per il controllore delle unità di elaborazione a singolo ciclo e con pipeline. Il paragrafo inizia con la descrizione delle macchine a stati finiti e

del controllore di una cache semplificata, e include la descrizione del controllore mediante un linguaggio di descrizione dell'hardware. Viene inoltre esaminato in dettaglio un esempio di protocollo che mantiene la coerenza delle cache, analizzando i problemi che possono sorgere nella sua implementazione.

5.13 Due casi reali: la gerarchia delle memorie del Cortex-A53 ARM e del Core i7 Intel

In questo paragrafo prenderemo in considerazione la gerarchia delle memorie degli stessi due microprocessori descritti nel Capitolo 4: il Cortex-A53 ARM e il Core i7 Intel. Questo paragrafo è basato sul Paragrafo 2.6 della sesta edizione del testo *Computer Architecture: A Quantitative Approach* degli stessi autori.

Il Cortex-A53 è un core configurabile che supporta l'architettura dell'insieme delle istruzioni ARMv8A, la quale comprende sia la modalità a 32 bit sia quella a 64 bit. Il Cortex A-53 è distribuito come un core IP (*Intellectual Property*, Proprietà Intellettuale) ed è utilizzato in un gran numero di tablet e smart phone. È stato progettato per essere particolarmente efficiente dal punto di vista energetico, un criterio fondamentale nei PMD che funzionano alimentati da una batteria, e, nei PMD di fascia alta, può essere configurato con più core per chip. La descrizione riportata in questo capitolo si concentra su un singolo core. Il Cortex A-53 può lanciare in esecuzione due istruzioni per ciclo di clock, a una frequenza di clock fino a 1,3 GHz.

L'i7 supporta l'architettura dell'insieme di istruzioni x86-64, un'estensione a 64 bit dell'architettura x86, ed è un processore con esecuzione fuori ordine che contiene quattro core. La descrizione riportata in questo capitolo si concentra sul sistema di memoria e sull'analisi delle prestazioni di un singolo core. Ciascun core dell'i7 può eseguire fino a quattro istruzioni x86 per ciclo di clock, utilizzando la pipeline a 16 stadi, a cammini di esecuzione multipli e schedulazione dinamica, che abbiamo descritto in dettaglio nel Capitolo 4. Il Core i7 supporta fino a tre canali di comunicazione con la memoria, ciascuno dei quali accede a un insieme distinto di DIMM e può trasferire i dati in parallelo. Utilizzando le memorie DDR3-1066, la larghezza di banda della memoria dell'i7 supera i 25 GB/s.

La **Figura 5.41** riporta la dimensione degli indirizzi e dei TLB dei due processori. L'A53 ha 3-TLB con uno spazio di indirizzamento virtuale e uno spazio di indirizzamento fisico, entrambi di 32 bit. Il Core i7 ha tre TLB con indirizzi virtuali su 48 bit e indirizzi fisici su 32 bit. Sebbene i registri a 64 bit del Core i7 possano contenere indirizzi virtuali più ampi, il software non ne ha bisogno e un indirizzo su 48 bit consente di ridurre la dimensione della memoria allocata per la tabella delle pagine e la dimensione dell'hardware del TLB.

La **Figura 5.42** mostra le cache di questi processori. Ciascun core possiede una cache L1 per le istruzioni e una cache L1 per i dati, entrambe contenenti 64 byte per blocco; questa cache è set-associativa a due vie per l'A53 e a otto vie per l'i7. La cache dati L1 del Core i7 contiene 32 KiB, mentre quella dell'A53 è configurabile e può contenere da 8 a 64 KiB. La cache istruzioni L1 di entrambi i processori è organizzata allo stesso modo: è una cache set-associativa a quattro vie, che contiene 32 KiB (per core).

Entrambi i processori utilizzano, per ogni core, una cache L2 set-associativa, unificata per istruzioni e dati, con blocchi di 64 byte; la dimensione della cache L2 dell'A53 può variare tra 128 KiB e 1 MiB, mentre la cache L2 del Core i7 ha una dimensione fissa, pari a 256 KiB. Dato che i Core i7 vengono utilizzati sui server, montano sul chip anche una cache L3, set-associativa

Caratteristica	Cortex-A53 ARM	Core i7 Intel
Indirizzo virtuale	48 bit	48 bit
Indirizzo fisico	40 bit	36 bit
Dimensione pagina	Variabile: 4, 16, 64 KiB, 1, 2 MiB, 1 GiB	Variabile: 4 KiB, 2/4 MiB
Organizzazione TLB	1 TLB per le istruzioni e 1 TLB per i dati	1 TLB per le istruzioni e 1 TLB per i dati per core
	Entrambi i micro-TLB L1 sono completamente associativi, con 10 elementi, sostituzione di tipo round robin (circolare) I TLB L2 sono unificati e sono set-associativi a quattro vie e contengono 512 elementi	Entrambi i TLB di L1 sono set-associativi a quattro vie, con sostituzione LRU
	Le miss dei TLB sono gestite in hardware	Il TLB Istruzioni di L1 ha 128 elementi per pagine piccole, 7 per ogni thread per pagine grandi
		Il TLB Dati ha 64 elementi per pagine piccole, 32 elementi per pagine grandi
		Il TLB di L2 è set associativo a quattro vie, con sostituzione LRU
		Il TLB L2 ha 512 elementi
		Le miss dei TLB sono gestite in hardware

Figura 5.41 Traduzione degli indirizzi e struttura hardware dei TLB del Cortex-A53 ARM e del Core i7 Intel. Entrambi i processori forniscono il supporto a pagine di grandi dimensioni, impiegate per esempio nella gestione del sistema operativo o nella mappatura della memoria video. Le pagine grandi evitano di utilizzare un elevato numero di elementi per mappare un singolo oggetto che è sempre presente in memoria.

Caratteristica	Cortex-A53 ARM	Core i7 Intel
Organizzazione cache L1	Cache separate per dati e istruzioni	Cache separate per dati e istruzioni
Dimensione cache L1	Configurabile, da 8 a 64 KiB ciascuna per istruzioni/dati	32 KiB per istruzioni/dati per ogni core
Grado associatività cache L1	Set-associativa a 2 vie (I), a 2 vie (D)	Set-associativa a 8 vie (I), a 8 vie (D)
Modalità sostituzione L1	Random	LRU approssimato
Dimensione blocco L1	64 byte	64 byte
Politica scrittura L1	Write-back, diverse politiche di allocazione (il default è write-allocate)	Write-back, no-write-allocate
Tempo hit L1 (utilizzo load)	Due cicli di clock	4 cicli di clock (pipeline)
Organizzazione cache L2	Un'unica cache per istruzioni e dati	Un'unica cache per istruzioni e dati
Dimensione cache L2	Da 128 KiB a 2 MiB	256 KiB (0,25 MiB)
Grado associatività cache L2	Set-associativa a 8 vie	Set-associativa a 4 vie
Modalità sostituzione L2	LRU approssimato	LRU approssimato
Dimensione blocco L2	64 byte	64 byte
Politica scrittura L2	Write-back, write-allocate	Write-back, write-allocate
Tempo hit L2	12 cicli di clock	12 cicli di clock
Organizzazione cache L3	–	Un'unica cache per istruzioni e dati
Dimensione cache L3	–	2 MiB / core, condivisa dai core
Grado associatività cache L3	–	Set-associativa a 16 vie
Modalità sostituzione L3	–	LRU approssimato
Dimensione blocco L3	–	64 byte
Politica scrittura L3	–	Write-back, write-allocate
Tempo hit L3	–	44 cicli di clock

Figura 5.42 Cache del Cortex-A53 ARM e del Core i7 Intel.

a 16 vie, unificata per istruzioni e dati, da 2 MiB per core, che è condivisa da tutti i core del chip.

Il Core i7 contiene delle ottimizzazioni ulteriori che consentono di ridurre la penalità di miss. La prima consiste nel restituire la parola che ha causato la miss prima di completare il trasferimento del blocco di memoria dal livello inferiore. Inoltre, può continuare a eseguire altre istruzioni anche se accedono alla cache dati mentre la gestione della miss è ancora in corso. Una cache di questo tipo, chiamata **cache non bloccante**, viene comunemente utilizzata dai progettisti per nascondere la latenza dovuta alle miss della cache durante l'esecuzione fuori ordine. Essi implementano due tipi di tecniche non bloccanti. La *hit sotto una miss* consente di ottenere altre hit quando la gestione della miss è in corso, mentre la *miss sotto una miss* consente di gestire miss multiple. Lo scopo della prima tecnica è di mascherare una parte della latenza dovuta a una miss permettendo al processore di svolgere altro lavoro, mentre lo scopo della seconda è di sovrapporre temporalmente la latenza di due diverse miss.

Cache non bloccante: una cache che permette al processore di effettuare accessi anche quando la cache sta gestendo una miss verificatasi precedentemente.

Per sovrapporre gran parte della latenza di miss per le miss multiple rilevanti, occorre disporre di un sistema di memoria con una larghezza di banda elevata, capace di gestire miss multiple in parallelo. Nei dispositivi mobili, il sistema di memoria sottostante è spesso in grado di organizzare gli accessi in pipeline, unire, riordinare e assegnare priorità diverse alle richieste in modo appropriato. I grandi server e i sistemi multiprocessori hanno spesso un sistema di memoria capace di gestire più di una miss rilevante in parallelo.

Il Core i7 possiede al suo interno un meccanismo hardware per il caricamento anticipato dei dati (pre-fetch). Questo meccanismo osserva la successione delle miss sui dati e utilizza questa informazione per cercare di predire il prossimo indirizzo da cui caricare il dato prima che la miss successiva venga generata. Tale sistema funziona generalmente in modo ottimale quando si accede a vettori all'interno di cicli. Nella maggior parte dei casi, la linea caricata in anticipo è semplicemente il blocco successivo della cache.

Le sofisticate gerarchie delle memorie di questi chip e la larga porzione di silicio dedicato alle cache e ai TLB illustrano chiaramente il grande sforzo dei progettisti per cercare di colmare il divario tra durata del ciclo di clock del processore e latenza della memoria.

Prestazioni della gerarchia delle memorie del Cortex-A53 e del Core i7

La gerarchia delle memorie del Cortex-A53 è stata valutata mediante i benchmark SPECInt2006 con 32 KiB di cache primaria e 1 MiB di cache L2. La frequenza di miss della cache istruzioni si è rivelata molto piccola, anche considerando solo la cache L1: prossima allo zero per la maggior parte dei benchmark e inferiore all'1% su tutti i benchmark. Questa frequenza di miss così bassa è probabilmente dovuta alla natura dei programmi del bechmark SPECCPU, che sono computazionalmente intensivi, e alla struttura a due vie della cache che consente di eliminare la maggior parte delle miss per conflitto.

La **Figura 5.43** riporta i risultati per la cache dati e mostra frequenze di miss L1 e L2 significative. La frequenza di miss della cache dati L1 varia di un fattore 75: va dallo 0,5% al 37,3%, con un valore medio del 6,4% e un valore mediano del 2,4%. La frequenza di miss (globale) della cache L2 varia di un fattore 180 ed è compresa tra lo 0,05% e il 9%, con una valore medio dell'1,3% e un valore mediano dello 0,3%. Utilizzando le penalità di miss riportate nella Figura 2.19, la **Figura 5.44** mostra la penalità media di accesso ai dati. Anche se la frequenza di miss della cache L1 è circa sette volte maggiore di quella della cache della cache L2, la penalità di miss di L2 è 9,5 volte maggiore, per cui

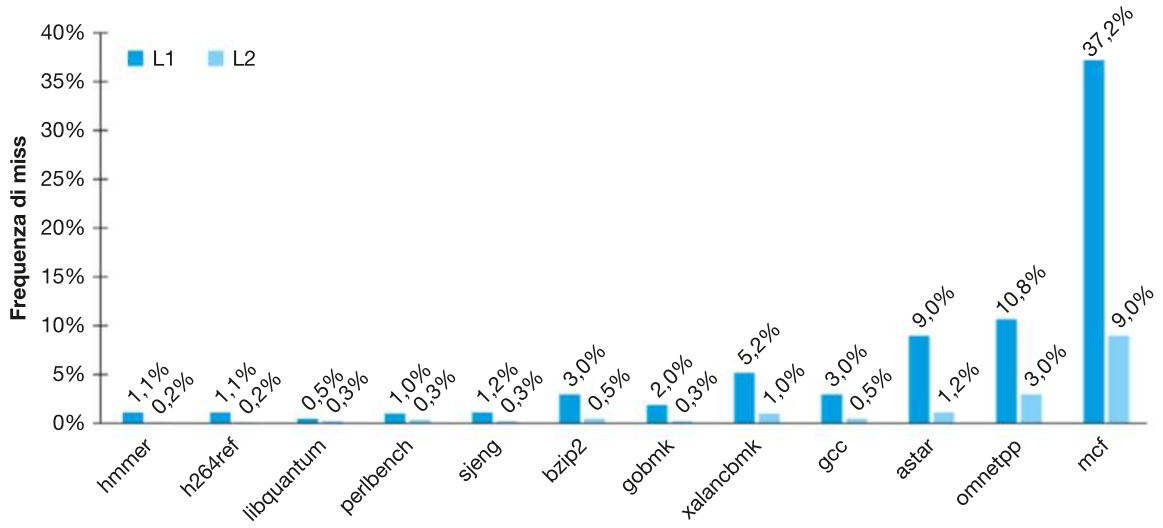


Figura 5.43 Frequenza di miss sui dati per un Cortex-A53 ARM con 32 KiB di cache L1 e una cache L2 da 1 MiB nell'esecuzione dei benchmark SPEC2006int è molto sensibile al programma utilizzato. Applicazioni che richiedono una maggiore occupazione di memoria tendono ad avere una frequenza di miss maggiore nelle cache L1 e L2. Si noti che la frequenza di miss nella cache L2 è una frequenza di miss globale, cioè si considerano tutti gli accessi alla memoria che hanno successo compresi quelli alla cache L1. Il programma mcf è noto come *cache buster* ("acchiappa cache").

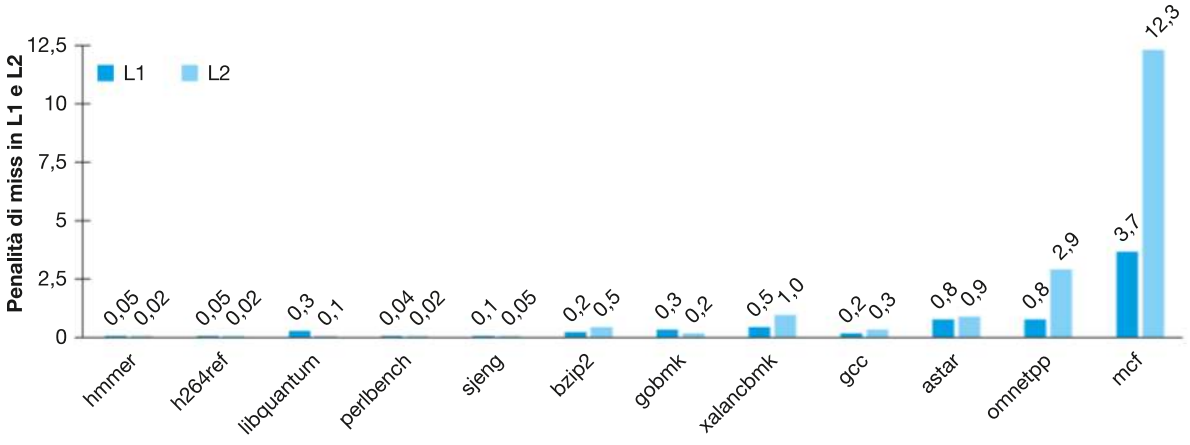


Figura 5.44 Penalità media di accesso alla memoria misurata in numero di cicli di clock per accesso alla memoria per gli accessi alla cache dati di primo (L1) e secondo livello (L2) di un processore Cortex A-53 nell'esecuzione dei benchmark SPEC2006int. Sebbene la frequenza di miss di L1 sia significativamente maggiore, la penalità di miss della cache L2, che è di oltre cinque volte maggiore, può dare un contributo significativo alla penalità di miss.

le miss di L2 risultano leggermente più costose nei benchmark che stressano il sistema di memoria.

L'unità di fetch delle istruzioni dell'i7 cerca di prelevare 16 byte a ogni ciclo di clock. Questo complica il confronto tra le frequenze di miss sulle istruzioni, poiché vengono prelevate più istruzioni per ogni ciclo di clock (circa 4,5 istruzioni in media). La cache istruzioni set-associativa o otto vie da 32 KiB porta ad avere una frequenza di miss molto bassa per i programmi dei benchmark SPECin2006: le frequenze di miss delle istruzioni sono tipicamente inferiori all'1%. La frequenza con cui viene messa in stallo l'unità di fetch, in attesa che vengano risolte le mss sulla cache istruzioni, è altrettanto piccola.

5.15 Come andare più veloci: blocchi di cache e moltiplicazione tra matrici

Il passo successivo per aumentare le prestazioni della funzione DGEMM adattandola progressivamente all'hardware a disposizione, consiste nell'utilizzo di blocchi di cache adeguati, da aggiungere alla parallelizzazione a livello di parola e di istruzioni, viste nei Capitoli 3 e 4. La **Figura 5.48** mostra la versione di DGEMM riportata nella Figura 4.82 modificata per sfruttare la gestione a blocchi della cache. Le modifiche sono le stesse apportate in precedenza quando siamo passati dalla versione DGEMM non ottimizzata della Figura 3.19 alla versione a blocchi della Figura 5.21. Questa volta prendiamo come base la versione di DGEMM utilizzata nel Capitolo 4, in cui i cicli sono stati espansi e la richiamiamo più volte sulle sottomatrici di `A`, `B` e `C`. Le linee 25–31 e 7–8 della Figura 5.48 sono uguali alle linee 14–20 e 5–6 della Figura 5.21, a eccezione dell'incremento dell'indice di fine ciclo alla linea 7, pari al fattore di espansione del ciclo.

Il beneficio dell'utilizzo di una cache a blocchi aumenta con la dimensione delle matrici. Dato che il numero di operazioni in virgola mobile eseguite su ciascun elemento delle matrici è lo stesso, indipendentemente dalle dimensioni delle metrici, possiamo ragionevolmente misurare le prestazioni attraverso il

```
 1 #include <x86intrin.h>
 2 #define ESPANDI (4)
 3 #define DIM_BLOCCO 32
 4 void esegui_blocco (int n, int si, int sj, int sk,
 5                double *A, double *B, double *C)
 6 {
 7   for (int i = si; i < si+DIM_BLOCCO; i+=ESPANDI*8)
 8     for (int j = sj; j < sj+DIM_BLOCCO; j++) {
 9       __m512d c[ESPANDI];
10       for (int r = 0; r < ESPANDI; r++)
11         c[r] = _mm512_load_pd(C+i+r*8+j*n);  //ESPANSIONE
12
13       for( int k = sk; k < sk+DIM_BLOCCO; k++)
14       {
15         __m512d bb = _mm512_broadcastsd_pd(_mm_load_sd(B+j*n+k));
16         for (int r = 0; r < ESPANDI; r++)
17           c[r] = mm512_fmadd_pd(_mm512_load_pd(A+n*k+r*8+i),bb,c[r]);
18       }
19
20       for (int r = 0; r < ESPANDI; r++)
21         _mm512_store_pd(C+i+r*8+j*n, c[r]);
22     }
23 }
24
25 void dgemm (int n, double* A, double* B, double* C)
26 {
27   for (int sj = 0; sj < n; sj += DIM_BLOCCO)
28     for (int si = 0; si < n; si += DIM_BLOCCO)
29       for (int sk = 0; sk < n; sk += DIM_BLOCCO)
30         esegui_blocco(n, si, sj, sk, A, B, C);
31 }
```

Figura 5.48 Versione di DGEMM della Figura 4.78 ottimizzata utilizzando blocchi di cache. Queste modifiche sono le stesse della Figura 5.21. Il linguaggio assembler prodotto dal compilatore per la funzione `esegui_blocco` è quasi identico a quello della Figura 4.79. Anche qui non c'è costo addizionale nella chiamata della funzione `esegui_blocco`, perché la funzione viene inserita direttamente nel codice (modalità inline).

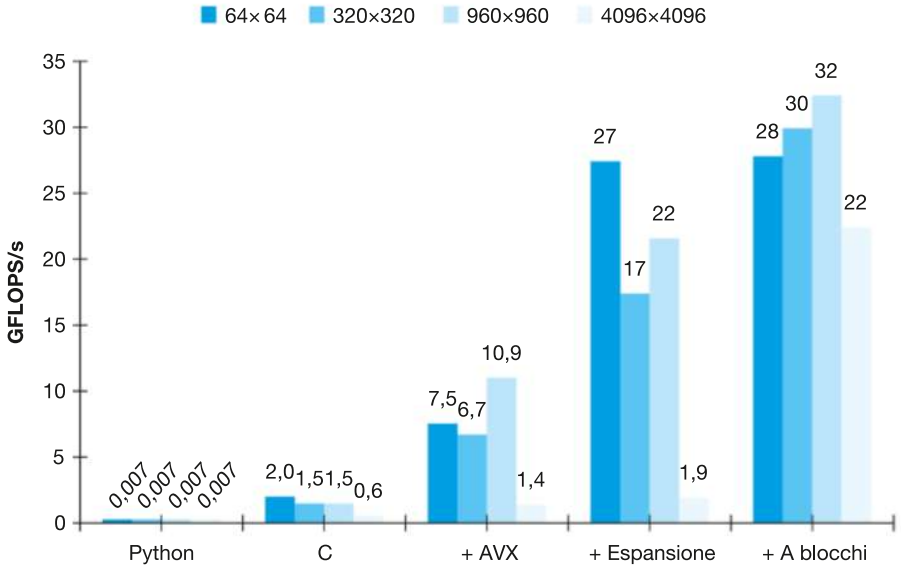


Figura 5.49 Prestazioni di diverse versioni di DGEMM in funzione della dimensione delle matrici, misurate in miliardi di operazioni in virgola mobile al secondo (GFLOPS/secondo). Il codice contenente tutte le ottimizzazioni è da 14 a 32 volte più veloce del codice in C non ottimizzato riportato nel Capitolo 2. Il codice Python viene eseguito a 0,007 GFlops / secondo per tutte le dimensioni delle matrici. L'hardware dell'i7 Intel implementa la speculazione precaricando i dati della cache L3 alle cache L1 e L2. Per questo motivo i benefici della suddivisione in blocchi non poi così grandi come in altri processori.

numero di operazioni in virgola mobile eseguite in un secondo. La Figura 5.49 mostra i risultati del confronto tra le prestazioni della versione di DGEMM originaria in C non ottimizzata, la versione ottimizzata sfruttando il parallelismo a livello di parola, quella ottimizzata sfruttando anche il parallelismo a livello di istruzioni, e quella ottimizzata sfruttando anche la gestione a blocchi della cache. La gestione a blocchi della cache consente di aumentare le prestazioni rispetto al codice AVX, nel quale i cicli sono stati espansi di un fattore compreso tra 1,5 e 1,7, per matrici di medie dimensioni, e di un fattore 10, su matrici di grandi dimensioni. Confrontando la versione non ottimizzata con la versione contenente tutte e tre le ottimizzazioni, si osserva un miglioramento delle prestazioni di un fattore compreso tra 14 e 41, con il miglioramento più ampio ottenuto per le matrici di dimensioni maggiori.

5.16 Errori e trabocchetti

Poiché le gerarchie di memorie sono uno degli aspetti delle architetture dove l'analisi quantitativa conta maggiormente, si potrebbe pensare che siano meno vulnerabili a errori e trabocchetti. Invece, sono stati compiuti molti errori (i cui effetti si sono trascinati per anni) e si è incappati in svariati trabocchetti, alcuni dei quali hanno avuto ricadute estremamente negative. Iniziamo questo paragrafo esaminando due trabocchetti in cui cadono spesso gli studenti quando svolgono gli esercizi.

> ***Trabocchetto**: ignorare il comportamento del sistema di memoria quando si scrivono i programmi o quando si genera il codice tramite compilatore.*

Questo trabocchetto può condurre facilmente all'errore: «I programmatori possono ignorare le gerarchie delle memorie quando scrivono i loro

 © 978-88-08-19966-9

programmi». La valutazione del programma di ordinamento della Figura 5.19 e dell'utilizzo dei blocchi della cache nel Paragrafo 5.14 dimostrano chiaramente che i programmatori possono facilmente raddoppiare le prestazioni dei loro programmi se considerano il comportamento del sistema di memoria quando progettano i loro algoritmi.

> ***Trabocchetto**: dimenticare di tenere conto dell'indirizzamento al byte o della dimensione dei blocchi quando si simula una cache.*

Quando si simula una cache, manualmente o con il calcolatore, bisogna tenere in considerazione l'indirizzamento al byte e il numero di parole di cui sono costituiti i blocchi quando si determina il blocco della cache corrispondente a un certo indirizzo. Per esempio, se abbiamo una cache a mappatura diretta di 32 byte con blocchi di 4 byte, l'indirizzo del byte numero 36 corrisponderà al blocco 1 della cache, dato che l'indirizzo del byte 36 viene mappato sul blocco avente indirizzo 9: 9 modulo 8 = 1. D'altra parte, se l'indirizzo 36 fosse un indirizzo di parola, allora corrisponderebbe al blocco: 36 modulo 8 = 4. Verificate sempre l'unità di misura di base rispetto alla quale viene espresso l'indirizzo.

In maniera analoga occorre tenere in considerazione la dimensione dei blocchi: si ipotizzi di avere una cache di 256 byte con blocchi di 32 byte. In quale blocco cade l'indirizzo 300 espresso in byte? Se scomponiamo l'indirizzo 300 nei diversi campi, la risposta è immediata:

L'indirizzo 300 in byte corrisponde al blocco:

$$\left[\frac{300}{32}\right] = 9$$

Il numero di blocchi nella cache è pari a:

$$\left[\frac{256}{32}\right] = 8$$

Il blocco numero 9 corrisponde pertanto al blocco della cache numero: 9 modulo 8 = 1.

Molte persone commettono questo errore, compresi gli autori di questo libro (che conteneva l'errore nelle bozze!) e i docenti che si dimenticano di specificare se gli indirizzi debbano essere intesi come indirizzi di parole, di byte o di blocco. Ricordatevi di questo trabocchetto quando svolgete gli esercizi.

> ***Trabocchetto:** avere un grado di associatività di una cache condivisa inferiore al numero di core o di thread (Capitolo 6) che condividono quella cache.*

Senza particolari attenzioni, un programma **parallelo** che viene eseguito su 2^n processori o che è costituito da 2^n thread potrebbe facilmente allocare le strutture dati a indirizzi che vengono mappati sulla stessa linea di una cache L2

condivisa. Se la cache ha un grado di associatività pari ad almeno 2^n, gli eventuali conflitti possono essere nascosti al programma dall'hardware; altrimenti, i programmatori si troverebbero a dovere capire l'origine di un calo delle prestazioni apparentemente inspiegabile, ma in realtà dovuto alle miss di conflitto della cache L2. Questo calo di prestazioni, per esempio, può insorgere quando si porta il codice da un'architettura a 16 core su un'architettura a 32 core mantenendo però una cache associativa L2 a 16 vie.

> ***Trabocchetto:*** *utilizzare il tempo di accesso medio alla memoria per valutare la gerarchia delle memorie di un processore fuori ordine.*

Se un processore va in stallo durante una miss della cache, è possibile calcolare separatamente il tempo di stallo dovuto alla memoria e il tempo di esecuzione del processore, e quindi valutare la gerarchia delle memorie indipendentemente utilizzando il tempo medio di accesso alla memoria (Paragrafo 5.4).

Se durante la gestione di una miss il processore continua a eseguire altre istruzioni e può anche gestire altre miss della cache, l'unica valutazione accurata della gerarchia delle memorie consiste nella simulazione del processore fuori ordine e della sua gerarchia delle memorie.

> ***Trabocchetto:*** *estendere lo spazio di indirizzamento aggiungendo altri segmenti a uno spazio di indirizzamento non segmentato.*

Durante gli anni '70 molti programmi crebbero a tal punto che non tutte le istruzioni e i dati potevano essere indirizzati utilizzando solamente i 16 bit a disposizione per l'indirizzo.

I calcolatori furono riprogettati in modo da offrire indirizzi su 32 bit attraverso uno spazio di indirizzamento non segmentato, detto anche *spazio di indirizzamento piatto* (*flat address space*), oppure aggiungendo agli indirizzi esistenti (su 16 bit) altri 16 bit per specificare un segmento. Dal punto di vista del mercato, l'aggiunta dei segmenti – che erano visibili ai programmatori e che obbligavano programmatori e compilatori a scomporre i programmi in segmenti – poteva risolvere il problema dell'indirizzamento. Si verificava però un problema ogni volta che un programma doveva indirizzare dati oltre la dimensione di un singolo segmento, per esempio quando bisognava scorrere mediante un indice gli elementi di un vettore di grandi dimensioni, oppure quando si utilizzavano puntatori senza restrizioni o senza indirizzare parametri. Inoltre, l'aggiunta dei segmenti può spezzare tutti gli indirizzi in due parole, una per il segmento e una per l'offset all'interno del segmento, causando dei problemi nell'utilizzo degli indirizzi che sono contenuti nei registri.

> ***Errore:*** *la frequenza dei guasti di un disco utilizzato in un calcolatore è uguale a quella riportata nelle specifiche.*

Due studi hanno preso in considerazione alcune grandi collezioni di dischi per valutare la relazione tra i risultati ottenuti sul campo e i dati riportati nelle specifiche. Il primo studio riguardava quasi 100 000 dischi che riportavano un MTTF compreso tra 1 000 000 e 1 500 000 ore, ossia un AFR compreso tra lo 0,6 e lo 0,8%; i ricercatori, invece, hanno trovato che l'AFR era compreso tra il 2 e il 4%, valori spesso da tre a cinque volte superiori a quelli riportati nelle specifiche [Schroeder e Gibson, 2007].

Un secondo studio su più di 100 000 dischi di Google, le cui specifiche riportavano un AFR dell'1,5%, ha misurato un AFR dell'1,7% per le unità disco nel loro primo anno di vita, che saliva all'8,6% nel loro terzo anno di vita,

un valore da cinque a sei volte superiore a quello riportato nelle specifiche [Pinheiro, Weber e Barroso, 2007].

> ***Errore:*** *i sistemi operativi sono il posto migliore per programmare gli accessi a disco.*

Come accennato nel Paragrafo 5.2, le interfacce ad alto livello dei dischi consentono al sistema operativo di indirizzare blocchi logici. Essendo presente questo alto livello di astrazione, la cosa migliore che il SO può fare per cercare di migliorare le prestazioni è ordinare gli indirizzi dei blocchi logici in ordine crescente. Tuttavia, poiché è il disco a sapere quale sia la mappatura degli indirizzi logici sui diversi componenti fisici del disco (settori, tracce e piatti), è il controllore del disco che può ridurre la latenza di rotazione e di ricerca riorganizzando gli accessi.

Per esempio, si supponga di avere un carico di lavoro di quattro letture [Anderson, 2003]:

Operazione	Indirizzo di partenza del blocco	Lunghezza
Lettura	724	8
Lettura	100	16
Lettura	9987	1
Lettura	26	128

Il calcolatore potrebbe riordinare le quattro letture a seconda dell'indirizzo logico dei blocchi:

Operazione	Indirizzo di partenza del blocco	Lunghezza
Lettura	26	128
Lettura	100	16
Lettura	724	8
Lettura	9987	1

In base alla posizione relativa dei dati sul disco, la riorganizzazione degli accessi potrebbe determinare un peggioramento delle prestazioni, come mostrato nella **Figura 5.50**. Le letture riorganizzate dal disco richiederebbero solamente tre quarti di un giro completo del disco, mentre la riorganizzazione fatta dal sistema operativo richiederebbe tre giri.

> ***Trabocchetto:*** *implementare il monitor di una macchina virtuale per un'architettura dell'insieme delle istruzioni che non è stata progettata per essere virtualizzabile.*

Molti progettisti negli anni '70 e '80 non consideravano con la dovuta attenzione il fatto che tutte le istruzioni che leggevano o scrivevano informazioni collegate alle risorse dell'hardware dovessero essere istruzioni privilegiate. Questo approccio "lassista" ha causato svariati problemi nella realizzazione dei VMM per molte architetture, compresa la x86 che utilizziamo qui come esempio. La **Figura 5.51** descrive le 18 istruzioni che sono problematiche per la virtualizzazione [Robin e Irvine, 2000]; esse possono essere suddivise in due ampie classi:

- istruzioni di lettura dei registri di controllo in modalità utente, che rivelano che il sistema operativo ospitato sta girando su una macchina virtuale, come l'istruzione POPF, già introdotta in questo capitolo;

- istruzioni di controllo delle protezioni (richieste dalle architetture che utilizzano la segmentazione) che suppongono che il sistema operativo sia in esecuzione con i privilegi più elevati.

Per semplificare le implementazioni dei VMM sugli x86, sia AMD sia Intel hanno introdotto alcune estensioni dell'architettura aggiungendo una nuova

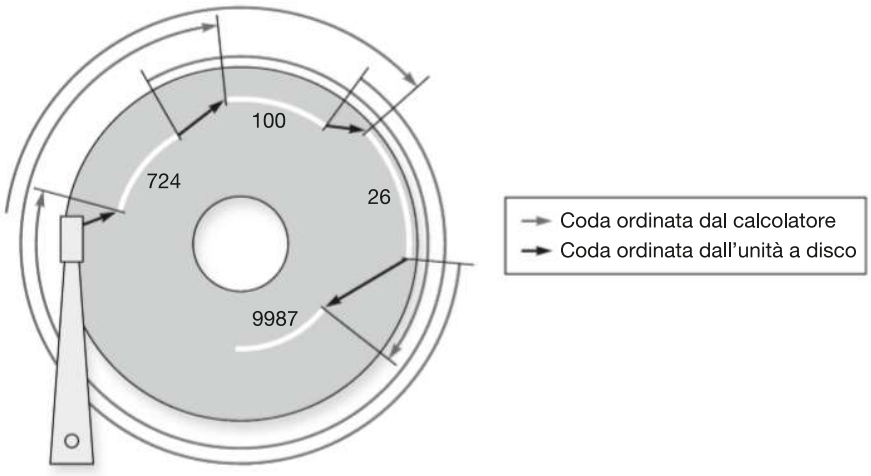


Figura 5.50 Esempio che mostra la differenza tra gli accessi ordinati dal SO e gli accessi ordinati dall'unità a disco. Nel primo caso sono richiesti tre giri del disco per completare le quattro letture, mentre nel secondo caso le letture possono essere effettuate utilizzando solamente tre quarti del tempo di una singola rotazione. (D Anderson, 2003.)

Categoria di problemi	Istruzioni x86 che causano problemi
Registri sensibili all'accesso, che non generano un'eccezione di trap quando l'esecuzione è in modalità utente	Memorizzazione del registro della tabella dei descrittori globali (SGDT) Memorizzazione del registro della tabella dei descrittori locali (SLDT) Memorizzazione del registro della tabella dei descrittori degli interrupt (SIDT) Memorizzazione della parola contenente lo stato della macchina (SMSW) Push dei flag (PUSHF, PUSHFD) Pop dei flag (POPF, POPFD)
Istruzioni che non superano il controllo di protezione dell'x86 quando si accede ai meccanismi della memoria virtuale in modalità utente	Caricamento dei diritti di accesso dal descrittore dei segmenti (LAR) Caricamento dei limiti dei segmenti dal descrittore dei segmenti (LSL) Verifica della leggibilità del descrittore dei segmenti (VERR) Verifica della scrivibilità del descrittore dei segmenti (VERW) Copia da stack dei registri di segmento (POP CS, POP SS, …) Scrittura dei registri di segmento in stack (PUSH CS, PUSH SS, …) Chiamata di tipo far a istruzioni con un diverso livello di privilegio (CALL) Ritorno da procedura di tipo far a istruzioni con un diverso livello di privilegio (RET) Salto di tipo far a istruzioni con un diverso livello di privilegio (JMP) Interrupt software (INT) Memorizzazione del registro di selezione di segmento (STR) Spostamento ai/dai registri di segmento (MOVE)

Figura 5.51 Elenco delle 18 istruzioni dell'x86 che creano problemi alla virtualizzazione. Le prime cinque istruzioni del primo gruppo consentono a un programma in modalità utente di leggere un registro di controllo, per esempio il registro di una tabella di descrizione, senza sollevare un'eccezione di trap. L'istruzione pop dei flag modifica un registro di controllo con informazioni sensibili, ma fallisce quando viene eseguita in modalità utente senza darne avviso. Il controllo delle protezioni dell'architettura segmentata dell'x86, nel secondo gruppo in basso, si è rivelato particolarmente disastroso, dato che ciascuna di queste istruzioni controlla il livello di privilegio in modo implicito durante l'esecuzione dell'istruzione stessa, leggendo uno dei registri di controllo. Il controllo presuppone che il SO si trovi al livello più alto di privilegio, cosa che non succede in una VM ospitata. Solamente l'istruzione di spostamento a uno dei registri di segmento cerca di modificare lo stato di controllo, ma il controllo delle protezioni fa comunque fallire il tentativo di modifica. (Da Robin e Irvine, 2000).

modalità di funzionamento. L'estensione Intel VT-x fornisce una nuova modalità di esecuzione da utilizzare per eseguire le VM, una definizione interna all'architettura dello stato di una VM, delle istruzioni per sostituire velocemente le VM e un ampio insieme di parametri per scegliere in quali circostanze occorra richiamare il VMM. Complessivamente, l'estensione VT-x aggiunge 11 nuove istruzioni all'insieme dell'x86 e l'estensione Pacifica di AMD utilizza soluzioni simili.

Un'alternativa alla modifica dell'hardware è quella di operare delle piccole modifiche al sistema operativo per evitare di utilizzare quelle parti dell'architettura che possono creare problemi. Questa tecnica si chiama *paravirtualizzazione* e il software open source Xen VMM ne è un buon esempio. Lo Xen VMM fornisce un SO ospitato contenente un'astrazione di macchina virtuale che utilizza solamente le parti facilmente "virtualizzabili" dell'hardware fisico dell'x86 sul quale eseguire il VMM.

Trabocchetto: *attacchi hardware possono compromettere la sicurezza*

Anche se i numerosi bachi software dei sistemi operativi sono il veicolo principale degli attacchi ai calcolatori, nel 2015 Google ha dimostrato che un programma utente può minare la protezione offerta dalla memoria virtuale sfruttando una debolezza dei chip di memoria DRAM DDR3.

Data la struttura interna bidimensionale delle DRAM e la piccola dimensione delle celle di memoria delle DRAM DDR3, i ricercatori hanno osservato come "martellare" su una linea di DRAM DDR3 scrivendo ripetutamente su di essa possa causare dei disturbi sulle linee adiacenti, che a possono provocare il cambiamento di uno o più bit. Un attacco ingegnoso potrebbe sfruttare la tecnica del "martellamento su una linea" per cambiare uno o più bit di protezione degli elementi della tabella delle pagine e fornire così al programma l'accesso a regioni di memoria che il sistema operativo sta cercando di proteggere. I microprocessori e le DRAM più recenti includono dei meccanismi per riconoscere gli attacchi da "martellamento su una linea" in modo da poterli sconfiggere.

Questo tipo di attacco ha sorpreso tanti ricercatori i quali, fino a quel momento, pensavano che l'hardware non fosse vulnerabile agli attacchi alla sicurezza. Come vedremo nei trabocchetti del Capitolo 6, il "martellamento su una linea" è stato solamente il primo di una serie di nuove modalità di attacco alla sicurezza dei calcolatori.

5.17 Note conclusive

La difficoltà nel costruire un sistema di memoria che possa tenere il passo di CPU sempre più veloci è dovuta al fatto che il componente di base utilizzato per la memoria principale, il chip di DRAM, è praticamente lo stesso nei calcolatori più veloci e in quelli più lenti ed economici.

È il principio di località che ci dà la possibilità di superare il limite imposto dai lunghi tempi di latenza degli accessi alla memoria, e l'efficacia di questa strategia è dimostrata a tutti i livelli della **gerarchia delle memorie**. Sebbene i diversi livelli di una gerarchia sembrino piuttosto diversi dal punto di vista quantitativo, essi funzionano mettendo in pratica strategie simili e sfruttano le stesse proprietà di località.

Le cache multilivello rendono possibile utilizzare più facilmente diverse ottimizzazioni della cache, per due ragioni.

In primo luogo, i parametri di progetto di una cache di livello più basso nella gerarchia sono differenti da quelli di una cache di primo livello. Per

esempio, poiché una cache di livello inferiore conterrà parole molto più ampie, sarà possibile utilizzare blocchi di dimensioni maggiori. In secondo luogo, una cache di livello più basso non è costantemente utilizzata dal processore, come avviene invece per una cache di primo livello. Questo permette di far fare qualcosa di utile a una cache di livello inferiore mentre è inattiva, per prevenire miss future.

Un'altra strategia utilizzata è quella di "farsi aiutare" dal software: gestire in modo efficace una gerarchia delle memorie ricorrendo a trasformazioni dei programmi o a particolari componenti hardware è uno degli argomenti più caldi nello sviluppo dei compilatori. In questo ambito vengono esplorate due idee principali.

La prima consiste nella riorganizzazione del codice per aumentarne la località spaziale e temporale. Questo approccio si focalizza sui programmi che fanno un uso intensivo dei cicli e che utilizzano vettori di grandi dimensioni come strutture principali (le operazioni di algebra lineare, come DGEMM, ne sono un esempio). Riorganizzando le istruzioni contenute nei cicli che accedono agli elementi dei vettori, si può migliorare significativamente la località e, quindi, le prestazioni delle cache.

Un altro approccio è rappresentato dal **caricamento anticipato** (*prefetching*), in cui un blocco di dati viene portato all'interno della cache prima ancora che venga effettivamente indirizzato. Molti microprocessori utilizzano un caricamento anticipato guidato dall'hardware, cercando di **predire** gli accessi ai dati che sarebbe difficile identificare via software.

Caricamento anticipato (prefetching): una tecnica nella quale i blocchi di dati che serviranno in futuro vengono scritti nella cache in anticipo, utilizzando delle istruzioni speciali che specificano l'indirizzo di questi blocchi.

Un terzo approccio è rappresentato da istruzioni speciali che tengono conto della struttura della cache per ottimizzare il trasferimento dalla memoria. Per esempio, i microprocessori descritti nel Paragrafo 6.10 adottano un'ottimizzazione per la quale quando un programma si accinge a scrivere in cache e si verifica una miss in scrittura, non viene caricato in cache il contenuto del blocco della memoria principale quando il programma si accinge a scrivere l'intero blocco.

Questa ottimizzazione riduce significativamente il traffico con la memoria a carico del processore.

Come vedremo nel Capitolo 6, i sistemi di memoria costituiscono un argomento chiave nella progettazione dei processori paralleli. L'importanza crescente delle gerarchie delle memorie nel determinare le prestazioni dell'intero sistema farà in modo che questo aspetto continuerà a ricevere, nei prossimi anni, una particolare attenzione sia da parte dei progettisti sia da parte dei ricercatori.

5.18 Inquadramento storico e approfondimenti

Questo paragrafo, disponibile online, fornisce una visione di insieme delle tecnologie delle memorie che si sono susseguite negli anni, dalle linee di ritardo al mercurio fino alle DRAM. Esso tratta l'invenzione della gerarchia delle memorie, i meccanismi di protezione e le macchine virtuali, e termina con una breve storia dei sistemi operativi, tra cui CTSS, MULTICS, UNIX, UNIX BSD, MS-DOS, Windows e Linux.

5.19 Aiuto allo studio

Più è meglio? La Figura 5.9 mostra lo stato di una piccola cache a mappatura diretta dopo nove accessi che terminano con l'accesso all'indirizzo 16. Supponiamo che i cinque accessi successivi provengano da un ciclo e riguardino

gli indirizzi 18, 20, 22, 24 e 26. Quanti di questi accessi risultano in una hit? Quale sarà il contenuto della cache al termine di questi accessi?

L'associatività è una proprietà desiderabile? Supponiamo che la cache nella Figura 5.9 sia set-associativa a due vie invece che a mappatura diretta; questo riesce a trasformare in hit le miss associate agli accessi alle celle di memoria di indirizzo 18, 20, 22, 24 e 26? Perché? Utilizzate il modello delle tre C per spiegare il vostro ragionamento.

Analogia con la catena del freddo. Dalla biblioteca alla lavanderia, ci siamo basati su delle analogie per spiegare alcuni argomenti della struttura degli elaboratori. Ora vi chiediamo di cercare di spiegare la similitudine tra la gerarchia delle memorie e la catena del freddo per la conservazione del cibo. Quali livelli e concetti della gerarchia delle memorie si possono associare ai meccanismi e ai dispositivi di refrigerazione per la conservazione del cibo?

1. Il frigorifero della cucina.
2. Il freezer integrato (di solito contenuto nella parte superiore o inferiore del frigorifero).
3. Congelatore a sé stante posto in garage o in cantina.
4. I freezer per il cibo surgelato presso il negozio di alimentari.
5. Grossisti che forniscono il cibo surgelato al negozio di alimentari.
6. Prendere il cibo dal frigorifero per cucinarlo.
7. Tempo richiesto per prendere il cibo dal frigorifero.
8. Mettere il cibo, dopo la cottura, nel frigorifero.
9. Spostare il cibo surgelato dal freezer integrato del frigorifero al frigorifero stesso per scongelarlo prima della cottura.
10. Tempo richiesto per scongelare il cibo contenuto nel freezer integrato nel frigorifero.
11. Spostare il cibo dal frigorifero al freezer integrato per preservarlo, in modo da poterlo mangiare in un secondo tempo.
12. Spostare il cibo surgelato tra il congelatore e il freezer integrato del frigorifero.
13. Acquistare altro cibo dal negozio di alimentari e metterlo nel freezer integrato del frigorifero.

Struttura della catena del freddo. Il Paragrafo 5.8 fornisce uno schema comune per le gerarchie delle memorie. Quali delle caratteristiche di questo schema si possono identificare nella struttura della catena del freddo e quali non si possono identificare?

Le C nella catena del freddo. Il Paragrafo 5.8 descrive anche il modello delle tre C, che è un modello intuitivo che spiega i diversi tipi di miss delle memorie cache. Quali tipi di miss si possono identificare nel caso della catena del freddo? Fornire un'analogia per ciascun tipo di miss e spiegare i motivi per cui eventualmente in alcuni casi non si può definire un'analogia.

I fallimenti nella catena del freddo. Fornire almeno tre esempi nei quali non si può identificare nessuna analogia vicina al funzionamento della gerarchia delle memorie di un'architettura.

Il "martellamento" nelle macchine virtuali. Perché le vulnerabilità dell'hardware agli attacchi quali, per esempio, il "Martellamento su una linea" descritto nel Paragrafo 5.15, sono particolarmente preoccupanti per le società che offrono servizi di cloud computing come Amazon Web Services?

Risposte ai quesiti

Più è meglio? Questi sono i cinque accessi successivi alla cache e il loro risultato:

Indirizzo decimale della cache	Indirizzo binario della cache	Hit o miss della cache	Blocco della cache assegnato (dove si trova)
18	10010	hit	$(10010_{due} \bmod 8) = 010_{due}$
20	10100	miss	$(10100_{due} \bmod 8) = 110_{due}$
22	10110	hit	$(10110_{due} \bmod 8) = 110_{due}$
24	11000	miss	$(11000_{due} \bmod 8) = 000_{due}$
26	11010	miss	$(11010_{due} \bmod 8) = 010_{due}$

In totale si verificano 2 hit e 3 miss per i 5 accessi. La cache appare così dopo l'accesso all'indirizzo 26:

Indice	V	Etichetta	Dati
000	sì	10_{due}	Memoria (11000_{due})
001	no		
010	sì	10_{due}	Memoria (11010_{due})
011	sì	00_{due}	Memoria (00011_{due})
100	sì	10_{due}	Memoria (10100_{due})
101	no		
110	sì	10_{due}	Memoria (10110_{due})
111	no		

L'associatività è una proprietà desiderabile? Le miss associate all'accesso agli indirizzi 20 e 24 sono miss legate a un primo accesso e sono quindi classificate miss obbligate secondo il modello delle tre C. In questo caso l'associatività non può aiutare.

Il blocco della memoria principale di indirizzo 26 era stato indirizzato originariamente nel secondo accesso alla memoria come riportato nel Paragrafo 5.3 e inserito nel blocco 2 della cache. Nella cache a mappatura diretta, questo blocco era poi stato sostituito dal blocco di indirizzo 18 dall'ottava istruzione, che aveva provocato quindi una miss da conflitto dato che il blocco della memoria principale viene mappato anch'esso nel blocco 2 della cache. Una cache set-associativa a due vie avrebbe evitato questa miss da conflitto e avrebbe così prodotto una hit della cache in più di quelle ottenute con la cache a mappatura diretta.

Per capire veramente quanti accessi diventerebbero hit e quanti miss, dobbiamo rivalutare non solo questi cinque accessi ma anche i nove accessi iniziali, per la memoria cache set-associativa a due vie. Dobbiamo cioè capire in quale dei due insiemi viene scritto il blocco a cui fa riferimento ogni accesso per capire l'impatto completo dell'associatività sulla cache, dato che la mappatura dell'indirizzo della memoria principale cambia con l'associatività. Lasciamo questo compito al lettore come esercizio e terminiamo con la semplice osservazione sull'opportunità di evitare una miss di conflitto sul blocco 26.

Analogia con la catena del freddo. Ci sono due interpretazioni plausibili della gerarchia a seconda che si consideri il congelatore come una cache di terzo livello o una memoria principale. Qui discuteremo la risposta basata sulla prima di queste due interpretazioni.

1. Cache di primo livello: il frigorifero della cucina.
2. Cache di secondo livello: il freezer integrato.
3. Cache di terzo livello: congelatore a sé stante posto in garage o in cantina.
4. Memoria principale: i freezer per il cibo surgelato presso nel negozio di alimentari.
5. Memoria secondaria: grossisti che forniscono il cibo surgelato al negozio di alimentari.
6. Lettura della cache di primo livello: prendere il cibo dal frigorifero per cucinarlo.
7. Tempo di hit della cache di primo livello: tempo richiesto per prendere il cibo dal frigorifero.
8. Scrittura della cache di primo livello: mettere il cibo, dopo la cottura, nel frigorifero.
9. Miss della cache di primo livello che richiede il trasferimento dalla cache di secondo livello: spostare il cibo surgelato dal freezer integrato del frigorifero al frigorifero stesso per scongelarlo prima della cottura.
10. Tempo di lettura associato a una hit nella cache di secondo livello: tempo richiesto per scongelare il cibo contenuto nel freezer integrato del frigorifero.
11. Traffico tra le cache di primo e secondo livello, come quello che si verifica quando si ha una miss della cache di primo livello o durante un write-back: spostare il cibo dal frigorifero al freezer integrato per preservarlo, in modo da poterlo mangiare in un secondo momento.
12. Traffico tra le cache di secondo e terzo livello, come quello che si verifica quando si ha una miss della cache di secondo livello o durante un write-back: spostare il cibo surgelato tra il congelatore e il freezer integrato del frigorifero.
13. Miss della cache di terzo livello e lettura del dato dalla memoria principale: acquistare altro cibo dal negozio di alimentari e metterlo nel freezer integrato del frigorifero.

Analogia con la catena del freddo.

- *Dove si può scrivere un blocco?* Non ci sono restrizioni sulla posizione del cibo in tutti livelli della nostra catena del freddo; la migliore analogia è quindi con una memoria completamente associativa a tutti i livelli. L'unica eccezione è il negozio di alimentari, che raggruppa il cibo a seconda del tipo e mantiene una tabella nella quale viene riportata l'associazione tra tipo di cibo e numero del congelatore.
- *Come si trova un blocco di memoria?* Dato che la memoria ha un posizionamento completamente associativo, dobbiamo cercare all'interno dell'intero componente della catena del freddo considerato (a eccezione del negozio di alimentari).
- *Quale blocco verrà sostituito nel caso di una miss della cache?* Plausibilmente sostituiremo il cibo comperato meno di recente, utilizzando la data di scadenza sulla confezione.
- *Cosa succede in una scrittura?* Una gerarchia della memoria generalmente copia i dati senza spostarli. Tuttavia, non abbiamo questa possibilità quando lavoriamo con oggetti fisici per cui l'analogia migliore è con la modalità di scrittura in write-back.

Le C nella catena del freddo. Le tre C sono:

1. Miss obbligate
2. Miss di capacità
3. Miss di conflitto

Una miss obbligata (molto triste) si verifica quando desiderate un gelato al cioccolato, ma non ce n'è nel frigorifero, nel freezer integrato o nel congelatore, per cui dovete recarvi al negozio di alimentari per acquistarne un po'. Anche se potreste non trovare il gelato al cioccolato neppure in negozio – ottenendo così una "page fault del freddo!", avete buone probabilità di trovarne almeno un po' così che potete soddisfare il vostro desiderio, anche se molto più lentamente di quanto avreste sperato inizialmente.

Anche le miss di capacità hanno senso. In queste miss l'articolo desiderato non si trova nel livello che desiderate perché non c'era abbastanza spazio per contenerlo. Dovrete perciò cercarlo nel componente inferiore della catena del freddo, quello immediatamente successivo.

Come nel caso delle cache reali, non si verificano miss di conflitto nel caso di cache completamente associative.

I fallimenti nella catena del freddo. Questi sono i casi nei quali questa analogia non funziona.

- *Dimensione del blocco fissa.* Il cibo può assumere le forme e dimensioni più diverse per cui non esiste un concetto equivalente al blocco di memoria cache. L'analogia migliore potrebbe essere quella con una gavetta di rancio militare, che non è esattamente quello che mangia la maggior parte delle persone!
- *Località spaziale.* Dato che non esiste la dimensione del blocco, è difficile immaginare un'analogia per la località spaziale. L'eccezione è rappresentata dal negozio di alimentari, nel quale diverse confezioni dello stesso cibo sono posizionate una di fianco all'altra, con una evidente località spaziale.
- *Write-back in una cache di terzo livello.* È abbastanza inverosimile che il vostro negozio di alimentari vi lasci restituire del cibo preso dal vostro congelatore con la spiegazione: "Non ho usato questo cibo per molto tempo e devo mettere qualcos'altro nel mio congelatore per cui potete tenere voi questo cibo fino a quando mi servirà?"
- *Miss di una cache di primo livello e integrità dei dati.* Anche se l'analogia funziona abbastanza bene tra i congelatori, la maggior parte del cibo non può essere scongelata e ricongelata senza rovinarla, per cui c'è un problema in questa analogia per le miss di una cache di primo livello. L'equivalente per le architetture degli elaboratori è che i dati vengano distrutti dopo un certo numero di miss della cache, cosa che sarebbe catastrofica se fosse vera e avrebbe impedito completamente l'introduzione delle memorie cache.
- *Inclusività attraverso i diversi livelli.* La politica più popolare di gestione dell'inclusività delle cache prevede che ciascun dato presente in una cache di un certo livello sia presente anche al livello inferiore, dato che è semplice fare una copia dei dati. (La politica write-back e altre situazioni possono portare a valori incoerenti, ma una versione, magari precedente, dei dati è sempre presente nei livelli inferiori.) Non possiamo creare istantaneamente una copia degli oggetti per i livelli inferiori, seguiamo perciò una politica di scrittura esclusiva secondo la quale i dati sono presenti in un solo livello della gerarchia.

Il "martellamento" nelle macchine virtuali. Società come Amazon Web Services possono offrire servizi cloud a prezzi bassi facendo condividere uno stesso server a molte macchine virtuali. Il problema è che la protezione offerta dalla memoria virtuale e dalle macchine virtuali consente ai programmi concorrenti di eseguire in modo sicuro il loro codice nello stesso istante e sullo stesso hardware, dato che i diversi programmi non possono accedere ai

dati sensibili degli altri programmi, fin tanto che AWS assicura che non ci siano bachi nei meccanismi della memoria virtuale. Un attacco hardware come il martellamento su una linea comporta che, anche se il software è stato scritto perfettamente, gli avversari possono impadronirsi del server e dei dati sensibili dei programmi concorrenti.

Il risultato di questa debolezza potenziale è che AWS offre ai suoi clienti la possibilità di garantire che solo i task della loro organizzazione vengano eseguiti sui server che stanno utilizzando, anche se questa opzione richiedeva, nel 2020, un 5% di aumento sulla tariffa oraria.

5.20 Esercizi

Si supponga che la memoria sia indirizzabile al byte e che le parole siano di 64 bit, a meno che non sia specificato diversamente.

5.1 In questo esercizio esamineremo le proprietà di località della memoria mediante il calcolo matriciale. Il seguente frammento di codice è scritto in C; gli elementi delle matrici posti sulla stessa riga sono contenuti in celle contigue della memoria. Si supponga che ciascuna parola sia costituita da interi su 64 bit.

```
for (I=0; I<8; I++)
  for (J=0; J<8000; J++)
    A[I][J] = B[I][0] + A[J][I];
```

5.1.1 [5] <5.1> Quanti interi su 64 bit possono essere contenuti in una linea di cache di 16 byte?

5.1.2 [5] <5.1> L'accesso a quali variabili gode della località temporale?

5.1.3 [5] <5.1> L'accesso a quali variabili gode della località spaziale?

La località è influenzata sia dall'ordine degli accessi sia dalla struttura dei dati. Lo stesso calcolo matriciale può essere scritto in Matlab come riportato sotto; il codice Matlab differisce dal codice C perché sono gli elementi delle colonne delle matrici a essere memorizzati in locazioni adiacenti.

```
for I=1:8
  for J=1:8000
    A(I,J) = B(I,0) + A(J,I);
  end
end
```

5.1.4 [5] <5.1> L'accesso a quali variabili gode della località temporale?

5.1.5 [5] <5.1> L'accesso a quali variabili gode della località spaziale?

5.1.6 [15] <5.1> Quante linee di cache contenenti 16 byte sono necessarie per memorizzare tutti gli elementi della matrice di 64 bit a cui il programma accede, utilizzando la modalità di memorizzazione di Matlab? Quante utilizzando la modalità di memorizzazione del C? Si supponga che ciascuna linea contenga più di un elemento.

5.2 Le cache sono importanti per fornire ai processori una gerarchia delle memorie ad alte prestazioni. Di seguito viene riportata una sequenza di indirizzi della memoria su 64 bit, espressi come indirizzi di parole.

```
0x03, 0xb4, 0x2b, 0x02, 0xbf, 0x58,
0xbe, 0x0e, 0xb5, 0x2c, 0xba, 0xfd
```

5.2.1 [10] <5.3> Per ciascuno di questi indirizzi di memoria, identificare il corrispondente indirizzo binario, il tag e l'indice per una cache a mappatura diretta contenente 16 blocchi di una parola. Riportare anche se ciascun accesso si risolve in una hit o una miss, supponendo che la cache sia inizialmente vuota.

5.2.2 [10] <5.3> Per ciascuno di questi indirizzi di memoria, identificare il corrispondente indirizzo binario, il tag e l'indice per una cache a mappatura diretta contenente otto blocchi di due parole ciascuno. Stabilire anche se ogni accesso si risolve in una hit o una miss, supponendo che la cache sia inizialmente vuota.

5.2.3 [20] <5.3, 5.4> Si deve ottimizzare la struttura della cache per la sequenza di accessi alla memoria riportata sopra. Si può scegliere tra tre tipi di cache a mappatura diretta, tutte contenenti dati per un totale di otto parole:

- la cache C1 ha blocchi di una parola;
- C2 ha blocchi di due parole;
- C3 ha blocchi di quattro parole.

5.3 Per convenzione, la dimensione di una cache indicata si riferisce alla quantità di dati che contiene (per es. una cache da 4 KiB può contenere 4 KiB di dati); tuttavia, le cache richiedono memoria SRAM anche per memorizzare dei metadati come i tag e i bit di validità. In questo esercizio, si esaminerà come la configurazione di una cache influenzi la quantità totale di SRAM richiesta per implementarla e le prestazioni della cache. Per tutti i problemi si supponga che le cache a mappatura diretta, siano indirizzabili al byte e che gli indirizzi e le parole siano su 32 bit.

5.3.1 [10] <5.3> Calcolare il numero totale di bit richiesti per implementare una cache da 32 KiB con blocchi di due parole.

5.3.2 [10] <5.3> Calcolare il numero totale di bit richiesti per implementare una cache da 64 KiB con blocchi da 16 parole. Di quanto è più grande questa cache rispetto alla cache da 32 KiB descritta nell'Esercizio 5.3.1? Si noti che cambiando la dimensione del blocco, si raddoppia la quantità di dati senza raddoppiare la dimensione totale della cache.

5.3.3 [5] <5-3> Spiegare perché questa cache da 64 KiB, nonostante la dimensione maggiore dei dati, potrebbe risultare più lenta della prima cache.

5.3.4 [10] <5.3, 5.4> Generare una serie di richieste di lettura che abbiano una frequenza di miss inferiore in una cache set-associativa a due vie da 32 KiB che nella cache descritta nell'Esercizio 5.3.1.

5.4 [15] <5.3> Il Paragrafo 5.3 mostra il tipico metodo per indicizzare una cache a mappatura diretta, e precisamente (indirizzo di blocco) modulo (numero di blocchi nella cache). Supponendo indirizzi su 32 bit e che la cache contenga 1024 blocchi, consideriamo una differente funzione di indicizzazione, precisamente (indirizzo di blocco[63:54]) XOR (indirizzo di blocco[53:44]). È possibile utilizzare questa funzione per indicizzare una cache a mappatura diretta? Se è possibile, spiegare perché e discutere tutte le modifiche necessarie da apportare alla cache. Se non è possibile spiegare perché.

5.5 I seguenti gruppi di bit degli indirizzi su 32 bit, suddivisi nei diversi campi, vengono utilizzati per l'accesso a una cache a mappatura diretta.

Tag	Indice	Offset
63–10	9–5	4–0

5.5.1 [5] <5.3> Qual è la dimensione della linea della cache (in numero di parole)?

5.5.2 [5] <5.3> Quanti elementi contiene la cache?

5.5.3 [5] <5.3> Qual è il rapporto tra il numero totale di bit contenuti in questa cache e il numero di bit utilizzati per memorizzare i dati?

A partire dall'accensione del calcolatore, si verifica una sequenza di accessi alla cache con i seguenti indirizzi, riferiti al byte (tabella riportata in fondo alla pagina).

5.5.4 [20] <5.3> Per ciascun indirizzo, riportare (1) il suo tag, indice e offset, (2) se l'accesso risulta in una hit o miss, e (3) quali byte vengono sostituiti (in caso di sostituzione).

5.5.5 [5] <5.3> Qual è la frequenza di hit?

5.5.6 [5] <5.3> Riportare lo stato finale della cache, indicando (per ogni blocco che contiene un dato valido) indice, tag e dato contenuto. Per esempio:

```
<0, 3, Mem[0xC00]-Mem[0xC1F]>
```

5.6 Si ricordi che sono due le possibili politiche di scrittura e di allocazione in scrittura, e che diverse loro combinazioni possono essere implementate nella cache L1 o L2. Supponiamo che siano state effettuate queste scelte:

L1	L2
Write-through, No-write-allocate	Write-back, Write-allocate

5.6.1 [5] <5.3, 5.8> I buffer vengono utilizzati per ridurre la latenza di accesso tra livelli di memoria adiacenti. Per le strategie riportate in tabella, elencare i buffer necessari tra la cache L1 e la cache L2 e tra la cache L2 e la memoria.

5.6.2 [20] <5.3, 5.8> Descrivere la procedura per la gestione delle miss in scrittura di L1, considerando il componente coinvolto e la possibilità di sostituire i blocchi che sono stati modificati (quelli per i quali è impostato a 1 il dirty bit).

Indirizzo												
Hex	00	04	10	84	E8	A0	400	1E	8C	C1C	B4	884
Dec	0	4	16	132	232	160	1024	30	140	3100	180	2180

5.6.3 [20] <5.3, 5.8> Per una cache multilivello di tipo esclusivo, cioè nella quale un blocco può essere contenuto o nella cache L1 o nella cache L2, descrivere la procedura per la gestione delle miss in scrittura di L1, considerando i componenti coinvolti e la possibilità di sostituire i blocchi che sono stati modificati.

5.7 Si consideri il programma e il comportamento della cache con le caratteristiche riportate nella tabella in fondo alla pagina.

5.7.1 [10] <5.3, 5.8> Si supponga che una CPU con una cache di tipo write-through e write-allocate, raggiunga un CPI di 2. Qual è la larghezza di banda in lettura e scrittura, misurata in numero di byte per ciclo di clock, tra la RAM e la cache? Si supponga che ciascuna miss generi una richiesta per ciascun blocco.

5.7.2 [10] <5.3, 5.8> Per una cache di tipo write-back e write-allocate, qual è la minima larghezza di banda in lettura e scrittura, richiesta per ottenere un CPI di 2, supponendo che il 30% dei blocchi di dati sostituiti siano blocchi modificati?

5.8 Le applicazioni multimediali che riproducono file audio o video, fanno parte di una famiglia di carichi di lavoro detti *streaming*, letteralmente "flusso continuo", perché acquisiscono dall'esterno una grande quantità di dati ma ne riutilizzano pochi. Si consideri il carico di lavoro di uno streaming video di 512 KiB che accede sequenzialmente ai seguenti indirizzi:

```
0, 1, 2, 3, 4, 5, 6, 7, 8, 9, ...
```

5.8.1 [10] <5.4, 5.8> Si supponga di avere a disposizione una cache a mappatura diretta di 64 KiB con linee di 32 byte. Quale sarà la frequenza di miss per la sequenza di indirizzi riportata in precedenza? Quanto è sensibile la frequenza di miss alla dimensione della cache o dello stream? Come si classificano le miss di questo carico di lavoro secondo il modello delle 3C?

5.8.2 [5] <5.1, 5.8> Ricalcolare la frequenza di miss quando la linea della cache diventa di 16 byte, di 64 byte o di 128 byte. Quale tipo di località viene sfruttata da questo carico di lavoro?

5.8.3 [10] <5.13> L'anticipazione del caricamento (*prefetching*) è una tecnica che fa leva sulla predicibilità delle sequenze di indirizzi per caricare in anticipo i dati nelle linee della cache, ogni volta che si accede a una linea della cache stessa. Un esempio di caricamento anticipato è rappresentato dal buffer di stream: quando vengono caricati i dati in una linea della cache, viene caricato anche il contenuto delle linee successive in un buffer separato. Se i dati si trovano nel buffer di prefetch, viene generata una hit, i dati vengono copiati nella cache e la linea successiva della cache viene caricata nel buffer di prefetch. Si supponga di avere un buffer di stream a due elementi e che la latenza della cache sia tale per cui una linea di cache può essere caricata prima che l'elaborazione del contenuto della linea precedente sia stata completata. Qual è la frequenza di miss per la sequenza di indirizzi riportata?

5.9 La dimensione del blocco della cache, che chiameremo B, influenza sia la frequenza di miss sia la latenza di miss. Data una macchina con un CPI pari a 1 e una media di 1,35 accessi cumulativi per le istruzioni e i dati, individuare la dimensione ottimale del blocco della cache, date le frequenze di miss riportate nella tabella seguente in funzione della dimensione dei blocchi.

8: 4%	16: 3%	32: 2%	64: 1,5%	128: 1%

5.9.1 [10] <5.3> Qual è la dimensione ottimale del blocco per una latenza di miss di 20 × B cicli di clock?

5.9.2 [10] <5.2> Qual è la dimensione ottimale del blocco per una latenza di miss di 24 + B cicli di clock?

5.9.3 [10] <5.2> Data una certa latenza di miss, costante, qual è la dimensione ottimale del blocco?

5.10 In questo esercizio esamineremo i diversi modi in cui la capacità di una cache influenza le prestazioni complessive. In generale, il tempo di accesso di una cache è proporzionale alla sua capacità. Si supponga che gli accessi alla memoria principale richiedano 70 ns e che costituiscano il 36% di tutte le istruzioni. La tabella seguente mostra i dati relativi a una cache L1 collegata a due processori diversi: P1 e P2.

	Dimensione L1	Frequenza di miss L1	Tempo di hit L1
P1	2 KiB	8,0%	0,66 ns
P2	4 KiB	6,0%	0,90 ns

Letture dati ogni 1000 istruzioni	Scritture dati ogni 1000 istruzioni	Frequenza di miss sulla cache istruzioni	Frequenza di miss sulla cache dati	Dimensione del blocco (byte)
250	100	0,30%	2%	64

5.10.1 [5] <5.4> Supponendo che il tempo di hit di L1 determini la durata del periodo del clock di P1 e P2, quale sarà la loro frequenza di clock?

5.10.2 [5] <5.4> Qual è l'*Average Memory Access Time* (AMAT, tempo medio di accesso alla memoria) di P1 e di P2 (in cicli di clock)?

5.10.3 [5] <5.4> Supponendo un CPI di base di 1,0 senza stalli della memoria, quale sarà il CPI totale per P1 e P2? Quale dei due processori è più veloce? (Quando si dichiara un "CPI di base pari a 1,0", intendiamo che le istruzioni vengono completate in un ciclo di clock, a meno che l'accesso ai dati in memoria da parte dell'istruzione non causi una miss.)

Per i tre problemi seguenti, si aggiunga una cache L2 a P1, per esempio per compensare la limitata capacità della cache L1. Utilizzare la capacità e il tempo di hit della cache L1 riportati nella precedente tabella per risolvere questi problemi. La frequenza di miss di L2 riportata nella tabella seguente è la frequenza di miss locale.

Dimensione L2	Frequenza di miss L2	Tempo di hit L2
1 MiB	95%	5,62 ns

5.10.4 [10] <5.4> Qual è l'AMAT di P1 quando si aggiunge la cache L2? L'AMAT è migliore o peggiore con la cache L2?

5.10.5 [5] <5.4> Supponendo un CPI di base di 1,0 senza stalli della memoria, quale sarà il CPI globale per P1 con l'aggiunta della cache L2?

5.10.6 [10] <5.4> Quale dovrebbe essere la frequenza di miss di L2 perché il processore P1 con una cache L2 risulti più veloce del processore P1 senza una cache L2?

5.10.7 [15] <5.4> Quale dovrebbe essere la frequenza di miss di L2 perché P1 con una cache L2 risulti più veloce del processore P2 senza una cache L2?

5.11 Questo esercizio esamina l'impatto di diversi schemi di cache; in particolare verranno confrontate le cache associative con le cache a mappatura diretta descritte nel Paragrafo 5.4. Per questi esercizi si faccia riferimento alla sequenza di indirizzi seguente:

```
0x03, 0xb4, 0x2b, 0x02, 0xbe, 058, 0xbf,
0x0e, 0x1f, 0xb5, 0xbf, 0xba, 0x2e, 0xce
```

5.11.1 [10] <5.4> Delineare la struttura di una cache set-associativa a tre vie con blocchi di due parole e una dimensione totale di 48 parole. La struttura dovrà essere simile a quella riportata nella Figura 5.18, ma dovrà essere indicata chiaramente l'ampiezza del campo tag e dei dati.

5.11.2 [10] <5.4> Riportare il comportamento della cache dell'Esercizio 5.11.1. Si supponga una politica di sostituzione completamente LRU. Per ciascun accesso a cache identificare:

- l'indirizzo binario della parola;
- il tag;
- l'indice;
- l'offset;
- se l'accesso si risolve in una hit o in una miss;
- quale tag è contenuto in ciascuna delle tre vie della cache quando è terminata la gestione dell'accesso alla cache.

5.11.3 [5] <5.4> Delineare la struttura di una cache completamente associativa con blocchi di una parola e una dimensione totale di otto parole. La struttura dovrà essere simile a quella riportata nella Figura 5.18, ma dovranno essere indicati chiaramente l'ampiezza del campo tag e dei dati.

5.11.4 [10] <5.4> Riportare il comportamento della cache dell'Esercizio 5.11.3. Si supponga una politica di sostituzione completamente LRU. Per ciascun accesso a cache identificare:

- l'indirizzo binario della parola;
- il tag;
- l'indice;
- l'offset;
- se l'accesso si risolve in una hit o in una miss;
- il contenuto della cache quando è terminata la gestione di ciascun accesso alla cache.

5.11.5 [5] <5.4> Delineare la struttura di una cache completamente associativa con blocchi di due parole e una dimensione totale di otto parole. La struttura dovrà essere simile a quella riportata nella Figura 5.18, ma dovranno essere indicati chiaramente l'ampiezza del campo tag e dei dati.

5.11.6 [10] <5.4> Riportare il comportamento della cache dell'Esercizio 5.11.5. Si supponga una politica di sostituzione completamente LRU. Per ciascun accesso a cache identificare:

- l'indirizzo binario della parola;
- il tag;
- l'indice;
- l'offset;
- se l'accesso si risolve in una hit o in una miss;
- il contenuto della cache quando è terminata la gestione di ciascun accesso alla cache.

5.11.7 [10] <5.4> Ripetere l'Esercizio 5.11.6 utilizzando la politica di sostituzione MRU (*Most Recently Used*, utilizzato più di recente).

5.11.8 [15] <5.4> Ripetere l'Esercizio 5.11.6 utilizzando la politica ottima di sostituzione (cioè quella che fornisce la frequenza di miss più bassa).

5.12 Le cache multilivello rappresentano una tecnica importante per superare il problema della dimensione limitata dello spazio che una cache di primo livello può mettere a disposizione senza perdere in velocità. Si consideri un processore con i parametri indicati nella tabella seguente.

CPI di base, nessuno stallo della memoria	Velocità del processore	Tempo di accesso alla memoria principale	Frequenza di miss per istruzione della cache di primo livello*	Velocità della cache di secondo livello a mappatura diretta	Frequenza di miss globale con una cache di secondo livello a mappatura diretta	Velocità della cache di secondo livello, set-associativa a otto vie	Frequenza di miss globale con una cache di secondo livello set-associativa a otto vie
1,5	2 GHz	100 ns	7%	12 cicli	3,5%	28 cicli	1,5%

* Frequenza di miss per istruzione per la cache di primo livello. Si supponga che il numero totale di miss della cache L1 (istruzioni e dati) sia uguale al 7% del numero di istruzioni.

5.12.1 [10] <5.4> Calcolare il CPI dei due processori descritti in tabella, utilizzando: (1) solamente una cache di primo livello, (2) una cache di secondo livello a mappatura diretta e (3) una cache di secondo livello set-associativa a otto vie. Come varia il CPI se il tempo di accesso alla memoria principale raddoppia? E se viene dimezzato? (Fornire ogni valore come CPI assoluto e percentuale di variazione.) Si noti fino a che punto una cache L2 può nascondere gli effetti di una memoria lenta.

5.12.2 [10] <5.4> È possibile avere una gerarchia delle memorie su un numero di livelli maggiore di due? Un progettista vorrebbe aggiungere una cache di terzo livello al processore riportato nella precedente tabella, dotato di una cache di secondo livello a mappatura diretta; questa cache di terzo livello richiederebbe 50 cicli di clock per l'accesso e produrrebbe una frequenza di miss globale all'1,3%. Questa modifica fornirebbe prestazioni migliori? In generale, quali sono i vantaggi e gli svantaggi nell'aggiungere un terzo livello di cache?

5.12.3 [20] <5.4> Nei vecchi processori, quali il Pentium di Intel o l'Alpha 21264, la cache di secondo livello era esterna al processore, contenuta in un chip separato da quello contenente il processore e la cache di primo livello. Questo consentiva di avere cache di secondo livello di grandi dimensioni, ma la latenza per accedere a queste cache era molto maggiore e la larghezza di banda tipicamente inferiore, perché la cache di secondo livello lavorava a una frequenza inferiore. Si supponga di avere a disposizione una cache di secondo livello di 512 KiB su un chip separato con una frequenza di miss del 4%. Se ogni 512 KiB aggiunti alla cache riducesse dello 0,7% la frequenza di miss e la cache secondaria avesse un tempo di accesso totale di 50 cicli di clock, quanto dovrebbe essere grande la cache per potere avere le stesse prestazioni della cache di secondo livello a mappatura diretta riportata nella tabella?

5.13 Il tempo medio tra due guasti (MTBF, *Mean Time Between Failures*), il tempo medio di sostituzione (MTTR, *Mean Time To Replacement*) e il tempo di funzionamento medio prima che si verifichi un guasto (MTTF, *Mean Time To Failure*) sono misure utili per valutare l'affidabilità e la disponibilità di una risorsa di memoria di massa. Esploreremo queste misure risolvendo alcuni problemi riguardanti un dispositivo che ha le seguenti specifiche:

MTTF	MTTR
3 anni	1 giorno

5.13.1 [5] <5.5> Calcolare l'MTBF per questo dispositivo.

5.13.2 [5] <5.5> Calcolare la disponibilità di questo dispositivo.

5.13.3 [5] <5.5> Che cosa succede alla disponibilità se l'MTTR si avvicina a 0? È una situazione realistica?

5.13.4 [5] <5.5> Che cosa succede alla disponibilità se l'MTTR diventa molto elevato, per esempio perché il dispositivo è difficile da riparare? Un MTTR molto elevato implica che il dispositivo abbia una disponibilità bassa?

5.14 Questo esercizio esamina il codice di Hamming per la *correzione dell'errore singolo e il rilevamento dell'errore doppio* (SEC/DED, *Single Error Correcting/Double Error Detection*).

5.14.1 [5] <5.5> Qual è il numero minimo di bit di parità richiesti per proteggere una parola di 128 bit utilizzando il codice SEC/DED?

5.14.2 [5] <5.5> Il Paragrafo 5.5 riporta che i moduli di memoria (DIMM) nei server moderni utilizzano il codice ECC SEC/DED per proteggere dati su 64 bit con 8 bit di parità. Determinare il rapporto costo/prestazioni di questo codice e del codice utilizzato nell'Esercizio 5.14.1. In questo caso, il costo è rappresentato dal numero relativo di bit di parità richiesti, mentre le prestazioni sono rappresentate dal numero relativo di errori che possono essere corretti. Quale dei due è migliore?

5.14.3 [5] <5.5> Si consideri un codice SEC che protegge parole di 8 bit con 4 bit di parità. Se leggiamo il numero 0x375 si è verificato un errore? Se sì, correggere l'errore.

5.15 Per un sistema ad alte prestazioni quale l'indicizzazione mediante B-albero (*B-tree*), o albero bilanciato, di un database, la dimensione della pagina è determinata principalmente dalla dimensione dei dati e dalle prestazioni del disco. Si supponga che, in media, una pagina contenente l'albero bilanciato degli indici sia piena al 70% e che contenga elementi di dimensione prefissata. L'utilità di una pagina sta nel livello di profondità del B-albero associato ed è pari al $\log_2$ (numero di elementi). La tabella seguente mostra che la dimensione ottimale della pagina è di 16 KiB, per dati su 16 byte su un disco vecchio di 10 anni con una latenza di 10 ms e una velocità di trasferimento di 10 MB/s.

Dimensione pagina (KiB)	Utilità di pagina o profondità del B-albero (n. di accessi a disco salvati)	Costo di accesso a indice di pagina (ms)	Utilità/ Costo
2	6,49 [o $\log_2(2048/16 \times 0{,}7)$]	10,2	0,64
4	7,49	10,4	0,72
8	8,49	10,8	0,79
16	9,49	11,6	0,82
32	10,49	13,2	0,79
64	11,49	16,4	0,70
128	12,49	22,8	0,55
256	13,49	35,6	0,38

5.15.1 [10] <5.7> Quale sarebbe la dimensione ottimale della pagina se gli elementi diventassero di 128 bit?

5.15.2 [10] <5.7> Facendo riferimento al ragionamento dell'Esercizio 5.15.1, quale sarebbe la dimensione ottimale della pagina se le pagine fossero piene a metà?

5.15.3 [20] <5.7> Seguendo il ragionamento dell'Esercizio 5.15.2, quale sarebbe la dimensione ottimale della pagina se si utilizzasse un disco più recente con una latenza di 3 ms e una velocità di trasferimento di 100 MB/s? Spiegare perché è verosimile che i server futuri abbiano pagine di dimensioni maggiori.

Mantenere nella DRAM le pagine utilizzate più di frequente, dette anche "pagine calde" (*hot page*), può ridurre il numero di accessi a disco; ma come possiamo determinare le hot page per un certo sistema? I progettisti utilizzano il rapporto tra il costo d'accesso alla DRAM e quello d'accesso a disco per determinare una soglia sul tempo di riutilizzo di una pagina; sotto questa soglia una pagina si può considerare "calda". Il costo di un accesso a disco viene misurato come costo del disco/accessi al secondo, mentre il costo per mantenere una pagina in DRAM viene misurato come costo per MiB della DRAM/dimensione della pagina. Il costo tipico delle DRAM e dei dischi e le dimensioni caratteristiche delle pagine dei database riferiti a tre anni particolari sono riportati nella tabella in fondo alla pagina.

5.15.4 [20] <5.7> Quali altri fattori si possono modificare per continuare a mantenere la stessa dimensione della pagina, evitando perciò di dover riscrivere il software? Discutere quanto siano adatti alla tecnologia corrente considerando il trend e i costi.

5.16 Come descritto nel Paragrafo 5.7, la memoria virtuale utilizza una tabella delle pagine per tracciare la traduzione degli indirizzi virtuali in indirizzi fisici. Questo esercizio mostra come questa tabella debba essere aggiornata quando gli indirizzi vengono utilizzati. La tabella in alto alla pagina successiva e quelle che la seguono contengono una sequenza di indirizzi virtuali visti da un sistema. Si supponga che la pagina

Anno	Costo DRAM ($/Mib)	Dimensione della pagina (Kib)	Costo del disco ($)	Frequenza di accesso a disco (accessi/s)
1987	5000	1	15000	15
1997	15	8	2000	64
2007	0,05	64	80	83

Decimale	4669	2227	13916	34587	48870	12608	49225
Esadecimale	0x123d	0x08b3	0x365c	0x871b	0xbee6	0x3140	0xc049

sia di 4 KiB, il TLB sia completamente associativo e contenga 4 elementi, e che venga utilizzata una strategia di sostituzione LRU non approssimata. Se una pagina deve essere copiata dal disco in memoria, incrementare il numero di pagina più grande, tra quelli delle pagine che seguono.

TLB

Valido	Tag	Numero di pagina fisica	Tempo dall'ultimo accesso
1	0xb	12	4
1	0x7	4	1
1	0x3	6	3
0	0x4	9	7

Tabella delle pagine

Indice	Valida	Numero di pagina fisica o su disco
0	1	5
1	0	Disco
2	0	Disco
3	1	6
4	1	9
5	1	11
6	0	Disco
7	1	4
8	0	Disco
9	0	Disco
a	1	3
b	1	12

5.16.1 [10] <5.7> Per ciascuno degli accessi mostrati, riportare:

- se si verifica una hit o una miss del TLB;
- se si verifica una hit o una miss della tabella delle pagine;
- se si verifica un page fault;
- lo stato aggiornato del TLB.

5.16.2 [15] <5.7> Risolvere l'Esercizio 5.16.1, utilizzando una pagina da 16 KiB invece che da 4 KiB. Qual è il vantaggio di utilizzare una pagina di dimensioni maggiori? Quali sono alcuni degli svantaggi?

5.16.3 [15] <5.7> Risolvere l'Esercizio 5.16.1, utilizzando una pagina da 4 KiB e un TLB set-associativo a due vie.

5.16.4 [15] <5.7> Risolvere l'Esercizio 5.16.1, utilizzando una pagina da 4 KiB e un TLB a mappatura diretta.

5.16.5 [10] <5.4, 5.7> Spiegare perché una CPU debba avere un TLB per fornire prestazioni elevate. Come verrebbero gestiti gli accessi alla memoria virtuale se non ci fosse un TLB?

5.17 Diversi sono i parametri che influenzano la dimensione totale della tabella delle pagine; alcuni parametri chiave sono riportati nella tabella seguente:

Dimensione dell'indirizzo virtuale	Dimensione della pagina	Dimensione dell'elemento della pagina
32 bit	8 KiB	4 byte

5.17.1 [5] <5.7> Dati i parametri riportati nella tabella precedente, calcolare la dimensione totale della tabella delle pagine per un sistema che sta eseguendo 5 applicazioni che utilizzano metà della memoria disponibile.

5.17.2 [10] <5.7> Dati i parametri riportati nella tabella precedente, calcolare la dimensione totale della tabella delle pagine per un sistema che sta eseguendo 5 applicazioni che utilizzano metà della memoria disponibile; si consideri un sistema con una tabella delle pagine su due livelli contenente 256 elementi. Si supponga che ciascun elemento della tabella delle pagine principale sia di 6 byte. Calcolare la minima e a massima quantità di memoria richiesta.

5.17.3 [10] <5.7> Un progettista vuole aumentare la dimensione di una memoria cache, attualmente di 4 KiB, contenente tag fisici e indicizzata virtualmente. Considerando le diverse dimensioni di pagina riportate nella tabella precedente, è possibile realizzare una cache a mappatura diretta da 16 KiB, contenente parole di 32 bit per ogni blocco? Che cosa dovrebbe fare il progettista per aumentare la quantità di memoria della cache riservata ai dati?

5.18 In questo esercizio esamineremo l'ottimizzazione delle tabelle delle pagine in termini di capacità e velocità. La tabella in testa alla pagina successiva mostra i parametri di un sistema di memoria virtuale.

Indirizzi virtuali (bit)	Memoria fisica DRAM, installata	Dimensione della pagina	Dimensione del PTE (byte)
43	16 GiB	4 KiB	4

5.18.1 [10] <5.7> Quanti elementi della tabella delle pagine (PTE, *Page Table Element*) sono contenuti in una tabella delle pagine con un singolo livello? Quanta memoria fisica è richiesta per memorizzare la tabella delle pagine?

5.18.2 [10] <5.7> Utilizzando una tabella delle pagine su più livelli si può ridurre la quantità di memoria fisica consumata dalla tabella delle pagine, mantenendo in memoria solamente i PTE che sono attivi. Quanti livelli deve avere la tabella delle pagine se le tabelle di segmento (le tabelle delle pagine del livello superiore) possono avere dimensioni illimitate? Quanti accessi alla memoria sono richiesti per tradurre un indirizzo che non si trova nel TLB?

5.18.3 [15] <5.7> Si supponga che i segmenti siano limitati a una dimensione della pagina di 4 KiB (in modo tale che possano essere paginati). Elementi di 4 byte sono sufficientemente ampi per tutte le tabelle delle pagine comprese quelle dei segmenti?

5.18.4 [10] <5.7> Quanti livelli di tabelle delle pagine sono richiesti se i segmenti sono limitati a una dimensione di pagina di 4 KiB?

5.18.5 [15] <5.7> Una tabella delle pagine invertita può essere utilizzata per ottimizzare ulteriormente lo spazio richiesto e la velocità. Quanti PTE sono necessari per memorizzare una tabella delle pagine? Supponendo un'implementazione mediante tabella *hash*, quale sarà il numero di accessi alla memoria richiesto per gestire una miss del TLB nel caso più comune e nel caso peggiore?

5.19 La tabella in fondo alla pagina mostra il contenuto di un TLB con quattro elementi.

5.19.1 [5] <5.7> In quale situazione verrebbe impostato a 0 il bit di validità associato all'elemento 3?

5.19.2 [5] <5.7> Che cosa succede quando un'istruzione scrive nella pagina virtuale il cui indirizzo è 30? In quali casi un TLB gestito via software si può rivelare più veloce di un TLB gestito via hardware?

5.19.3 [5] <5.7> Che cosa succede quando un'istruzione scrive nella pagina con indirizzo virtuale 200?

5.20 In questo esercizio esamineremo l'impatto sulla frequenza di miss delle diverse politiche di sostituzione. Si supponga una cache set-associativa a 2 vie contenente 4 blocchi di una parola. Considerare la seguente sequenza di indirizzi di parola:

0, 1, 2, 3, 4, 2, 3, 4, 5, 6, 7, 0, 1, 2, 3, 4, 5, 6, 7, 0

5.20.1 [5] <5.4, 5.8> Supponendo che venga utilizzata una strategia di sostituzione di tipo LRU, quante hit si verificano in questa sequenza di accessi?

5.20.2 [5] <5.4, 5.8> Supponendo che venga utilizzata una strategia di sostituzione di tipo MRU (*Most recently used*, blocco utilizzato più di recente), quante hit si verificano in questa sequenza di accessi?

5.20.3 [5] <5.4, 5.8> Simulare una politica di sostituzione casuale ottenuta lanciando in aria una moneta; per esempio, se esce "testa" viene eliminato il primo blocco di un insieme e se esce "croce" viene eliminato il secondo blocco di un insieme. Quante hit si verificano in questa sequenza di accessi?

5.20.4 [10] <5.4, 5.8> Quale blocco deve essere eliminato a ogni sostituzione in modo da massimizzare il numero di hit? Quante hit si verificherebbero in questa sequenza di accessi se si seguisse questa strategia ottimale?

5.20.5 [10] <5.4, 5.8> Spiegare perché è difficile implementare una politica di sostituzione dei blocchi di una cache che risulti ottimale per tutte le possibili sequenze di accessi.

5.20.6 [10] <5.4, 5.8> Si supponga di poter decidere per ogni accesso alla memoria se inserire in cache il dato richiesto o meno. Quale impatto avrebbe ciò sulla frequenza di miss?

5.21 Uno dei maggiori ostacoli alla diffusione delle macchine virtuali è la diminuzione delle prestazioni dovute all'esecuzione della macchina virtuale stessa. Nella tabella in testa alla pagina successiva sono riportati alcuni parametri di valutazione delle prestazioni e alcune caratteristiche delle applicazioni.

ID elemento	Valido	Ind Virt Pagina	Modificata	Protezione	Ind Fisico Pagina
1	1	140	1	RW	30
2	0	40	0	RX	34
3	1	200	1	RO	32
4	1	280	0	RW	31

CPI di base	Accessi a istruzioni privilegiate del SO ogni 10 000 istruzioni	Impatto sulle prestazioni delle eccezioni di trap sul SO ospitato	Impatto sulle prestazioni delle eccezioni di trap sul VMM	Accesso all'I/O ogni 10 000 istruzioni	Tempo di accesso dell'I/O, comprendente il tempo per la gestione delle eccezioni di trap per il SO ospitato
1,5	120	15 cicli di clock	175 cicli di clock	30	1100 cicli di clock

5.21.1 [10] <5.6> Calcolare il CPI per il sistema riportato nella tabella in testa alla pagina supponendo che non ci siano accessi al sistema di I/O. Qual è il CPI se raddoppia l'impatto sulle prestazioni del VMM? E se viene ridotto della metà? Se una società volesse produrre software per macchine virtuali che venga eseguito con una diminuzione delle prestazioni del 10%, quale sarebbe la massima durata della gestione dell'eccezione di trap per richiamare il VMM?

5.21.2 [15] <5.6> Gli accessi all'I/O hanno spesso un grande impatto sulle prestazioni complessive del sistema. Calcolare il CPI di una macchina utilizzando i parametri di prestazione riportati nella tabella in testa alla pagina, supponendo un sistema non virtualizzato. Calcolare il CPI per un sistema virtualizzato. Come cambierebbe il CPI se il sistema effettuasse la metà degli accessi di I/O? Spiegare perché la virtualizzazione ha un impatto minore sulle applicazioni legate all'I/O. Spiegare perché applicazioni limitate dall'I/O soffrono di un impatto inferiore della virtualizzazione.

5.22 [15] <5.6, 5.7> Confrontare e discutere i concetti su cui sono basate la memoria virtuale e le macchine virtuali. Confrontare gli obiettivi dei due sistemi. Quali sono i vantaggi e gli svantaggi di ciascuno dei due sistemi? Riportare alcuni casi nei quali è desiderabile avere a disposizione una memoria virtuale e altri nei quali è desiderabile avere a disposizione una macchina virtuale.

5.23 [10] <5.6> Il Paragrafo 5.6 tratta la virtualizzazione sotto l'ipotesi che il sistema virtualizzato sia basato sulla stessa ISA dell'hardware su cui viene eseguito. Tuttavia, uno dei possibili utilizzi della virtualizzazione è l'emulazione delle ISA non native. Un esempio è il QEMU, che è un emulatore di diverse ISA, tra cui MIPS, SPARC e PowerPC. Quali sono alcuni dei problemi che si incontrano in questo tipo di virtualizzazione? È possibile che un sistema emulato possa girare più velocemente che sull'ISA nativo?

5.24 In questo esercizio esamineremo l'unità di controllo di una cache per un processore dotato di buffer di scrittura. Utilizzare la Figura 5.39 come punto di partenza per progettare la macchina a stati finiti. Si supponga che l'unità di controllo della cache debba gestire la semplice cache a mappatura diretta discussa nel Paragrafo 5.9 con l'aggiunta di un buffer di scrittura delle dimensioni di un blocco.

Si ricordi che lo scopo del buffer di scrittura è fornire un supporto alla memorizzazione temporanea dei blocchi in modo tale che il processore non debba aspettare due accessi alla memoria quando si verifica una miss su un blocco di cache che è stato modificato. Invece di scrivere questo blocco direttamente nella memoria principale, il processore scrive nel buffer di scrittura il blocco da scaricare dalla cache in modo tale da iniziare subito a leggere il nuovo blocco dalla memoria principale. Il blocco scaricato verrà poi trasferito dal buffer di scrittura nella memoria principale mentre il processore sta lavorando.

5.24.1 [10] <5.8, 5.9> Che cosa succederebbe se il processore inviasse una richiesta che genera una hit della cache mentre si sta trasferendo un blocco dal buffer di scrittura alla memoria principale?

5.24.2 [10] <5.8, 5.9> Che cosa succederebbe se il processore inviasse una richiesta che genera una miss della cache mentre si sta trasferendo un blocco dal buffer di scrittura alla memoria principale?

5.24.3 [30] <5.8, 5.9> Progettare una macchina a stati finiti che utilizzi un buffer di scrittura.

5.25 La coerenza della cache riguarda il modo in cui i blocchi di una cache vengono visti da diversi processori. La tabella seguente mostra due processori e le loro operazioni di lettura/scrittura su due diverse parole del blocco X della cache. Si supponga inizialmente X[0] = X[1] = 0.

P1	P2
`X[0] ++; X[1] = 3;`	`X[0] = 5; X[1] += 2;`

5.25.1 [15] <5.10> Elencare i valori che può assumere un blocco della cache con un'implementazione corretta di un protocollo che forzi la coerenza. Elencare almeno uno dei valori che può assumere il blocco considerato se il protocollo non garantisce la coerenza.

5.25.2 [15] <5.10> Per un protocollo di snooping, riportare una sequenza valida di operazioni su

ciascuna coppia processore/cache per completare le operazioni di lettura/scrittura riportate nella tabella dell'Esercizio 5.25.

5.25.3 [10] <5.10> Qual è il numero minimo e massimo di miss della cache richiesto per completare le istruzioni di lettura/scrittura riportate sopra?

La consistenza della memoria riguarda il modo in cui vengono visti dati multipli. La tabella che segue mostra due processori e le loro operazioni di lettura/scrittura su diversi blocchi di cache; si supponga che A e B contengano inizialmente 0.

P1	P2
`A = 1; B = 2; A+=2; B++;`	`C = B; D = A;`

5.25.4 [15] <5.10> Elencare i valori che possono assumere C e D per un'implementazione che soddisfa i requisiti di consistenza della memoria elencati all'inizio del Paragrafo 5.10.

5.25.5 [15] <5.10> Riportare una o più coppie di valori di C e D per le quali quei requisiti non sono soddisfatti.

5.25.6 [15] <5.3, 5.10> Quali combinazioni di strategie di scrittura e di allocazione in scrittura (write-allocate) rendono più semplice l'implementazione del protocollo?

5.26 I multiprocessori su singolo chip (CMP, *Chip MultiProcessors*) contengono più core, ciascuno con la propria cache montata sullo stesso chip. Sui CMP, la cache L2 è il risultato di interessanti compromessi: la tabella seguente mostra la frequenza di miss e la latenza di hit per due benchmark quando si utilizza una cache L2 privata oppure una cache L2 condivisa. Si supponga che la cache L1 abbia una frequenza di miss del 3% e un tempo di accesso di 1 ciclo di clock.

	Privata	Condivisa
Benchmark A, miss per istruzione	10%	4%
Benchmark B, miss per istruzione	2%	1%

La tabella seguente mostra le latenze di hit:

Cache privata	Cache condivisa	Memoria principale
5	20	180

5.26.1 [15] <5.13> Qual è il modello migliore di cache per ciascuno di questi due benchmark? Utilizzate dei dati per supportare le vostre conclusioni.

5.26.2 [15] <5.13> La larghezza di banda per le comunicazioni tra il chip CMP e gli altri componenti dell'architettura diventa il collo di bottiglia con l'aumentare del numero di core del CMP. Come vengono influenzate le cache private e le cache condivise da questo collo di bottiglia? Scegliere il modello migliore quando la latenza della memoria esterna al chip raddoppia.

5.26.3 [10] <5.13> Discutere i vantaggi e gli svantaggi di una cache L2 privata e di una cache L2 condivisa, per un carico di lavoro a singolo thread, multi-thread e multiprogramma; ripetere il ragionamento supponendo di avere a disposizione sul chip del processore anche una cache L3.

5.26.4 [10] <5.13> Una cache L2 non bloccante fornirebbe un miglioramento superiore su un CMP con una cache L2 condivisa o una cache L2 privata? Perché?

5.26.5 [10] <5.13> Si supponga che le nuove generazioni di processori raddoppino il numero di core ogni 18 mesi. Di quanto dovrebbe aumentare la larghezza di banda della memoria esterna al chip nei processori che verranno prodotti fra tre anni per mantenere lo stesso livello di prestazioni per singolo core?

5.26.6 [15] <5.13> Si consideri l'intera gerarchia delle memorie. Quali tipi di ottimizzazione possono ridurre il numero di miss concorrenti?

5.27 In questo esercizio mostreremo la definizione del log (registrazione degli eventi) di un web server ed esamineremo le ottimizzazioni del codice per migliorare la velocità di elaborazione del log. La struttura dati utilizzata per il log è definita come segue:

```
struct entry {
  int IPsorgente // indirizzo IP remoto
  char URL[128]; // URL richiesto, per esempio
                 // "GET index.html"
  long long tempoRif; // tempo di riferimento
  int stato; // stato della connessione
  char browser[64]; // nome del browser
                    // del client
} log[NUM_ELEM];
```

Si supponga la seguente funzione di elaborazione del log:

```
topK_IPsorgente(int ora);
```

Questa funzione determina l'IP richiesto più di frequente in un'ora.

Miss del TLB ogni 1000 istruzioni	Latenza delle miss del TLB per la tabella delle pagine annidata	Page fault ogni 1000 istruzioni	Incremento del tempo di gestione dei page fault nella paginazione ombra
0,2	200 cicli di clock	0,001	30 000 cicli di clock

5.27.1 [5] <5.15> A quali campi di una variabile contenente il log occorre accedere per eseguire la funzione riportata sopra? Supponendo che ci siano blocchi di cache di 64 byte e che non ci sia caricamento anticipato, quante miss della cache per ogni log, in media, sono causate da questa funzione?

5.27.2 [5] <5.15> Come può essere riorganizzata la struttura dati per migliorare l'utilizzazione della cache e la località degli accessi?

5.27.3 [10] <5.15> Fornire un altro esempio di funzione di elaborazione del log che utilizzi preferibilmente una diversa struttura dati. Se entrambe le funzioni fossero importanti, come si potrebbe riscrivere il programma per migliorare le prestazioni globali? Supportare la risposta con frammenti di codice e dati.

5.28 Per i problemi riportati di seguito, utilizzare i dati pubblicati in "Cache Performance for SPEC CPU2000 Benchmarks" (www.cs.wisc.edu/multifacet/misc/spec2000cache-data/) e, in particolare, i dati dei benchmark seguenti:

a.	`Mesa/gcc`
b.	`mcf/swim`

5.28.1 [10] <5.15> Per una cache contenente 64 KiB di dati con diversi gradi di associatività, qual è la frequenza di miss per i diversi tipi di miss (a freddo, di capacità e di conflitto) per ciascuno dei due benchmark?

5.28.2 [10] <5.15> Scegliere il grado di associatività da utilizzare per una cache dati L1 da 64 KiB, utilizzata in entrambi i benchmark. Se la cache L1 deve essere a mappatura diretta, scegliere il grado di associatività per la cache L2 da 1 MiB.

5.28.3 [20] <5.15> Fornire un esempio nel quale un aumento del grado di associatività comporta una maggiore frequenza di miss. Costruire una configurazione di cache e una sequenza di indirizzi che dimostri questa affermazione.

5.29 Per supportare macchine virtuali multiple, sono necessari due livelli di virtualizzazione della memoria. Ciascuna macchina virtuale continua a controllare la mappatura degli *indirizzi virtuali* (VA, *Virtual Address*) negli *indirizzi fisici* (PA, *Physical Address*), mentre il supervisore mappa gli *indirizzi fisici* (PA) di ciascuna macchina virtuale negli *indirizzi della macchina* (MA, *Machine Address*) effettivi del calcolatore utilizzato. Per accelerare queste mappature, un approccio software chiamato *shadow paging* (paginazione ombra) duplica la tabella delle pagine di ciascuna macchina virtuale nel supervisore e intercetta le modifiche della mappatura da VA a PA in modo da mantenere consistenti entrambe le copie. Per eliminare la complessità delle tabelle delle pagine ombra, un approccio hardware chiamato *tabella delle pagine annidata* (NPT, *Nested Page Table*) supporta in modo esplicito due tipi di tabelle delle pagine: la prima trasforma i VA in PA e la seconda trasforma i PA in MA; queste tabelle possono essere interrogate dall'hardware.

Si consideri la seguente sequenza di operazioni: (1) creazione di un processo; (2) miss del TLB; (3) page fault; (4) cambio di contesto.

5.29.1 [10] <5.6, 5.7> Per questa sequenza di operazioni, che cosa succede nella tabella ombra delle pagine e nella tabella delle pagine annidata?

5.29.2 [10] <5.6, 5.7> Supponendo che entrambe le tabelle delle pagine dell'x86, sia quella annidata che quella ospitata dalla macchina virtuale, siano su quattro livelli, quanti accessi alla memoria servono per gestire una miss del TLB per la tabella delle pagine nativa e per quella annidata?

5.29.3 [15] <5.6, 5.7> Quali sono le misure più importanti per le tabelle delle pagine ombra tra: frequenza di miss del TLB, latenza di miss del TLB, frequenza di page fault e latenza per la gestione dei page fault? Quali sono le più importanti per le tabelle delle pagine annidate?

La tabella in alto alla pagina mostra i parametri di un sistema di paginazione ombra.

5.29.4 [10] <5.6> Quale diventa il CPI per un benchmark con un CPI di esecuzione nativo di 1, se si utilizzano tabelle delle pagine ombra o tabelle delle pagine annidate, supponendo che l'incremento del tempo di esecuzione sia solamente dovuto alla virtualizzazione della tabella delle pagine?

5.29.5 [10] <5.6> Quali tecniche possono essere utilizzate per ridurre l'incremento del tempo di esecuzione dovuto alle tabelle delle pagine ombra?

5.29.6 [10] <5.6> Quali tecniche si possono utilizzare per ridurre l'incremento del tempo di esecuzione dovuto all'utilizzo delle tabelle delle pagine annidate?

Risposte alle domande di autovalutazione

Paragrafo 5.1, pagina 343 – 1. e 4.: la 3 è falsa perché il costo della gerarchia delle memorie varia a seconda del calcolatore, ma nel 2013 il costo più elevato era solitamente dovuto alla DRAM.

Paragrafo 5.3, pagina 362 – 1. e 4.: una penalità di miss inferiore autorizza blocchi di dimensioni inferiori, dato che non c'è una grande latenza da ammortizzare; tuttavia, una larghezza di banda della memoria più ampia è associata solitamente a blocchi più ampi, dato che la penalità di miss diventa solo di poco più grande.

Paragrafo 5.4, pagina 379 – 1.

Paragrafo 5.8, pagina 419 – 2.: sia i blocchi di grandi dimensioni sia il prefetching possono ridurre le miss obbligate, perciò l'affermazione 1 è falsa.